BCSE303L - Operating Systems



Module 3 – Scheduling

Module:3	Scheduling	9 hours
Processes Scheduling - CPU Scheduling: Pre-emptive, non-pre-emptive - Multiprocessor scheduling – Deadlocks - Resource allocation and management - Deadlock handling mechanisms: prevention, avoidance, detection, recovery.		

Dr. C.R.Dhivyaa

Assistant Professor Senior Grade-2

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

CPU Scheduling

- CPU Scheduling is the basis of Multi Programmed OS
- By switching the CPU among processes, the OS can make the computer more productive.
- **Multiprogramming:**
  - **Objective** – to have some process running all the time – to maximize CPU utilization
  - A process is executed until it must wait – typically for the completion of I/O requests.
  - Several processes are kept in the memory at one time – ready queue, waiting queue…
  - When one process has to wait, the OS takes the CPU away from that process and gives the CPU to other process and this pattern continues.

CPU and I/O Burst Cycles

- Process Execution consists of a cycle of CPU execution and I/O wait
- Processes alternates between these two states.
- Process Execution begins with

CPU burst That is followed by an

I/O burst, which is followed by another

CPU burst, then another

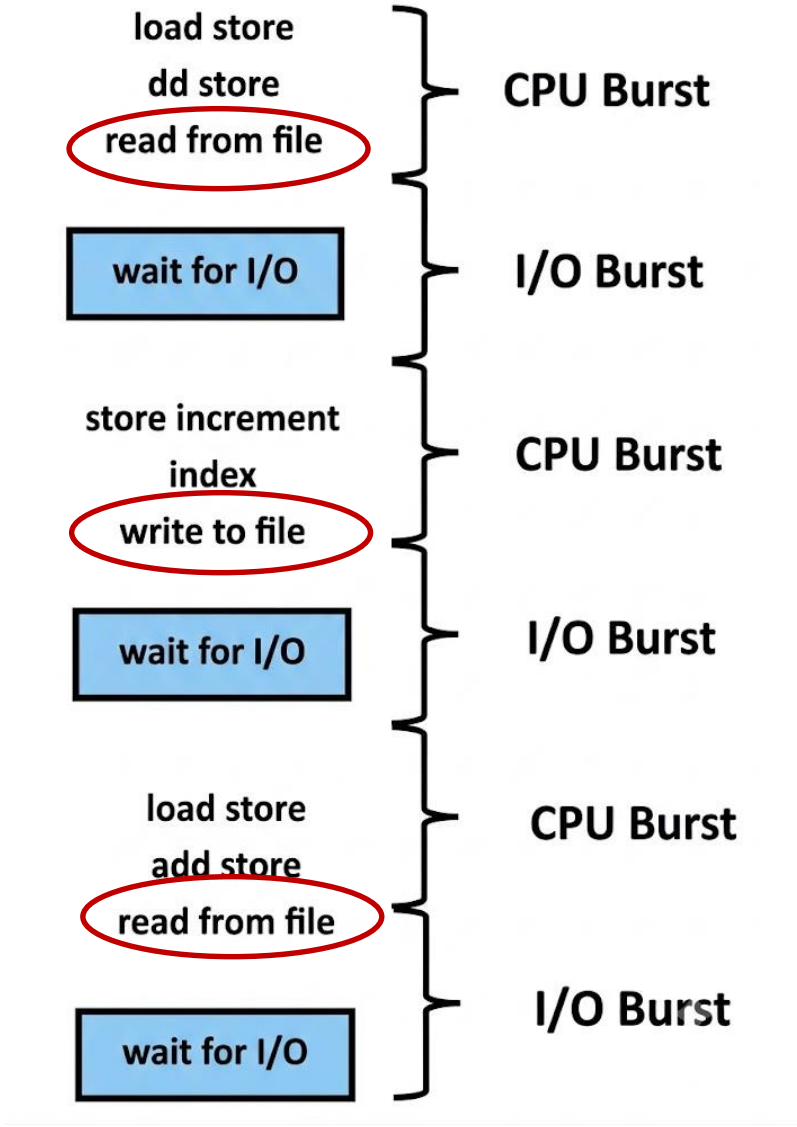
I/O burst,

and so on...

CPU burst is when the process is being executed in the CPU

I/O burst is when the CPU is waiting for I/O for further execution

CPU and I/O Burst Cycles



Alternative Sequence of CPU and I/O Bursts

CPU Scheduler & Dispatcher

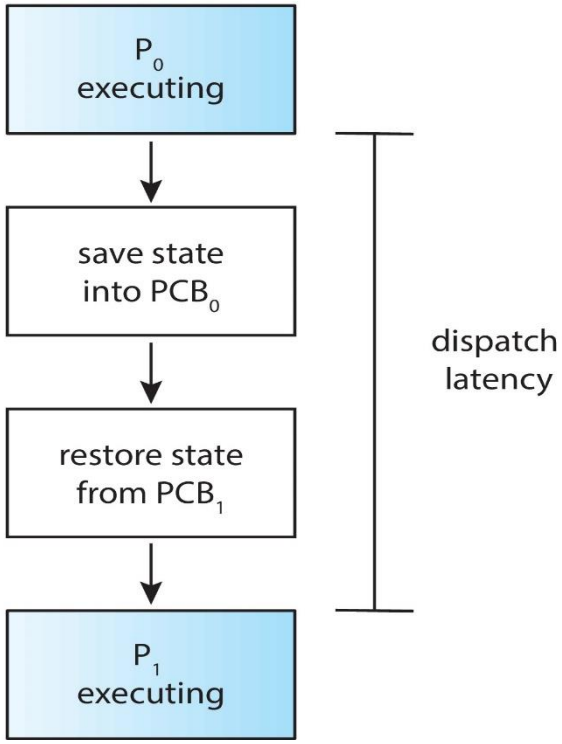
CPU Scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the short-term scheduler (or CPU scheduler). The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

Dispatcher

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler.

The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.



Scheduler – Selects the process

Dispatcher – Dispatches the selected process to CPU – should be done faster

Types of Schedulers

Scheduler	Function	Frequency
Long-Term Scheduler	Selects processes from **secondary storage** and **loads them into memory**	Rarely
Medium-Term Scheduler	**Suspends and resumes processes** to manage memory	Occasionally
Short-Term Scheduler (CPU Scheduler)	Selects the **next ready process to execute**	Very frequently (milliseconds)

Scheduling – Why it is Needed

Modern operating systems support multiprogramming, where many processes reside in memory simultaneously however:

- There is usually only one CPU core (or a limited number of cores).
- Multiple processes cannot execute on the same CPU core simultaneously.
- Therefore, the OS must decide which process gets the CPU next.

The goals of scheduling are				
Maximize CPU utilization	Improve system throughput	Reduce waiting time	Improve response time	Ensure fairness among processes

CPU Scheduling & Process Scheduling

Process Scheduling	CPU Scheduling
Overall management of process execution	Decides which ready process gets the CPU
Includes admission, suspension, and termination	Focuses only on CPU allocation
Performed by long-, medium-, and short-term schedulers	Performed by the short-term scheduler
Concerned with the entire process lifecycle	Concerned with selecting the next process to run
Related to all Process States	Mostly related to running State

CPU Scheduling

CPU scheduling decisions may take place under the following four circumstances:

1. When a process switches from the **running state to the waiting state**.
2. When a process switches from the **running state to the ready state** (for example, **when an interrupt occurs**).
3. When a process switches from the **waiting state to the ready state** (for example, **at completion of I/O**).
4. When a **process terminates**.

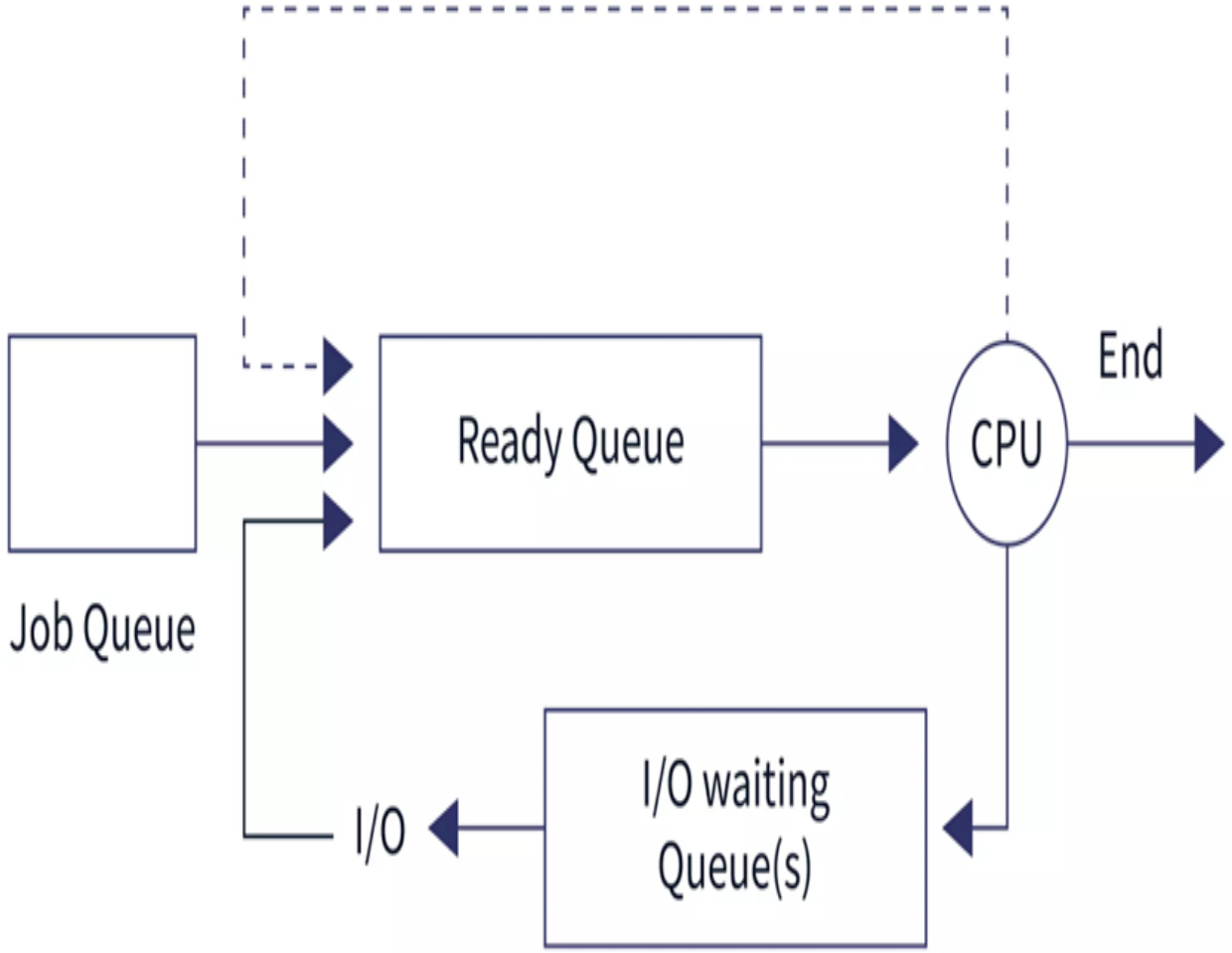
Note:

- **For situations 1 and 4, there is no choice in terms of scheduling.**
  - A new process (if one exists in the ready queue) **must** be selected for execution.
- **However, there is a choice for situations 2 and 3.**

CPU Scheduling – How it works

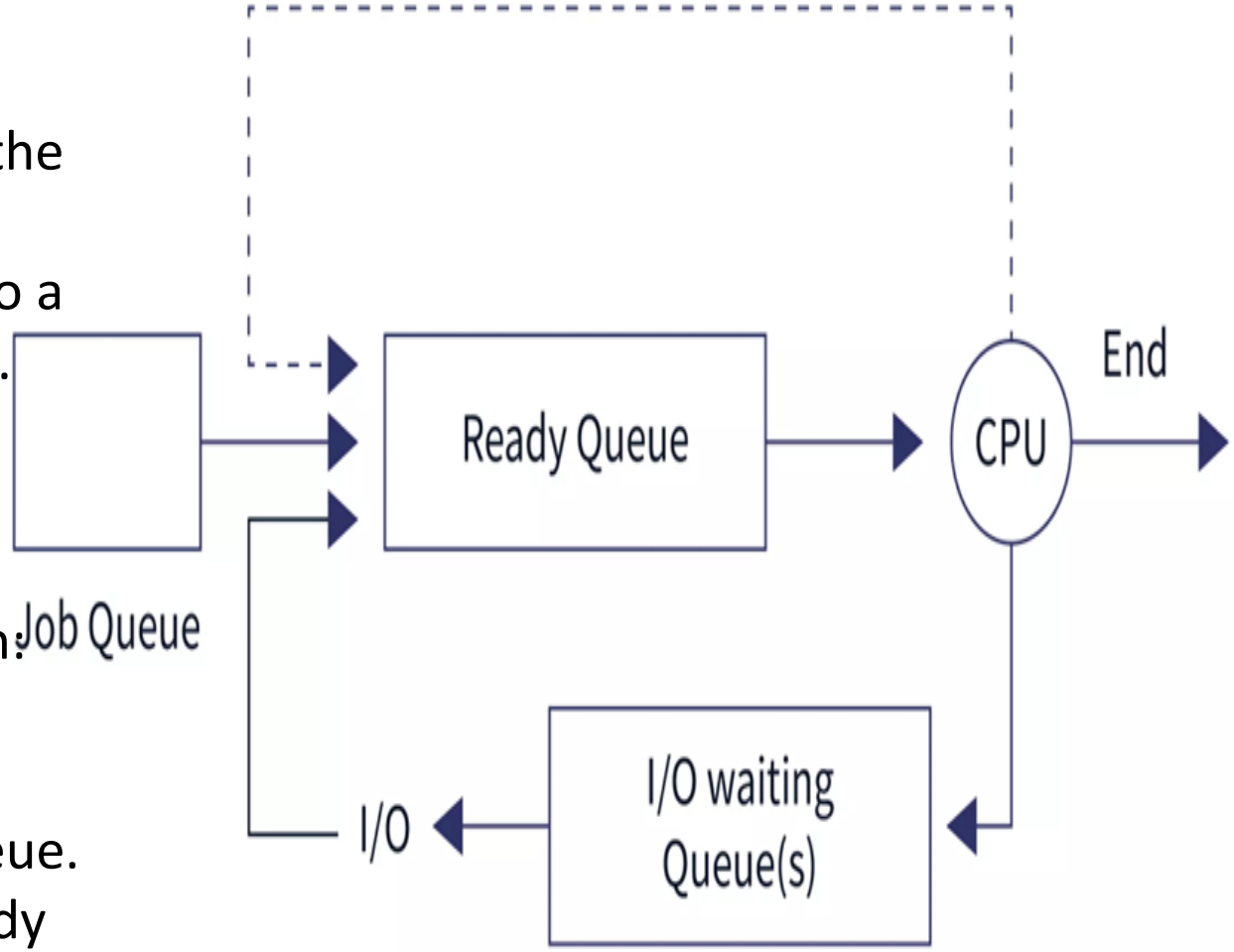
The **CPU scheduler** operates whenever:

- A process is created.
- A process terminates.
- A process requests I/O and blocks.
- An I/O operation completes.
- A timer interrupt occurs (preemptive scheduling).



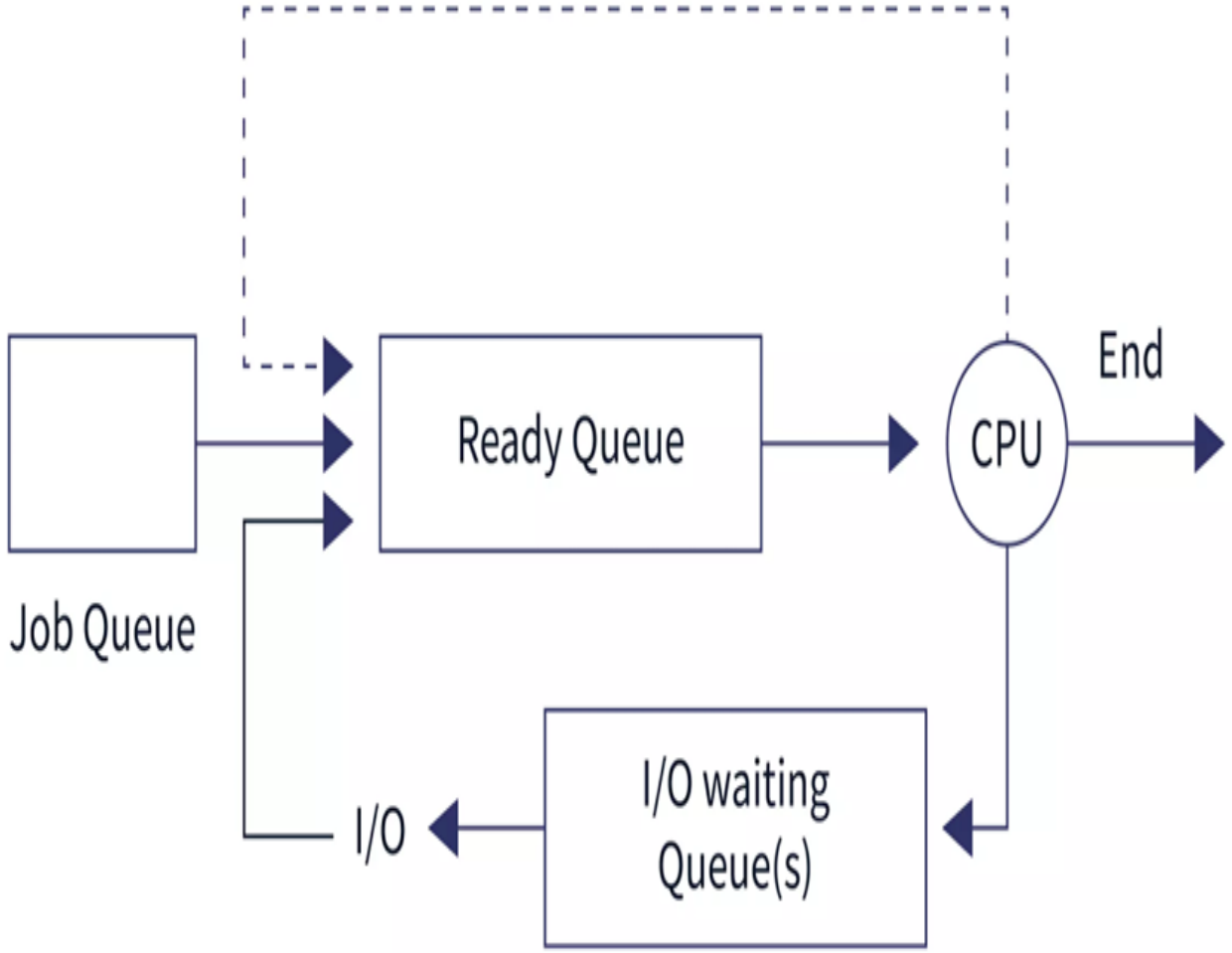
CPU Scheduling – How it works

- Multiple processes are loaded into memory.
- Processes waiting for CPU execution are placed in the **Ready Queue**.
- The CPU Scheduler selects one process according to a scheduling algorithm (e.g., FCFS, SJF, Round Robin).
- The selected process is assigned to the CPU by the **Dispatcher**.
- The process executes.
- During execution, one of the following may happen:
  - The process completes.
  - It requests I/O and moves to the Waiting state.
  - It is preempted and returned to the Ready Queue.
- The scheduler repeats the process for the next ready process.



CPU Scheduling – How it works

- **Ready Queue**
  - Stores all processes waiting for CPU execution.
- **CPU Scheduler**
  - Chooses the next process from the Ready Queue based on the scheduling algorithm.
- **Dispatcher**
  - Transfers CPU control to the selected process.
  - Dispatcher responsibilities include:
    - Context switching
    - Switching to user mode
    - Jumping to the appropriate instruction

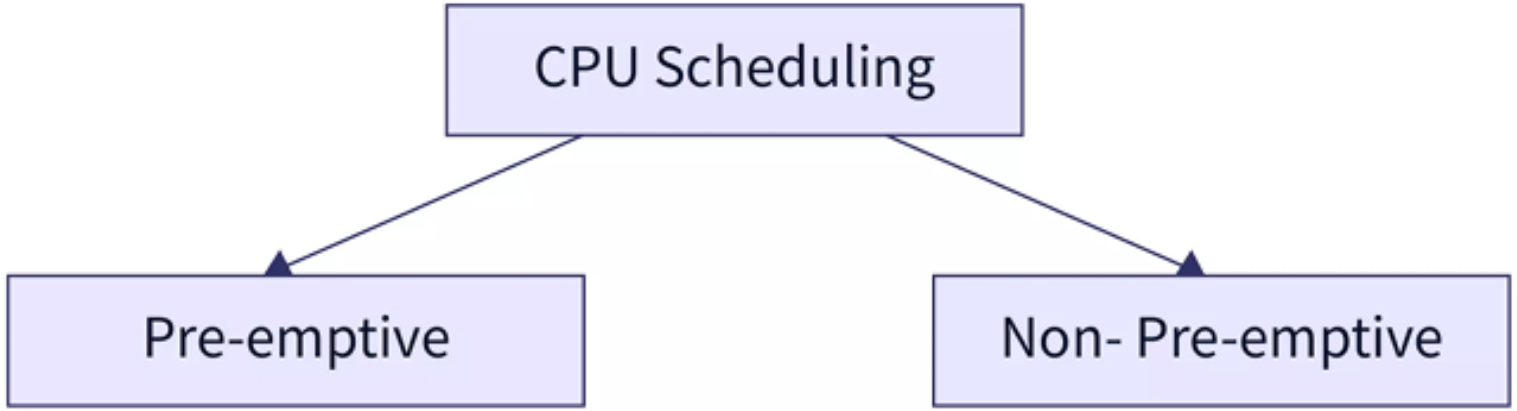


CPU Scheduling – Burst Time

Burst Time is the **amount of time a process spends executing a particular operation** before it switches to another operation.

Feature	CPU Burst Time	I/O Burst Time
Definition	Time during which a process executes instructions on the CPU.	Time during which a process waits for an input/output device to complete an operation.
Who Performs the Work?	CPU (Processor)	I/O Devices (Disk, Keyboard, Network, Printer, etc.)
Process State	Running	Waiting (Blocked)
CPU Usage	CPU is actively executing the process.	CPU is free to execute another process while the current process waits.
Resource Utilized	CPU	I/O Devices
Scheduler Involvement	CPU scheduler selects which process gets the CPU.	I/O scheduler/device controller manages the I/O request.
Typical Operations	Arithmetic calculations, loops, sorting, encryption, program execution	Reading files, writing files, database access, network communication, printer output
Ends When	CPU execution finishes or the process requests I/O.	I/O operation completes and the process becomes ready again.
Measured In	Milliseconds (ms) or CPU cycles	Milliseconds (ms)
Example Duration	4 ms of computation	20 ms waiting for disk read

CPU Scheduling



Preemptive Scheduling	Non-Preemptive Scheduling
CPU can be taken away from a running process	Running process keeps CPU until completion or blocking
Better responsiveness	Simpler implementation
Suitable for interactive systems	Suitable for batch systems
Example: Round Robin	Example: FCFS

CPU Scheduling
Preemptive and Non-Preemptive Scheduling

- When scheduling takes place only under circumstances 1 and 4, we say that the scheduling scheme is **non-preemptive or cooperative**;
  - Only the CPU is take away, once it is terminated or taken to waiting state
  - When a process is running, it will not be disturbed.
- otherwise, it is **preemptive**.
  - Even when the process is running and ready to be executed, it will be interrupted and CPU is given to other process.
  - CPU can be taken away while it is running state.

Note:

- Selection of Preemptive/Non-preemptive scheduling – depends on the OS and the scenarios. Both are preferred. --- Priority tasks needs preemptive scheduling..

Virtually all modern operating systems including Windows, MacOS, Linux, and UNIX use preemptive scheduling algorithms.

CPU Scheduling - Scheduling Criteria

CPU utilization

keep the CPU as busy as possible

We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).

Throughput

of processes that complete their execution per time unit

If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput.

Turnaround time

total time taken by a process from its submission to its completion

From the point of view of a particular process, the important criterion is how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the turnaround time. Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.

CPU Scheduling - Scheduling Criteria

Waiting time →

Amount of time a process has been waiting in the ready queue

The CPU scheduling algorithm does not affect the amount of time during which a process executes or does I/O; it affects only the amount of time that a process spends waiting in the ready queue. Waiting time is the sum of the periods spent waiting in the ready queue.

Response time →

Amount of time it takes from when a request was submitted until the first response is produced.

In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first response is produced. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response. The turnaround time is generally limited by the speed of the output device.

CPU Scheduling – Performance Metrics

Metric	Definition	Goal	Formula
CPU Utilization	Percentage of time the CPU is busy	Maximize	$\frac{CPU\ Busy\ Time}{Total\ Time} \times 100$
Throughput	Number of processes completed per unit time	Maximize	$\frac{Number\ of\ Completed\ Processes}{Total\ Time}$
Turnaround Time (TAT)	Total time from process arrival to completion	Minimize	$Completion\ Time\ - Arrival\ Time$
Waiting Time (WT)	Total time a process spends waiting in the Ready Queue	Minimize	$Turnaround\ Time\ - CPU\ Burst\ Time$
Response Time (RT)	Time from process arrival until it first gets the CPU	Minimize	$First\ CPU\ Allocation\ Time - Arrival\ Time$
Context Switches	Number of times the CPU switches between processes	Minimize	It introduces overhead because no useful computation is performed during the switch
Fairness	Ensures each process receives an equitable share of CPU time	Maximize	

CPU Scheduling - Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

TAT = Waiting Time + CPU Burst Time + I/O Burst Time

TAT = Waiting Time + CPU Burst Time

In **non-preemptive scheduling**, **Response Time = Waiting Time** because the process starts execution only once.

In **preemptive scheduling**, **Response Time ≤ Waiting Time** since a process may be preempted after its first execution.

CPU Scheduling Algorithms

- **First-Come, First-Served (FCFS) :** The simplest CPU scheduling algorithm in which processes are executed in the order in which they arrive.
- **Shortest Job First (SJF) :** This algorithm schedules processes based on their expected CPU burst time. The process with the shortest expected burst time is executed first.
- **Priority Scheduling :** This algorithm assigns a priority value to each process, and processes with higher priority values are executed first.
- **Round Robin (RR) :** This algorithm allocates a fixed time slice (**time quantum**) to each process in a cyclic order. If a process has not completed its execution within its allotted time slice, it is preempted, and the next process in the ready queue is executed.
- **Multilevel Queue Scheduling :** This algorithm divides processes into separate queues based on their characteristics. Each queue has its own scheduling algorithm, and processes are assigned permanently to one of these queues.
- **Multilevel Feedback Queue Scheduling (MLFQ) :** This algorithm is an extension of multilevel queue scheduling that allows processes to move between queues based on their behavior. For example, a process that uses a lot of CPU time may be moved to a lower-priority queue to make room for other processes.

CPU Scheduling Algorithms – Non Preemptive

Feature	FCFS (First-Come, First-Served)	SJF (Shortest Job First)	Priority Scheduling (Non-Preemptive)
Scheduling Type	Non-Preemptive	Non-Preemptive	Non-Preemptive
How it Works	The CPU executes the process that arrives first. Once execution begins, the process runs until completion.	Among all available processes, the one with the smallest CPU burst time is selected. Once started, it runs to completion.	The scheduler selects the ready process with the highest priority (usually the lowest priority number). The selected process executes until it finishes.
Selection Basis	Arrival Time	Shortest Burst Time	Highest Priority
CPU Allocation	Until completion	Until completion	Until completion
Need to Know Burst Time?	No	Yes	No
Need Priority Value?	No	No	Yes
Response to Newly Arrived Process	Waits until current process finishes	Waits until current process finishes	Waits until current process finishes
Starvation Possibility	No	Yes (long processes may wait indefinitely)	Yes (low-priority processes may starve)
Convoy Effect	High	Very Low	Moderate
Waiting Time	Usually High	Minimum among non-preemptive algorithms	Depends on assigned priorities
Turnaround Time	High	Low	Depends on priorities
Implementation Complexity	Very Simple	Moderate	Moderate
Advantages	Easy to implement, fair according to arrival order	Produces minimum average waiting time	Suitable for critical or important tasks
Disadvantages	Poor response time, convoy effect	Requires burst time prediction, starvation possible	Starvation of low-priority processes
Best Used For	Batch processing with simple scheduling	Batch systems where burst time is known	Real-time or priority-based systems

CPU Scheduling Algorithms – Preemptive

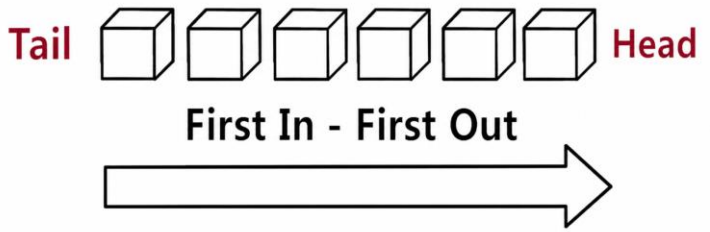
Feature	SRTF (Shortest Remaining Time First)	Priority Scheduling (Preemptive)	Round Robin (RR)	Multilevel Feedback Queue (MLFQ)
How it Works	The CPU always executes the process with the shortest remaining burst time. If a newly arrived process has a shorter remaining time, it immediately preempts the current process.	The CPU executes the highest-priority process. If a newly arrived process has a higher priority, it immediately preempts the running process.	Each process is given a fixed time quantum. If it does not finish within that quantum, it is preempted and moved to the end of the ready queue.	Processes are organized into multiple queues with different priority levels and time quanta. CPU schedules the highest-priority queue first. Processes may move between queues based on their CPU usage and behavior.
Selection Basis	Shortest Remaining Burst Time	Highest Priority	Circular Ready Queue	Queue Priority and Scheduling Policy
Preemption	Yes	Yes	Yes	Yes
CPU Allocation	Until preempted or completed	Until preempted or completed	Fixed Time Quantum	Depends on queue rules
Need to Know Burst Time?	Yes	No	No	Usually No
Need Priority Value?	No	Yes	No	Dynamic priorities
Response to Newly Arrived Process	Immediately checks remaining time and preempts if shorter	Immediately checks priority and preempts if higher	New process joins ready queue and waits for its turn	New process generally enters the highest-priority queue
Starvation Possibility	Yes (long processes)	Yes (low-priority processes)	No	Very Low (aging and queue promotion help)
Context Switches	High	Moderate to High	High	Very High
Waiting Time	Very Low	Depends on priorities	Moderate	Low
Turnaround Time	Very Low	Depends on priorities	Moderate	Low
Response Time	Excellent	Excellent for high-priority processes	Good	Excellent
Implementation Complexity	Moderate	Moderate	Simple	Very High
Advantages	Produces minimum average waiting and turnaround time	Important tasks receive immediate CPU access	Fair CPU sharing among all processes	Excellent balance between responsiveness and throughput
Disadvantages	Frequent context switches, starvation possible	Starvation of low-priority processes	Performance depends on time quantum; excessive context switches	Difficult to implement and tune
Best Used For	Interactive systems with predictable burst times	Real-time and priority-driven systems	Time-sharing and multi-user operating systems	Modern general-purpose operating systems (e.g., UNIX-like schedulers)

CPU Scheduling - FCFS

First Come First Serve Scheduling

FCFS Policy - is Non-Preemptive

- Simplest CPU-scheduling algorithm.
- The process that requests the CPU first is allocated the CPU first.
- The implementation of the FCFS policy is easily managed with a FIFO queue.
- FIFO (First In – First Out):
  - New processes are added at the Tail of the queue.
  - The process at the Head of the queue is executed first.



- When a process enters the ready queue, its PCB is linked onto the tail of the queue.
- When the CPU is free, it is allocated to the process at the head of the queue.
- The running process is then removed from the queue.

Note: The Average Waiting time with FCFS Policy - is High
Not suitable for Time-Sharing Systems

CPU Scheduling - FCFS

- **Example 1**

FCFS Policy - is Non-Preemptive

Process	Burst Time
P_1	24
P_2	3
P_3	3

Assume that – All process arrive at the same time at 0 milliseconds.

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
- The Gantt Chart for the schedule is:



- Waiting time for $P_1 = 0; P_2 = 24; P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

Suppose that the processes arrive in the order:

P_2 , P_3 , P_1

- The Gantt chart for the schedule is:



- Waiting time for $P_1 = 6; P_2 = 0, P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case

Convoy Effect

If processes with higher burst time arrived before the processes with smaller burst time, then, smaller processes have to wait for a long time for longer processes to release the CPU.

CPU Scheduling - FCFS

- **Example 1**

FCFS Policy - is Non-Preemptive

Process	Burst Time
P_1	24
P_2	3
P_3	3

Assume that – All process arrive at the same time at 2ms

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
- The Gantt Chart for the schedule is:

- Waiting time for P_1 = 0; P_2 = 24; P_3 = 27
- Average waiting time: (0 + 24 + 27)/3 = 17

CPU Scheduling - FCFS

- **Example 2**

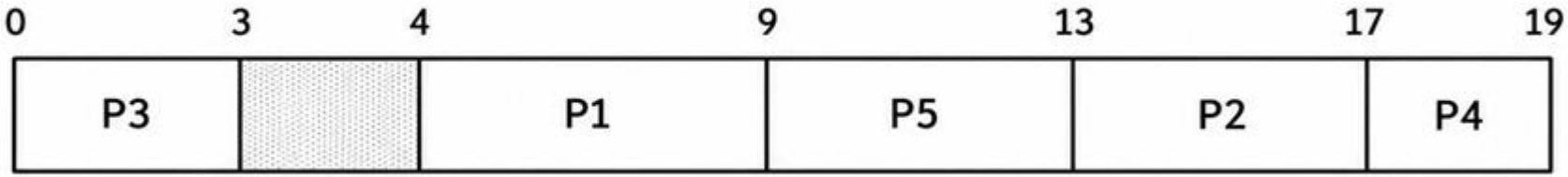
Consider the set of 5 processes whose arrival time and burst time are given below:

Process ID	Arrival Time	Burst Time
P1	4	5
P2	6	4
P3	0	3
P4	6	2
P5	5	4

Calculate the average waiting time and average turnaround time, if FCFS Scheduling Algorithm is followed.

Solution:

Gantt Chart:



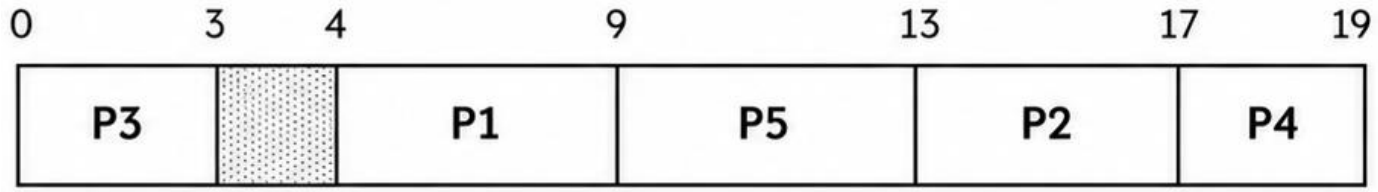
The shaded box represents the idle time of CPU

CPU Scheduling - FCFS

- **Example 2**

Process ID	Arrival Time	Burst Time
P1	4	5
P2	6	4
P3	0	3
P4	6	2
P5	5	4

Gantt Chart:



Turn Around time = Completion time – Arrival time

Waiting time = Turn Around time – Burst time

Process ID	Completion Time	Turnaround Time (CT – AT)	Waiting Time (TAT – BT)
P1	9	9 – 4 = 5	5 – 5 = 0
P2	17	17 – 6 = 11	11 – 4 = 7
P3	3	3 – 0 = 3	3 – 3 = 0
P4	19	19 – 6 = 13	13 – 2 = 11
P5	13	13 – 5 = 8	8 – 4 = 4

CPU Scheduling - FCFS

- **Example 2**

Process ID	Arrival Time	Burst Time
P1	4	5
P2	6	4
P3	0	3
P4	6	2
P5	5	4

Process ID	Completion Time	Turnaround Time (CT – AT)	Waiting Time (TAT – BT)
P1	9	9 – 4 = 5	5 – 5 = 0
P2	17	17 – 6 = 11	11 – 4 = 7
P3	3	3 – 0 = 3	3 – 3 = 0
P4	19	19 – 6 = 13	13 – 2 = 11
P5	13	13 – 5 = 8	8 – 4 = 4

Average Turn Around time = (5 + 11 + 3 + 13 + 8) / 5
= 40 / 5
= 8 units

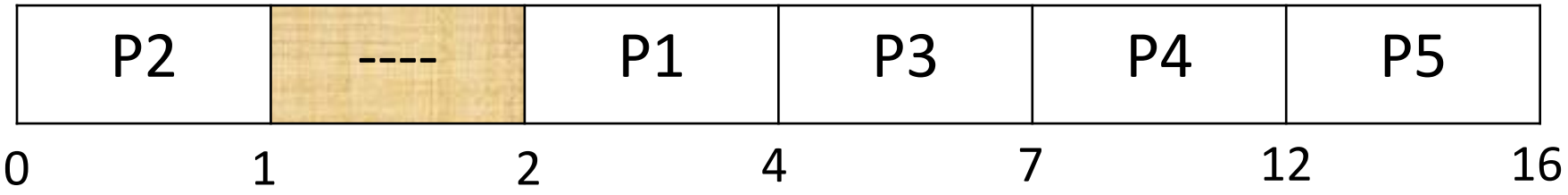
Average waiting time = (0 + 7 + 0 + 11 + 4) / 5
= 22 / 5
= 4.4 units

- Example 3

CPU Scheduling - FCFS

FCFS Policy - is Non-Preemptive

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT-AT)	Waiting Time (TAT-BT)	Response Time (1st CPU Allocation Time – AT)
P1	2	2	4	2	0	0
P2	0	1	1	1	0	0
P3	2	3	7	5	2	2
P4	3	5	12	9	4	4
P5	4	4	16	12	8	8
			Total: 40	29	14	14
			Avg: 40/5 =8	29/5 =5.8	14/5 =2.8	14/5=2.8

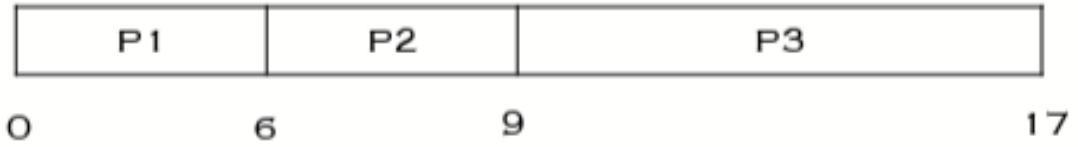


CPU Scheduling - FCFS

- **Example 4 - Process Arrive at Same Time**

  - Consider the three processes. Their arrival time is 0.

Process	Burst Time
P1	6
P2	3
P3	8



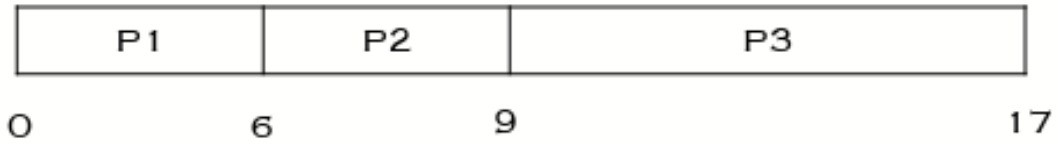
  - Turnaround Time of P1 = completion time – Arrival Time = 6 – 0 = 6
  - Turnaround Time of P2 = 9 – 0 = 9
  - Turnaround Time of P3 = 17 – 0 = 17
  - Waiting Time of P1 = Turnaround Time – Burst Time = 6 – 6 = 0
  - Waiting Time of P2 = 9 – 3 = 6
  - Waiting Time of P3 = 17 – 8 = 9
  - Avg Turnaround time = 6 + 9 + 17/3 = 10.67
  - Avg Waiting time = 0 + 6 + 9/3 = 5

CPU Scheduling - FCFS

- **Example 5 - Process Arrive at Different Time**

❑ Consider the three processes.

Process	Arrival Time	Burst Time
P1	0	6
P2	4	3
P3	6	8



❑ Turnaround Time of P1 = completion time – Arrival Time = 6 – 0 = 6

❑ Turnaround Time of P2 = 9 – 0 = 9

❑ Turnaround Time of P3 = 17 – 0 = 17

❑ Waiting Time of P1 = Turnaround Time – Burst Time = 6 – 6 = 0

❑ Waiting Time of P2 = 9 – 3 = 6

❑ Waiting Time of P3 = 17 – 8 = 9

❑ Avg Turnaround time = 6 + 9 + 17/3 = 10.67

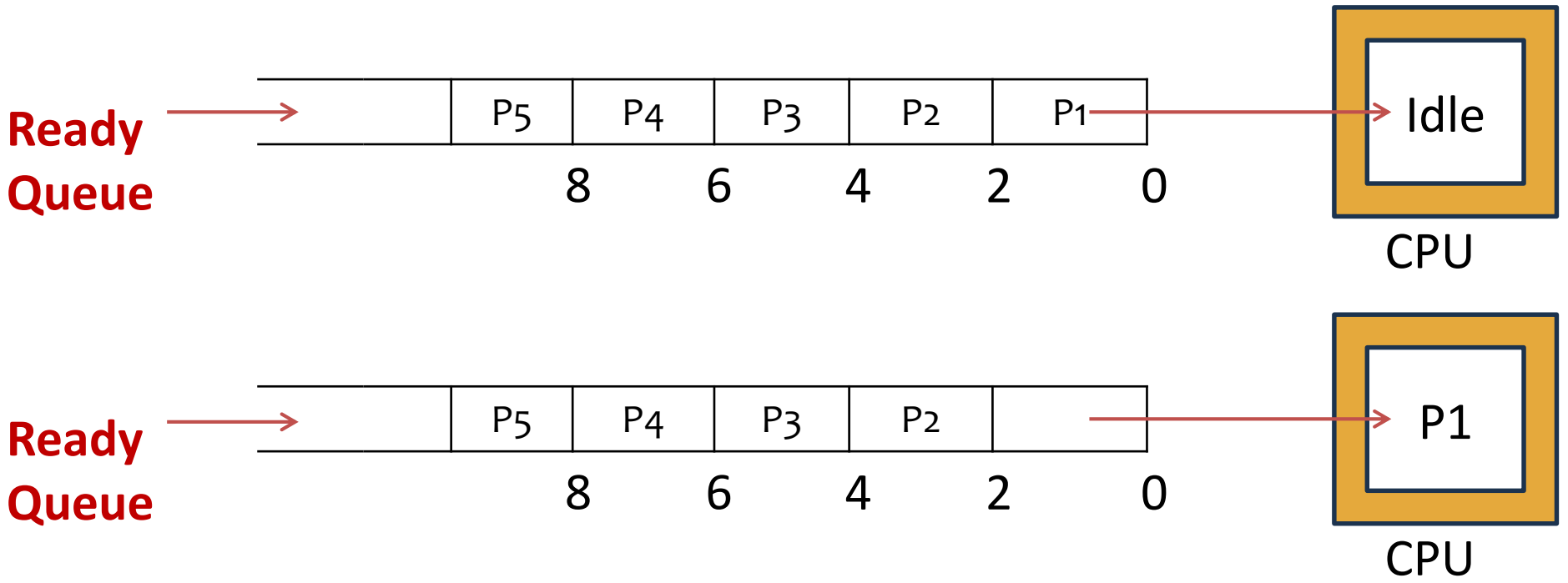
❑ Avg Waiting time = 0 + 6 + 9/3 = 5

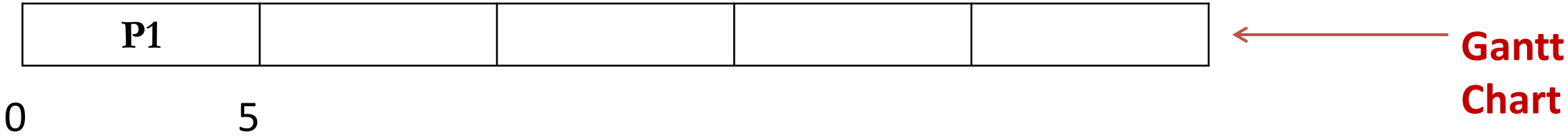
CPU Scheduling – FCFS - Working

Example 6 – All metrics

Consider the system image at the current moment and suggest how FCFS will work in this:

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2



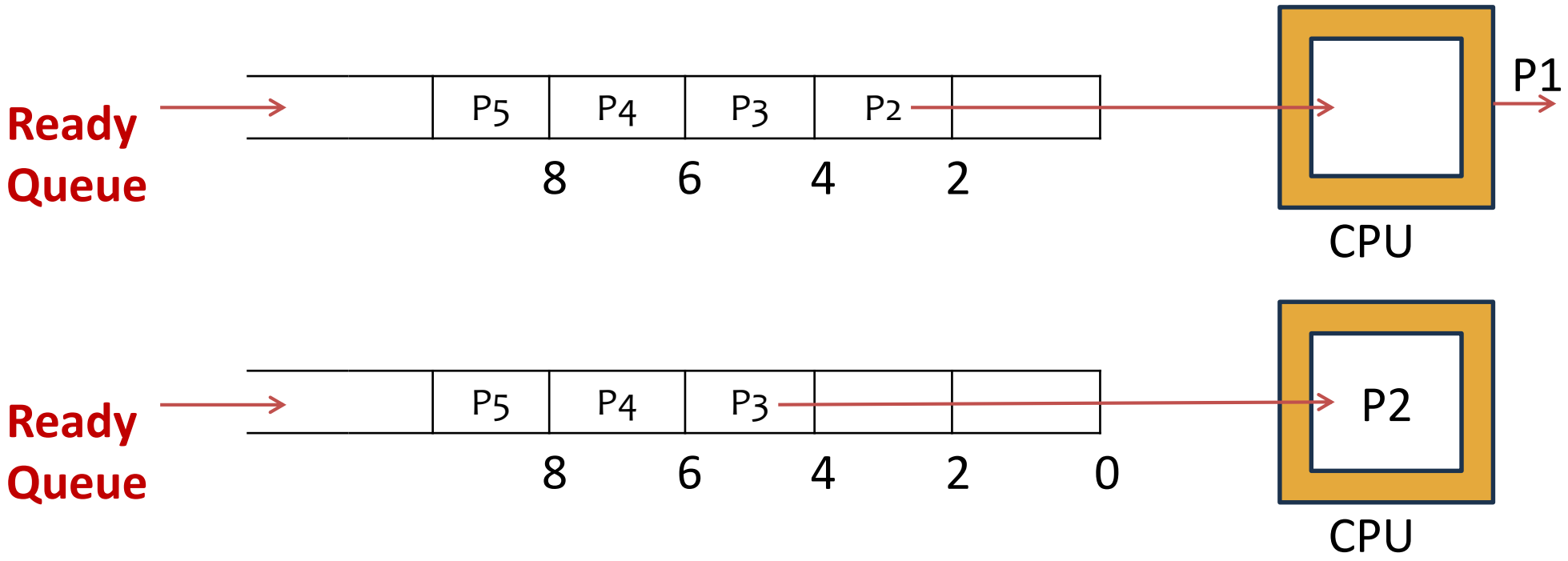


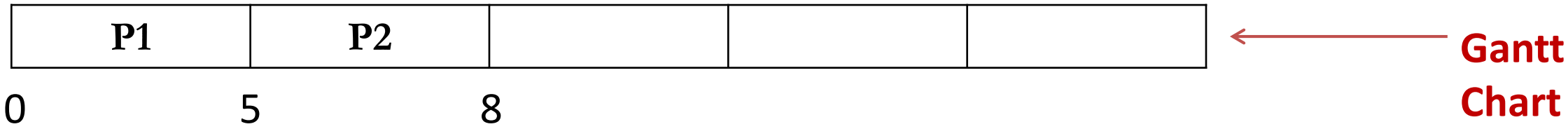
CPU Scheduling – FCFS - Working

Example 6 – All metrics

Consider the system image at the current moment and suggest how FCFS will work in this:

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2



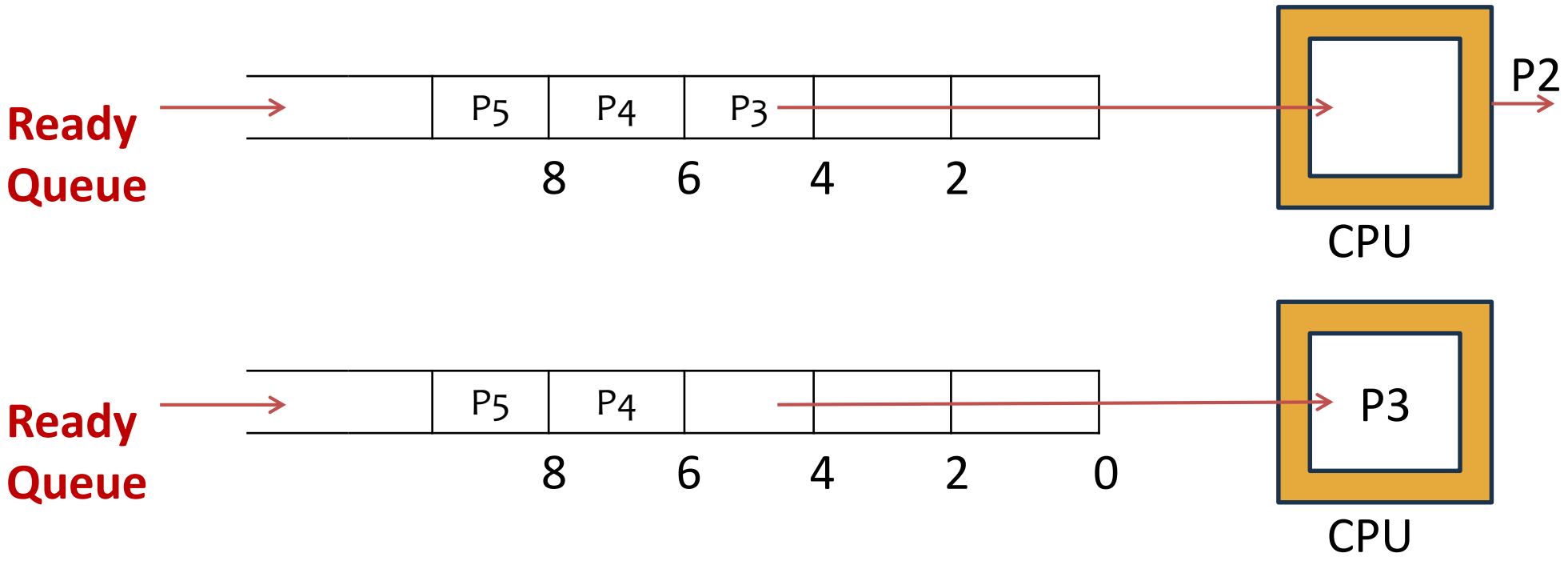


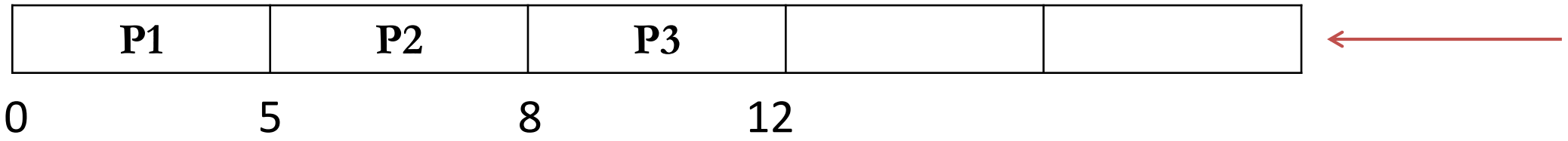
CPU Scheduling – FCFS - Working

Example 6 – All metrics

Consider the system image at the current moment and suggest how FCFS will work in this:

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2



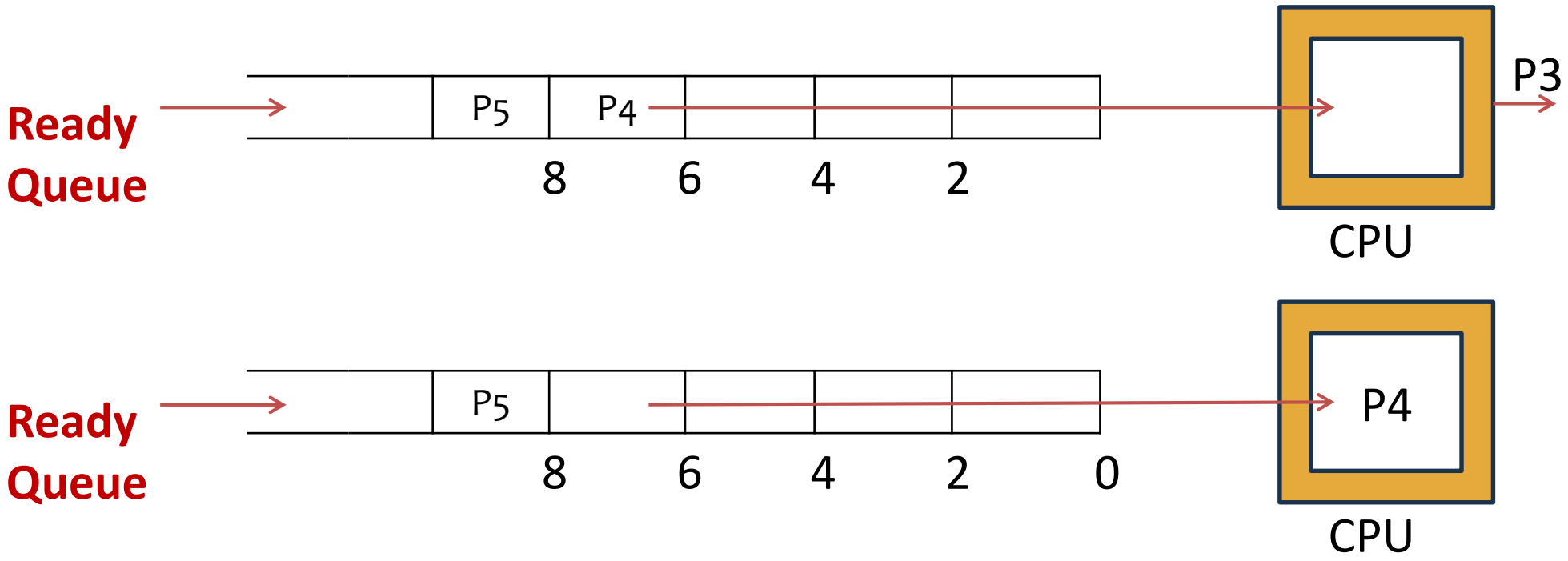


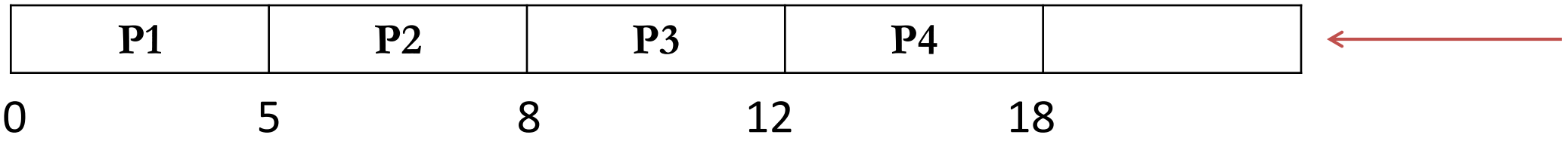
CPU Scheduling – FCFS - Working

Example 6 – All metrics

Consider the system image at the current moment and suggest how FCFS will work in this:

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2



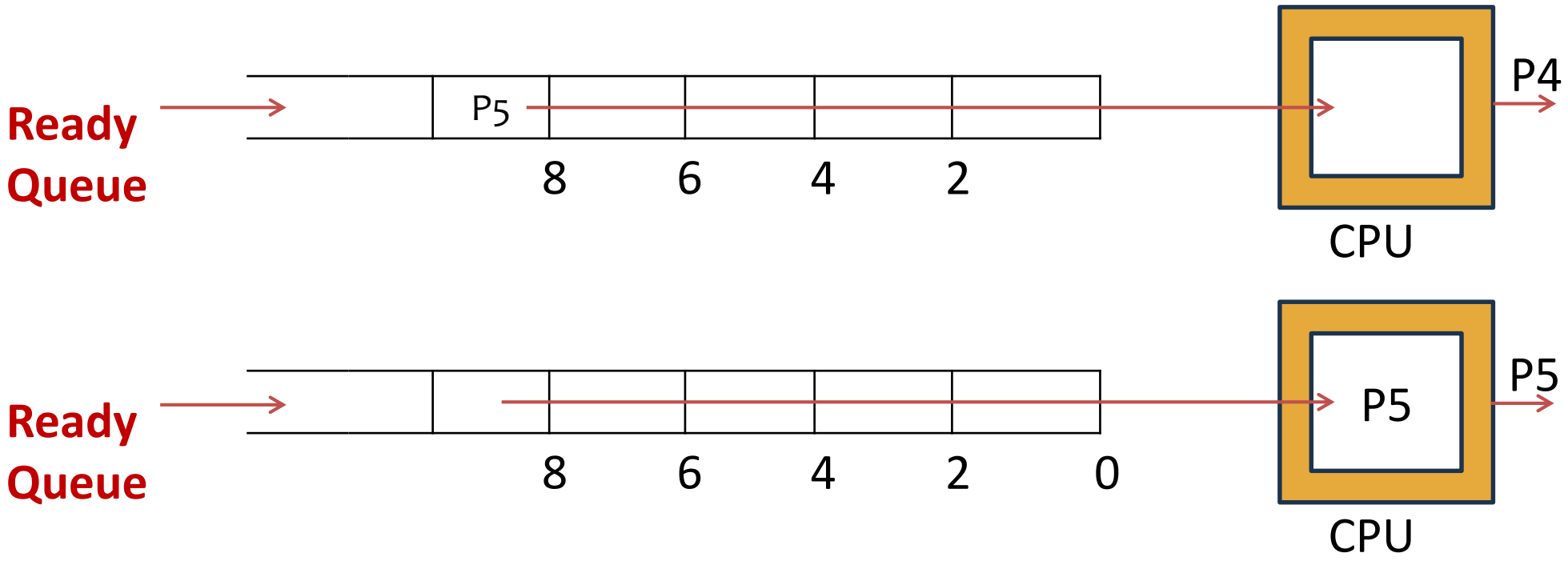


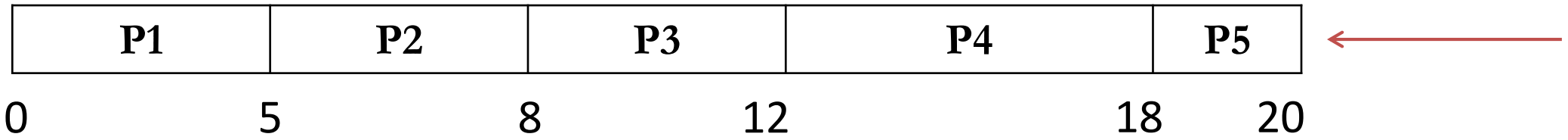
CPU Scheduling – FCFS - Working

Example 6 – All metrics

Consider the system image at the current moment and suggest how FCFS will work in this:

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2





CPU Scheduling – FCFS - Working

Example 6 – All metrics

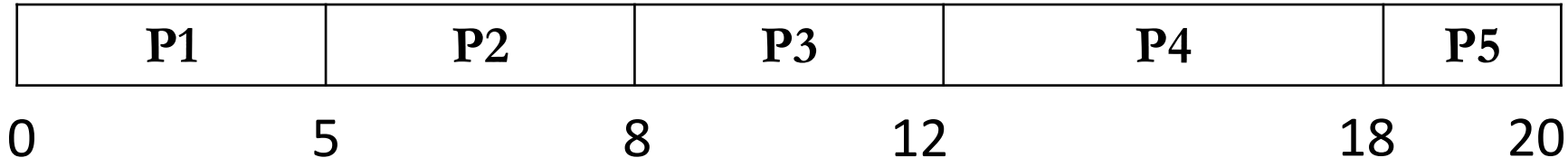
Performance of FCFS

Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU* Allocation *Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	0	5	0	5	5	0	0
P2	2	**3**	5	8	6	3	3
P3	4	4	8	12	8	4	4
P4	6	6	12	18	12	6	6
P5	8	2	18	20	12	10	10
SUM					**43**	**23**	**23**
AVG					**43/5 = 8.6 ms**	**23/5 = 4.6 ms**	**23/5 = 4.6 ms**

CPU Scheduling – FCFS - Working

Example 6 – All metrics

Performance of FCFS



Total CPU Burst Time

$$5 + 3 + 4 + 6 + 2 = 20 \text{ ms}$$

Total System Time

$$20 - 0 = 20 \text{ ms}$$

CPU utilization → 20/20 * 100 = 100%

Throughput= Number of Processes / Total System Time $= \frac{5}{20} = 0.25$

Throughput =0.25 processes/ms

No. of Context switches without Preemption in FCFS : N – 1 = 5 – 1 = 4

CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

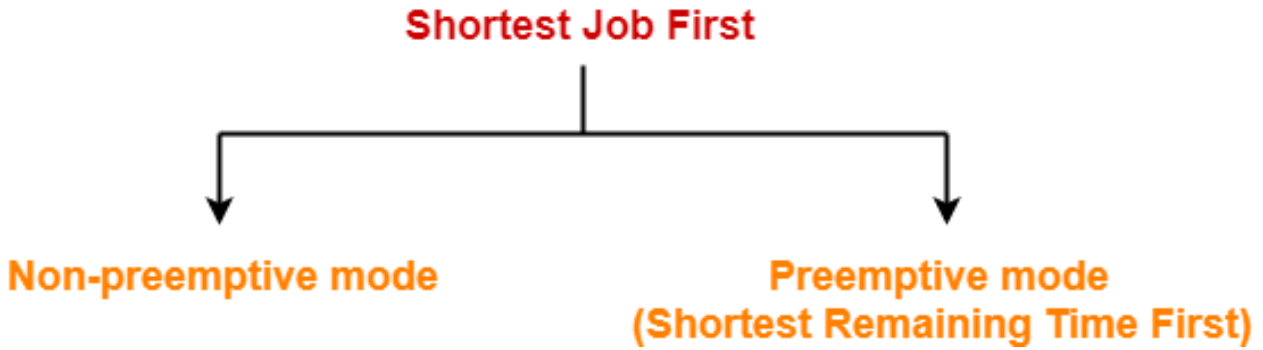
Shortest Job First (SJF) Scheduling

- This algorithm associates with each process the length of the process's next CPU burst.
- When the CPU is available, it is assigned to the process that has the smallest next CPU burst.
- If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

The SJF algorithm can be either preemptive or nonpreemptive.

Note:

A more appropriate term for this scheduling method would be the Shortest-Next-CPU-Burst Algorithm because scheduling depends on the length of the next CPU burst of a process, rather than its **total length**.



- Example 1

CPU Scheduling - SJF

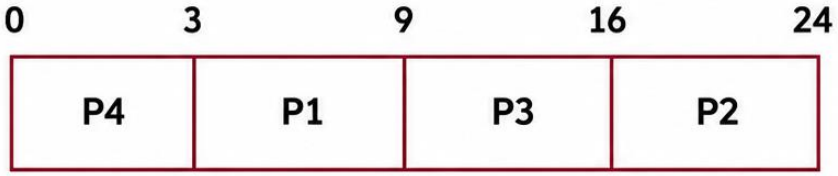
SJF – Preemptive/ Non-Preemptive

Shortest Job First (SJF) Scheduling – Non Preemptive (SJN)

Consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process ID	Burst Time (ms)
P1	6
P2	8
P3	7
P4	3

Gantt Chart:



Waiting Time for P1 = 3 ms

Waiting Time for P2 = 16 ms

Waiting Time for P3 = 9 ms

Waiting Time for P4 = 0 ms

Average Waiting Time

= (3 + 16 + 9 + 0) / 4 = 7 ms

By comparison, if we were using the FCFS scheduling scheme, the average waiting time would be 10.25 milliseconds.

(0+6+14+21)/4 = 41/4 = 10.25

- Example 2

CPU Scheduling - SJF

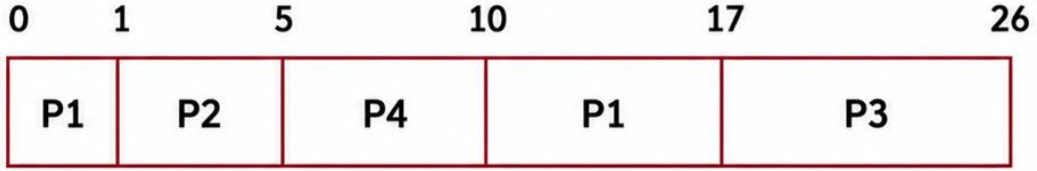
Shortest Job First (SJF) Scheduling – Preemptive (SRTF)

SJF – Preemptive/ Non-Preemptive

Consider the following four processes, with the length of the CPU burst given in milliseconds and the processes arrive at the ready queue at the times shown:

Process ID	Arrival Time	Burst Time
P1	0	~~8~~ 7
P2	1	4
P3	2	9
P4	3	5

Gantt Chart:



Waiting Time = Total waiting Time – No. of milliseconds Process executed – Arrival Time

Waiting Time for P1 = (10 – 1 – 0) = 9 ms

Waiting Time for P2 = (1 – 0 – 1) = 0 ms

Waiting Time for P3 = (17 – 0 – 2) = 15 ms

Waiting Time for P4 = (5 – 0 – 3) = 2 ms

Average Waiting Time

= (9 + 0 + 15 + 2) / 4 = 6.5 ms

- Example 2

CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

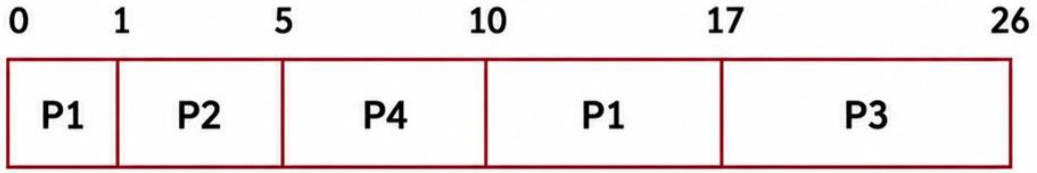
Shortest Job First (SJF) Scheduling – Preemptive (SRTF)

Consider the following four processes, with the length of the CPU burst given in milliseconds and the processes arrive at the ready queue at the times shown:

Process ID	Arrival Time	Burst Time
P1	0	~~8~~ 7
P2	1	4
P3	2	9
P4	3	5

Completion Time	TAT (CT-AT)	WT (TAT-BT)	RT (1st CPU-AT)
17	**17**	**9**	**0**
5	**4**	**0**	**0**
26	**24**	**15**	**15**
10	**7**	**2**	**2**
Total: 58	**52**	**26**	17
Avg: 58/4 =14.5	52/4 =13	26/4 =6.5	17/4=4.25

Gantt Chart:



CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

Problems with SJF Scheduling

- The real difficulty with the SJF algorithm is knowing the length of the next CPU request.
- Although the SJF algorithm is optimal, it cannot be implemented at the level of short-term CPU scheduling.
- There is no way to know the length of the next CPU burst.

One approach is:

- To try to approximate SJF scheduling.
- We may not know the length of the next CPU burst, but we may be able to predict its value.
- We expect that the next CPU burst will be similar in length to the previous ones.
- Thus, by computing an approximation of the length of the next CPU burst, we can pick the process with the shortest predicted CPU burst.

- Example 3

CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

Shortest Job First (SJF) Scheduling – Preemptive (SRTF)

An operating system uses shortest remaining time first scheduling algorithm for pre-emptive scheduling of processes. Consider the following set of processes with their arrival times and CPU burst times (in milliseconds):

Process ID	Arrival Time (ms)	Burst Time (ms)
P1	0	12
P2	2	4
P3	3	6
P4	8	5

The average waiting time (in milliseconds) of the processes is _____.

(A) 4.5 (B) 5.0 (C) 5.5 (D) 6.5

- Example 3

CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

Shortest Job First (SJF) Scheduling – Preemptive (SRTF)

Process ID	Arrival Time (ms)	Burst Time (ms)
P1	0	~~12~~ 10
P2	2	4
P3	3	6
P4	8	5

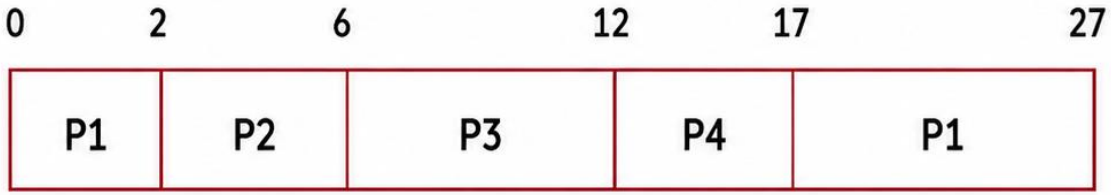
Waiting Time for P1 = (17 – 2 – 0) = 15 ms

Waiting Time for P2 = (2 – 0 – 2) = 0 ms

Waiting Time for P3 = (6 – 0 – 3) = 3 ms

Waiting Time for P4 = (12 – 0 – 8) = 4 ms

Solution: Gantt Chart:



Average Waiting Time

= (15 + 0 + 3 + 4) / 4 = 5.5 ms

Waiting Time = Total waiting Time – No. of milliseconds Process executed – Arrival Time

- Example 4

CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

Shortest Job First (SJF) Scheduling – Preemptive (SRTF)

Consider the following processes, with the arrival time and the length of the CPU burst given in milliseconds. The scheduling algorithm used is preemptive shortest remaining-time first.

Process ID	Arrival Time (ms)	Burst Time (ms)
P1	0	10
P2	3	6
P3	7	1
P4	8	3

The average turn around time of these processes is _____ milliseconds.

(A) 8.25 (B) 10.25 (C) 6.35 (D) 4.25

CPU Scheduling - SJF

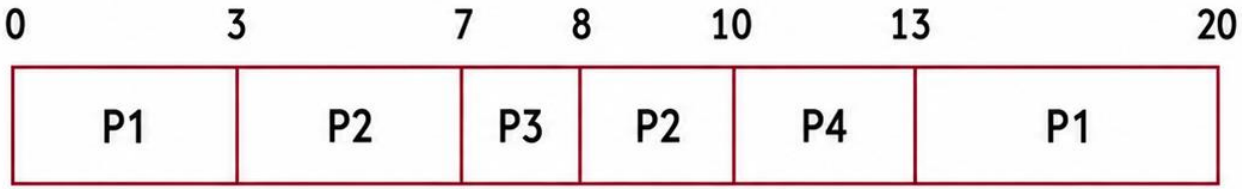
SJF – Preemptive/ Non-Preemptive

- **Example 4**

Shortest Job First (SJF) Scheduling – Preemptive (SRTF)

Process ID	Arrival Time (ms)	Burst Time (ms)
P1	0	~~10~~ 7
P2	3	~~6~~ 2
P3	7	1
P4	8	3

Solution: **Gantt Chart:**



Turn Around time = Completion time – Arrival time

Turnaround Time for P1 = (20 – 0) = 20 ms

Turnaround Time for P2 = (10 – 3) = 7 ms

Turnaround Time for P3 = (8 – 7) = 1 ms

Turnaround Time for P4 = (13 – 8) = 5 ms

Average Turnaround Time

= (20 + 7 + 1 + 5) / 4

= 33 / 4

= 8.25 ms

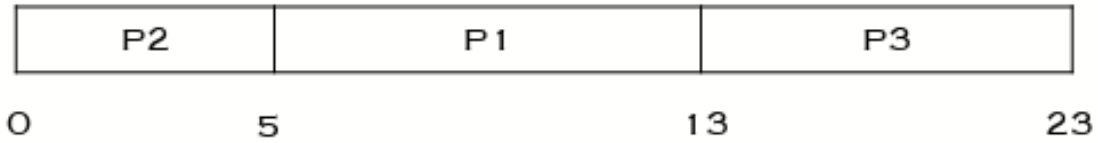
CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

- **Example 5 – Process arrive at Same Time**

❑ Consider the three processes. Their arrival time is 0.

Process	Arrival Time	Burst Time
P1	0	10
P2	0	5
P3	0	8



❑ Turnaround Time of P1 = completion time – Arrival Time = 13 – 0 = 13

❑ Turnaround Time of P2 = 5 – 0 = 5

❑ Turnaround Time of P3 = 23 – 0 = 23

❑ Waiting Time of P1 = Turnaround Time – Burst Time = 13 – 10 = 3

❑ Waiting Time of P2 = 5 – 5 = 0

❑ Waiting Time of P3 = 23 – 8 = 15

❑ Avg Turnaround time = 13 + 5 + 23/3 = 13.67

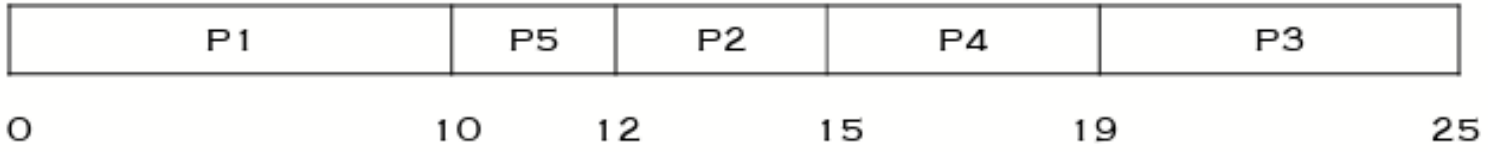
❑ Avg Waiting time = 3 + 0 + 15/3 = 6

CPU Scheduling - SJF

SJF – Preemptive/ Non-Preemptive

- **Example 6 – Process arrive at Different Time**

❑ Consider the three processes.



Process	Arrival Time	Burst Time
P1	0	10
P2	1	3
P3	2	6
P4	3	4
P5	4	2

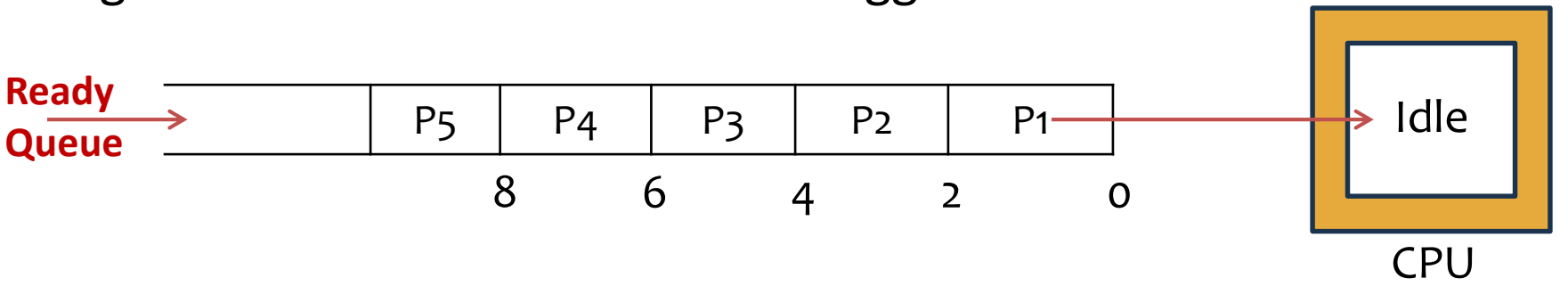
❑ Turnaround Time of P1 = completion time – Arrival Time = 10 – 0 = 10
❑ Turnaround Time of P2 = 15 – 1 = 14
❑ Turnaround Time of P3 = 25 – 2 = 23
❑ Turnaround Time of P4 = 19 – 3 = 16
❑ Turnaround Time of P5 = 12 – 4 = 8
❑ Waiting Time of P1 = Turnaround Time – Burst Time = 10 – 10 = 0
❑ Waiting Time of P2 = 14 – 3 = 11
❑ Waiting Time of P3 = 23 – 6 = 17
❑ Waiting Time of P4 = 16 – 4 = 12
❑ Waiting Time of P5 = 8 – 2 = 6
❑ Avg Turnaround time = (10 + 14 + 23 + 16 + 8)/ 5= 14.2
❑ Avg Waiting time = (0 + 11 + 17 + 12 + 6) / 5 = 9.2

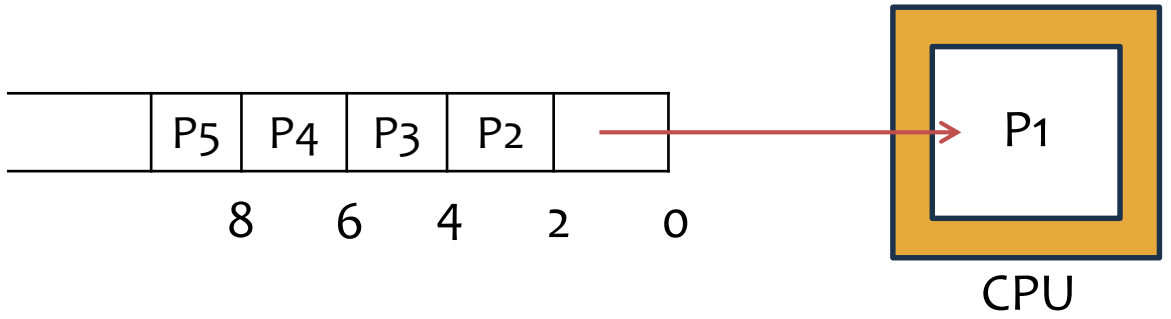
CPU Scheduling – SJF - Working

Example 7 – All Metrics

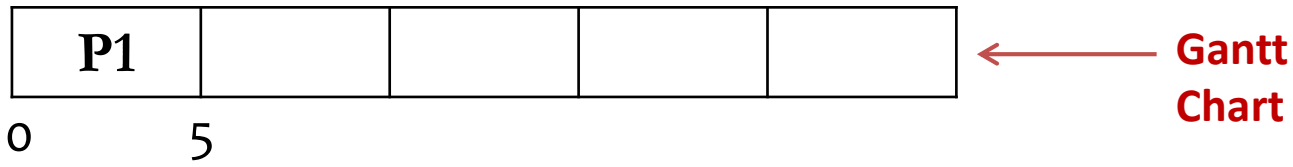
Consider the system image at the current moment and suggest how SJF will work in this

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2





Scheduling Decision Logic		
At Time	Processes in Ready Queue	Which Process Should be Executed
0	P1	Execute P1
1	No Process is there	Continue Execute P1
2	P2	Continue Execute P1
3	P2	Continue Execute P1
4	P3, P2	Continue Execute P1
5	P3, P2	P1 Terminates

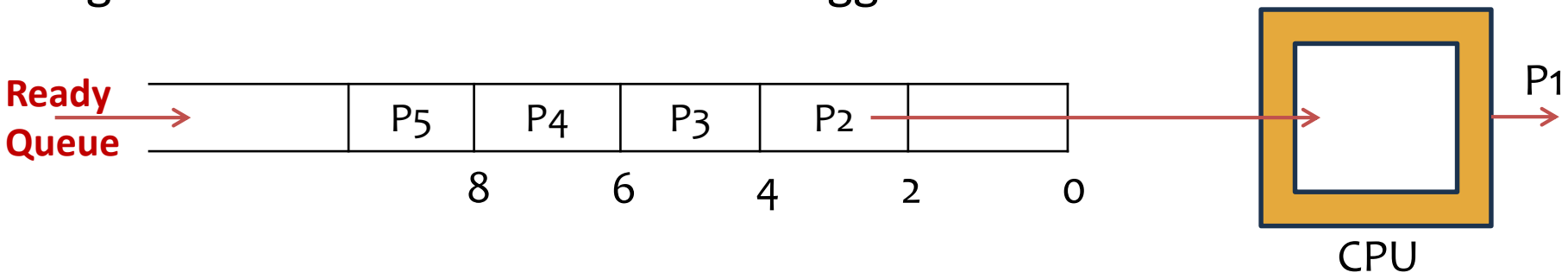


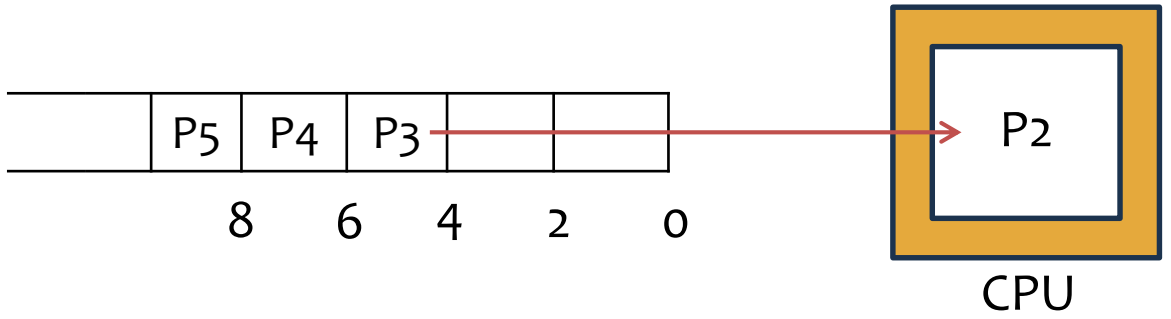
CPU Scheduling – SJF - Working

Example 7 – All Metrics

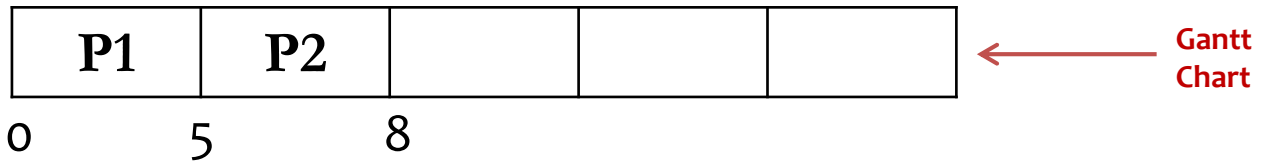
Consider the system image at the current moment and suggest how SJF will work in this

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2





Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
5	P3, P2	BT(P2) < BT(P3), Execute P2
6	P4, P3	Continue Execute P2
7	P4, P3	Continue Execute P2
8	P5, P4, P3	P2 Terminates

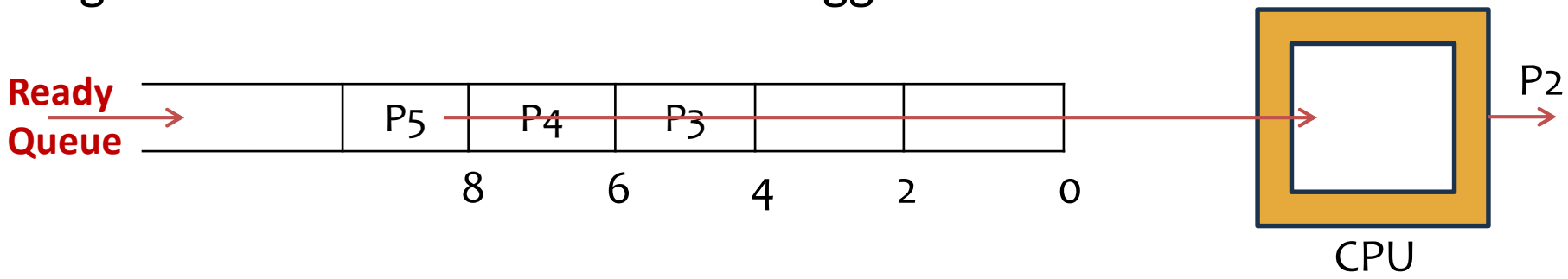


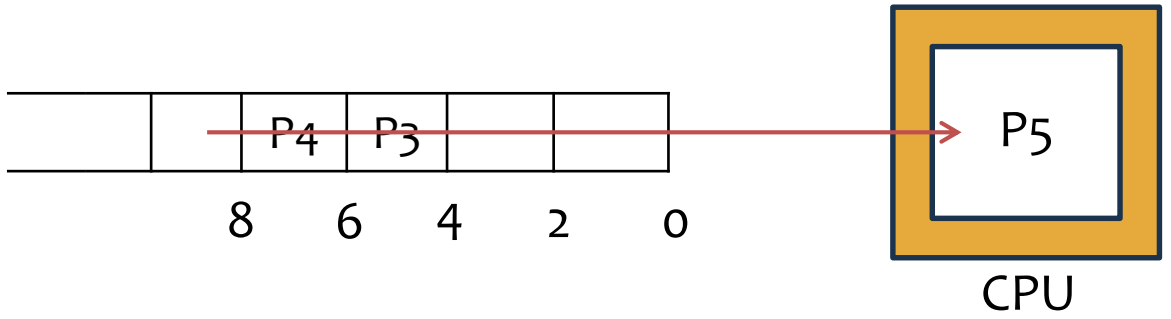
CPU Scheduling – SJF - Working

Example 7 – All Metrics

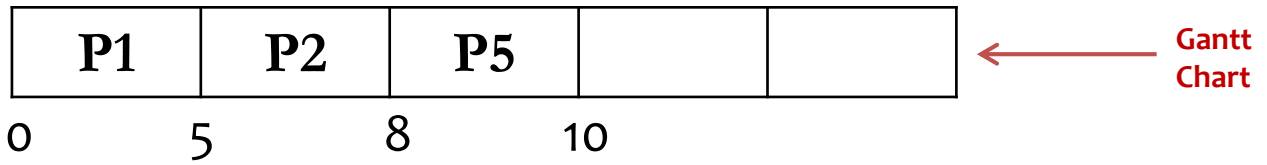
Consider the system image at the current moment and suggest how SJF will work in this

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2





Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
8	P5, P4, P3	BT(P5) < BT(P3) < BT(P4), Execute P5
9	P4, P3	Continue Execute P5
10	P4, P3	P5 Terminates

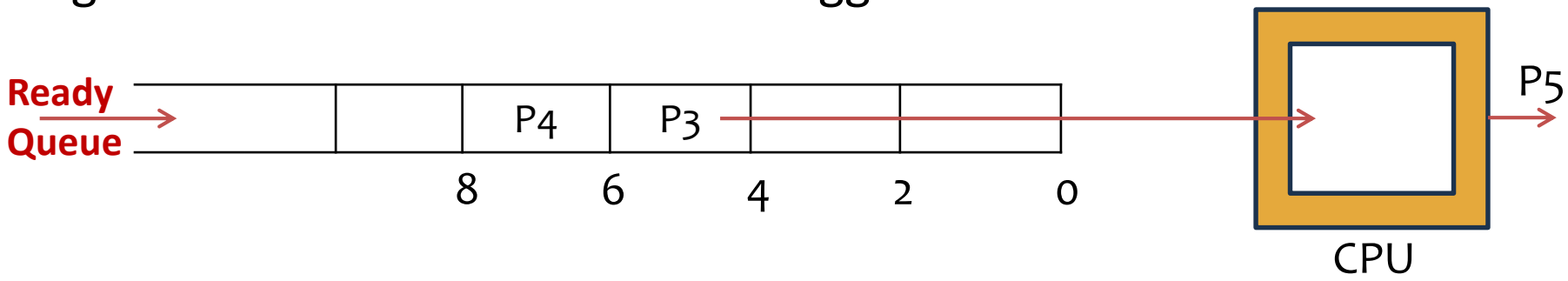


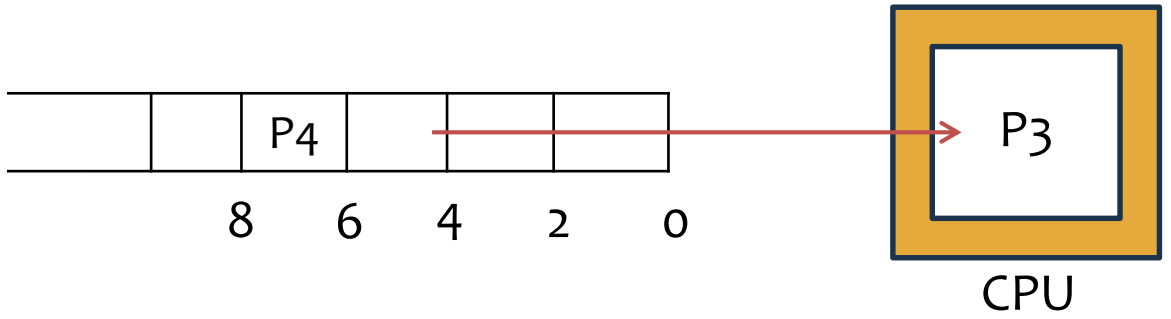
CPU Scheduling – SJF - Working

Example 7 – All Metrics

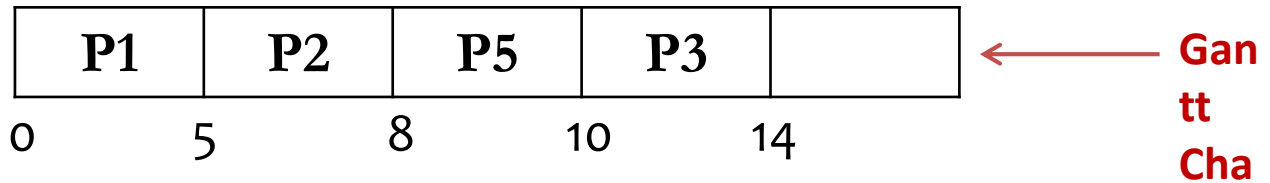
Consider the system image at the current moment and suggest how SJF will work in this

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2





Scheduling Decision Logic		
At Time	Processes in Ready Queue	Which Process Should be Executed
10	P4, P3	BT(P3) < BT(P4), Execute P3
11	P4	Continue Execute P3
12	P4	Continue Execute P3
13	P4	Continue Execute P3
14	P4	Continue Execute P3

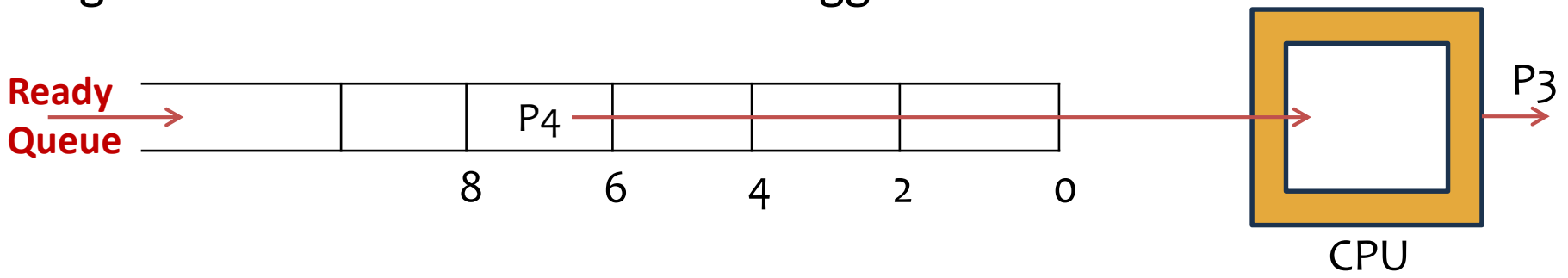


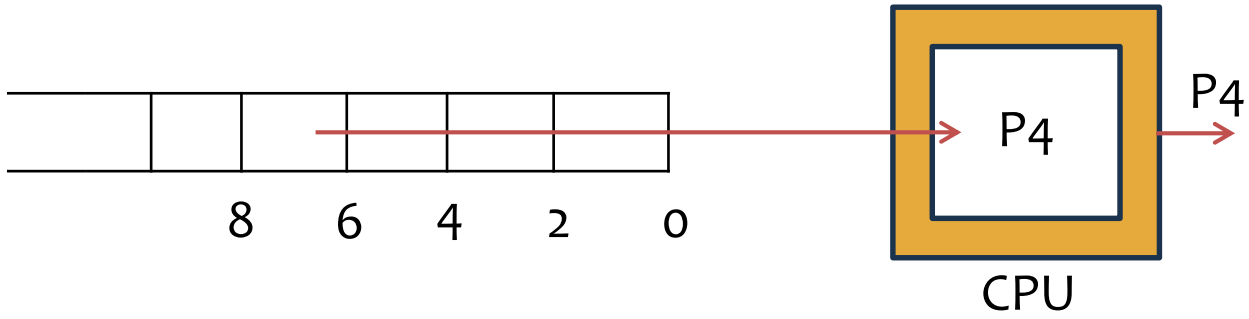
CPU Scheduling – SJF - Working

Example 7 – All Metrics

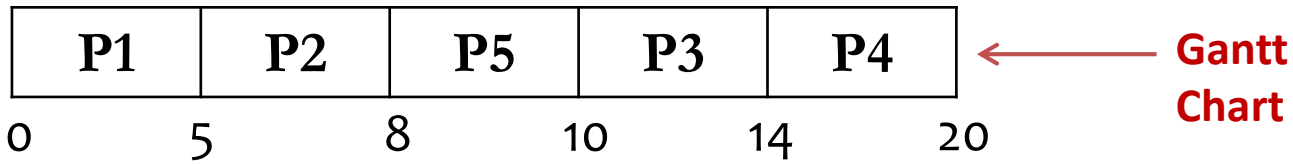
Consider the system image at the current moment and suggest how SJF will work in this

Process	Arrival Time	CPU Burst
P1	0	5
P2	2	3
P3	4	4
P4	6	6
P5	8	2





Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
14	No Process	Execute P4
15	No Process	Continue Execute P4
16	No Process	Continue Execute P4
17	No Process	Continue Execute P4
18	No Process	Continue Execute P4
19	No Process	Continue Execute P4
20	No Process	Continue Execute P4



CPU Scheduling – SJF - Working

Example 7 – All Metrics

Performance of SJF

Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU Allocation Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	0	5	0	5	5	0	0
P2	2	3	5	8	6	3	3
P3	4	4	10	14	10	6	6
P4	6	6	14	20	14	8	8
P5	8	2	8	10	2	0	0
SUM					**37**	**17**	**17**
AVG					**37/5 = 7.4 ms**	**17/5 = 3.4 ms**	**17/5 = 3.4 ms**

CPU Scheduling – SJF - Working

Example 7 – All Metrics

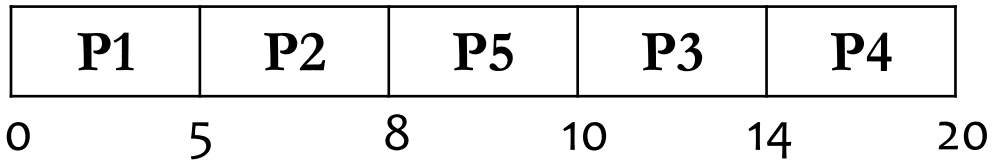
Performance of SJF

Total CPU Burst Time

$$5 + 3 + 4 + 6 + 2 = 20 \text{ ms}$$

Total System Time

$$20 - 0 = 20 \text{ ms}$$



CPU utilization → 20/20 * 100 = 100%

Throughput= Number of Processes / Total System Time $= \frac{5}{20} = 0.25$

Throughput =0.25 processes/ms

No. of Context switches without Preemption in SJF = 4

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

Priority Scheduling

- A priority is associated with each process, and the CPU is allocated to the process with the highest priority.
- Equal-priority processes are scheduled in FCFS (First-Come, First-Served) order.
- An **SJF (Shortest Job First)** algorithm is simply a priority algorithm where the priority is the inverse of the (predicted) next CPU burst.
  - The larger the CPU burst, the lower the priority, and vice versa.
- Priority scheduling can be either preemptive or nonpreemptive.

- A preemptive priority scheduling algorithm will preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.
- A nonpreemptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

Priority Scheduling

- (smallest integer ≡ highest priority)
- (largest integer ≡ highest priority)

- Problem: **Starvation** – low priority processes may never execute
- Solution : **Aging** – as time progresses increase the priority of the process

- Example 1

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

Priority Scheduling – Non Preemptive

Consider the following set of processes, assumed to have arrived at time 0, in the order P1, P2, P3, P4, P5, with the length of the CPU burst given in milliseconds:

Process ID	Burst Time (ms)	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

(smallest integer ≡ highest priority)

Waiting Time for P1 = 6 ms

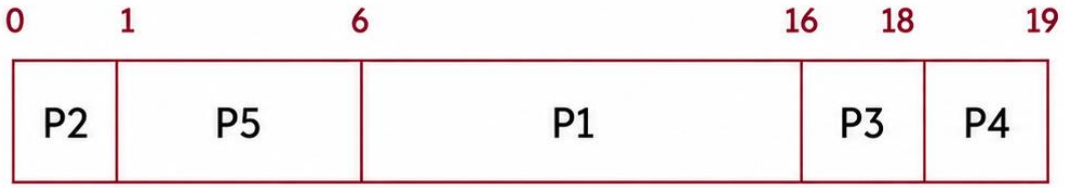
Waiting Time for P2 = 0 ms

Waiting Time for P3 = 16 ms

Waiting Time for P4 = 18 ms

Waiting Time for P5 = 1 ms

Gantt Chart:



Average Waiting Time = (6 + 0 + 16 + 18 + 1) / 5

= 41 / 5 ms

= 8.2 ms

- Example 2

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

Priority Scheduling – Preemptive

Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority (0 is the highest priority) shown below.

None of the processes have I/O burst time.

(smallest integer ≡highest priority)

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

The average waiting time (in milliseconds) of all the processes using preemptive priority scheduling algorithm is _____.

(A) 29 (B) 30 (C) 31 (D) 32

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

- Example 2

Priority Scheduling – Preemptive

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

Waiting Time for P1 = (40 – 2 – 0) = 38 ms

Waiting Time for P2 = (5 – 0 – 5) = 0 ms

Waiting Time for P3 = (49 – 0 – 12) = 37 ms

Waiting Time for P4 = (33 – 3 – 2) = 28 ms

Waiting Time for P5 = (51 – 0 – 9) = 42 ms

Average Waiting Time

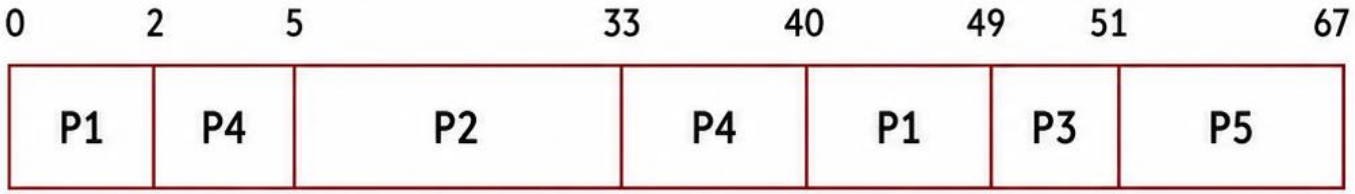
= (38 + 0 + 37 + 28 + 42) / 5

= 145 / 5 ms

= 29 ms

Solution:

Gantt Chart:



Waiting Time = Total waiting Time – No. of milliseconds Process executed – Arrival Time

Process ID	Arrival Time	Burst Time	Priority (0 is highest)
P1	0	11 → 9	2
P2	5	28	0
P3	12	2	3
P4	2	10 → 7	1
P5	9	16	4

- Example 3

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

Priority Scheduling - Preemptive

Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority shown below: (Higher number represents higher priority)

Process ID	Arrival Time (ms)	Burst Time (ms)	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

(smallest integer ≡ highest priority)

If the CPU scheduling policy is priority non-preemptive, calculate the average waiting time and average turn around time.

Average Waiting Time = (Sum of waiting times of all processes) / (Total number of processes)

Average Turn Around Time = (Sum of turn around times of all processes) / (Total number of processes)

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

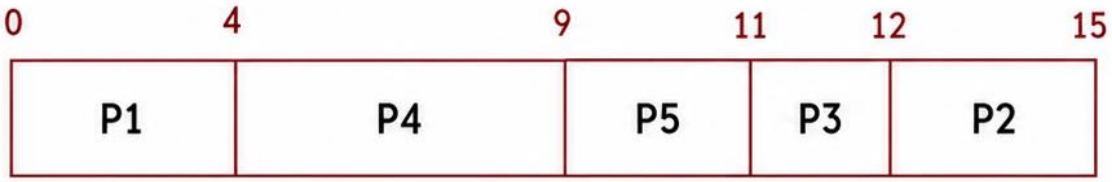
- **Example 3**

Priority Scheduling - Preemptive

Process ID	Arrival Time	Burst Time	Priority (Higher number = Higher priority)
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

Solution:

Gantt Chart:



Turn Around time = Completion time – Arrival time

Waiting time = Turn Around time – Burst time

Process ID	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
P1	4	4 – 0 = 4	4 – 4 = 0
P2	15	15 – 1 = 14	14 – 3 = 11
P3	12	12 – 2 = 10	10 – 1 = 9
P4	9	9 – 3 = 6	6 – 5 = 1
P5	11	11 – 4 = 7	7 – 2 = 5

CPU Scheduling - Priority

Priority – Preemptive/ Non-Preemptive

- Example 3

Priority Scheduling - Preemptive

Process ID	Completion Time	Turnaround Time (Completion Time – Arrival Time)	Waiting Time (Turnaround Time – Burst Time)
P1	4	4 – 0 = 4	4 – 4 = 0
P2	15	15 – 1 = 14	14 – 3 = 11
P3	12	12 – 2 = 10	10 – 1 = 9
P4	9	9 – 3 = 6	6 – 5 = 1
P5	11	11 – 4 = 7	7 – 2 = 5

Average Turn Around time

= (4 + 14 + 10 + 6 + 7) / 5

= 41 / 5 = **8.2 ms**

Average waiting time

= (0 + 11 + 9 + 1 + 5) / 5

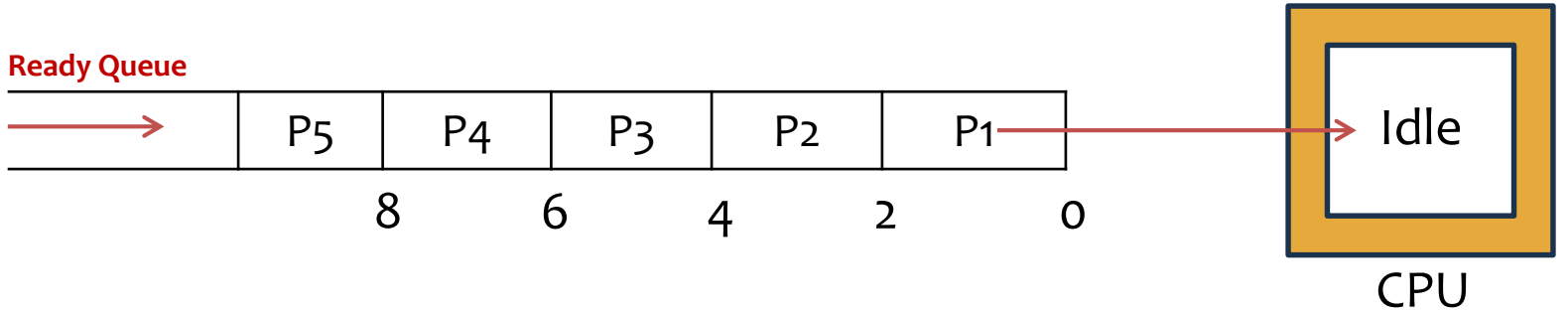
= 26 / 5 = **5.2 ms**

CPU Scheduling – Priority - Working

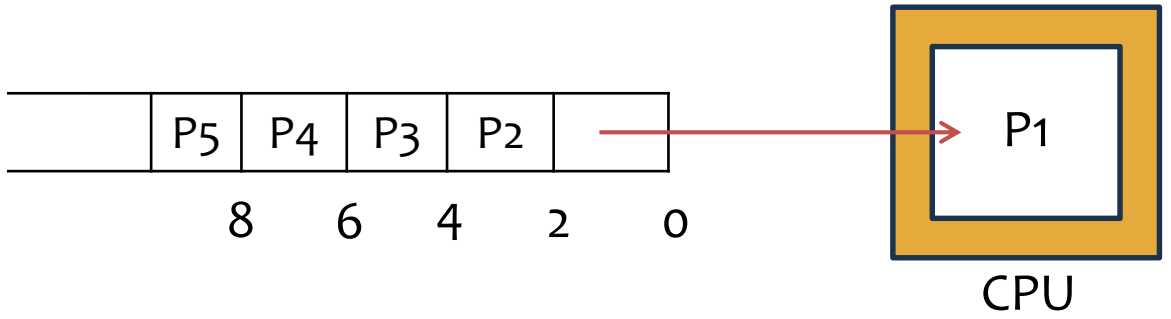
Example 4

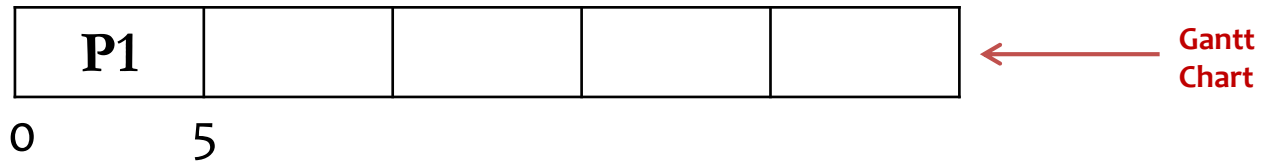
Consider the system image at the current moment and suggest how Priority (Non-Preemptive) will work in this:

Process	Arrival Time	CPU Burst	Priority
P1	0	5	2
P2	2	3	1
P3	4	4	3
P4	6	6	4
P5	8	2	2



Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
0	P1	Execute P1
1	No Process is there	Continue Execute P1
2	P2	Continue Execute P1
3	P2	Continue Execute P1
4	P3, P2	Continue Execute P1
5	P3, P2	P1 Terminates



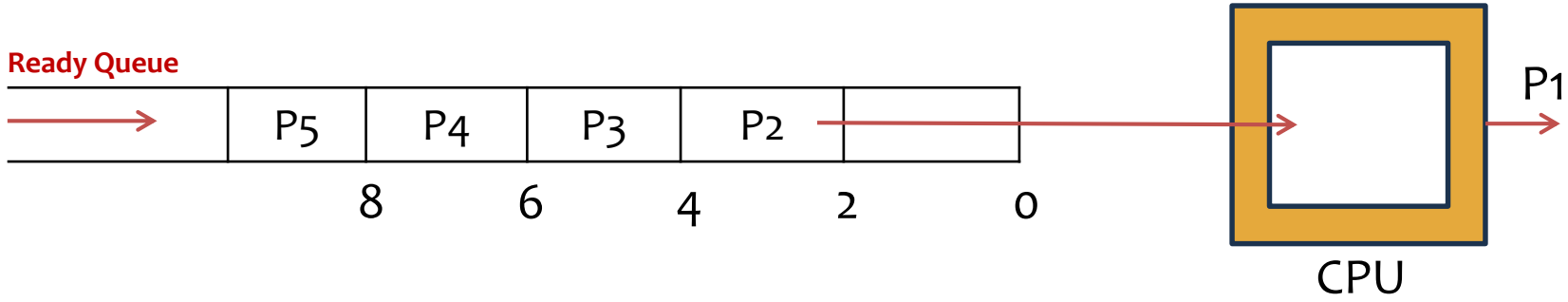


CPU Scheduling – Priority - Working

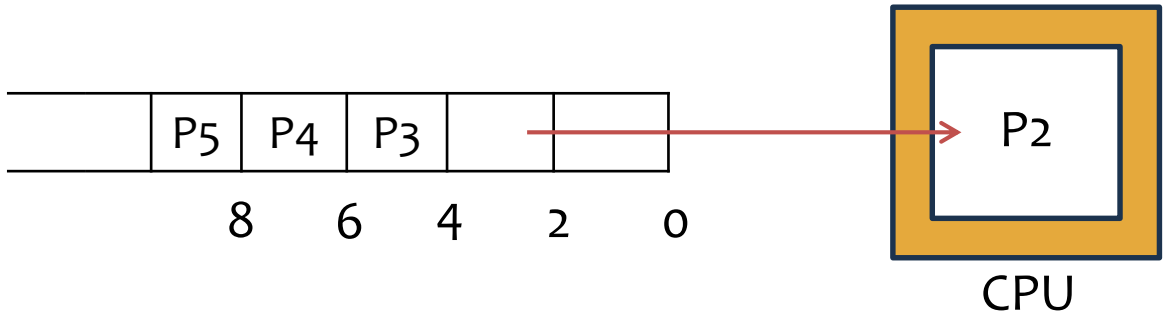
Example 4

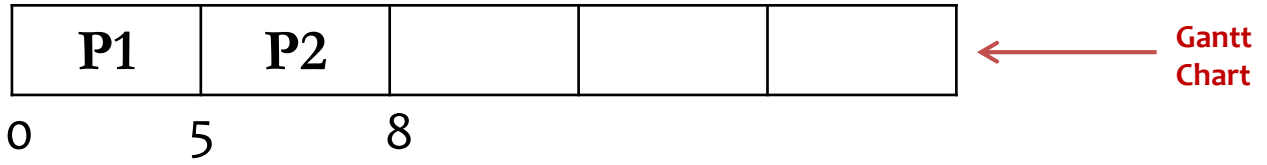
Consider the system image at the current moment and suggest how Priority (Non-Preemptive) will work in this:

Process	Arrival Time	CPU Burst	Priority
P1	0	5	2
P2	2	3	1
P3	4	4	3
P4	6	6	4
P5	8	2	2



Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
5	P3, P2	P(P2) > P(P3), Execute P2
6	P4, P3	Continue Execute P2
7	P4, P3	Continue Execute P2
8	P5, P4, P3	P2 Terminates



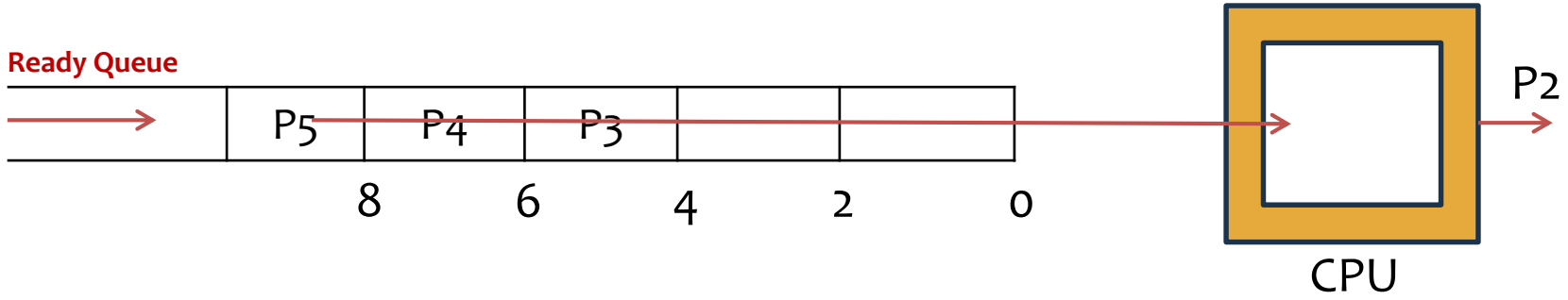


CPU Scheduling – Priority - Working

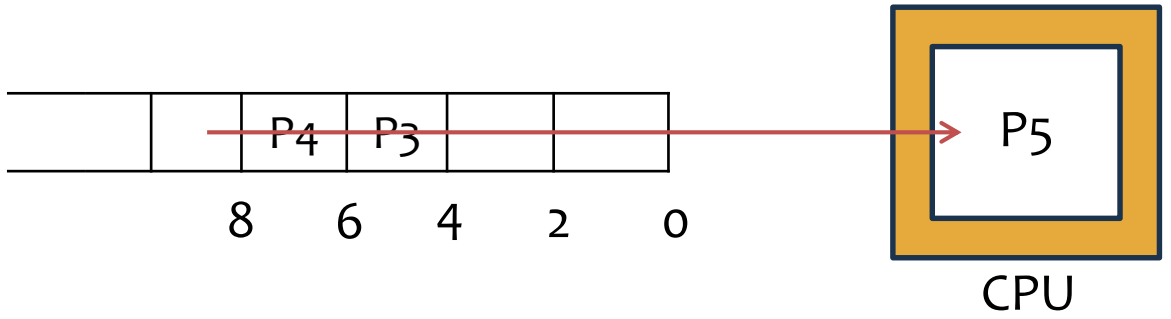
Example 4

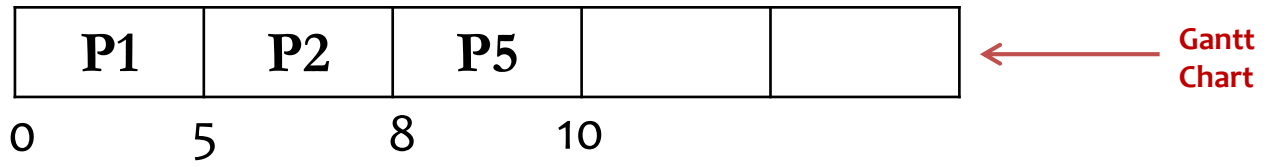
Consider the system image at the current moment and suggest how Priority (Non-Preemptive) will work in this:

Process	Arrival Time	CPU Burst	Priority
P1	0	5	2
P2	2	3	1
P3	4	4	3
P4	6	6	4
P5	8	2	2



Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
8	P5, P4, P3	P(P5) > P(P3) > P(P4), Execute P5
9	P4, P3	Continue Execute P5
10	P4, P3	P5 Terminates
11	P4, P3	



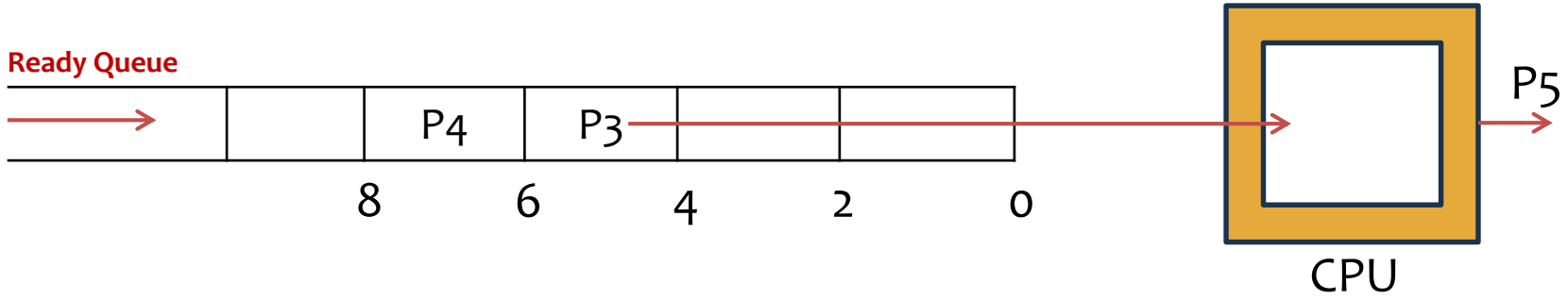


CPU Scheduling – Priority - Working

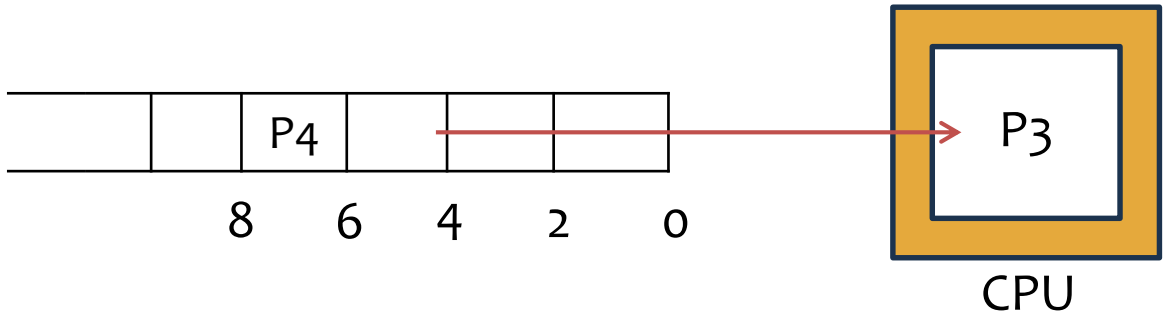
Example 4

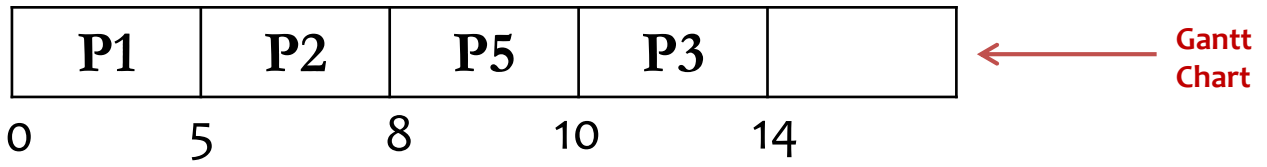
Consider the system image at the current moment and suggest how Priority (Non-Preemptive) will work in this:

Process	Arrival Time	CPU Burst	Priority
P1	0	5	2
P2	2	3	1
P3	4	4	3
P4	6	6	4
P5	8	2	2



Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
10	P4, P3	P(P3) > P(P4), Execute P3
11	P4	Continue Execute P3
12	P4	Continue Execute P3
13	P4	Continue Execute P3
14	P4	P3 Terminates



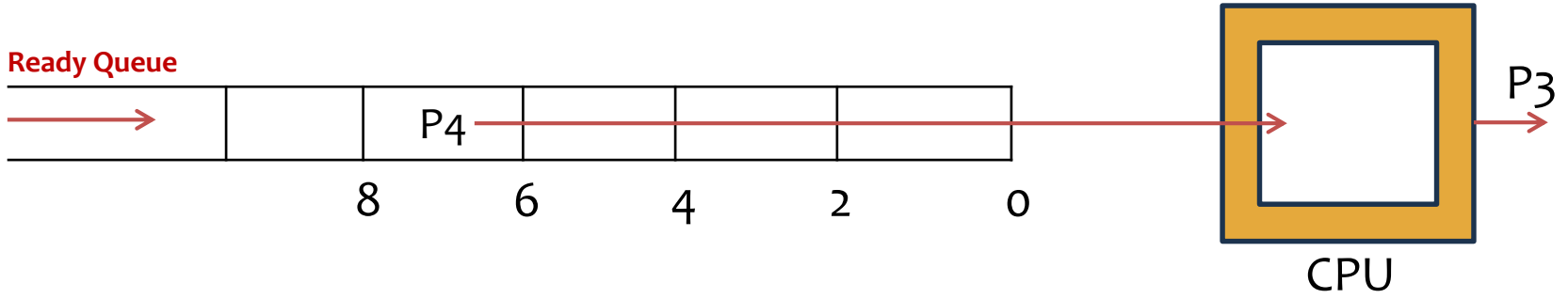


CPU Scheduling – Priority - Working

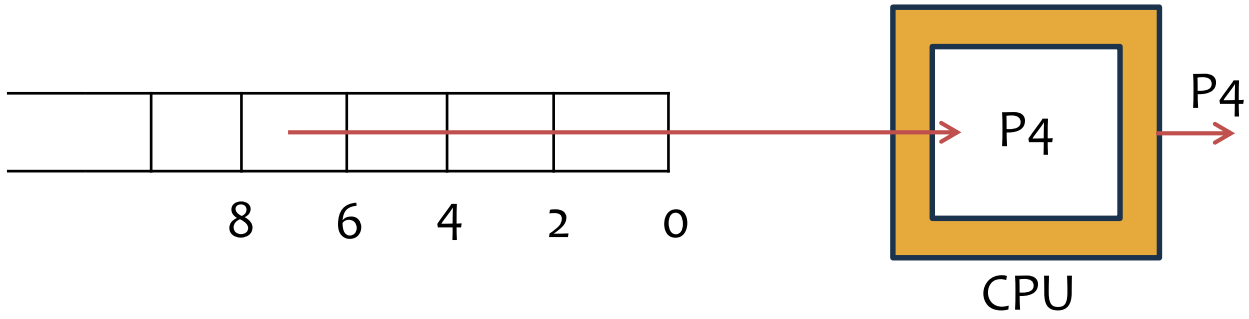
Example 4

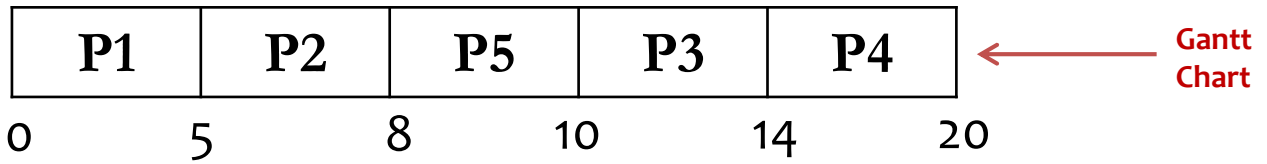
Consider the system image at the current moment and suggest how Priority (Non-Preemptive) will work in this:

Process	Arrival Time	CPU Burst	Priority
P1	0	5	2
P2	2	3	1
P3	4	4	3
P4	6	6	4
P5	8	2	2



Scheduling Decision Logic		
At Time	**Processes in Ready Queue**	**Which Process Should be Executed**
14	P4	Execute P4
15	No Process	Continue Execute P4
16	No Process	Continue Execute P4
17	No Process	Continue Execute P4
18	No Process	Continue Execute P4
19	No Process	Continue Execute P4
20	No Process	P4 Terminates





CPU Scheduling – Priority - Working

Performance of Priority (Non-Preemptive)

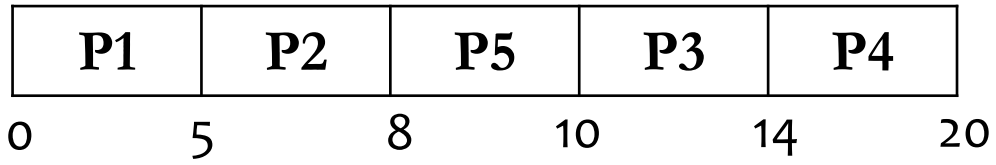
Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU* Allocated *Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	0	5	0	5	5	0	0
P2	2	3	5	8	6	3	3
P3	4	4	10	14	10	6	6
P4	6	6	14	20	14	8	8
P5	8	2	8	10	2	0	0
SUM					**37**	**17**	**17**
AVG					**37/5 = 7.4 ms**	**17/5 = 3.4 ms**	**17/5 = 3.4 ms**

CPU Scheduling – Priority - Working

Performance of Priority (Non-Preemptive)

Total CPU Burst Time

$$5 + 3 + 4 + 6 + 2 = 20 \text{ ms}$$



Total System Time

$$20 - 0 = 20 \text{ ms}$$

CPU utilization → 20/20 * 100 = 100%

Throughput= Number of Processes / Total System Time $= \frac{5}{20} = 0.25$

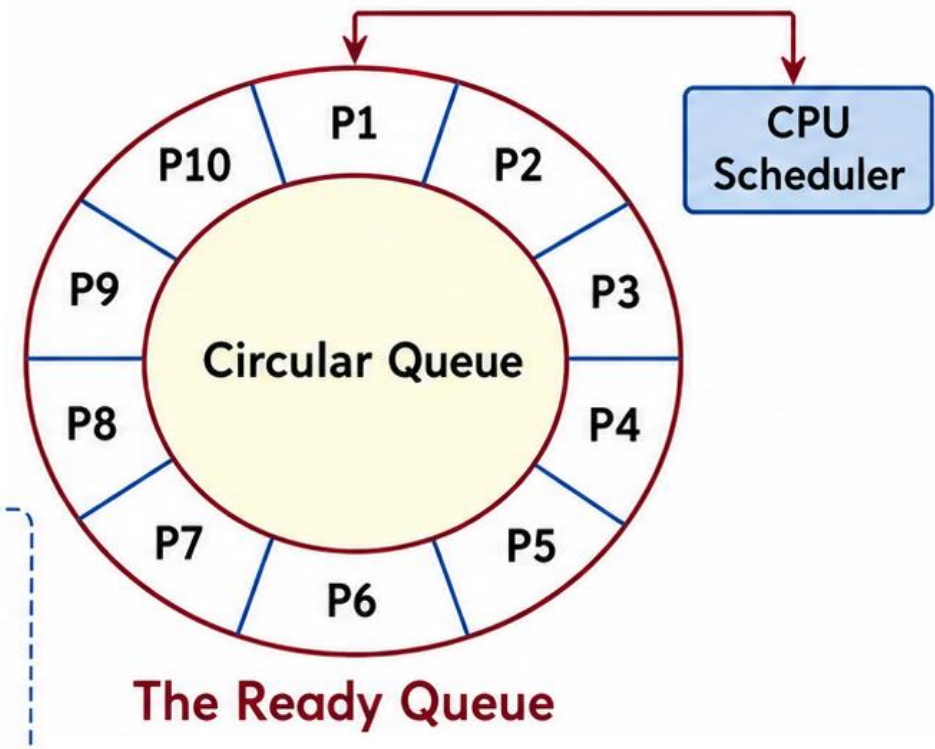
Throughput =0.25 processes/ms

No. of Context switches without Preemption in Priority = 4

CPU Scheduling – Round Robin

Round-Robin (RR) Scheduling

- The Round-Robin (RR) scheduling algorithm is designed especially for time-sharing systems.
- It is similar to FCFS scheduling, but *preemption* is added to switch between processes.
- A small unit of time, called a ***time quantum*** **or** ***time slice***, is defined (generally from 10 to 100 milliseconds).
- The ready queue is treated as a circular queue.



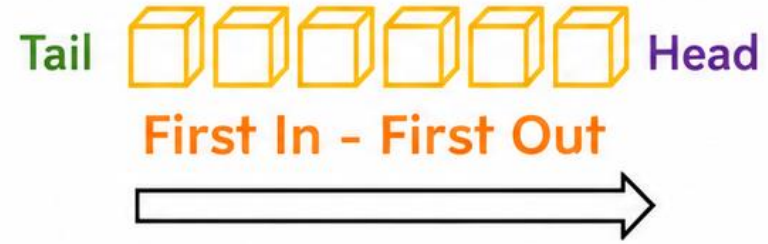
The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

Time Quantum (Time Slice): The maximum amount of CPU time allocated to a process in one turn before it is preempted.

Circular Queue: The ready queue is organized in a circular manner so that every process gets a fair chance to execute.

CPU Scheduling – Round Robin

Implementation of Round-Robin (RR) Scheduling



- We keep the ready queue as a FIFO queue of processes.
- New processes are added to the **tail** of the ready queue.
- The CPU scheduler picks the **first process** from the ready queue, **sets a timer to interrupt after one time quantum**, and **dispatches the process**.

if Time Quantum > burst time
---- Similar to FCFS

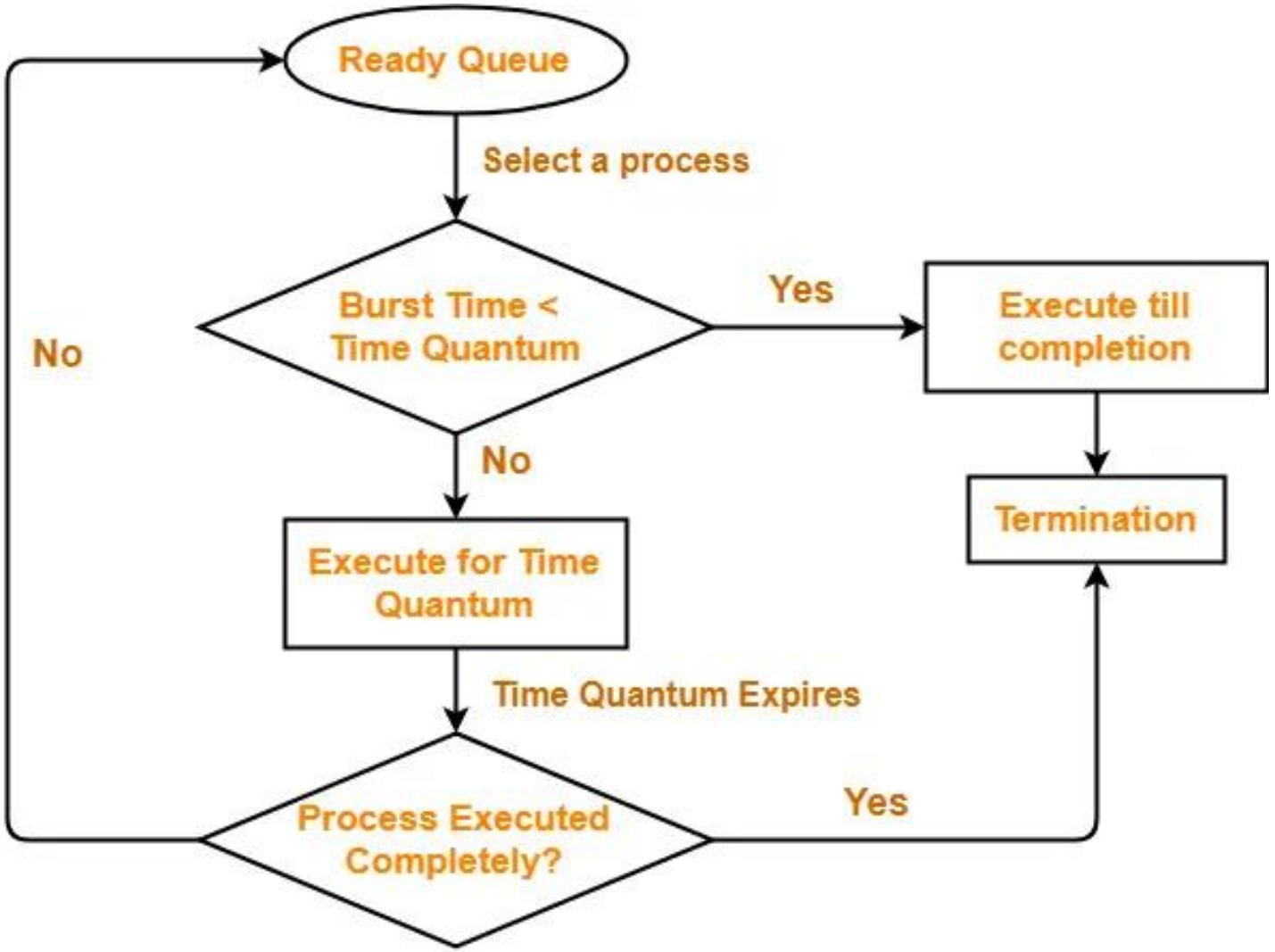
if Time Quantum < burst time
Causes several context switch -- - overloads OS.

One of two things will then happen

1. The process may have a CPU burst of less than 1 time quantum
   - Non-Preemptive
   - The process itself will release the CPU voluntarily
   - The CPU scheduler will then proceed to the next process in the ready queue

2. The CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the O.S.
   - Preemptive
   - A context switch will be executed, and the process will be put at the tail of the ready queue
   - The CPU scheduler will then select the next process in the ready queue

CPU Scheduling – Round Robin





Round Robin Scheduling

- Example 1

CPU Scheduling – Round Robin

Calculation of Turn Around Time and Waiting Time

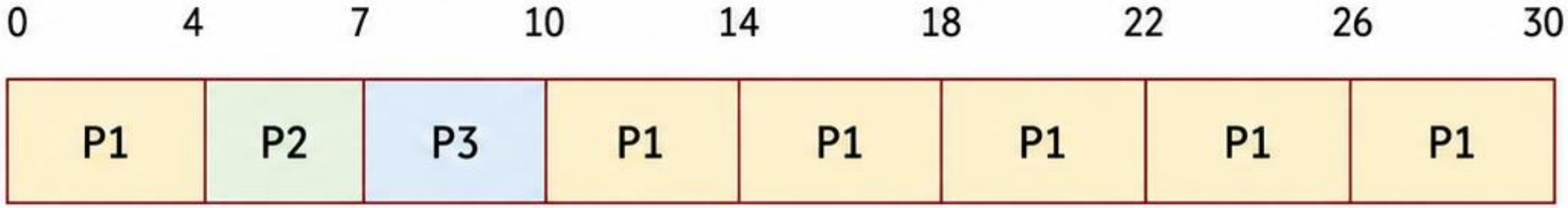
Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds and time quantum taken as 4 milliseconds for RR Scheduling:

Process ID	Burst Time (ms)
P1	24
P2	3
P3	3



Time Quantum

Gantt Chart:



- Example 1

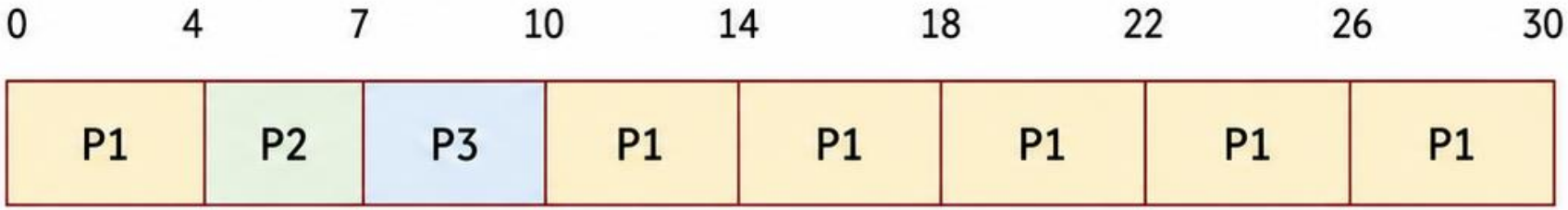
CPU Scheduling – Round Robin



Time Quantum

Calculation of Turn Around Time and Waiting Time

Process ID	Burst Time (ms)
P1	24
P2	3
P3	3



Method 1

Turn Around time = Completion time – Arrival time

Waiting time = Turn Around time – Burst time

Process ID	Completion Time (ms)	Turnaround Time (ms) (CT – AT)	Waiting Time (ms) (TAT – BT)
P1	30	30 – 0 = 30	30 – 24 = 6
P2	7	7 – 0 = 7	7 – 3 = 4
P3	10	10 – 0 = 10	10 – 3 = 7

Method 2

Waiting time = Last Start Time – Arrival Time – (Preemption × Time Quantum)

Process ID	Waiting Time (ms)
P1	26 – 0 – (5 × 4) = 6
P2	4 – 0 – (0 × 4) = 4
P3	7 – 0 – (0 × 4) = 7

- Example 1

CPU Scheduling – Round Robin



Time Quantum

Calculation of Turn Around Time and Waiting Time

Method 1

Turn Around time = Completion time – Arrival time

Waiting time = Turn Around time – Burst time

Process ID	Completion Time (ms)	Turnaround Time (ms) (CT – AT)	Waiting Time (ms) (TAT – BT)
P1	30	30 – 0 = 30	30 – 24 = 6
P2	7	7 – 0 = 7	7 – 3 = 4
P3	10	10 – 0 = 10	10 – 3 = 7

Average Turn Around time

= (30 + 7 + 10) / 3

= 47 / 3 = 15.66 ms

Average waiting time

= (6 + 4 + 7) / 3

= 17 / 3 = 5.66 ms

Method 2

Waiting time = Last Start Time – Arrival Time – (Preemption × Time Quantum)

Process ID	Waiting Time (ms)
P1	26 – 0 – (5 × 4) = 6
P2	4 – 0 – (0 × 4) = 4
P3	7 – 0 – (0 × 4) = 7

Average waiting time

= (6 + 4 + 7) / 3

= 17 / 3 = 5.66 ms

- **Example 2**

CPU Scheduling – Round Robin

Consider the set of 5 processes whose arrival time and burst time are given below:

Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

If the CPU scheduling policy is Round Robin with time quantum = 2 units, calculate the average waiting time and average turn around time.

https://www.youtube.com/watch?v=QlCmgBOMjlI

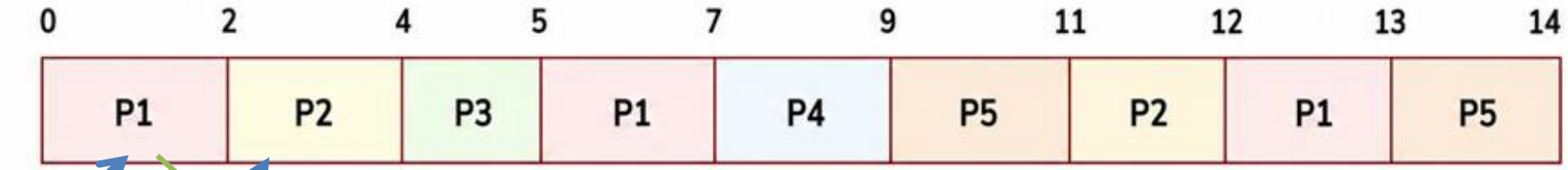
- Example 2

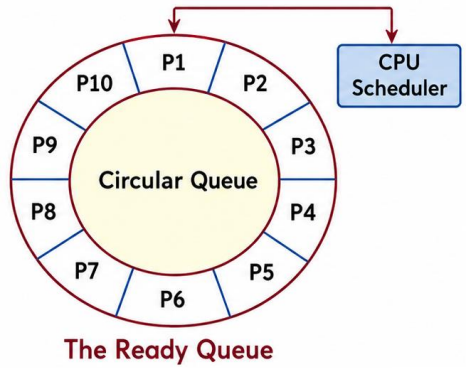
CPU Scheduling – Round Robin

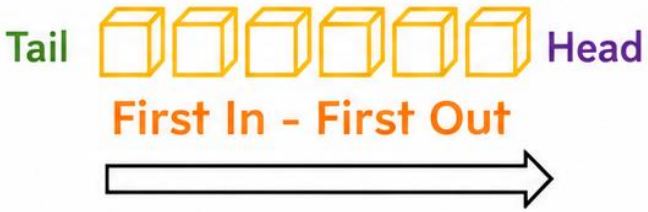
Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Time Quantum
2 Units

Gantt Chart:

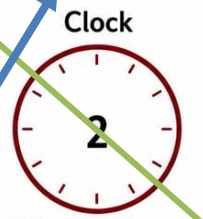








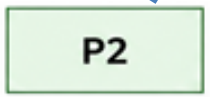




(((Alarm Rings)))











Process ID	Arrival Time	Burst Time
P1	0	5 → 3
P2	1	3
P3	2	1
P4	3	2
P5	4	3

- Example 2

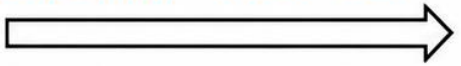
CPU Scheduling – Round Robin

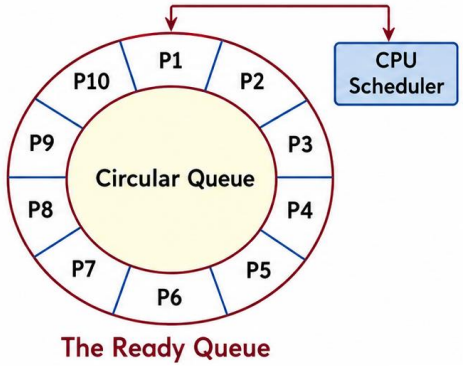
Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Time Quantum
2 Units

Tail  Head

First In - First Out





Gantt Chart:

0	2	4	5	7	9	11	12	13	14
P1	P2	P3	P1	P4	P5	P2	P1	P5	

Clock 3

P4	P1	P3

Clock 4

(((Alarm Rings)))

P2	P5	P4	P1

Clock 5

P2	P5	P4

Process ID	Arrival Time	Burst Time
P1	0	5 → 3
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3
P2	1	3 → 1
P3	2	1
P4	3	2
P5	4	3

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3
P2	1	3 → 1
P3	2	~~1~~
P4	3	2
P5	4	3

- Example 2

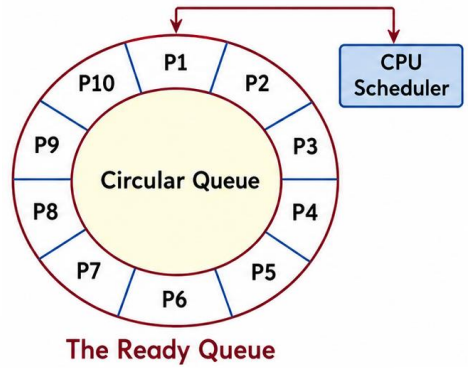
CPU Scheduling – Round Robin

Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Time Quantum
2 Units

Gantt Chart:





0 2 4 5 7 9 11 12 13 14

P1	P2	P3	P1	P4	P5	P2	P1	P5

Clock 5

P2	P5	P4

Clock 6

P2	P5	P4

Clock 7

(((Alarm Rings)))

P1	P2	P5

Clock 8

P1	P2	P5

Clock 9

(((Alarm Rings)))

P1	P2

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3
P2	1	3 → 1
P3	2	~~1~~
P4	3	2
P5	4	3

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → 1
P2	1	3 → 1
P3	2	~~1~~
P4	3	2
P5	4	3

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → 1
P2	1	3 → 1
P3	2	~~1~~
P4	3	~~2~~
P5	4	3

- Example 2

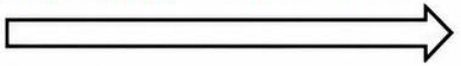
CPU Scheduling – Round Robin

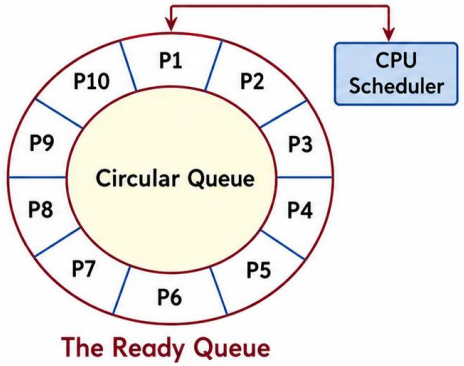
Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Time Quantum
2 Units

Tail  Head

First In - First Out





Gantt Chart:

0	2	4	5	7	9	11	12	13	14
P1	P2	P3	P1	P4	P5	P2	P1	P5	

Clock 9 (((Alarm Rings)))

P1	P2

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → 1
P2	1	3 → 1
P3	2	~~1~~
P4	3	~~2~~
P5	4	3

Clock 10

P1	P2

Clock 11 (((Alarm Rings)))

P5	P1

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → 1
P2	1	3 → 1
P3	2	~~1~~
P4	3	~~2~~
P5	4	3 → 1

Clock 12

P5

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → 1
P2	1	3 → ~~1~~
P3	2	~~1~~
P4	3	~~2~~
P5	4	3 → 1

- Example 2

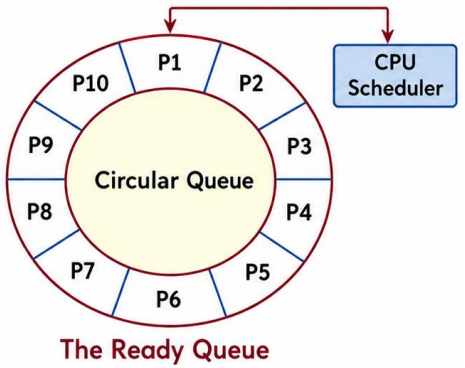
CPU Scheduling – Round Robin

Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Time Quantum
2 Units

Tail  Head

First In - First Out



Gantt Chart:

0	2	4	5	7	9	11	12	13	14
P1	P2	P3	P1	P4	P5	P2	P1	P5	

Clock 12 — P5

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → 1
P2	1	3 → ~~1~~
P3	2	~~1~~
P4	3	~~2~~
P5	4	3 → 1

Clock 13 — Ready Queue: EMPTY

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → ~~1~~
P2	1	3 → ~~1~~
P3	2	~~1~~
P4	3	~~2~~
P5	4	3 → 1

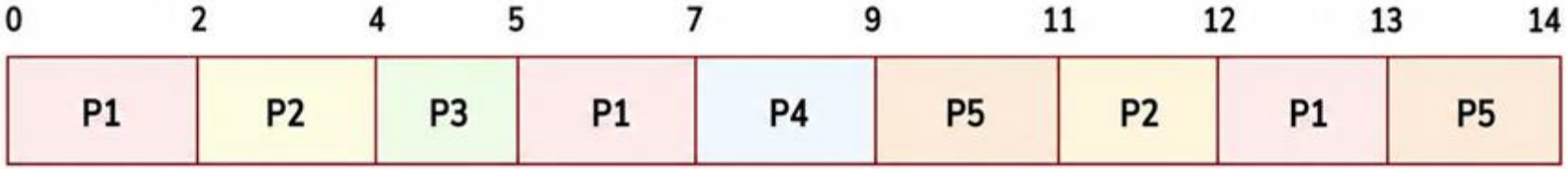
Clock 14 — Ready Queue: EMPTY

Process ID	Arrival Time	Burst Time (Remaining)
P1	0	5 → 3 → ~~1~~
P2	1	3 → ~~1~~
P3	2	~~1~~
P4	3	~~2~~
P5	4	3 → ~~1~~

Example 2

CPU Scheduling – Round Robin

Gantt Chart:



Turn Around time = Completion time – Arrival time

Waiting time = Turn Around time – Burst time

Process ID	Completion Time	Turnaround Time	Waiting Time
P1	13	13 – 0 = 13	13 – 5 = 8
P2	12	12 – 1 = 11	11 – 3 = 8
P3	5	5 – 2 = 3	3 – 1 = 2
P4	9	9 – 3 = 6	6 – 2 = 4
P5	14	14 – 4 = 10	10 – 3 = 7

Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Average Turn Around time

= (13 + 11 + 3 + 6 + 10) / 5

= 43 / 5 = 8.6 units

Average waiting time

= (8 + 8 + 2 + 4 + 7) / 5

= 29 / 5 = 5.8 units

- Example 3

CPU Scheduling – Round Robin

- Consider the three processes. The time quantum or time slice is 3 ms

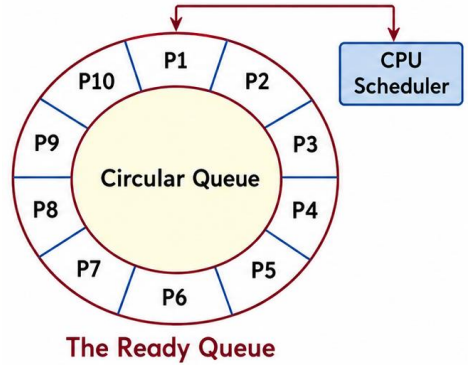
Process	Arrival Time	Burst Time
P1	0	8
P2	1	4
P3	2	9
P4	3	5
P5	4	2

P1	P2	P3	P4	P5	P1	P2	P3	P4	P1	P3

0 3 6 9 12 14 17 18 21 23 25 28

- Turnaround Time of P1 = completion time – Arrival Time = 25 – 0 = 25
- Turnaround Time of P2 = 18 – 1 = 17
- Turnaround Time of P3 = 28 – 2 = 26
- Turnaround Time of P4 = 23 – 3 = 20
- Turnaround Time of P5 = 14 – 4 = 10
- Waiting Time of P1 = Turnaround Time – Burst Time = 25 – 8 = 17
- Waiting Time of P2 = 17 – 4 = 13
- Waiting Time of P3 = 26– 9 = 17
- Waiting Time of P4 = 20 – 5 = 15
- Waiting Time of P5 = 10– 2 = 8
- Avg Turnaround time = (25 + 17 + 26 + 20 + 10)/ 5= 19.6
- Avg Waiting time = (17 + 13 + 17 + 15 + 8) / 5 = 14



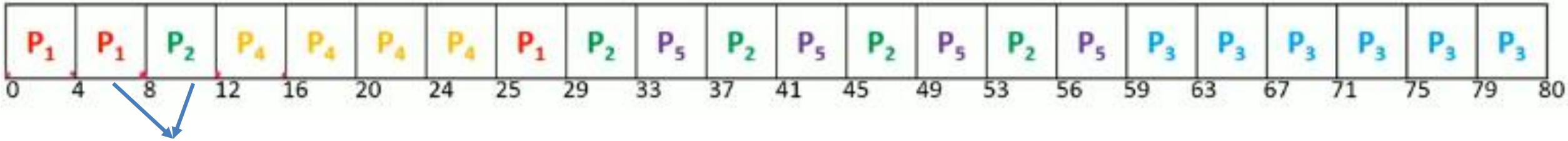


▪ Example 4 CPU Scheduling – Round Robin with Priority

Processes	Arrival Time	Burst Time	Priority
P_1	0	12	3
P_2	5	19	3
P_3	8	21	5
P_4	11	13	2
P_5	15	15	3

CT	TAT	WT	RT
29	29	17	0
56	51	32	3
80	72	51	51
25	14	1	1
59	44	29	18
Total:	210	130	73

Time Quantum = 4



Round Robin treats processes arriving with same priority equally --- it gives 4ms to P1 and next 4ms to P2

CPU Scheduling – with I/O Burst -FCFS

Example

Consider five processes with the following behaviour. Each process requires an initial CPU burst, followed by an I/O burst, and finally a second CPU burst to terminate.

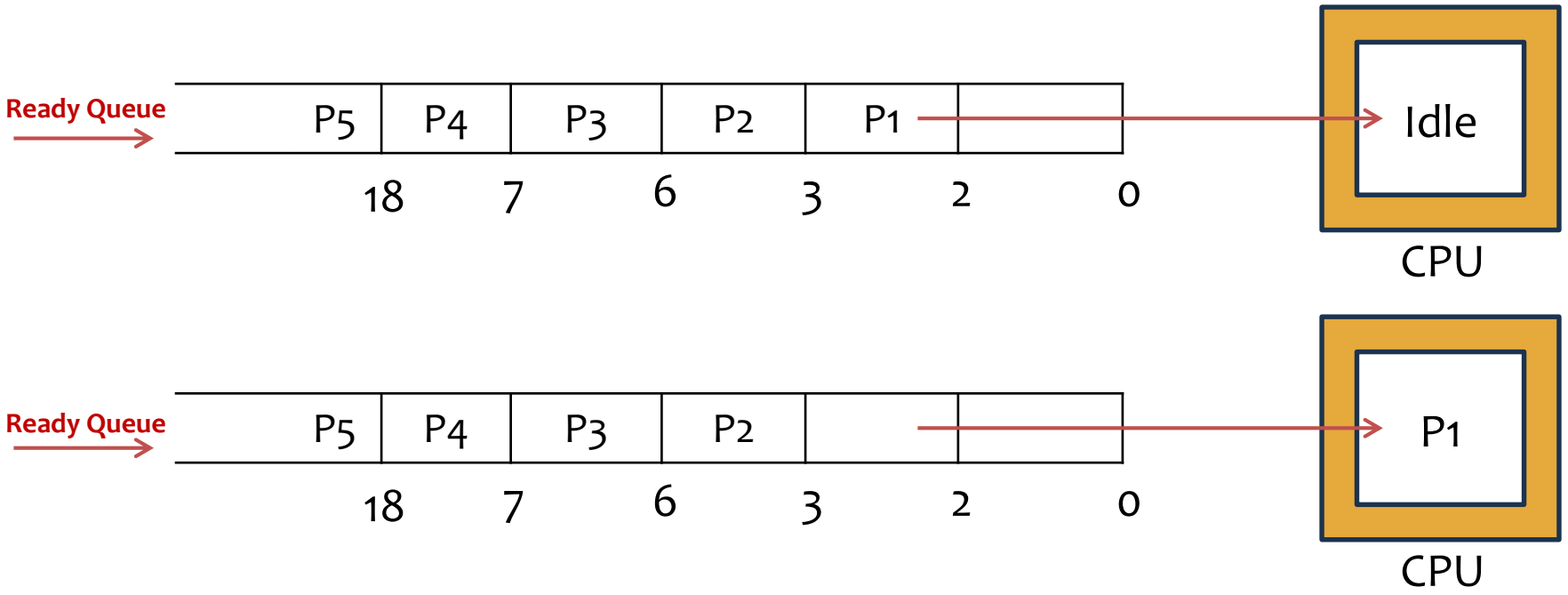
Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

- Calculate all the appropriate Performance Metrics to evaluate the Non Preemptive scheduling algorithms like FCFS.

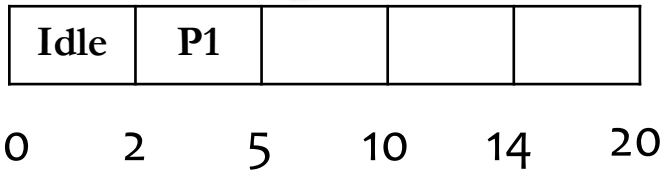
Note: I/O Processing can happen in parallel.

Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

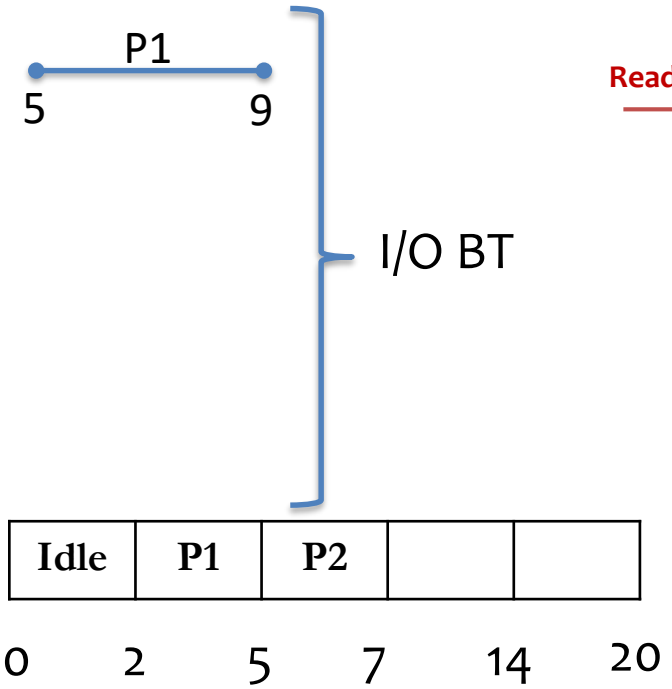


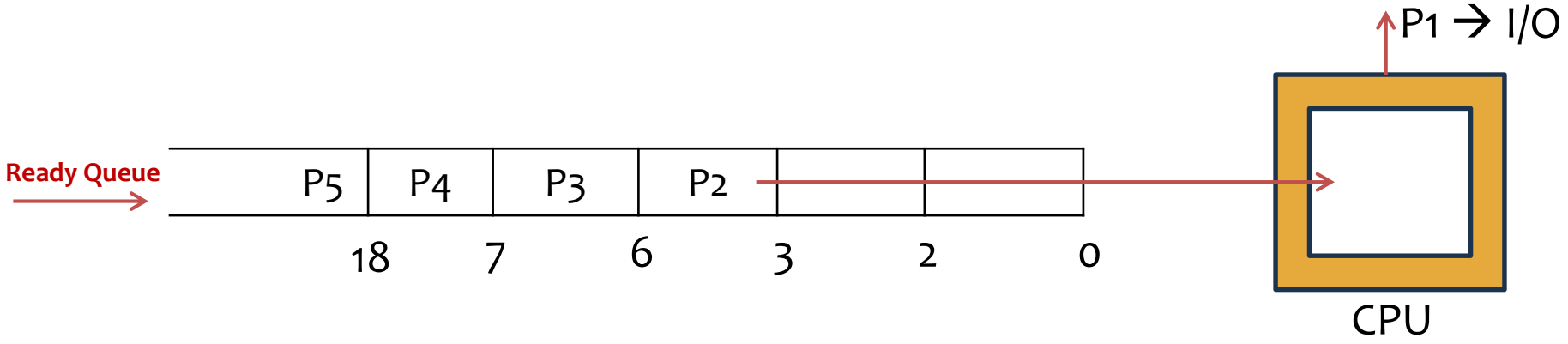
I/O BT

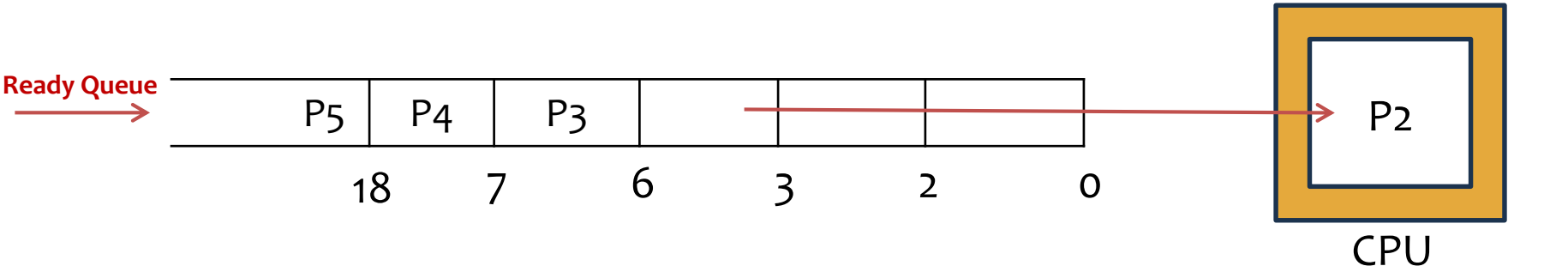


Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

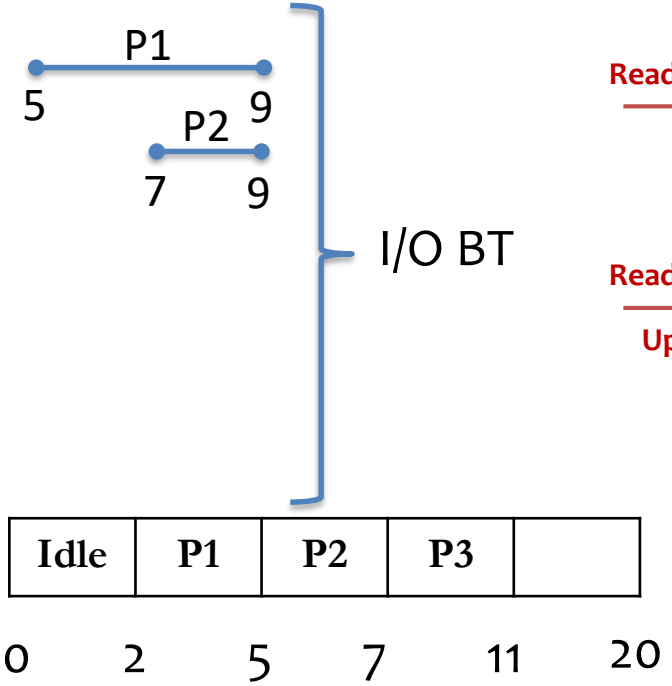


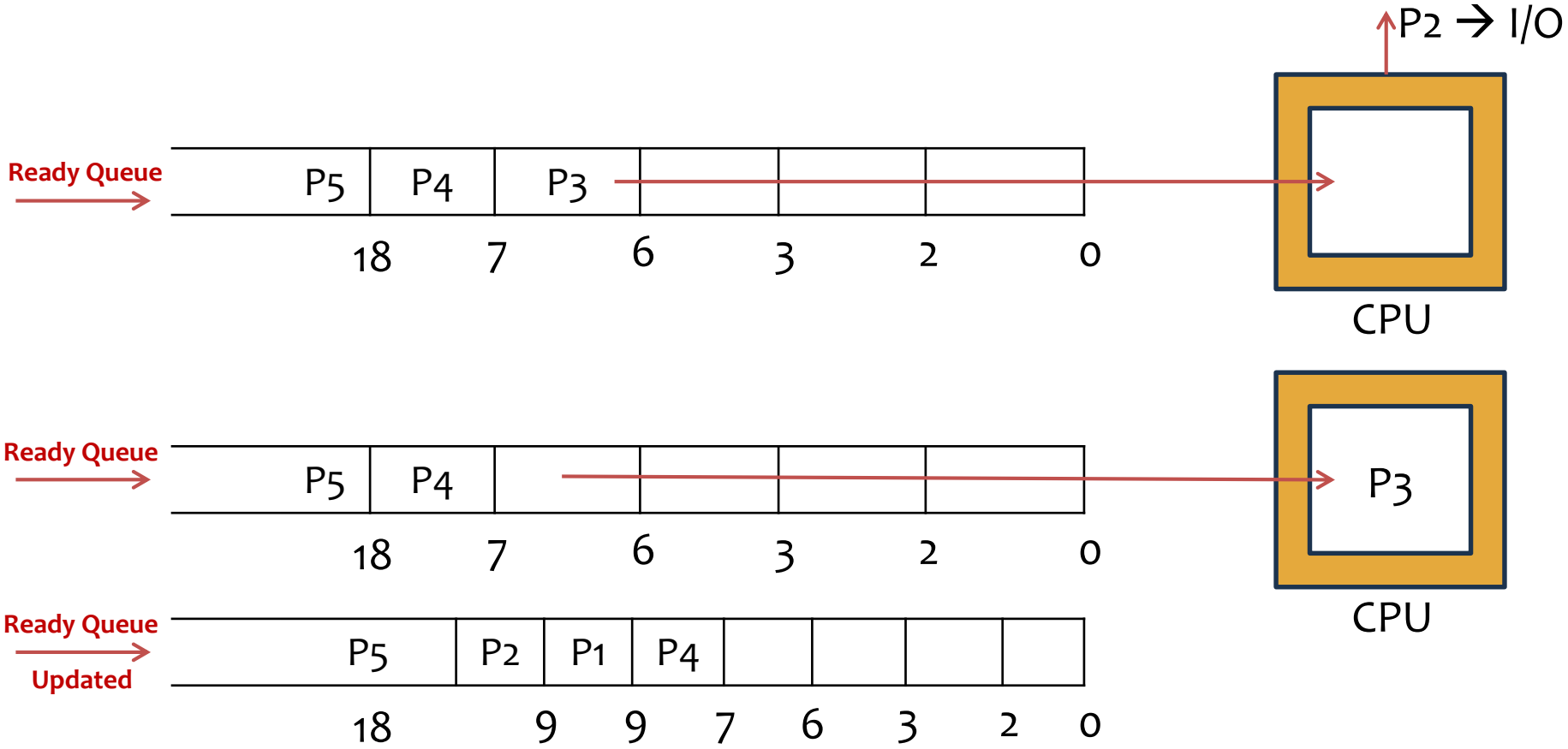




Example CPU Scheduling – with I/O Burst -FCFS

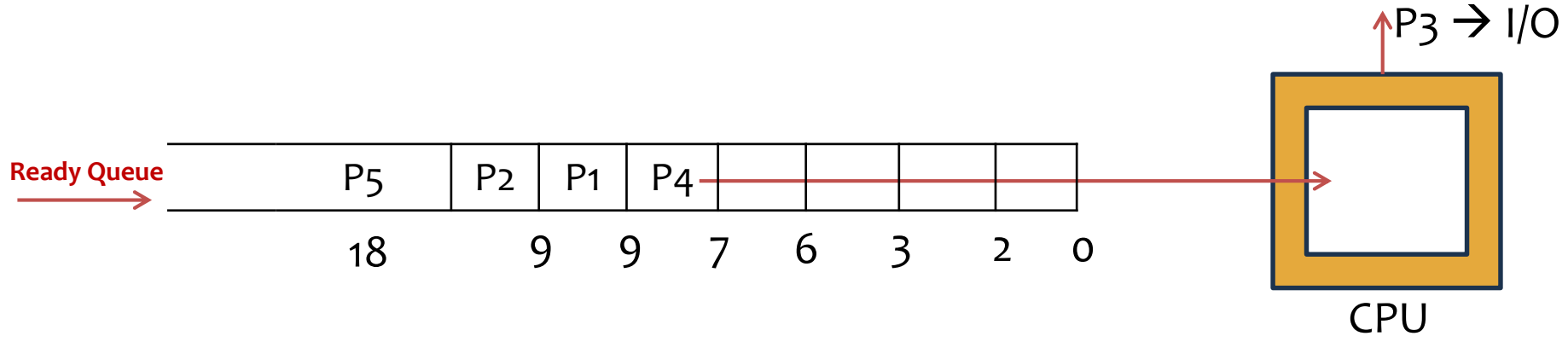
Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

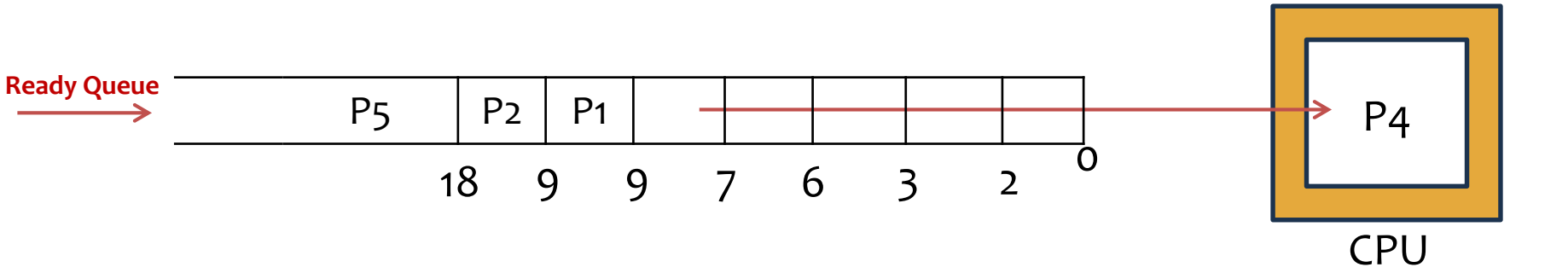


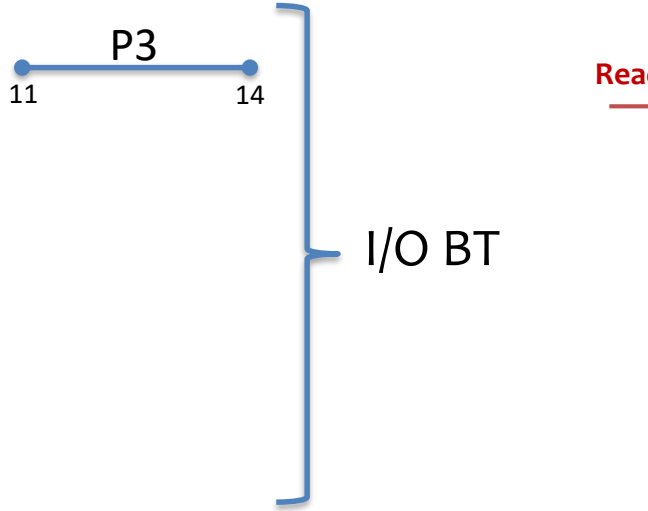


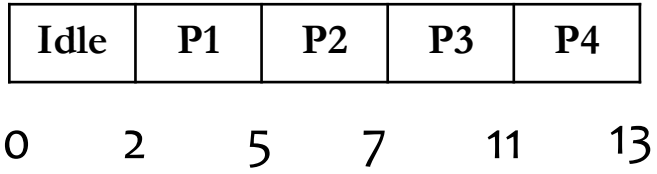
Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3



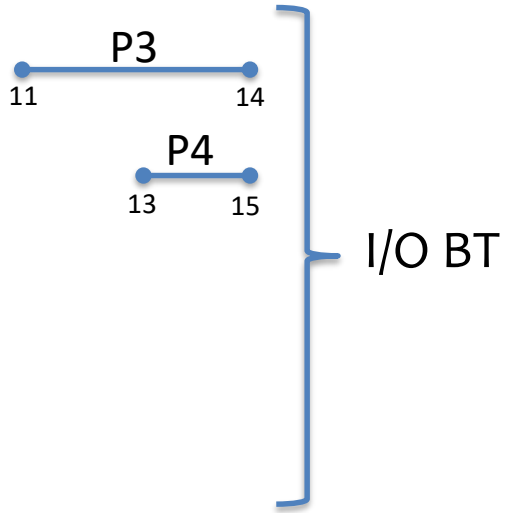


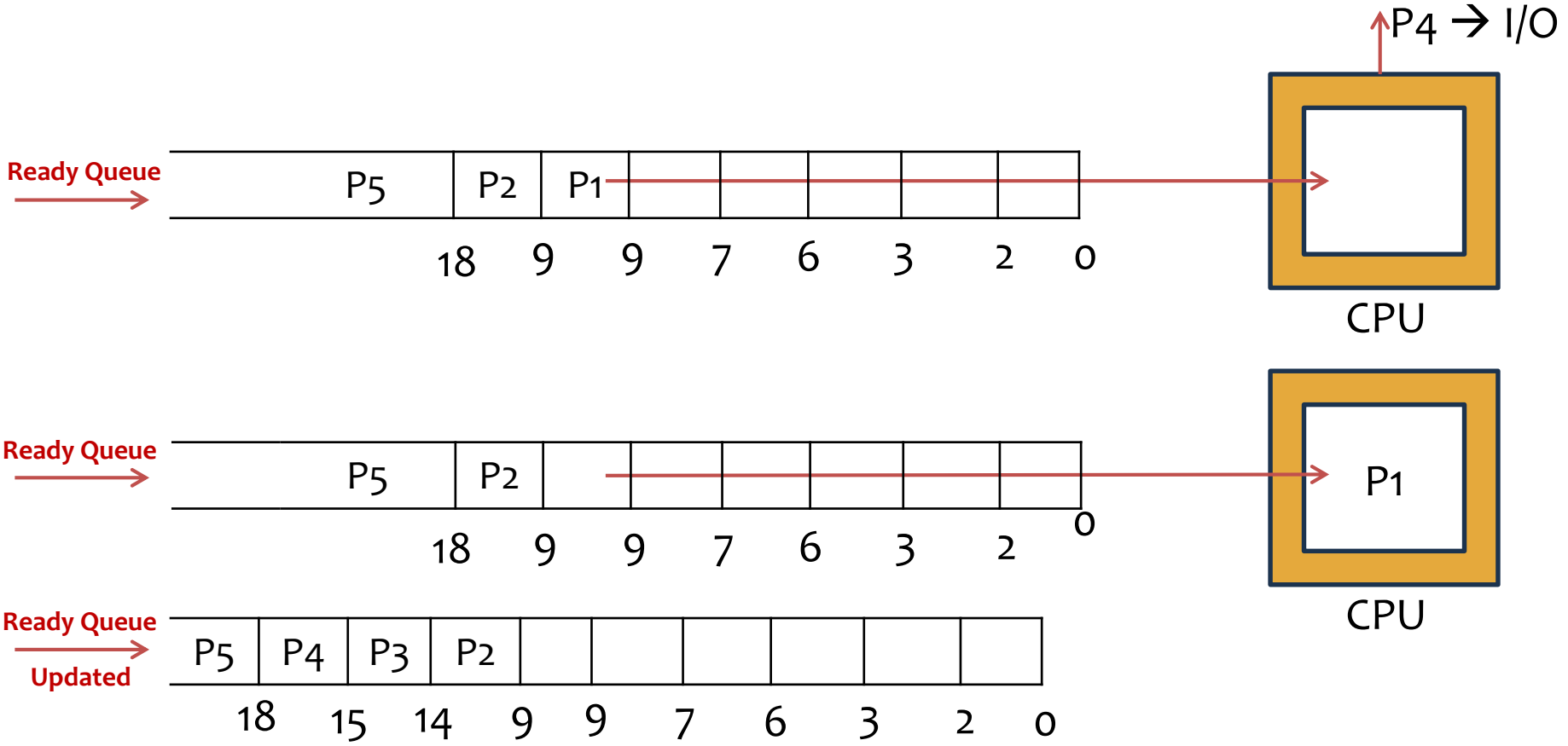


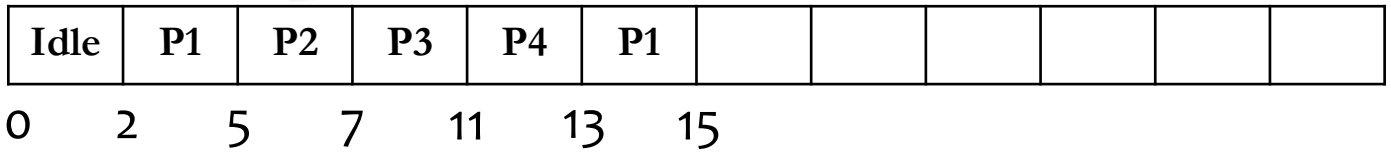


Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

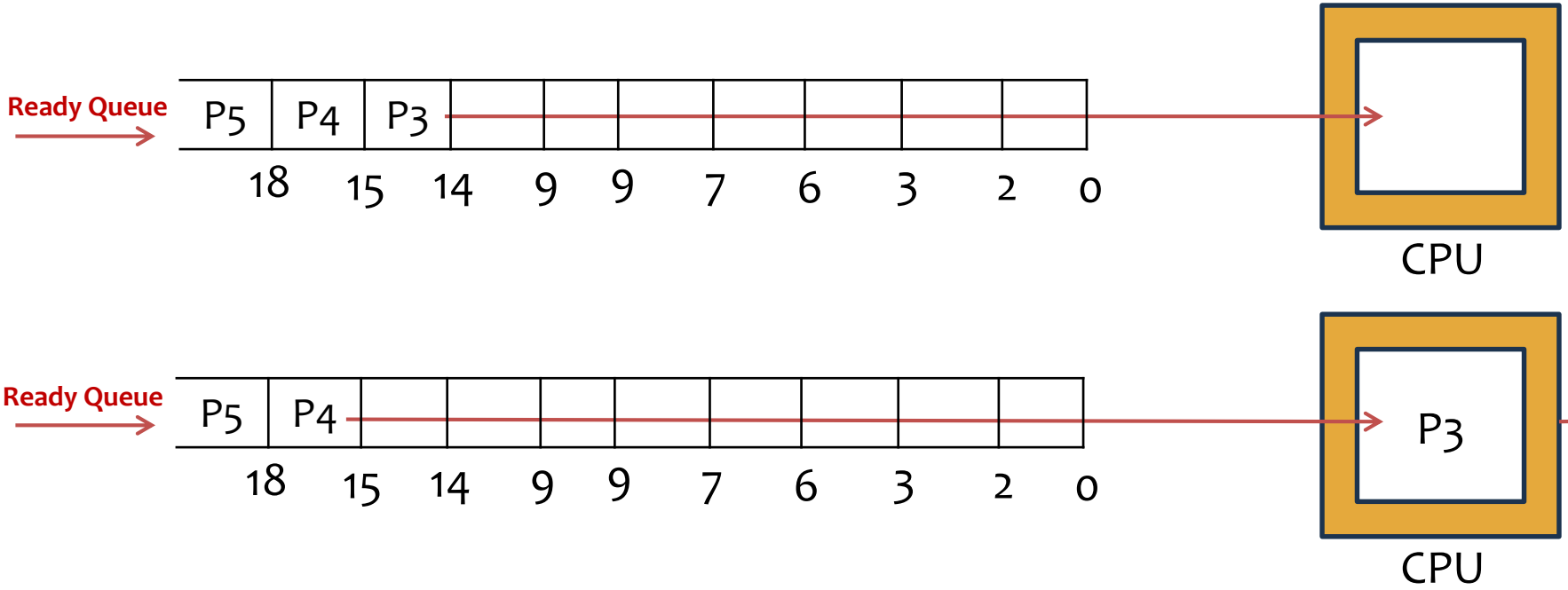




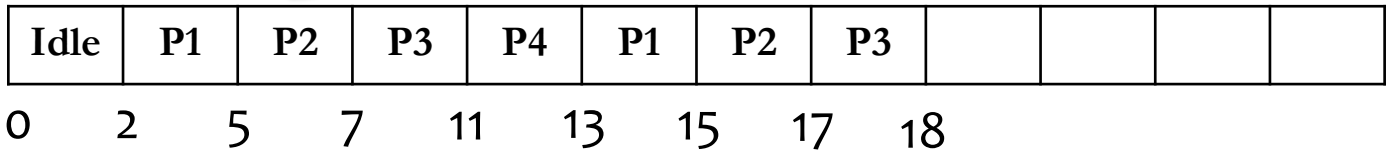


Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

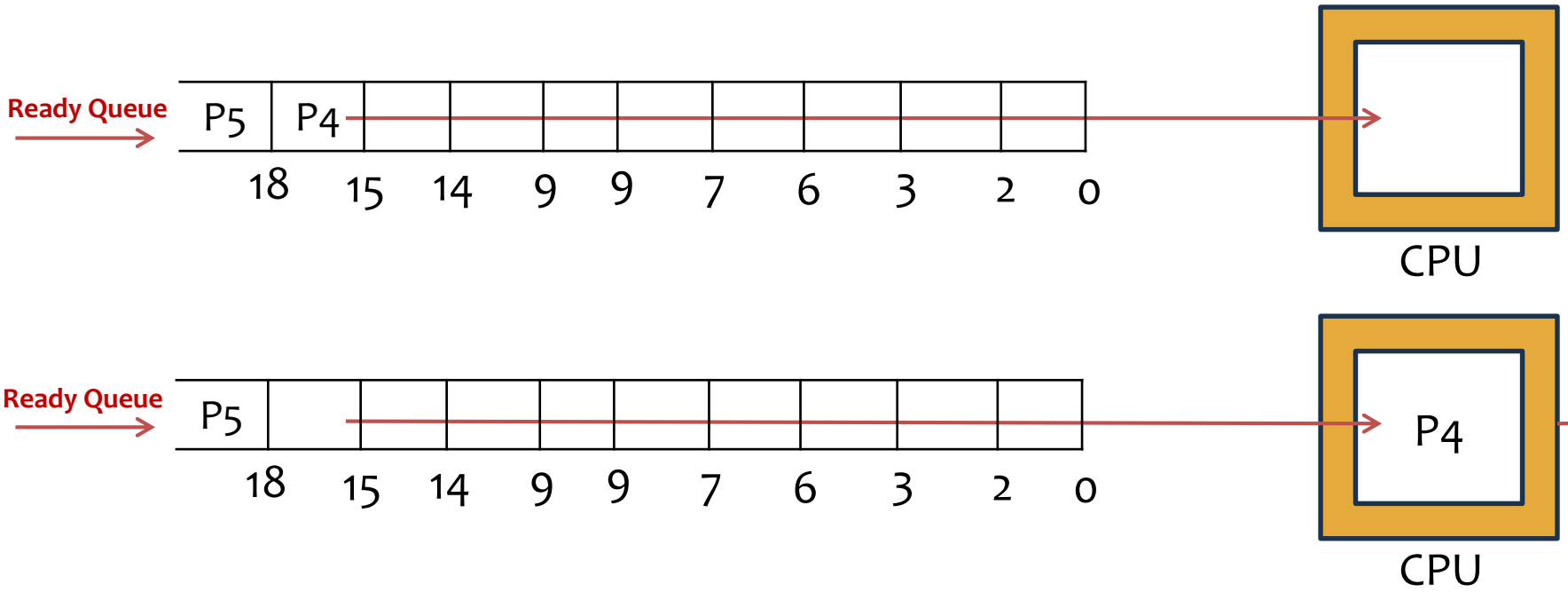


I/O BT

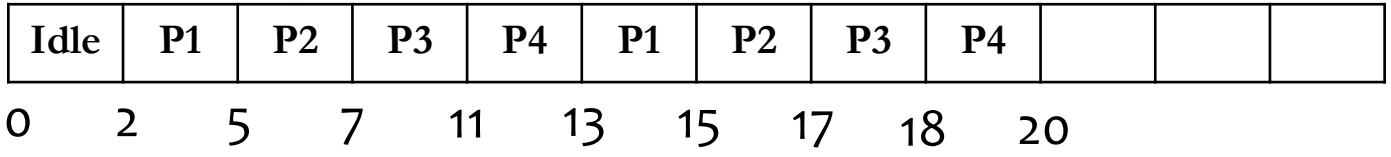


Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3

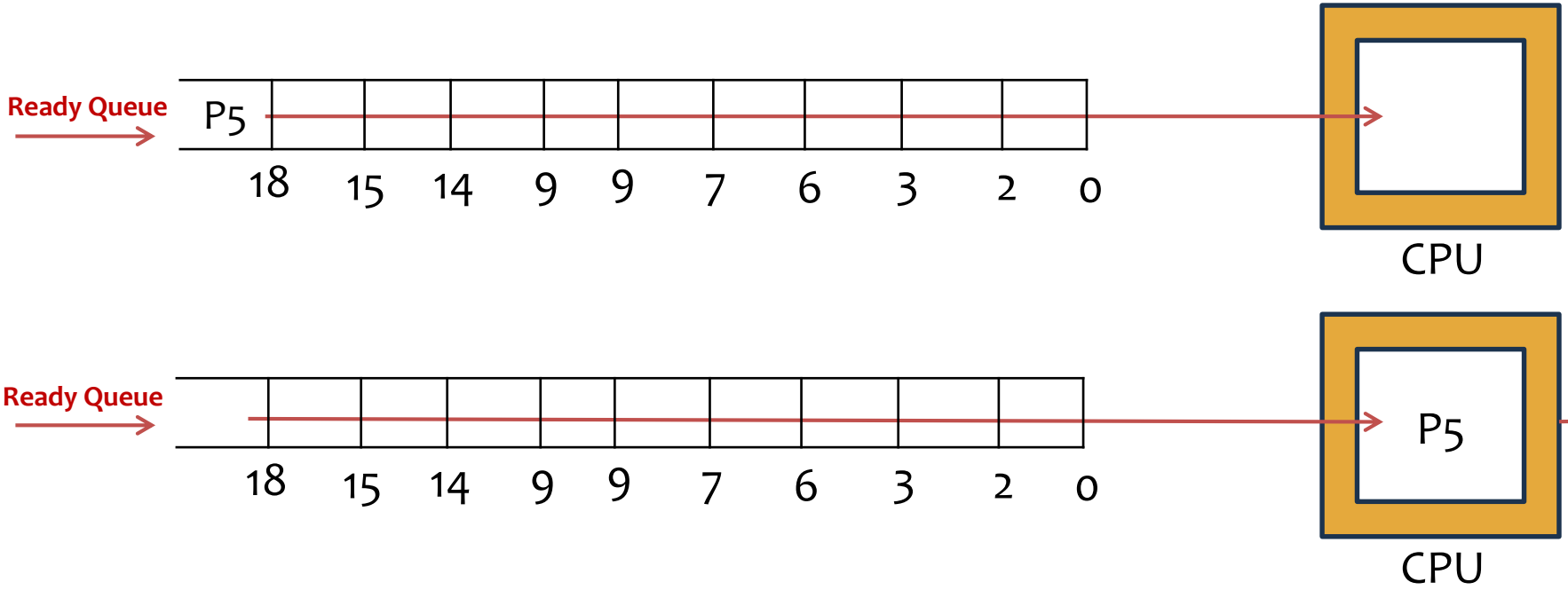


I/O BT



Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3



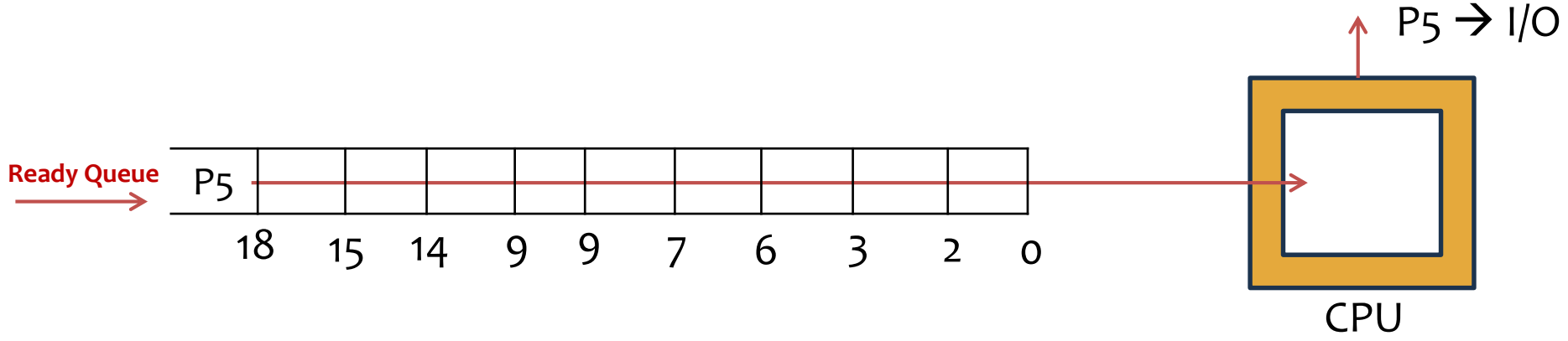
I/O BT

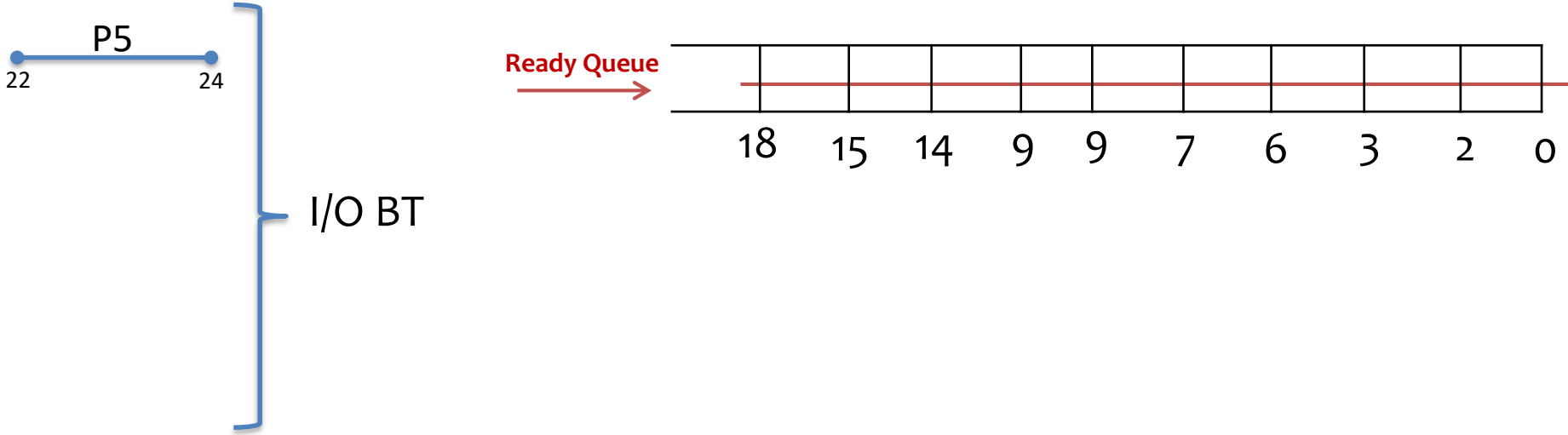
Idle	P1	P2	P3	P4	P1	P2	P3	P4	P5		

0 2 5 7 11 13 15 17 18 20 22

Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3



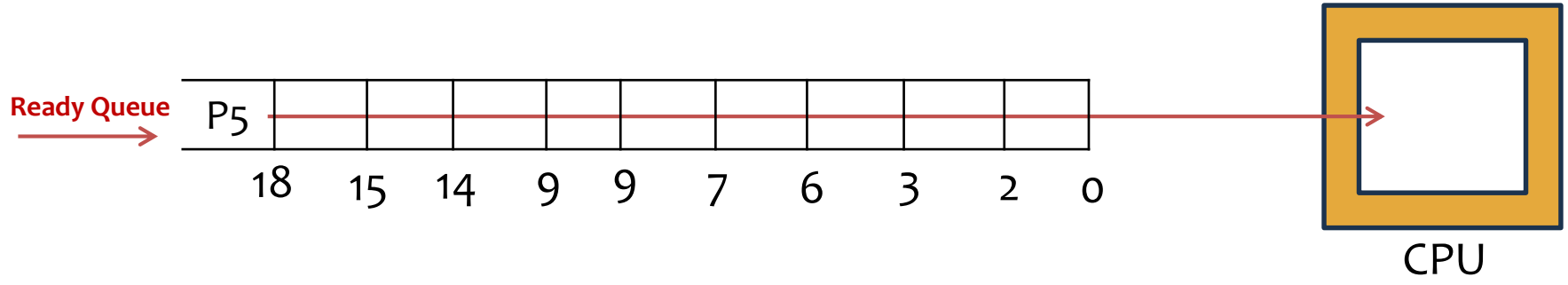


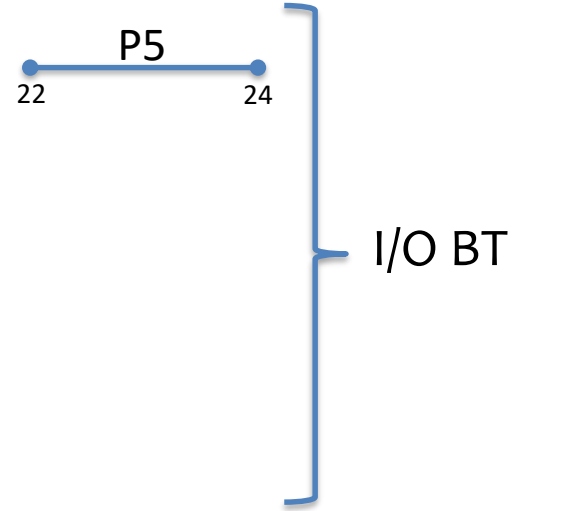
Idle	P1	P2	P3	P4	P1	P2	P3	P4	P5	Idle	

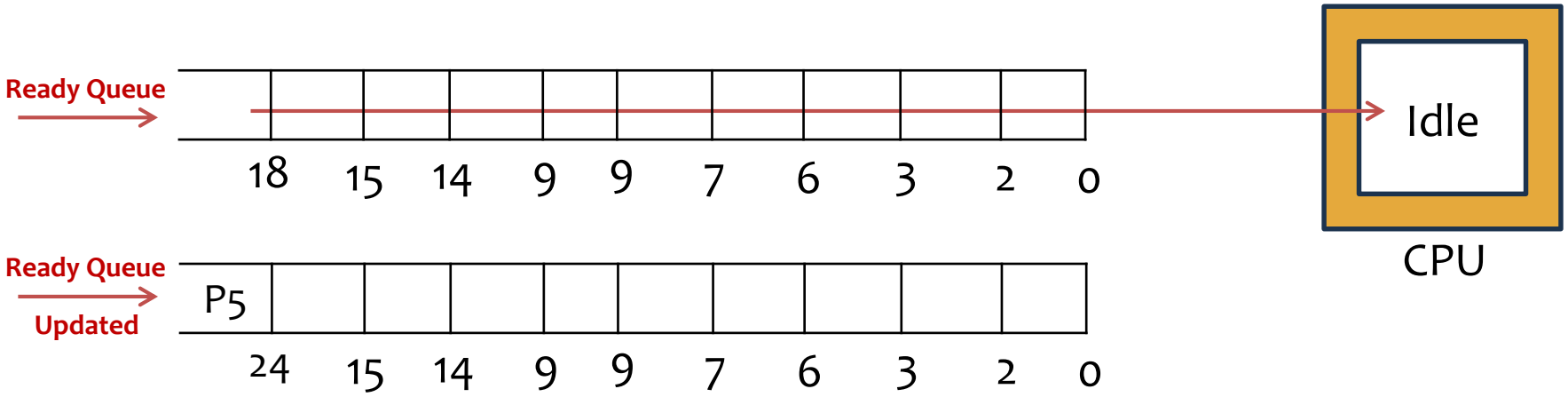
0 2 5 7 11 13 15 17 18 20 22 24

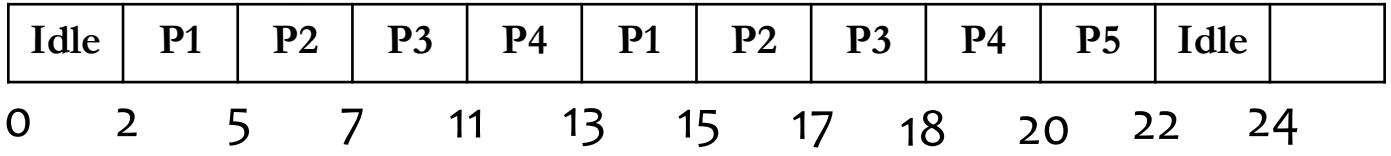
Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3



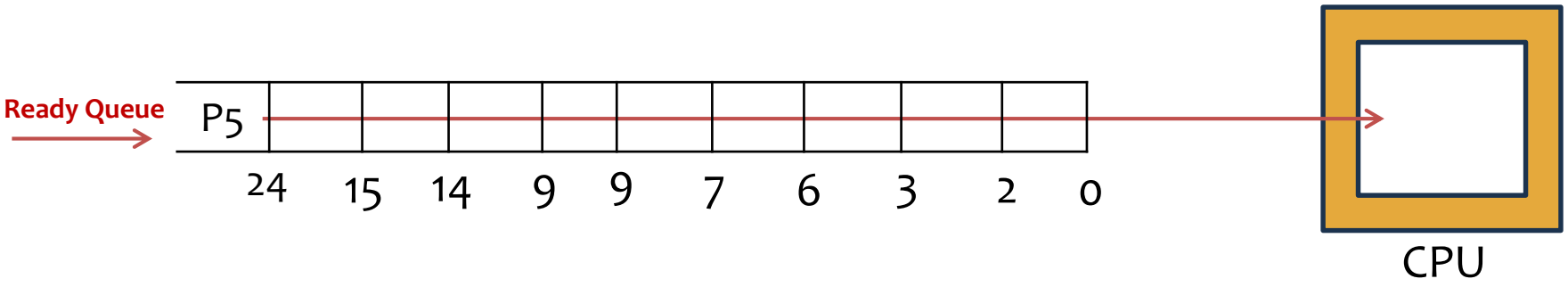


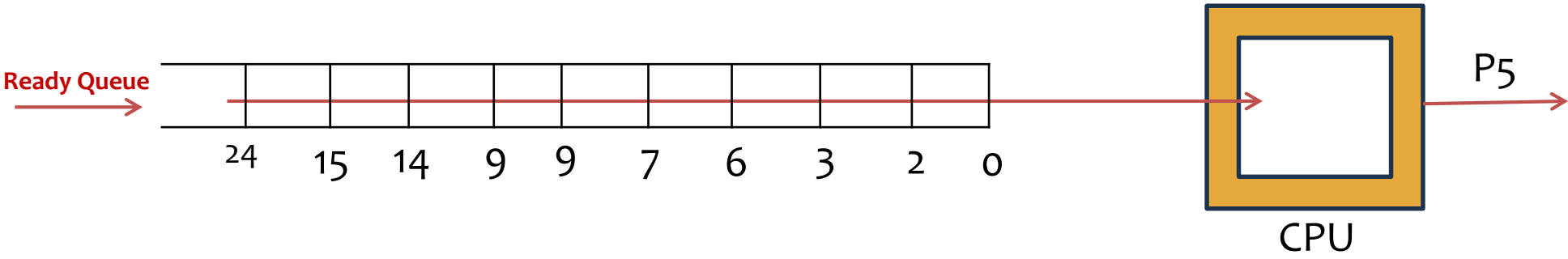




Example CPU Scheduling – with I/O Burst -FCFS

Process ID	Arrival Time	CPU Burst Time 1	I/O Burst Time	CPU Burst Time 2	Total CPU Burst Time
P1	2	3	4	2	5
P2	3	2	2	2	4
P3	6	4	3	1	5
P4	7	2	2	2	4
P5	18	2	2	1	3





I/O BT

Idle	P1	P2	P3	P4	P1	P2	P3	P4	P5	Idle	P5

0 2 5 7 11 13 15 17 18 20 22 24 25

Example

CPU Scheduling – with I/O Burst -FCFS

Performance of the FCFS

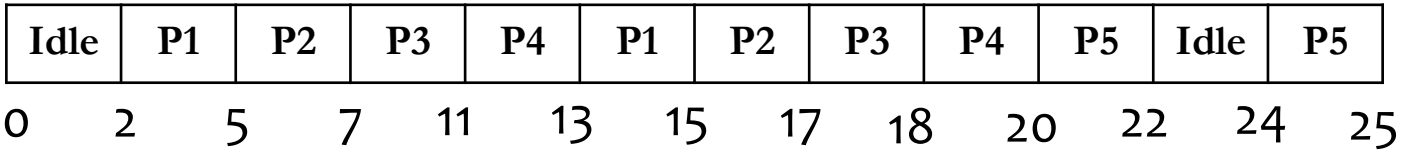
Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU* Allocation *Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	2	5	2	15	13	8	0
P2	3	4	5	17	14	10	2
P3	6	5	7	18	12	7	1
P4	7	4	11	20	13	9	4
P5	18	3	20	25	7	4	2
SUM					**59**	**38**	**9**
AVG					**59/5 = 11.8 ms**	**38/5 = 7.6 ms**	**9/5 = 1.8 ms**

Example CPU Scheduling – with I/O Burst -FCFS

Performance of the FCFS

Total CPU Burst Time

$$5 + 4 + 5 + 4 + 3 = 21 \text{ ms}$$



Total System Time

$$25 \text{ ms}$$

CPU utilization → 21/25 * 100 = 84%

Throughput= Number of Processes / Total System Time $= \frac{5}{25} = 0.2$

Throughput =0.2 processes/ms

No. of Context switches without Preemption in FCFS : 8

Example CPU Scheduling – with I/O Burst - SRTF

Process Specifications

P1: Arrives at t = 2. Requires a single continuous CPU burst of 4 units.

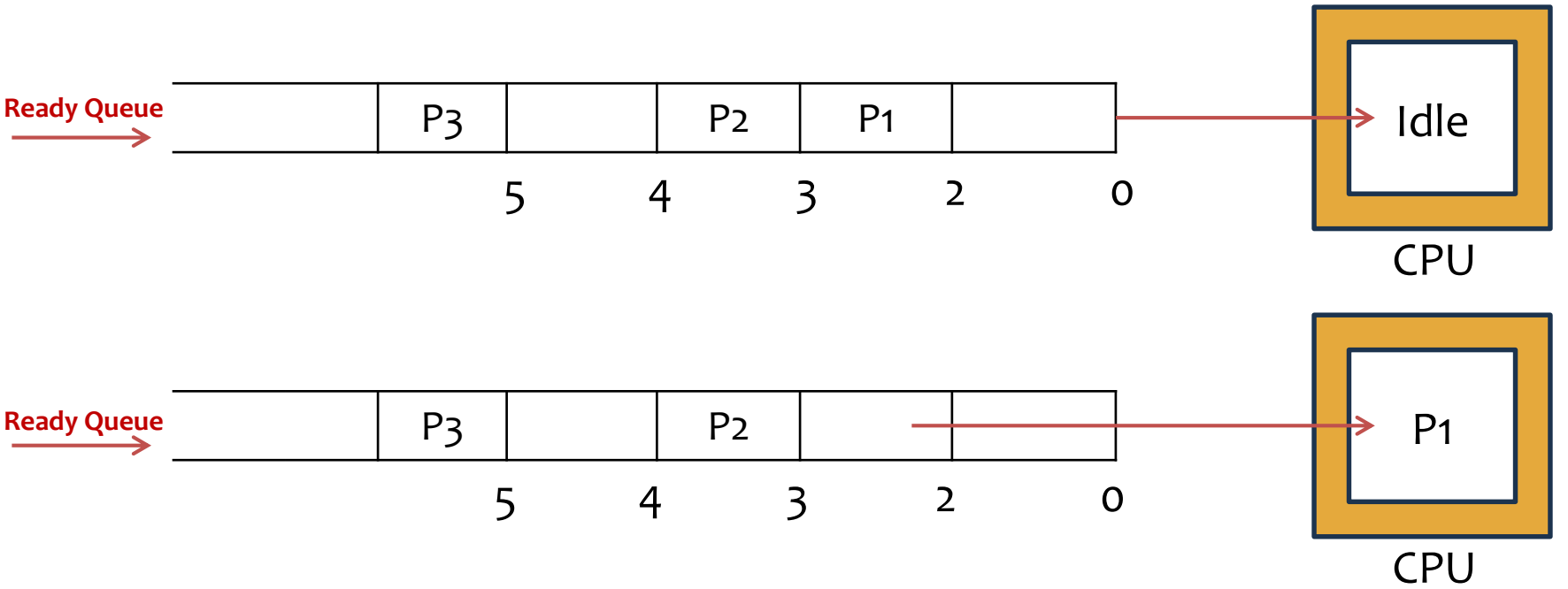
P2: Arrives at t = 3. Its lifecycle is split: 2 units of CPU burst, followed immediately by 3 units of I/O burst, and finishing with 2 units of a second CPU burst.

P3: Arrives at t = 5. Requires a single continuous CPU burst of 1 unit.

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		

Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		



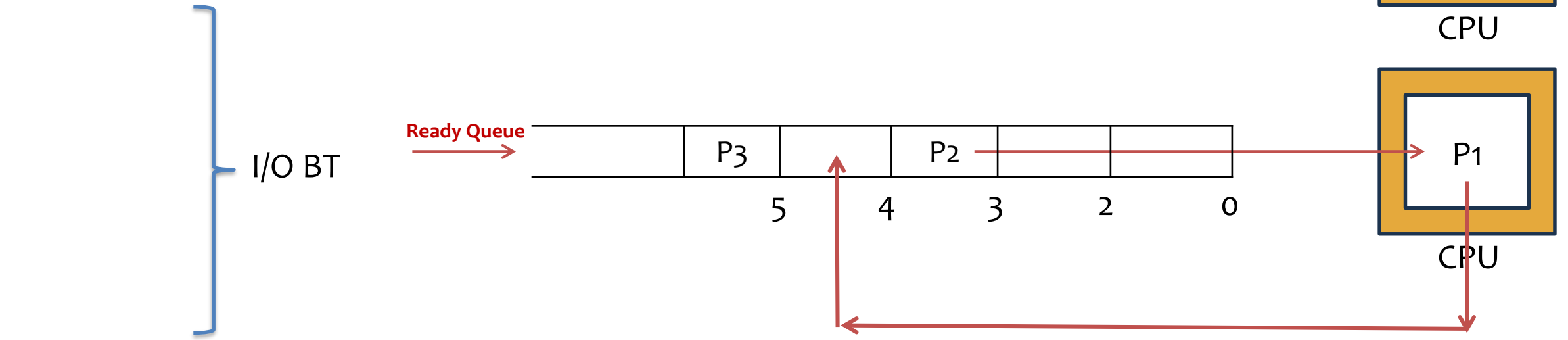
At time 0 – 2, No process in Ready Queue

Idle	P1										

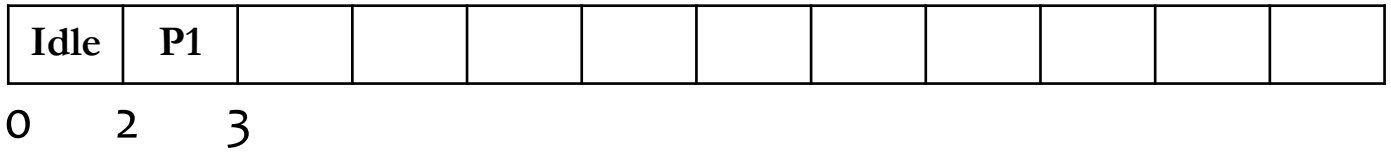
0 2 3

Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		



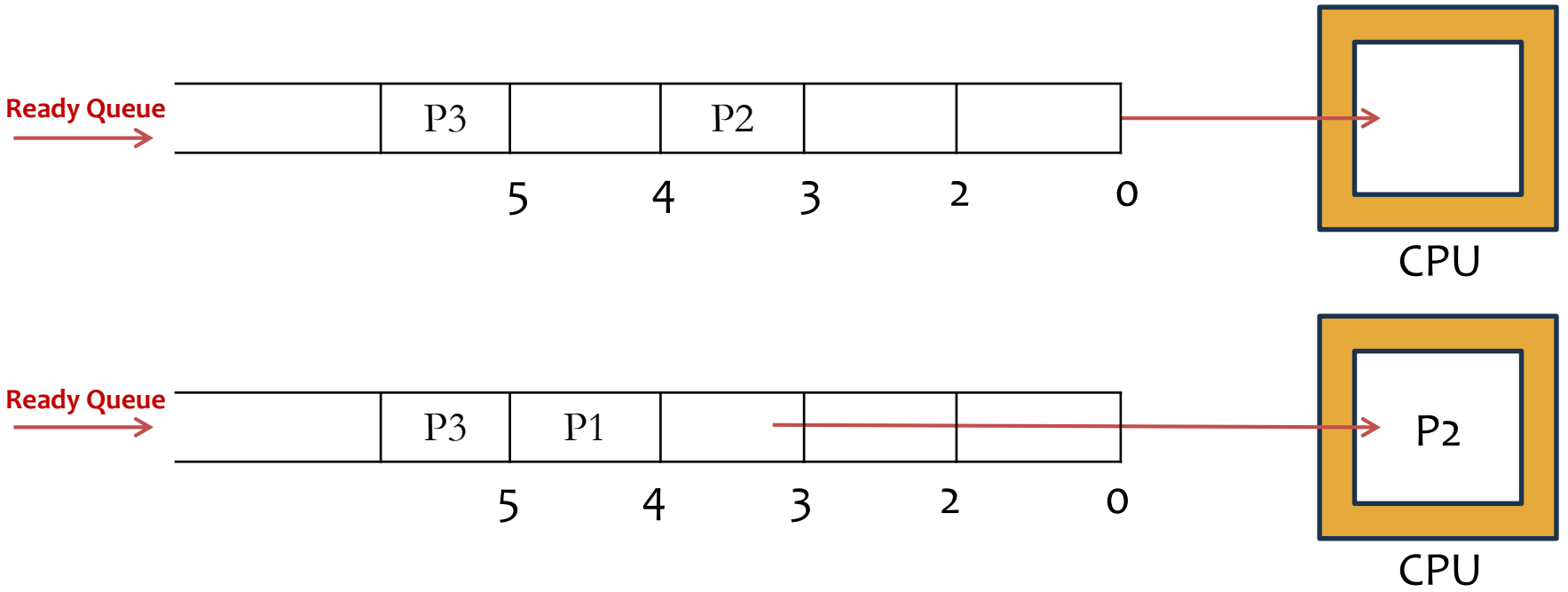
At time 3, Compare remaining BT of P2 & P1
P2(BT) < P1(BT), move P1 to Ready Queue



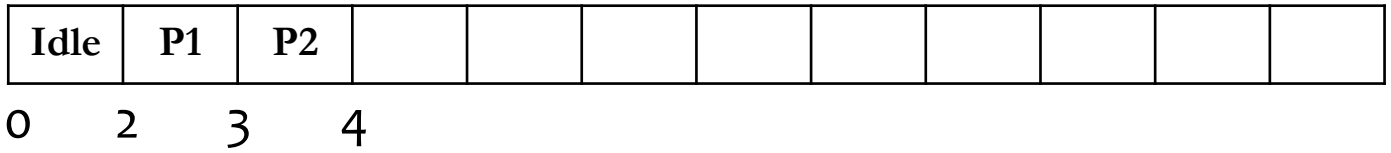
Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		

I/O BT



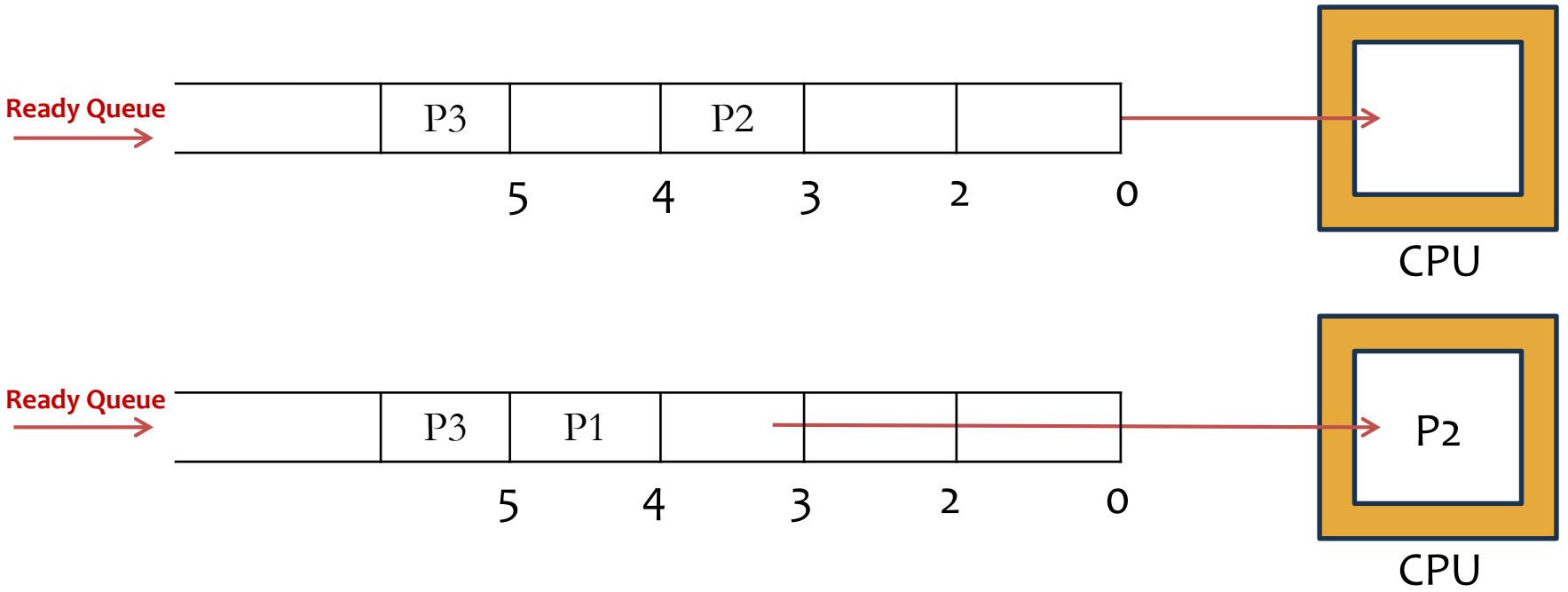
At time 3, Compare remaining BT of P2 & P1
P2(BT) < P1(BT), Preempt P1 and give CPU to P2



Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		

I/O BT



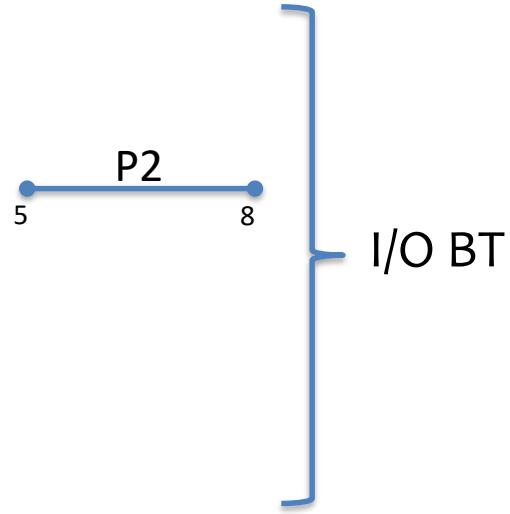
At time 4, Compare remaining BT of P2 & P1
P2(BT) < P1(BT), Continue P2 running

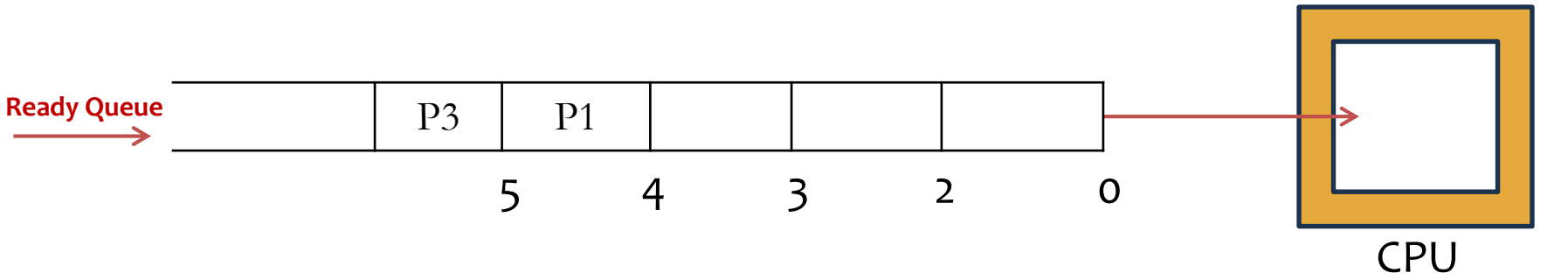
Idle	P1	P2	P2								

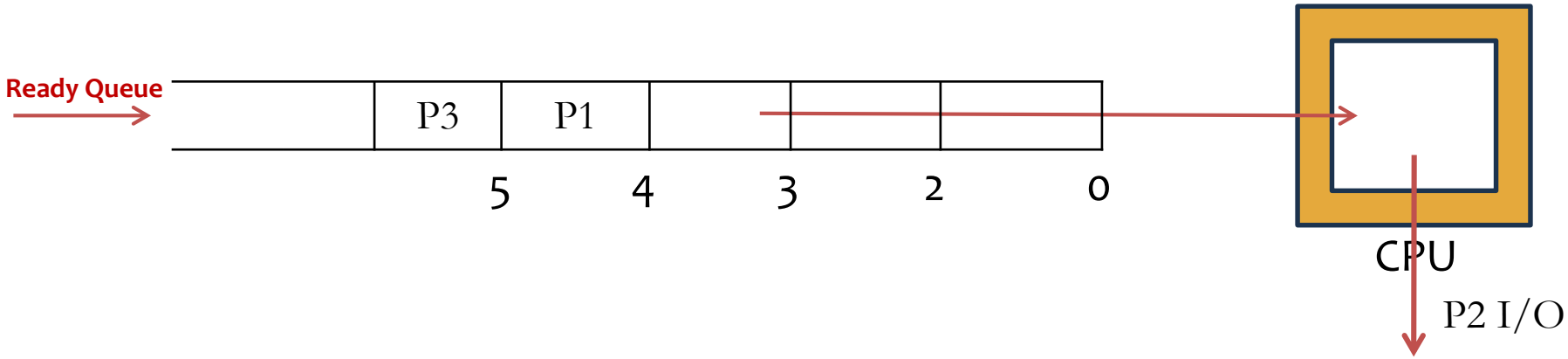
0 2 3 4 5

Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		





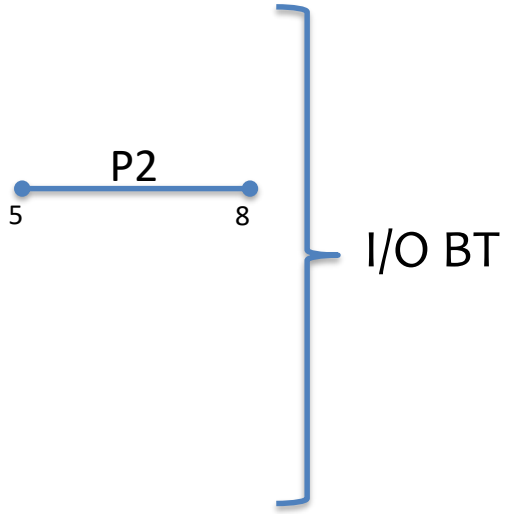


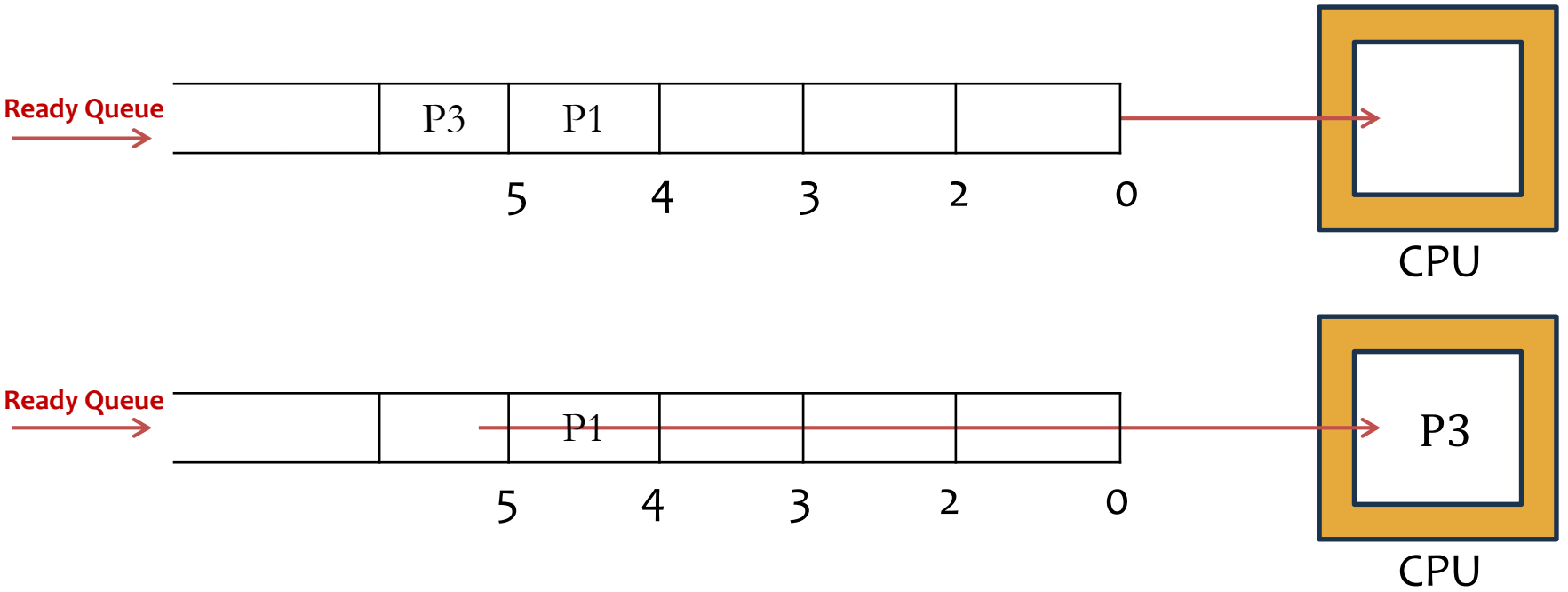
At time 5, P2 completes 1st BT & enters I/O



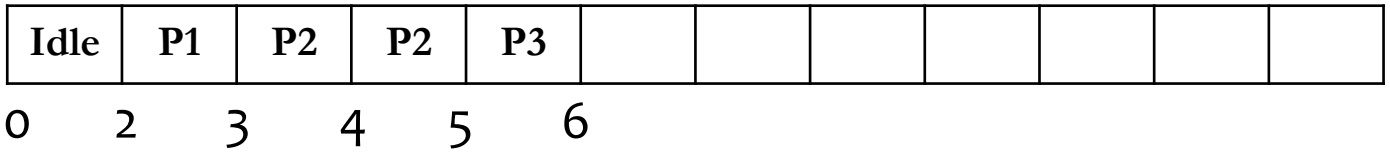
Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		



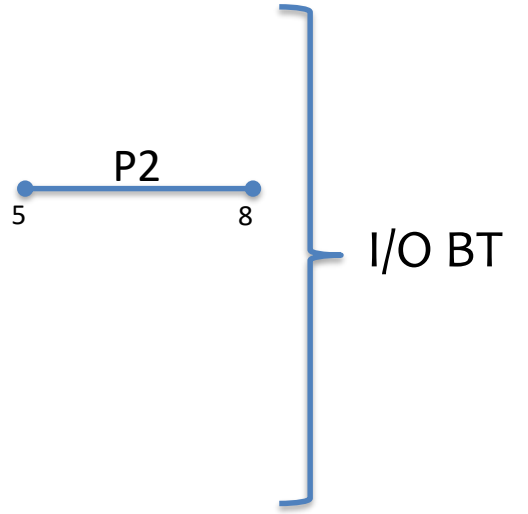


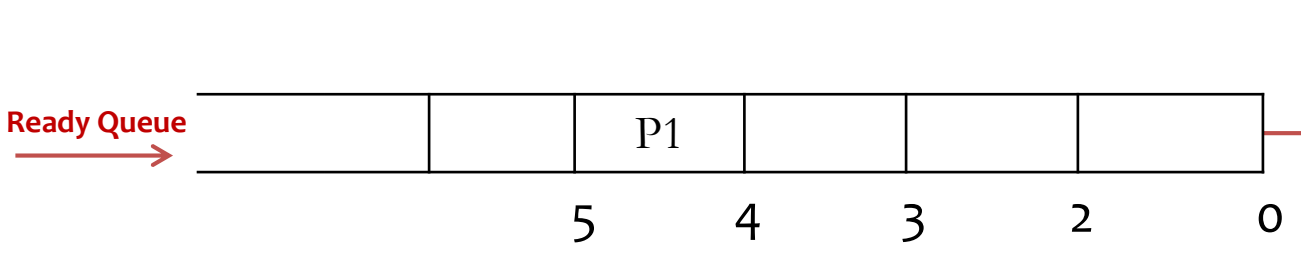
At time 5, Compare remaining BT time of P1 & P3. P3(BT) < P1(BT), P3 gets CPU

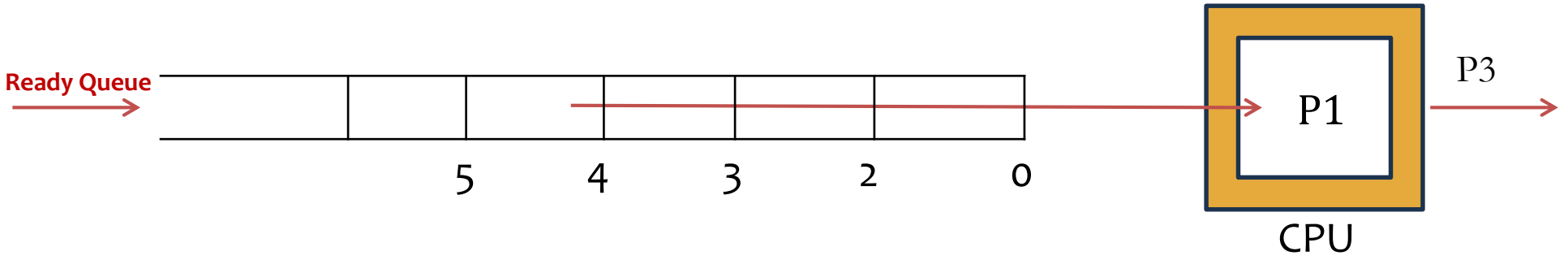


Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		







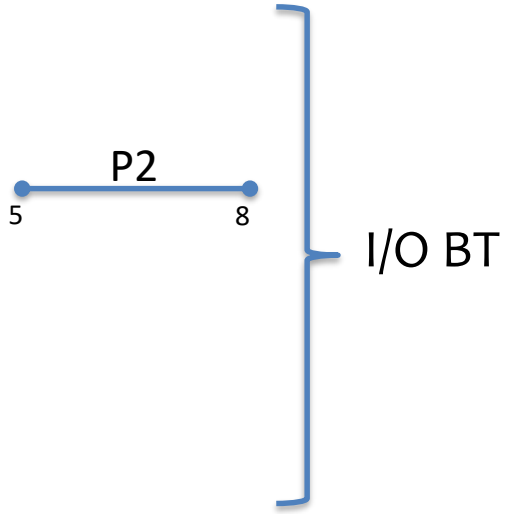
At time 6, Only P1 is there in ready Queue
Assign CPU to it

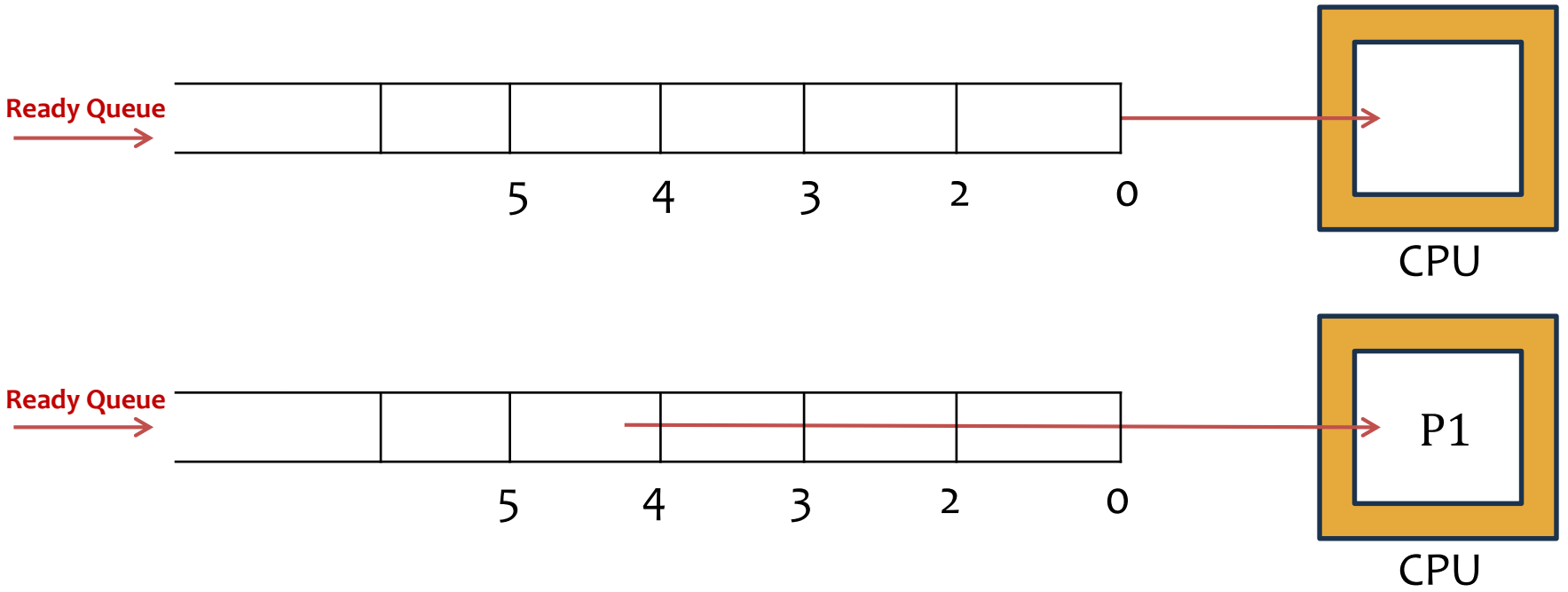
Idle	P1	P2	P2	P3	P1						

0 2 3 4 5 6 7

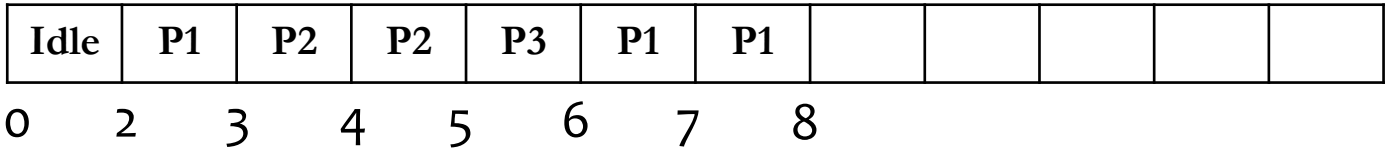
Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		



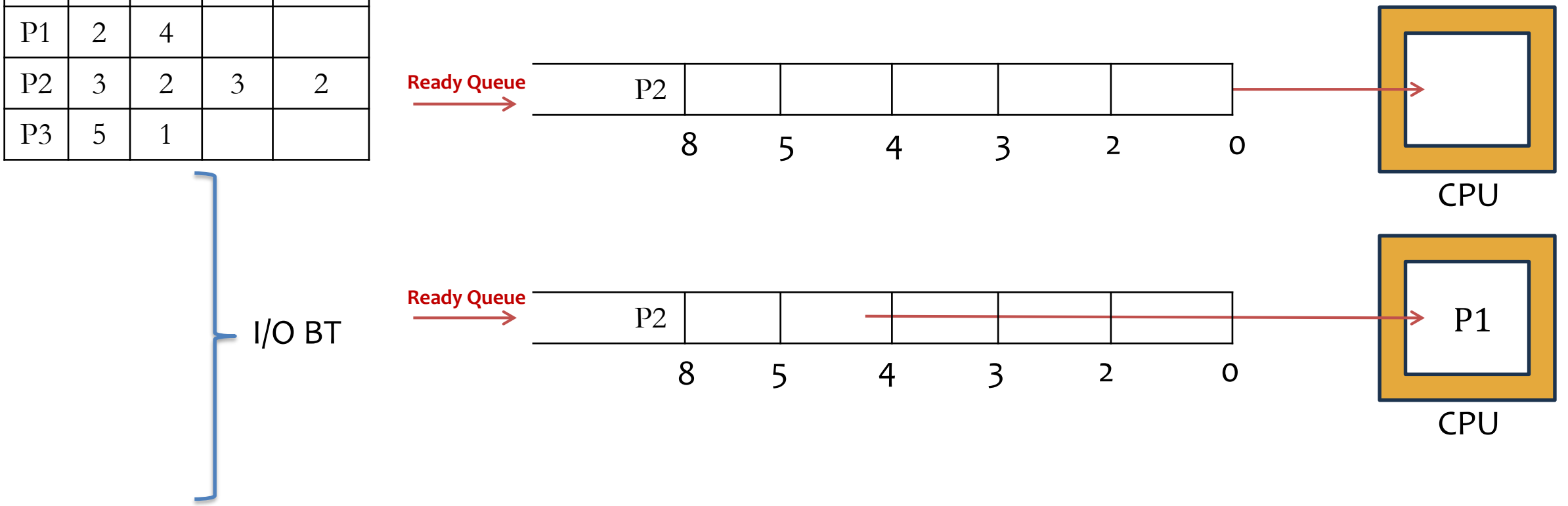


At time 7, No process in Ready Queue
P1 continues



Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		

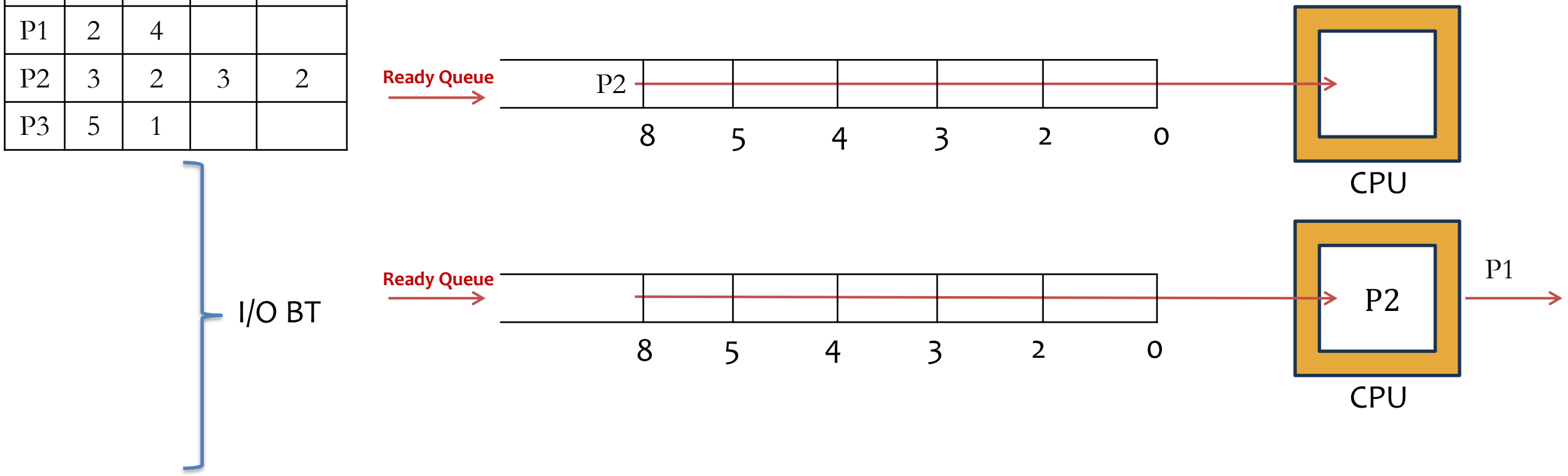


At time 8, P2 is in ready queue. Compare remaining BT time of P1 & P2.
P2(BT) > P1(BT). Continue with P1

Idle	P1	P2	P2	P3	P1	P1	P1				
0	2	3	4	5	6	7	8	9			

Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		



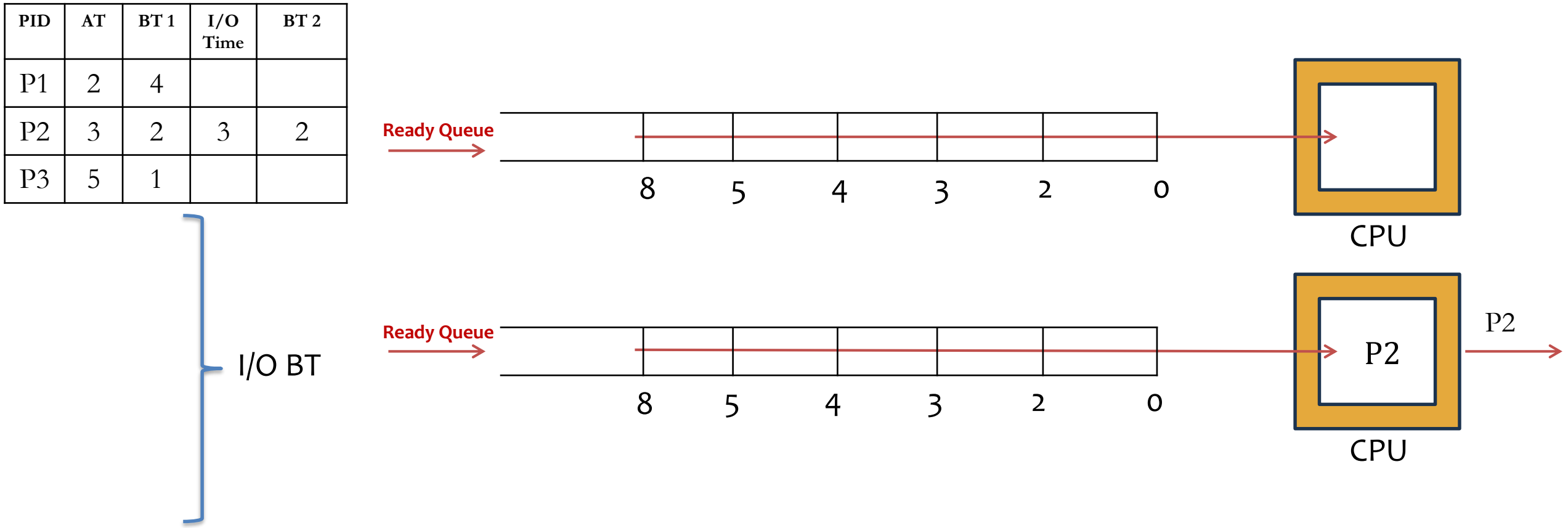
At time 9, P1 Terminates, Only P2 in ready Queue
P2 executes till time 11

Idle	P1	P2	P2	P3	P1	P1	P1	P2			

0 2 3 4 5 6 7 8 9 11

Example CPU Scheduling – with I/O Burst -SRTF

PID	AT	BT 1	I/O Time	BT 2
P1	2	4		
P2	3	2	3	2
P3	5	1		



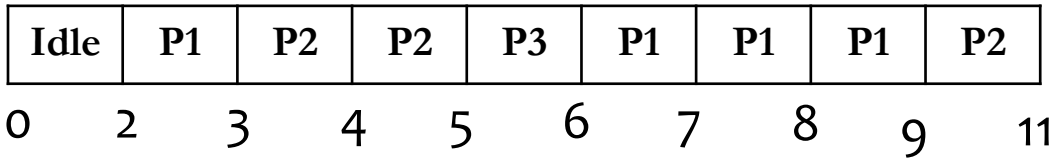
At time 11, P2 terminates, No process in Ready queue

Idle	P1	P2	P2	P3	P1	P1	P1	P2

0 2 3 4 5 6 7 8 9 11

Example CPU Scheduling – with I/O Burst -SRTF

Performance of the SRTF



Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU Start Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	2	4	2	9	7	3	0
P2	3	4	3	11	8	4	0
P3	5	1	5	6	1	0	0
SUM					**16**	**7**	**0**
AVG					**16/3 = 5.3 ms**	**7/3 = 2.3 ms**	**0**

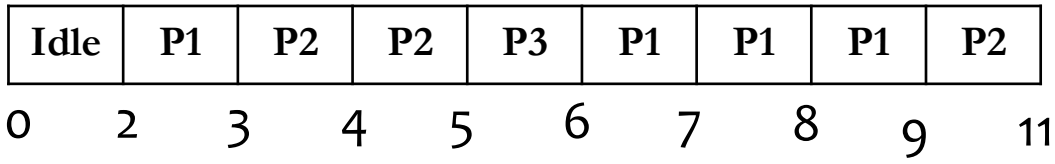
Example

CPU Scheduling – with I/O Burst -SRTF

Performance of the SRTF

Total CPU Burst Time

$$9ms$$



Total System Time

$$11 \text{ ms}$$

CPU utilization → 9/11 * 100 = 81.8%

Throughput= Number of Processes / Total System Time

$$= \frac{3}{11} = 0.3$$

Throughput =0.3 processes/ms

No. of Context switches in SRTF – 4

CPU Scheduling – with I/O Burst – Round Robin

Example

Process Specifications:

Time Quantum TQ = 2 units

P1: Arrives at t = 1. Continuous CPU burst of 3 units.

P2 : Arrives at t = 2. Phase 1 CPU burst of 2 units, followed by an I/O burst of 2 units, finishing with a Phase 2 CPU burst of 2 units.

P3: Arrives at t = 3. Continuous CPU burst of 1 unit.

P4: Arrives at t = 5. Continuous CPU burst of 2 units.

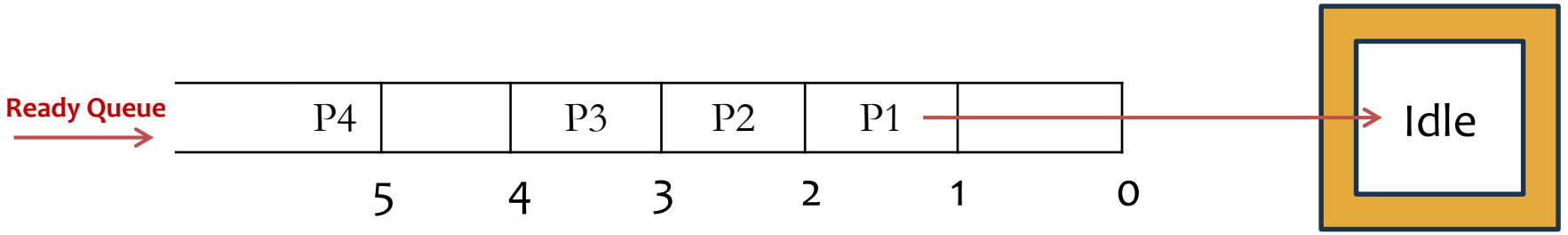
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		

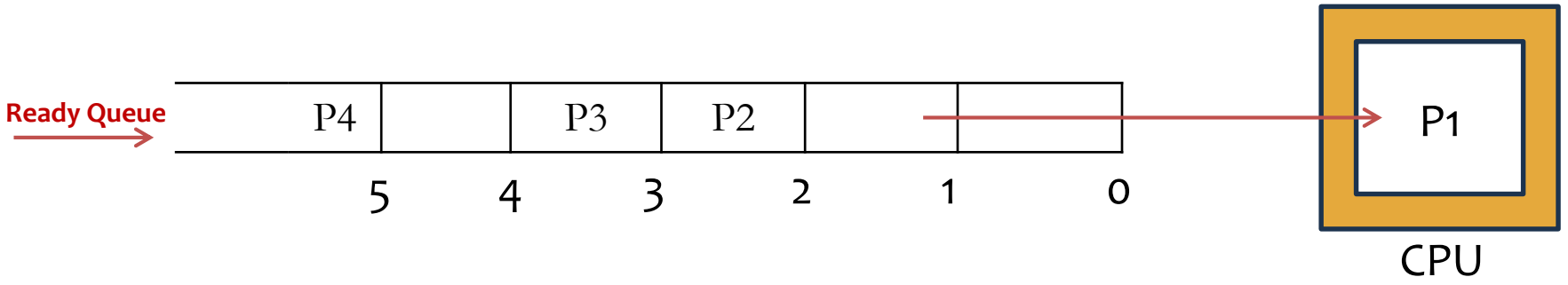
CPU Scheduling – with I/O Burst – Round Robin

Example

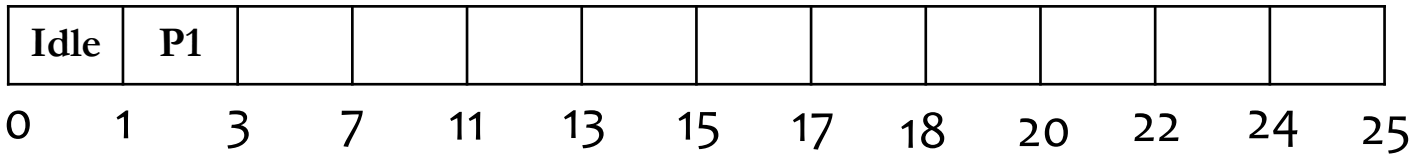
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		

I/O BT





At t=1 P1 gets CPU.
Executes till time Quantum 2

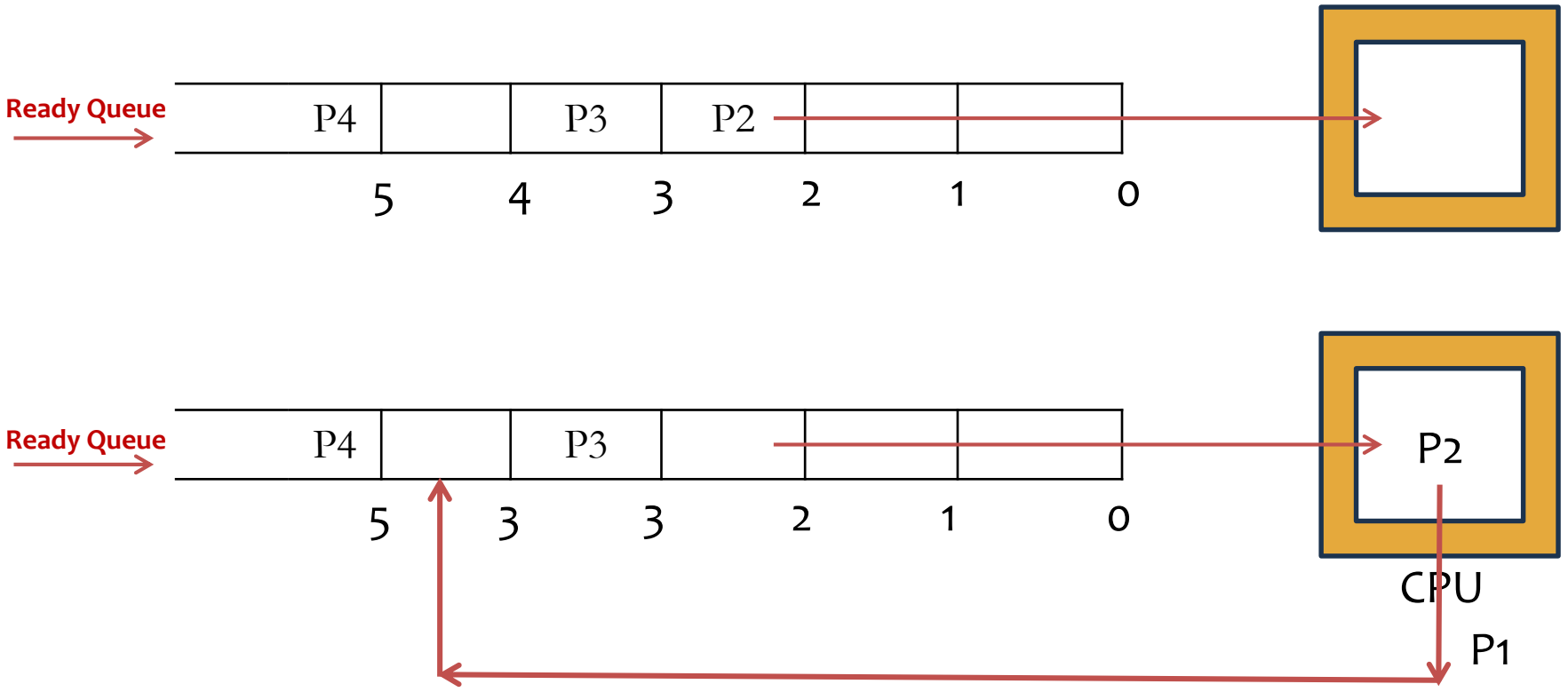


CPU Scheduling – with I/O Burst – Round Robin

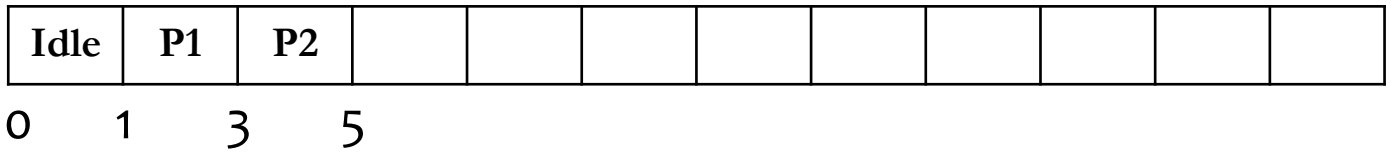
Example

PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		

I/O BT



At t = 3, P1 is suspended & moved to RQ
P2 gets the CPU and executes

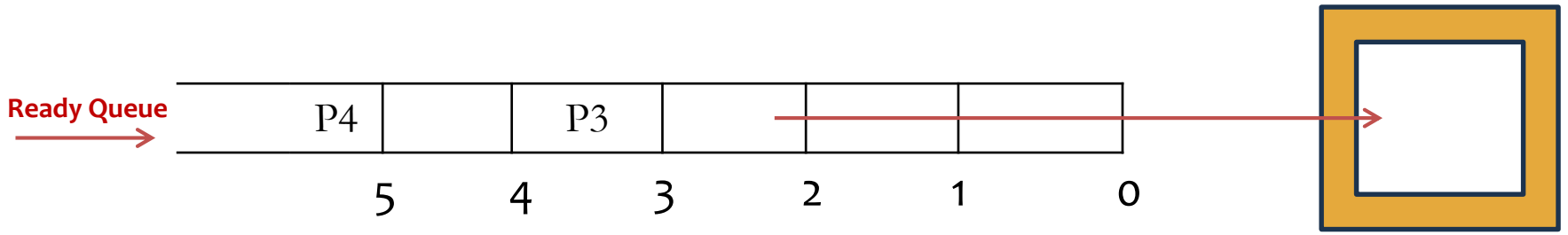


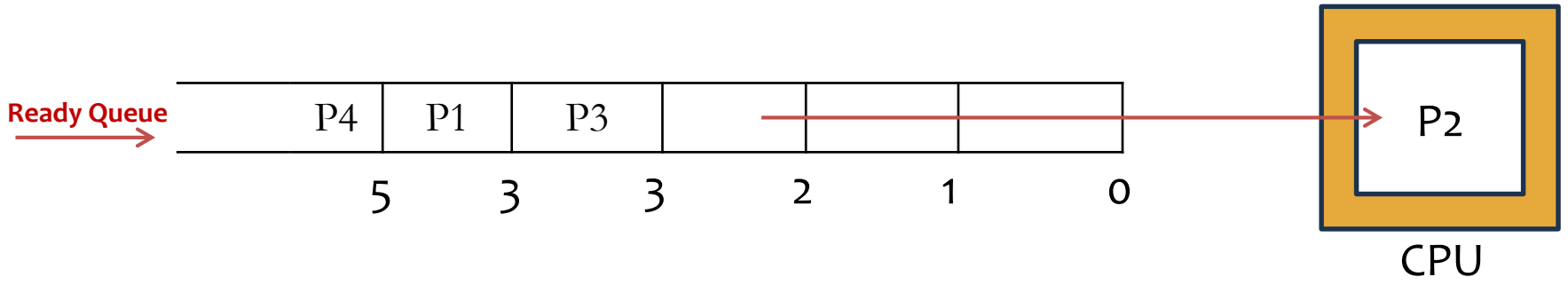
CPU Scheduling – with I/O Burst – Round Robin

Example

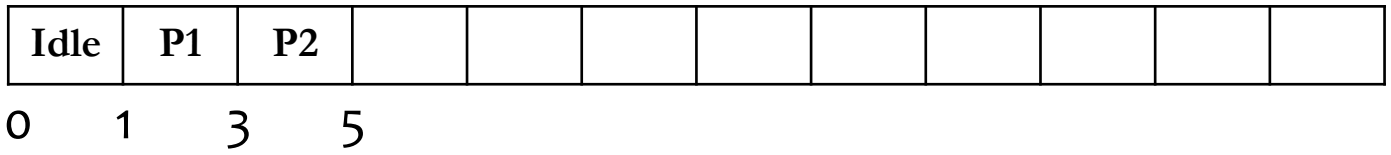
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		

I/O BT





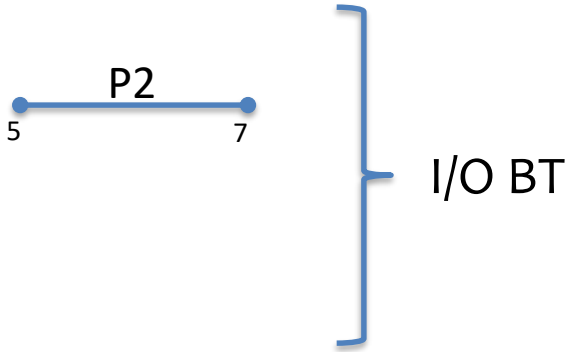
At t=4, TQ is pending so
P2 continues its execution till 5



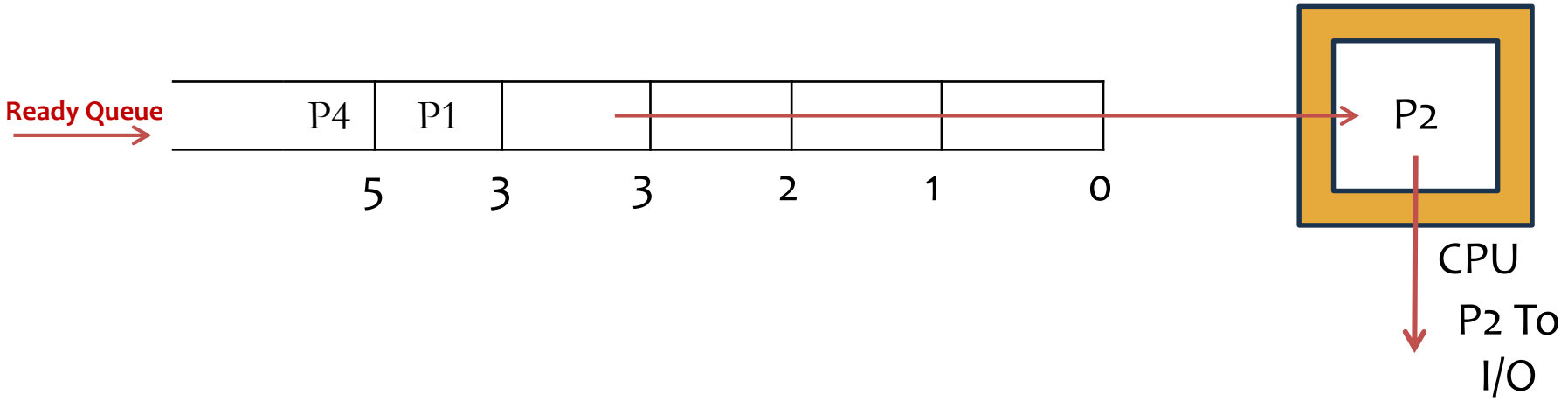
CPU Scheduling – with I/O Burst – Round Robin

Example

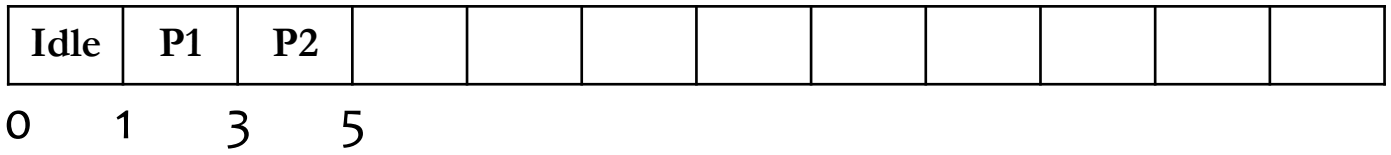
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		







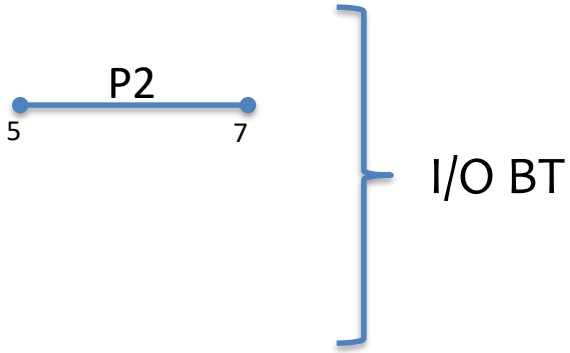
At t=4, TQ is pending so
P2 continues its execution till 5
P2 switches to I/O



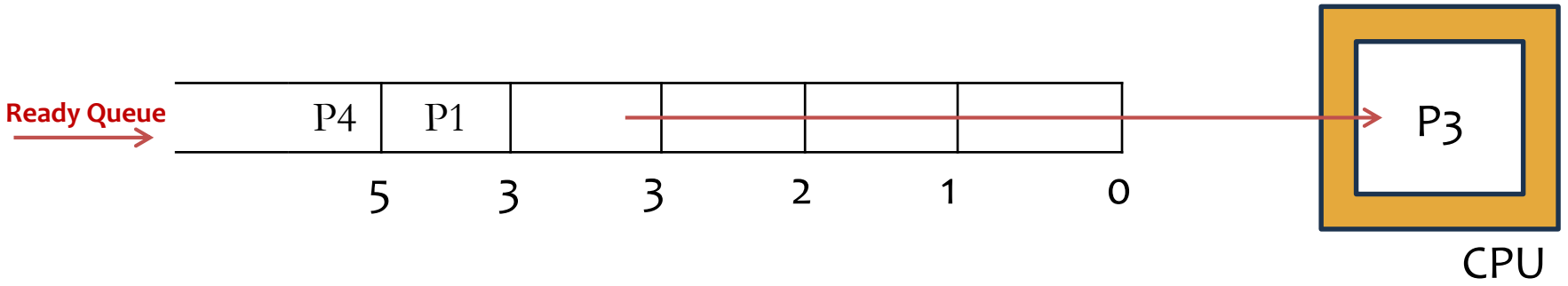
CPU Scheduling – with I/O Burst – Round Robin

Example

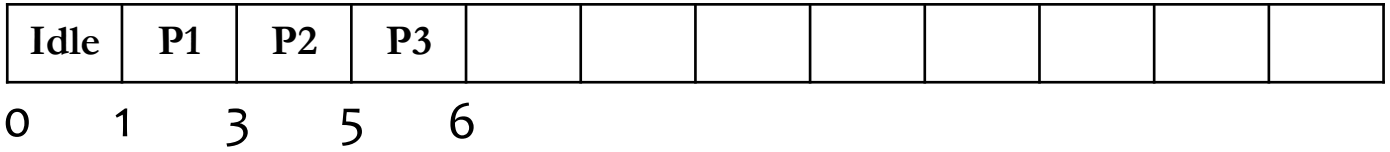
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		







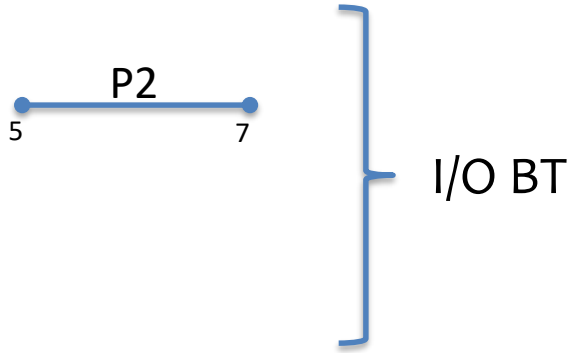
At t=5, P3 is in RQ. P3 runs till 6 and terminates

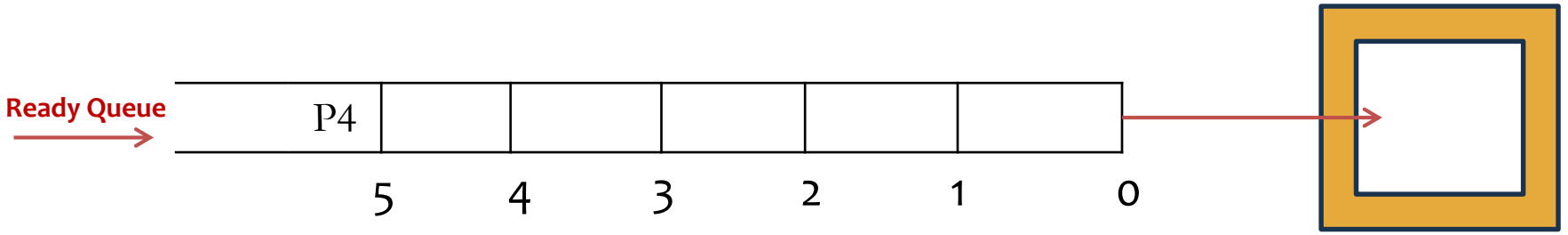


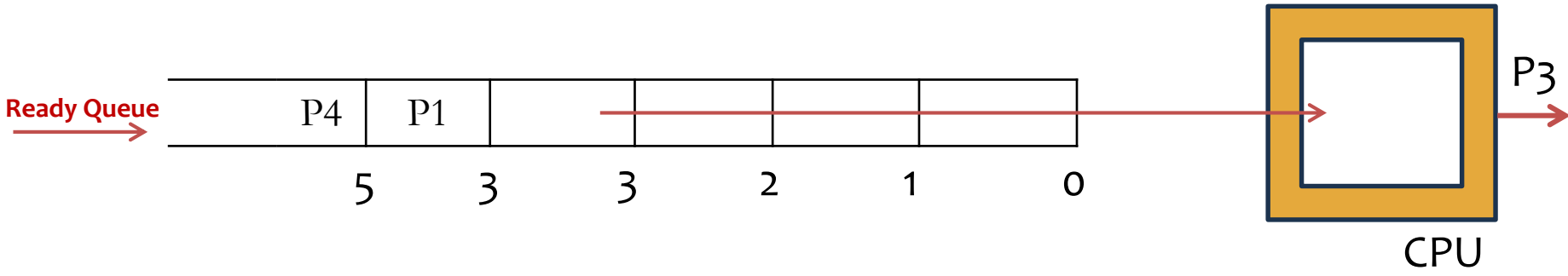
CPU Scheduling – with I/O Burst – Round Robin

Example

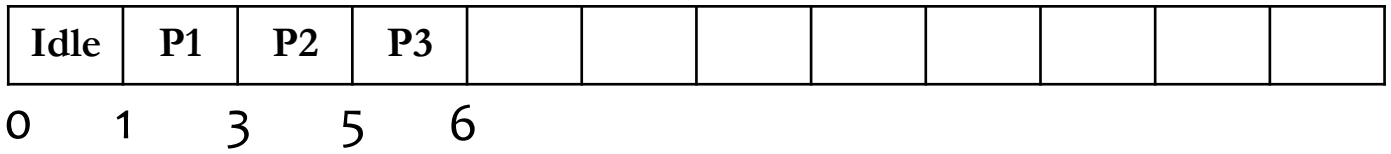
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		







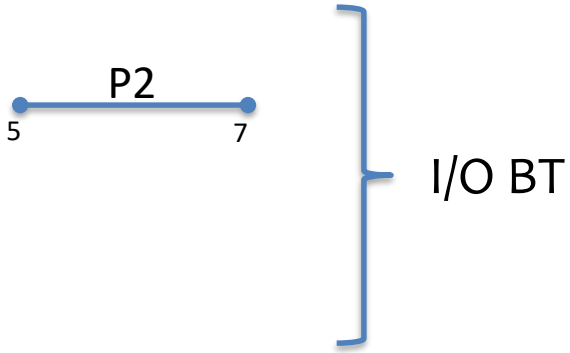
At t=5, P3 is in RQ. P3 runs till 6 and terminates



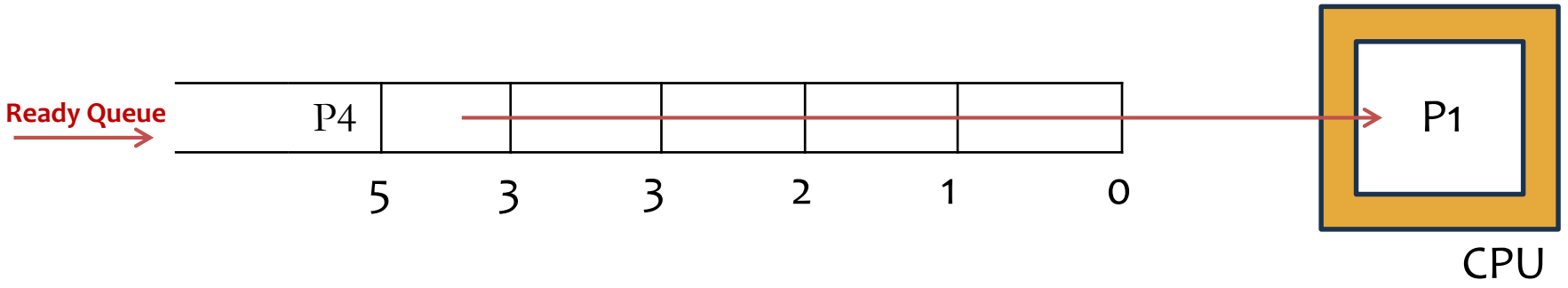
CPU Scheduling – with I/O Burst – Round Robin

Example

PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		







At t=6, P1 is in RQ. P1 runs till 7 and terminates

Idle	P1	P2	P3	P1							

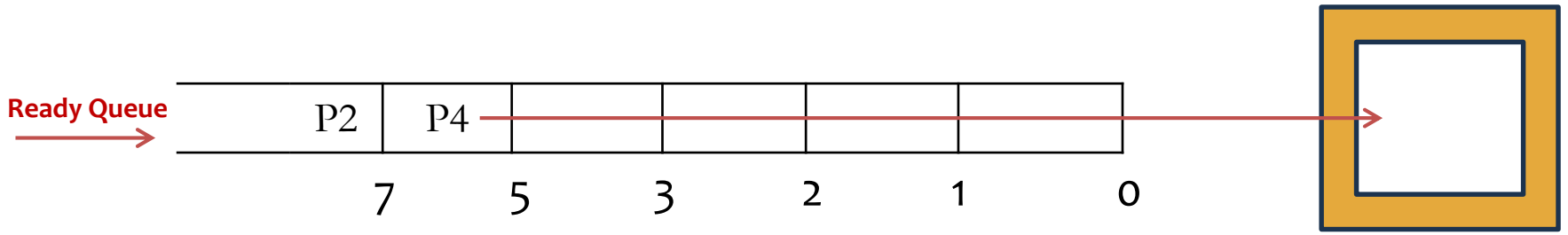
0 1 3 5 6 7

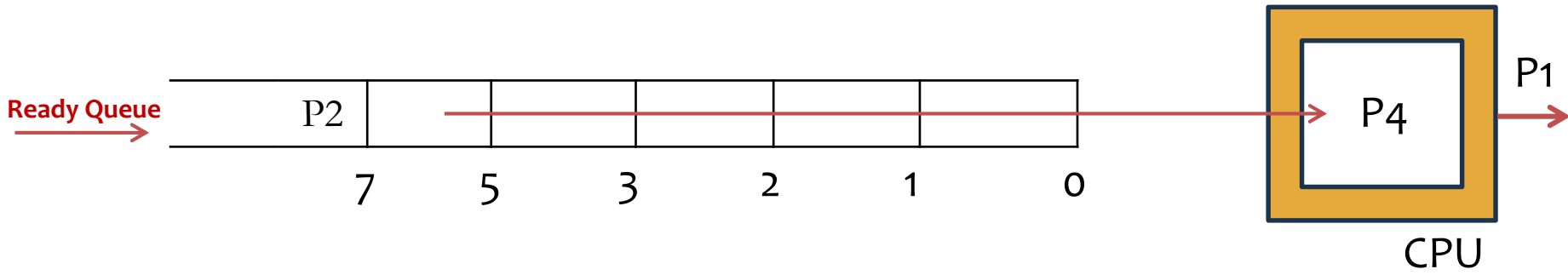
CPU Scheduling – with I/O Burst – Round Robin

Example

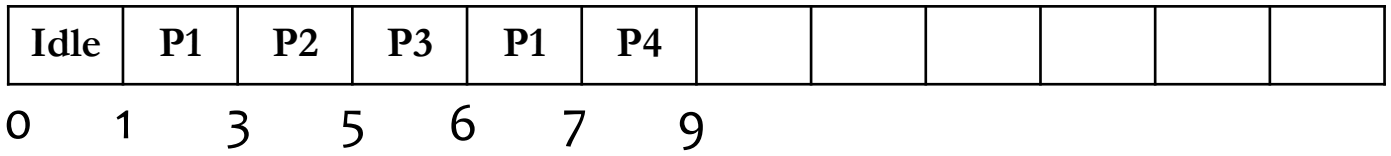
PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		

I/O BT





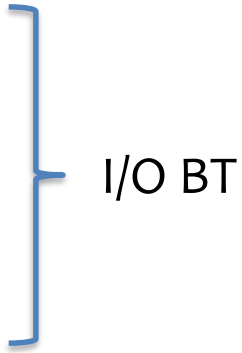
At t=7, P2 enters RQ. P4 is picked from RQ Executed for its BT.

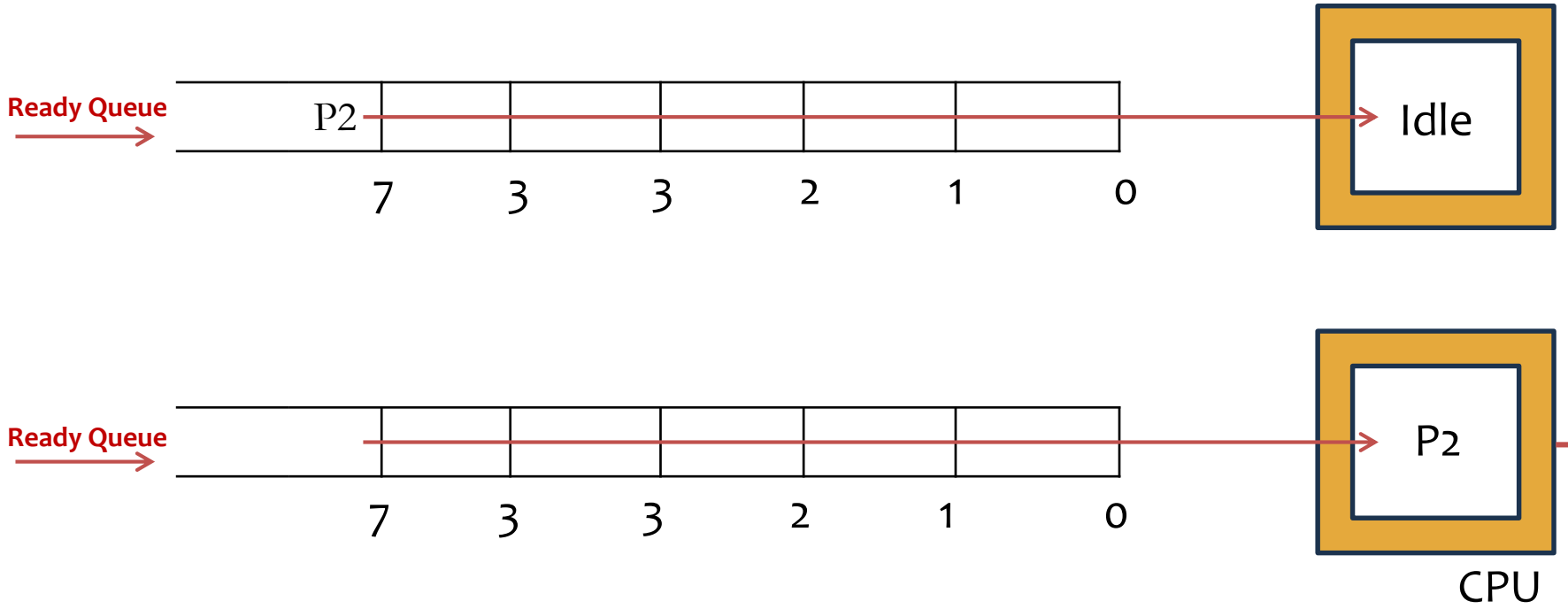


CPU Scheduling – with I/O Burst – Round Robin

Example

PID	AT	BT 1	I/O Time	BT 2
P1	1	3		
P2	2	2	2	2
P3	3	1		
P4	5	2		





At t=9, P4 terminates
P2 gets CPU and runs till time 11.

Idle	P1	P2	P3	P1	P4	P2

0 1 3 5 6 7 9 11

CPU Scheduling – with I/O Burst – Round Robin

Example

Performance of the RR

Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU Start Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	1	3	1	7	6	3	0
P2	2	4	3	11	9	5	1
P3	3	1	5	6	3	2	2
P4	5	2	7	9	4	2	2
SUM					**22**	**12**	**5**
AVG					**22/4 = 5.5 ms**	**12/4 = 3 ms**	**5/4 = 1.25 ms**

CPU Scheduling – with I/O Burst – Round Robin

Example

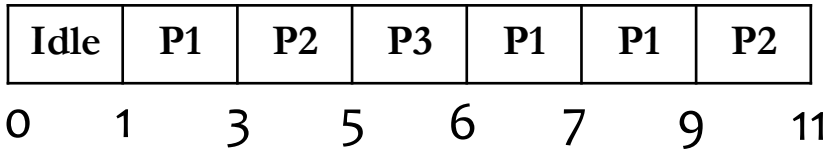
Performance of the RR

Total CPU Burst Time

$10ms$

Total System Time

11 ms



CPU utilization → 10/11 * 100 = 90.9%

Throughput= Number of Processes / Total System Time $= \frac{4}{11} = 0.36$

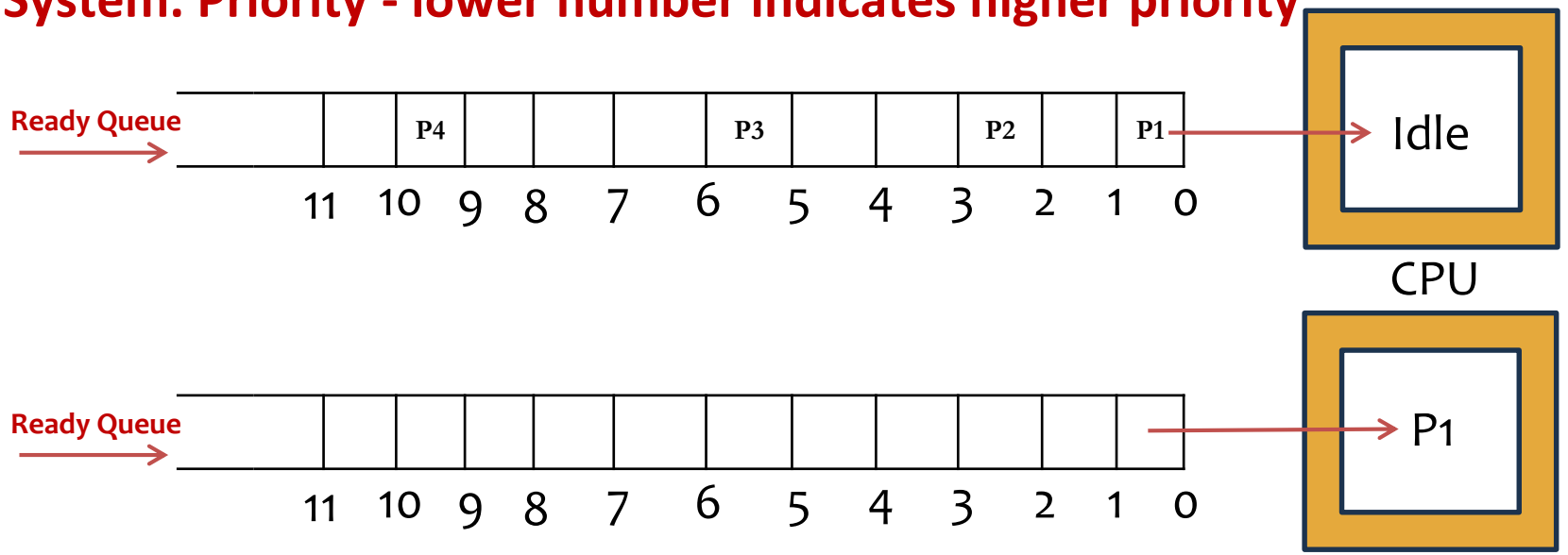
Throughput =0.4 processes/ms

No. of Context switches in RR – 5

Example CPU Scheduling – with I/O Burst -Priority

Consider the Snapshot of the System. Priority - lower number indicates higher priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

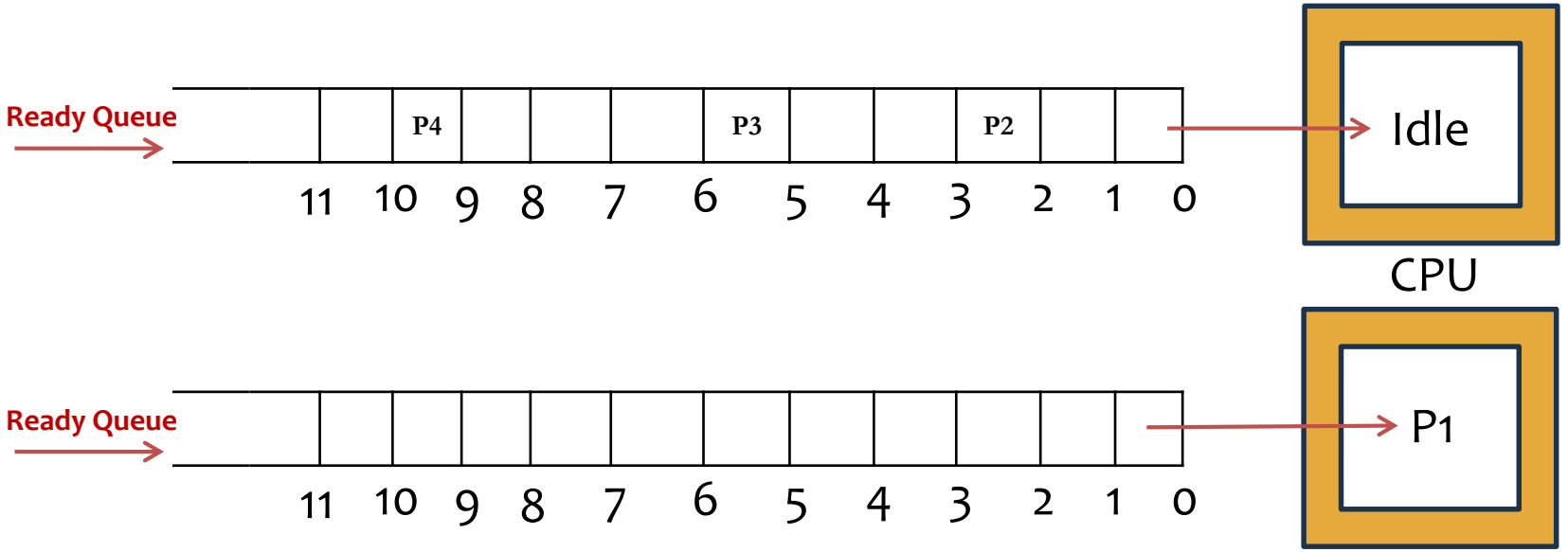


At T=0, only p1 arrives in RQ,
Allocate P1 to CPU,
P1 runs from 0 to 1



Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

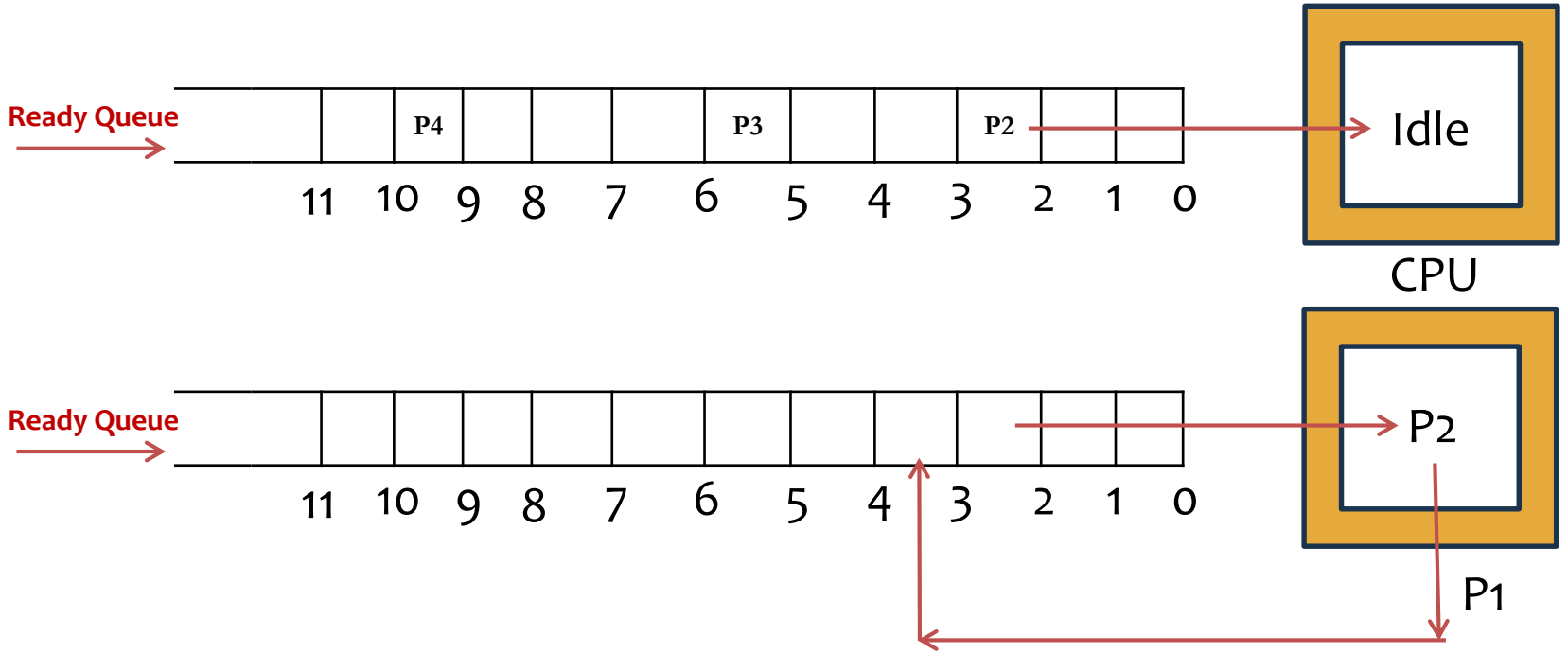


At T=1, P1 runs from 1 to 2



Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

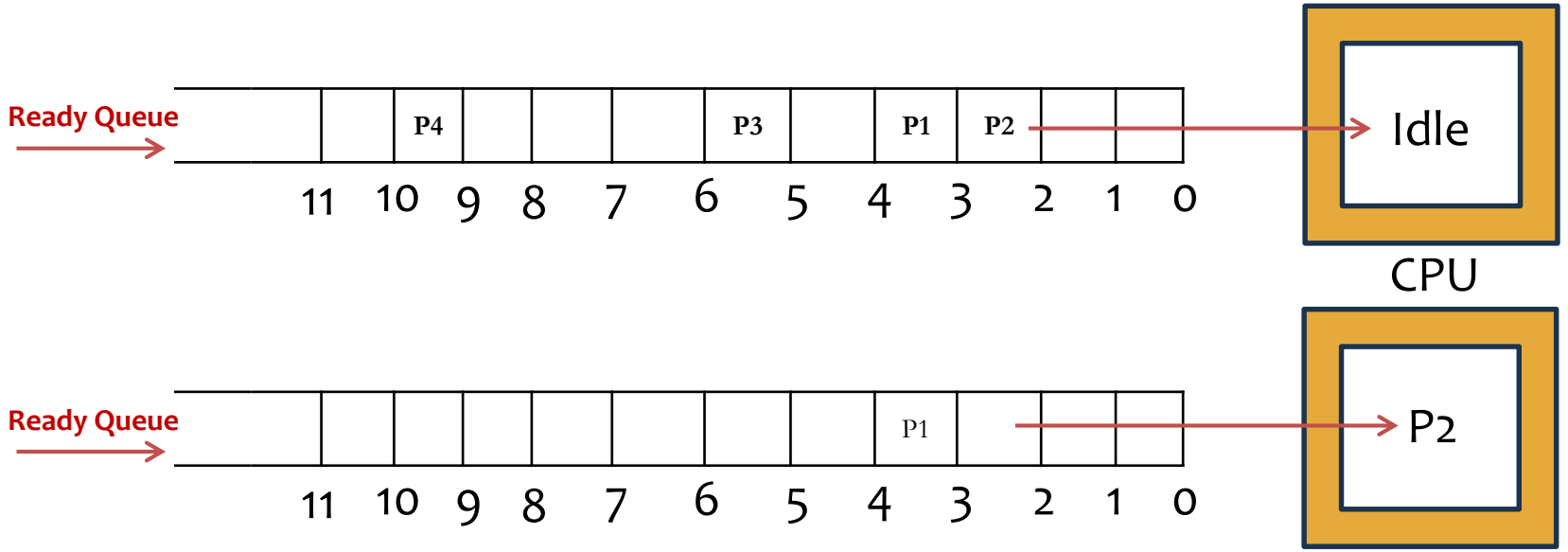


At T=2, P2 arrives. Compare Priority
– P1(P) < P2(P)
P1 is preempted, P2 gets CPU



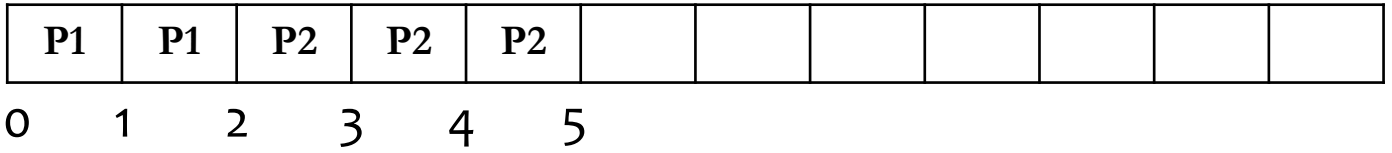
Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4



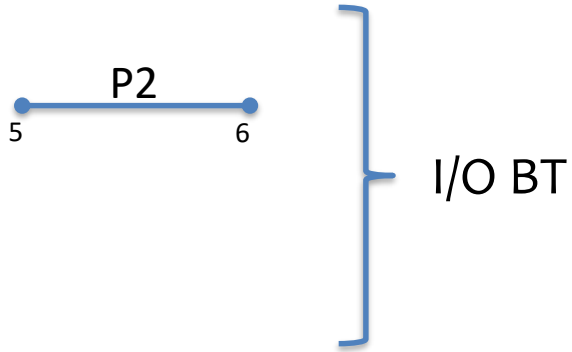
At T=3, P2 arrives. Only P1 Process is in RQ

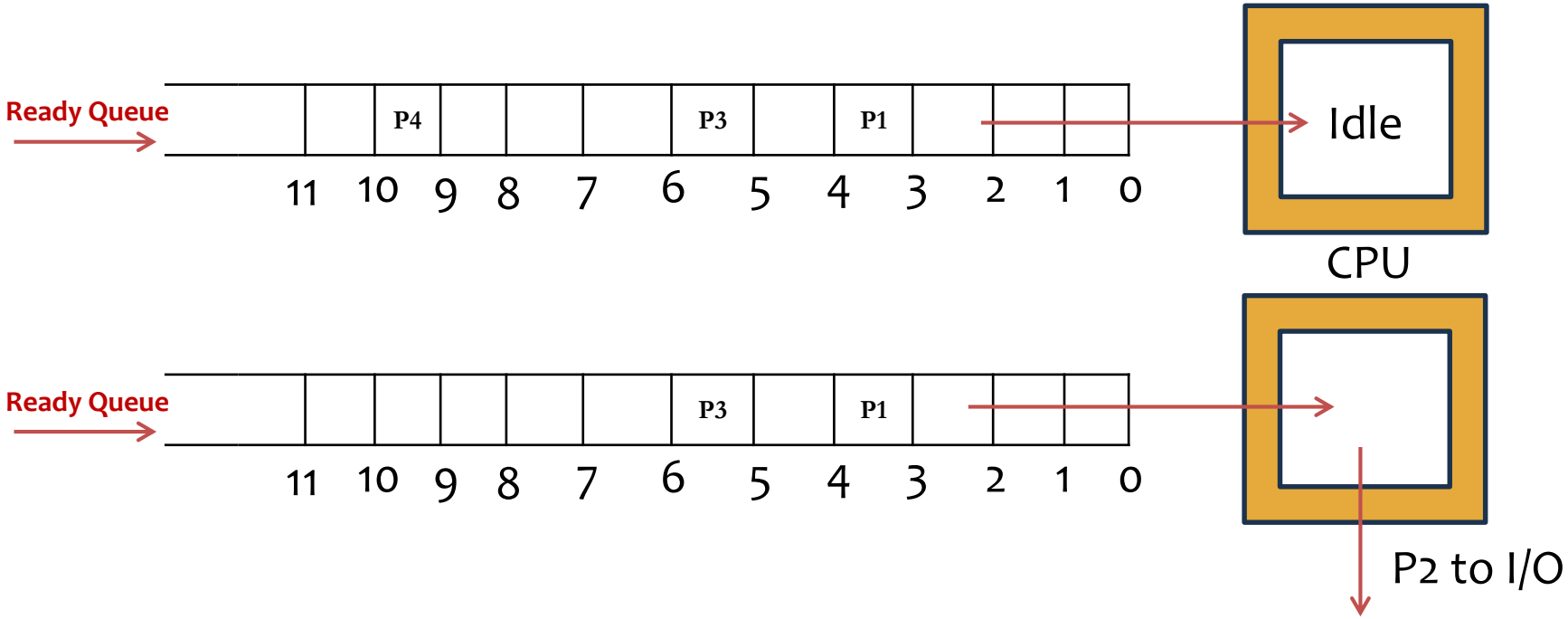
P2 Continues Execution till 5



Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4



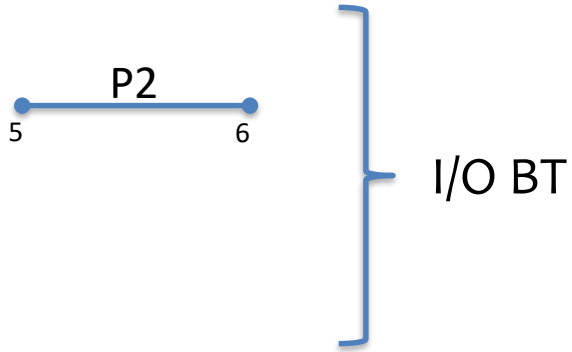


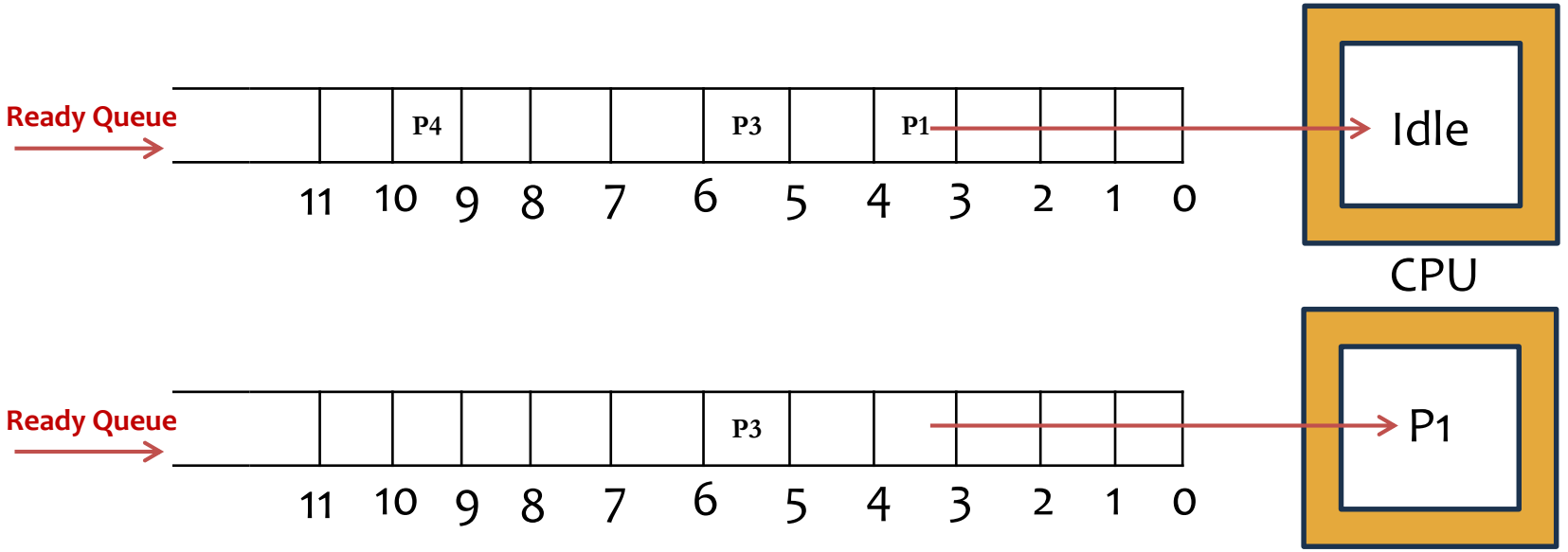
At T=5, P3 arrives.
P2 is suspended to I/O



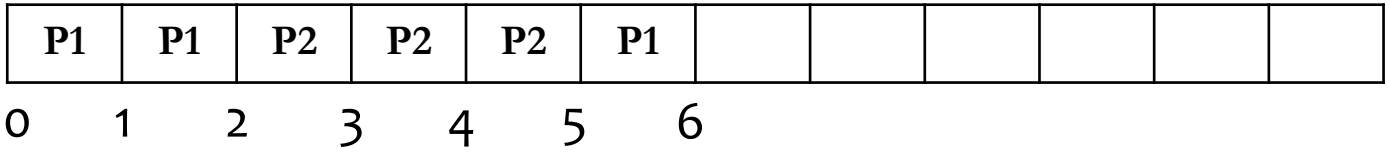
Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4



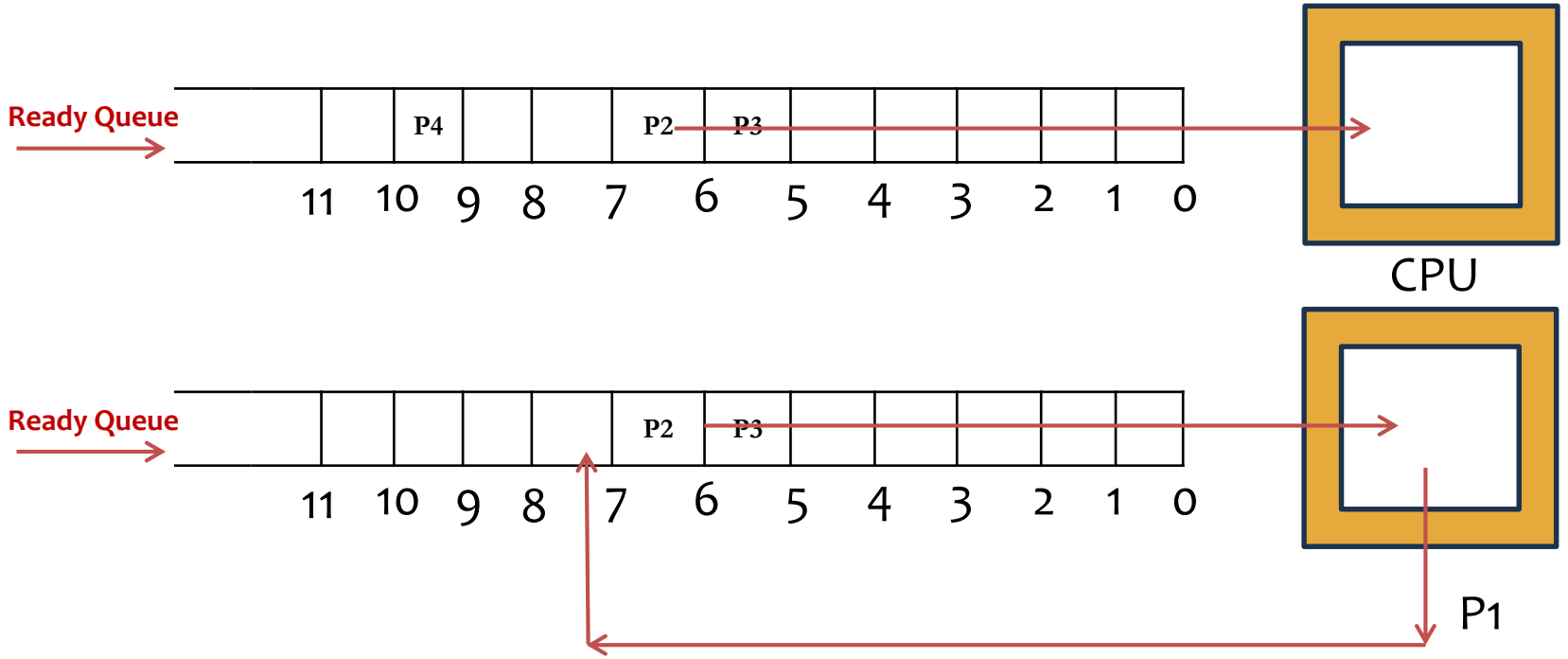


At T=5, P3 arrives.
Compare Priority value of P3 & P1
P3(P) < P1(P)
P1 gets CPU and executes till 6

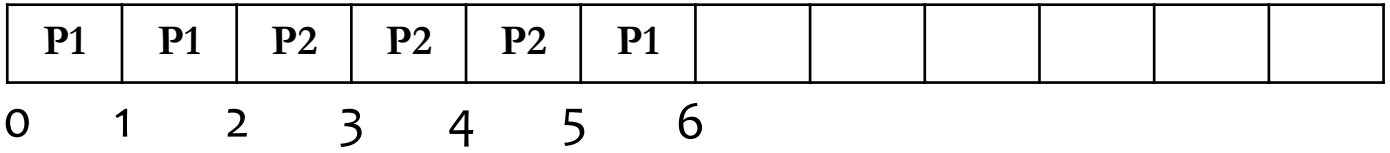


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

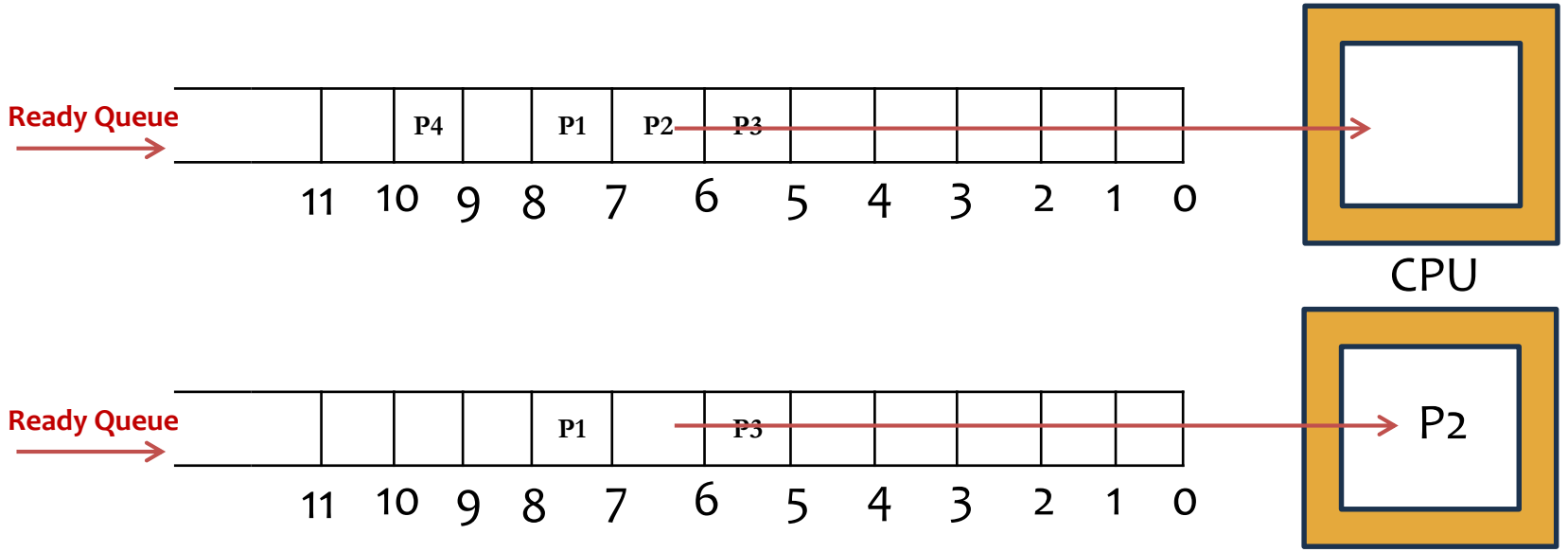


At T=6, P2 arrives in RQ.
Compare Priority value of P3, P2 & P1
P2(P) < P1(P) < P3(P)
P1 is added to RQ

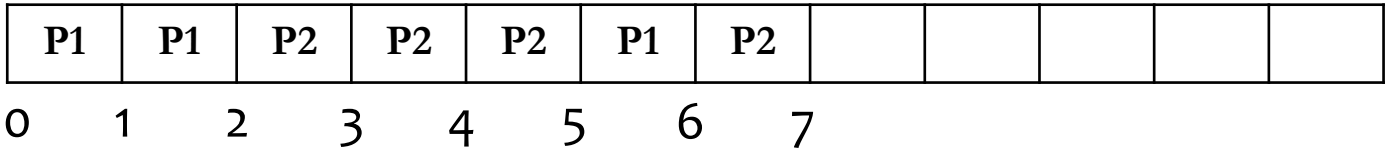


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

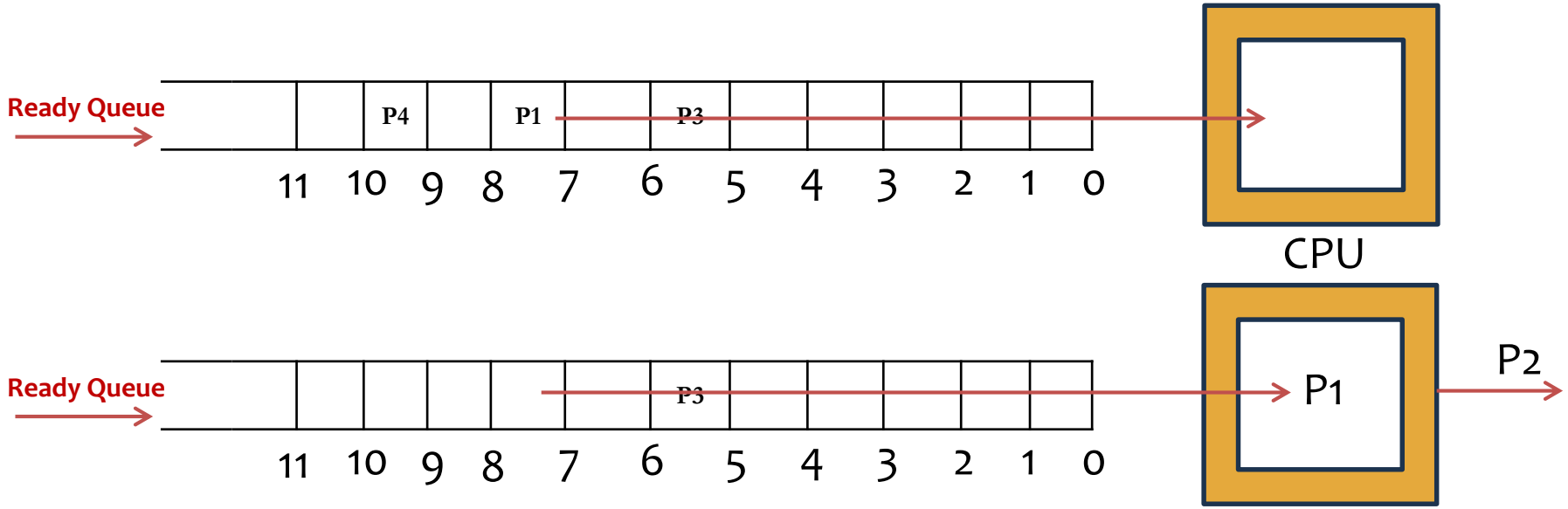


At T=6, P2 arrives in RQ.
Compare Priority value of P3, P2 & P1
P2(P) < P1(P) < P3(P)
P1 is added to RQ
P2 gets CPU and executes till 7

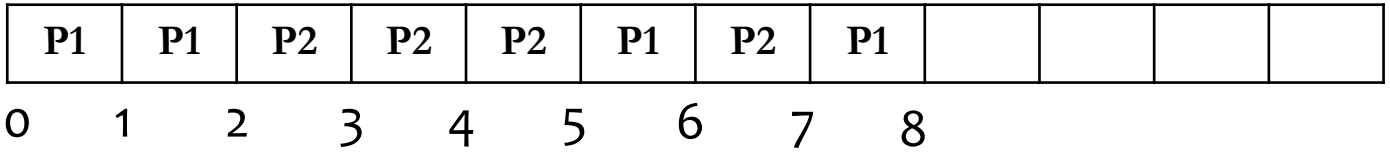


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

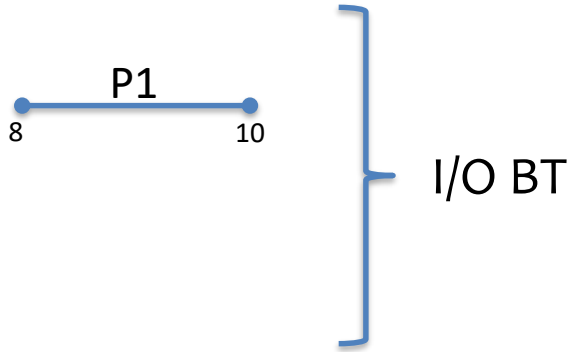


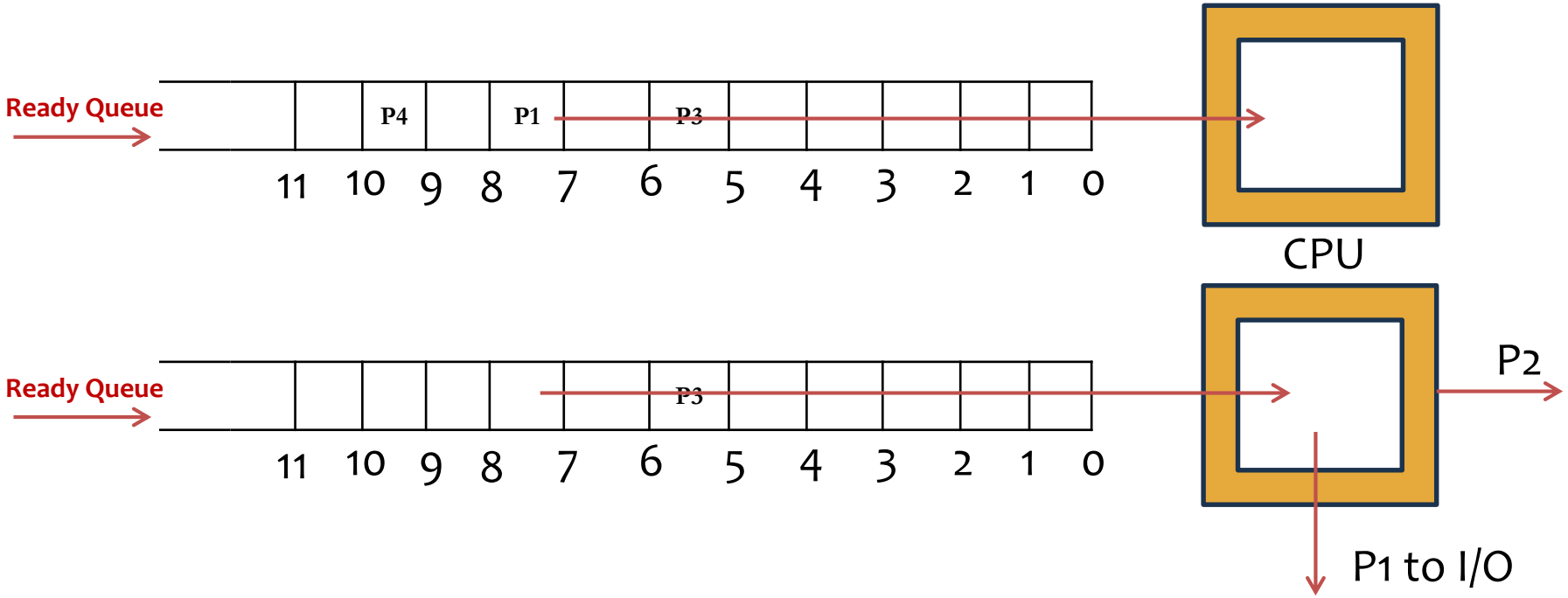
At T=7, P2 Terminates
Compare Priority value of P3, P1
P1(P) < P3(P)
P1 gets CPU and executes till 8



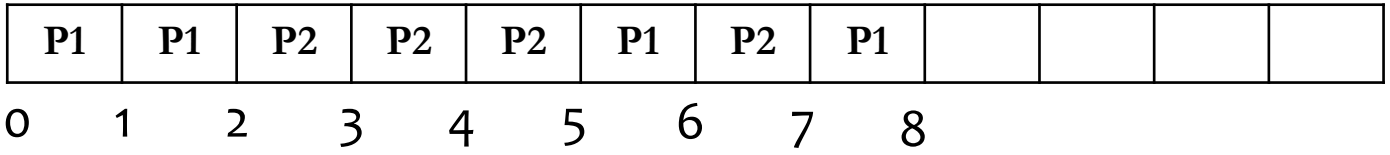
Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4



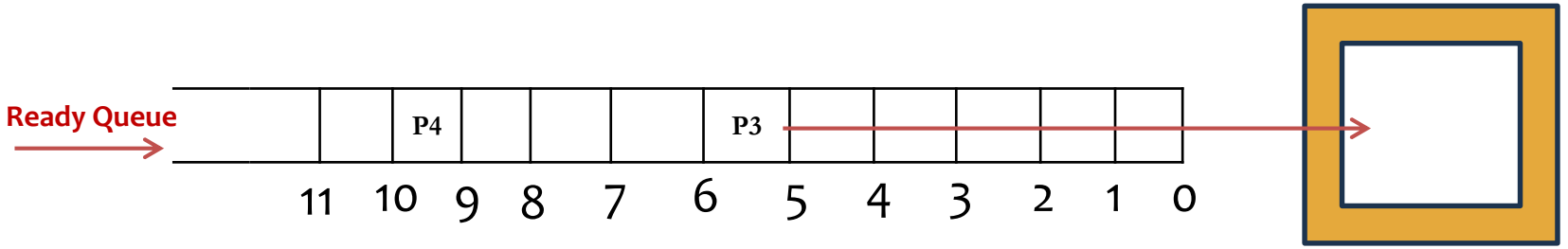


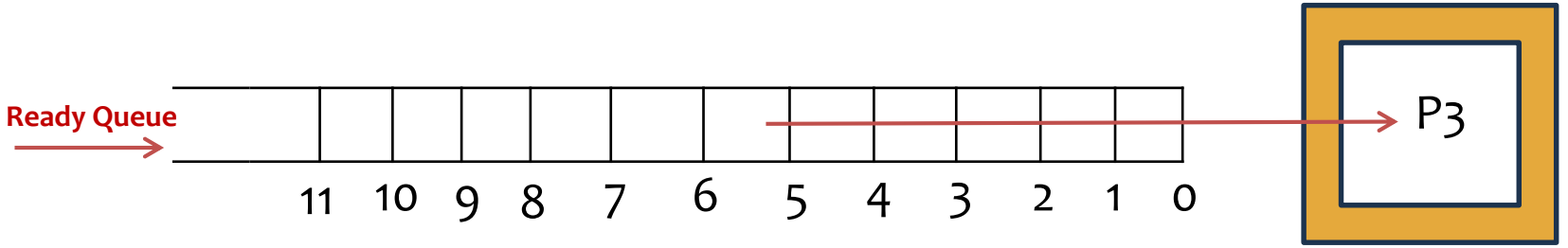
At T=8, P1 Suspends for I/O
Only P3 in RQ,

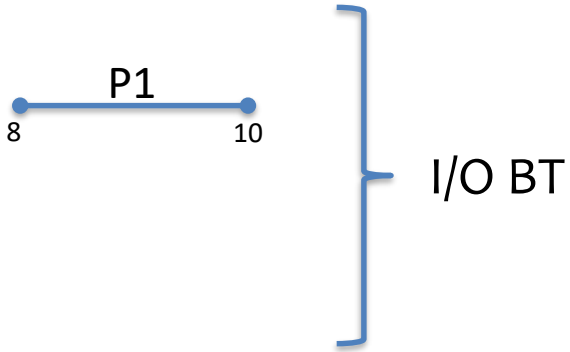


Example CPU Scheduling – with I/O Burst -Priority

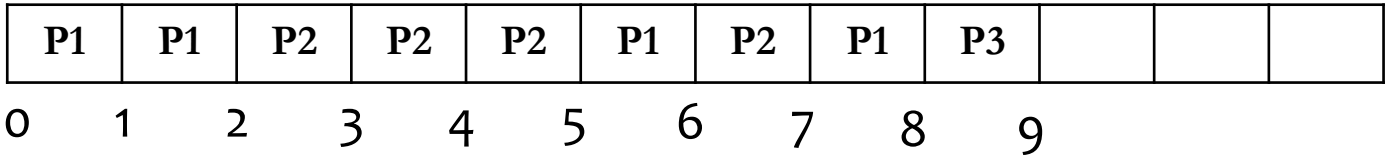
PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4





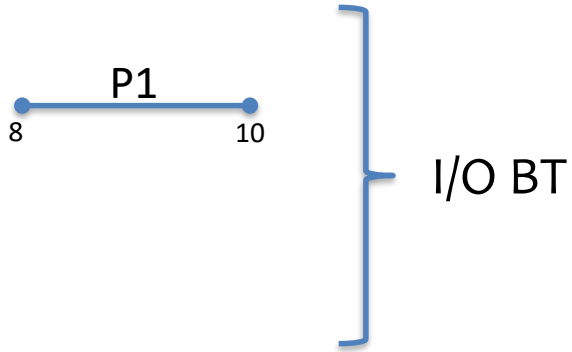


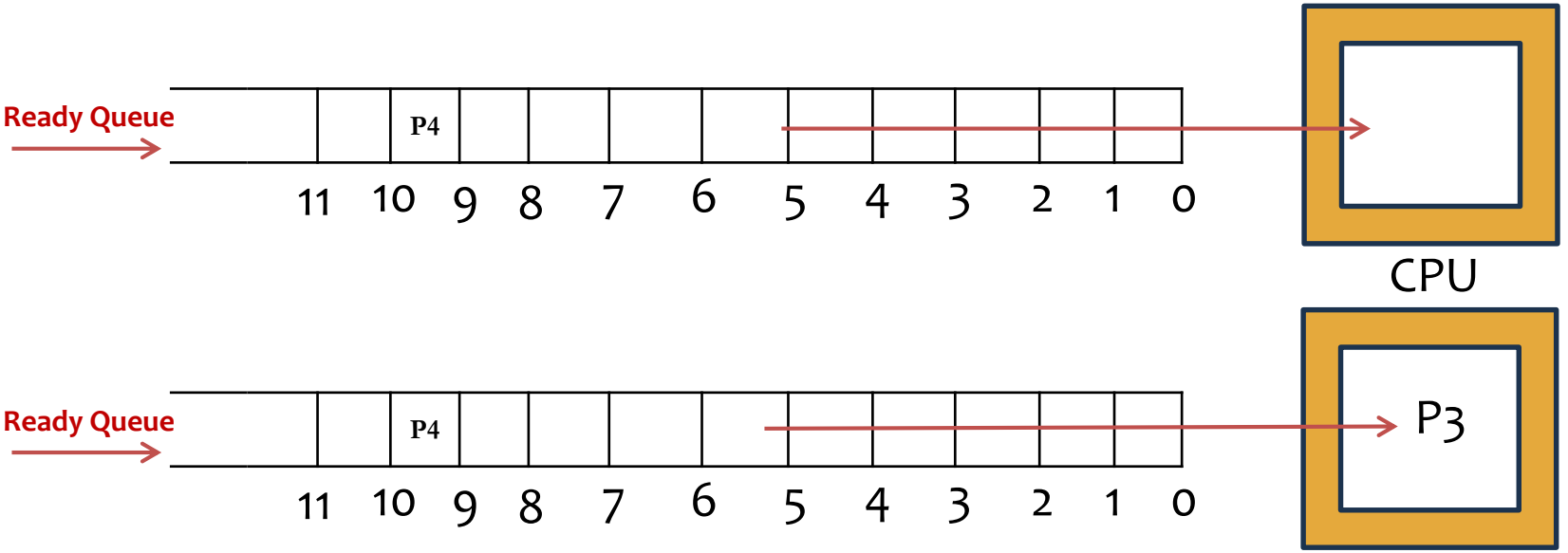
At T=8, P1 Suspends for I/O
Only P3 in RQ,
P3 gets CPU and Runs till 9



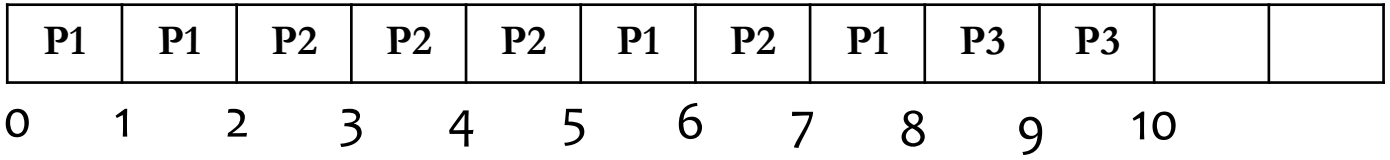
Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4



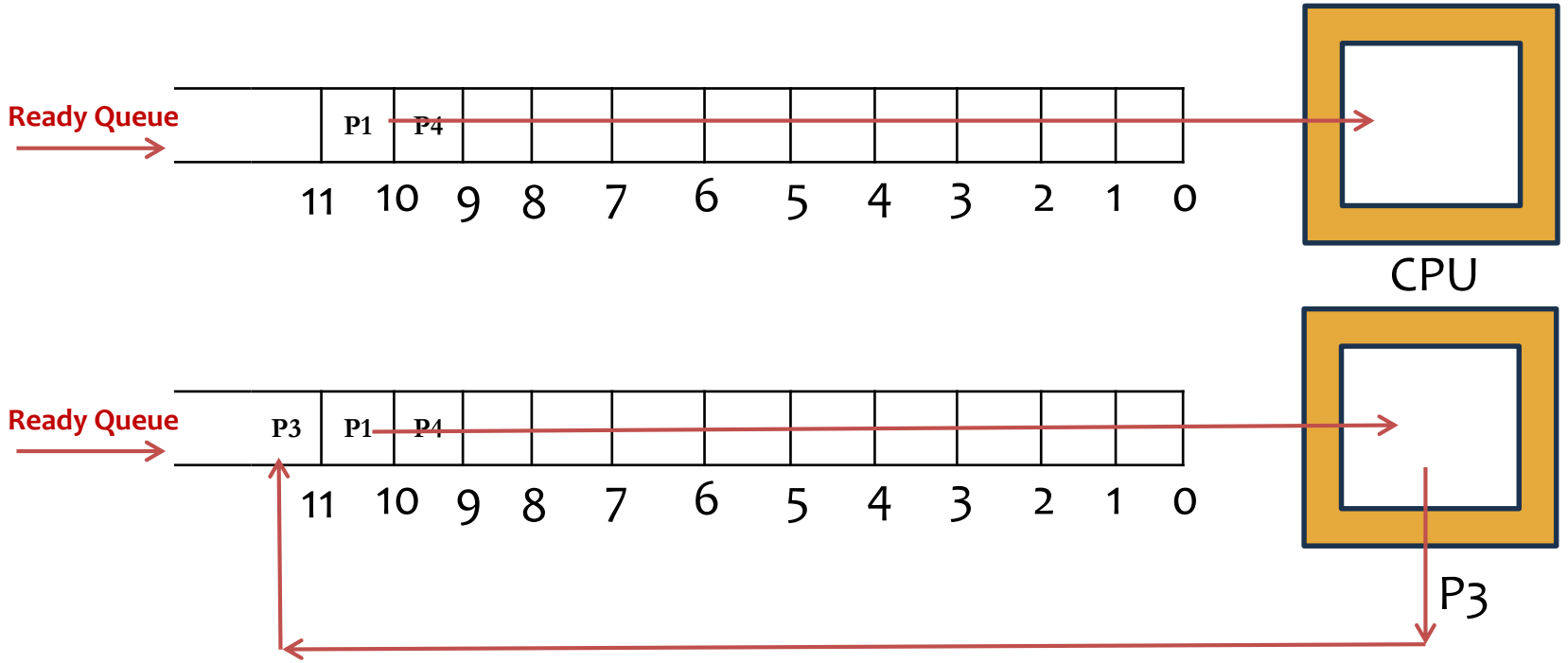


At T=9, P4 arrives
Compare Priority of P4 & P3
P4(P) < P3(P)
P3 Continues running till 10

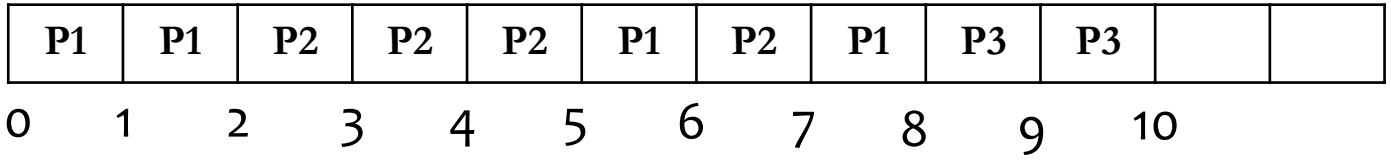


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

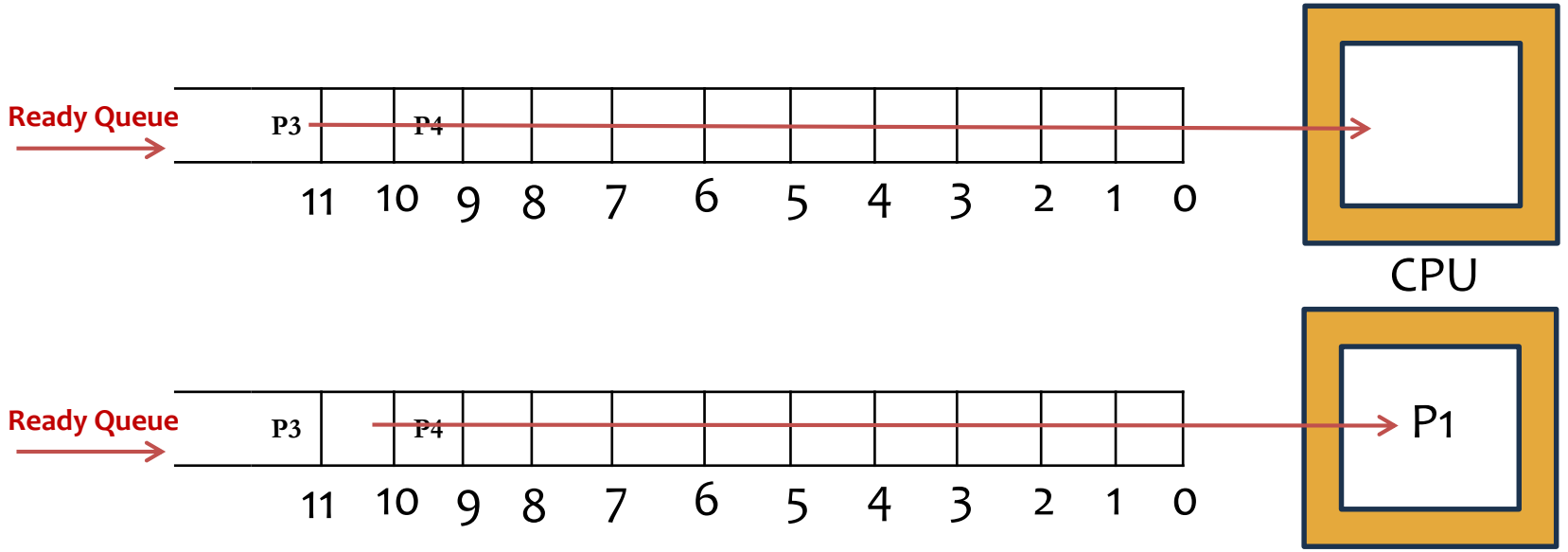


At T=10, P1 arrives to Ready Queue
Compare Priority of P1, P4 & P3
P4(P) < P3(P) < P1(P)
P3 is moved to Ready queue
P1 gets the CPU

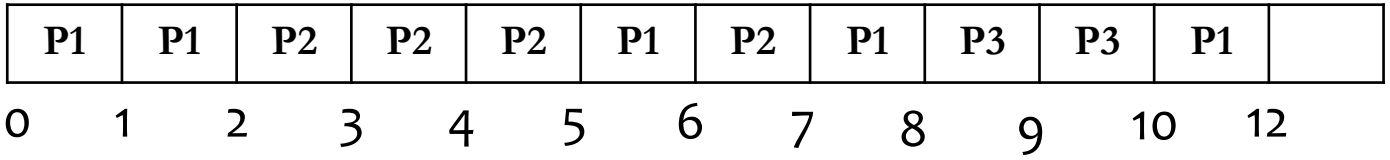


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

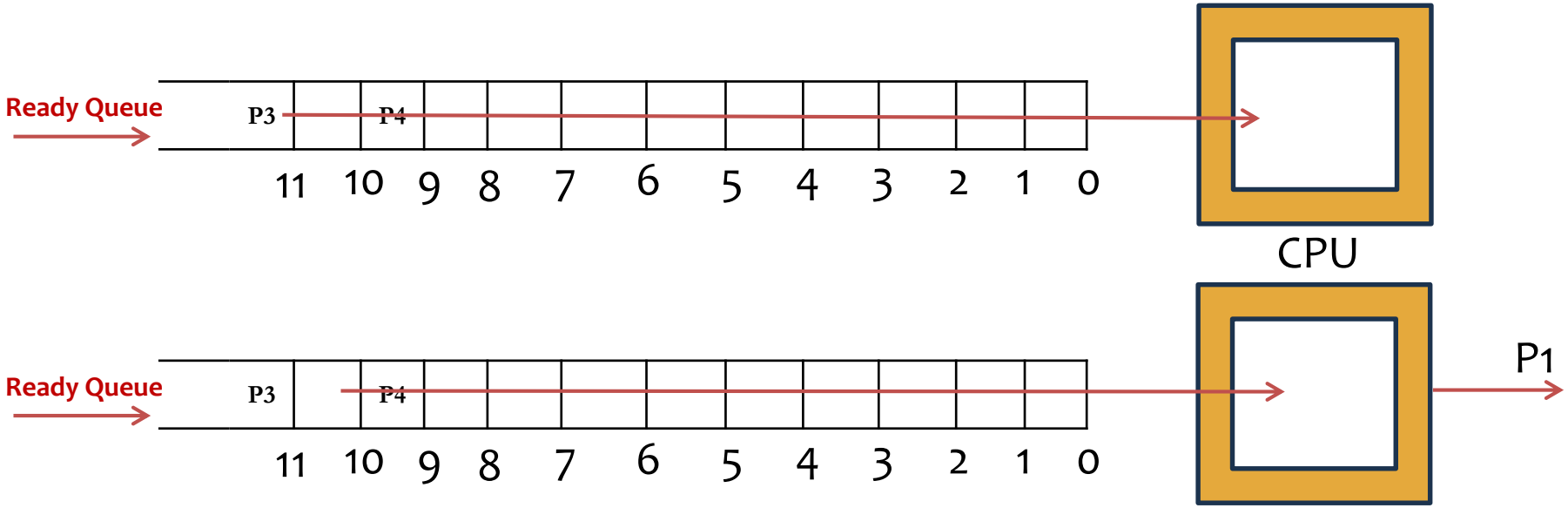


At T=10, P1 arrives to Ready Queue
Compare Priority of P1, P4 & P3
P4(P) < P3(P) < P1(P)
P3 is moved to Ready queue
P1 gets the CPU till T=12

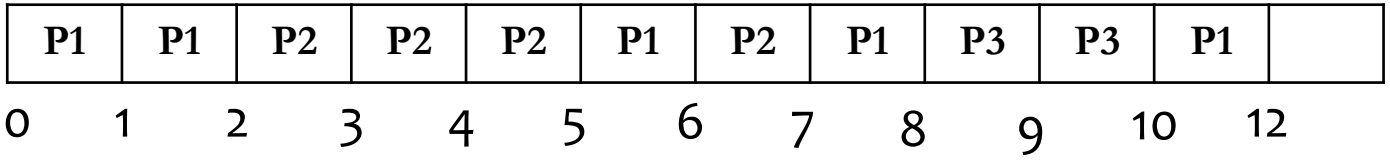


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

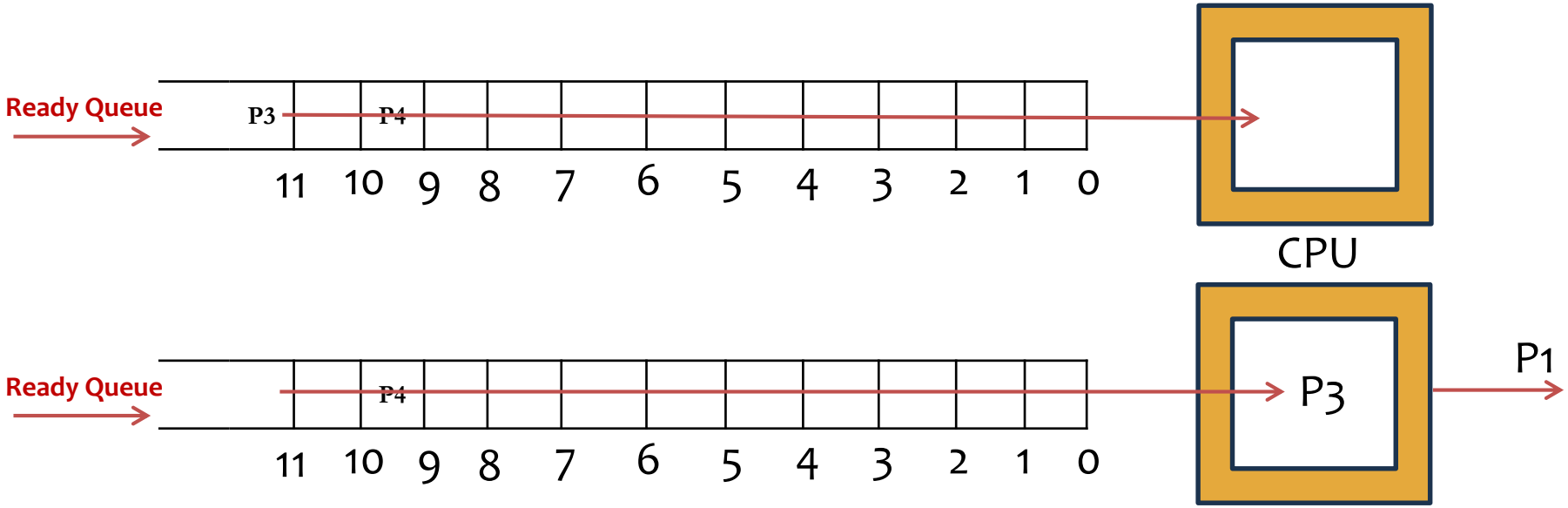


At T=12, P1 Terminates
Compare Priority of P4 & P3
P4(P) < P3(P)
P3 gets CPU, P3 runs the remaining cycle

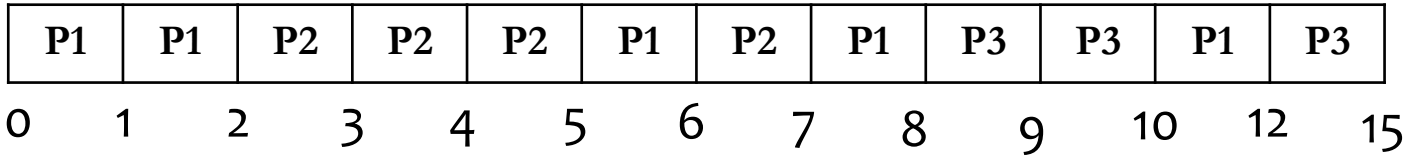


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

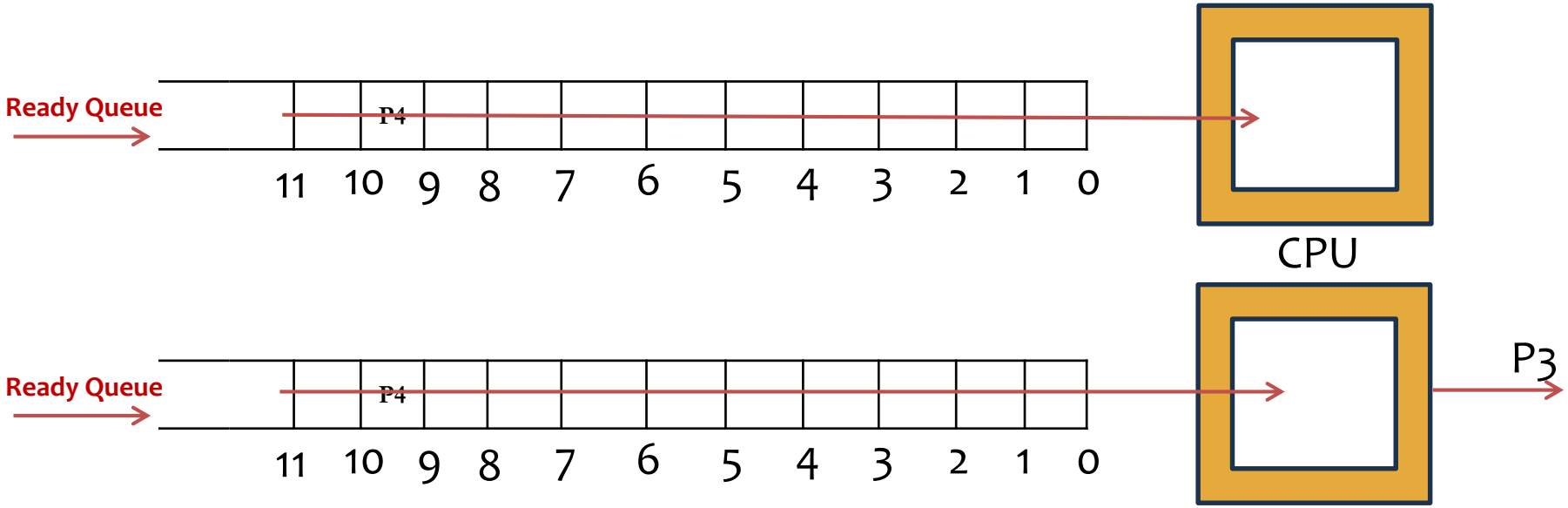


At T=12, P1 Terminates
Compare Priority of P4 & P3
P4(P) < P3(P)
P3 gets CPU, P3 runs the remaining cycle

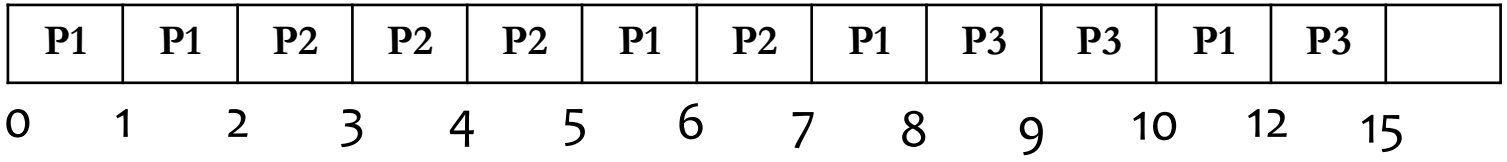


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

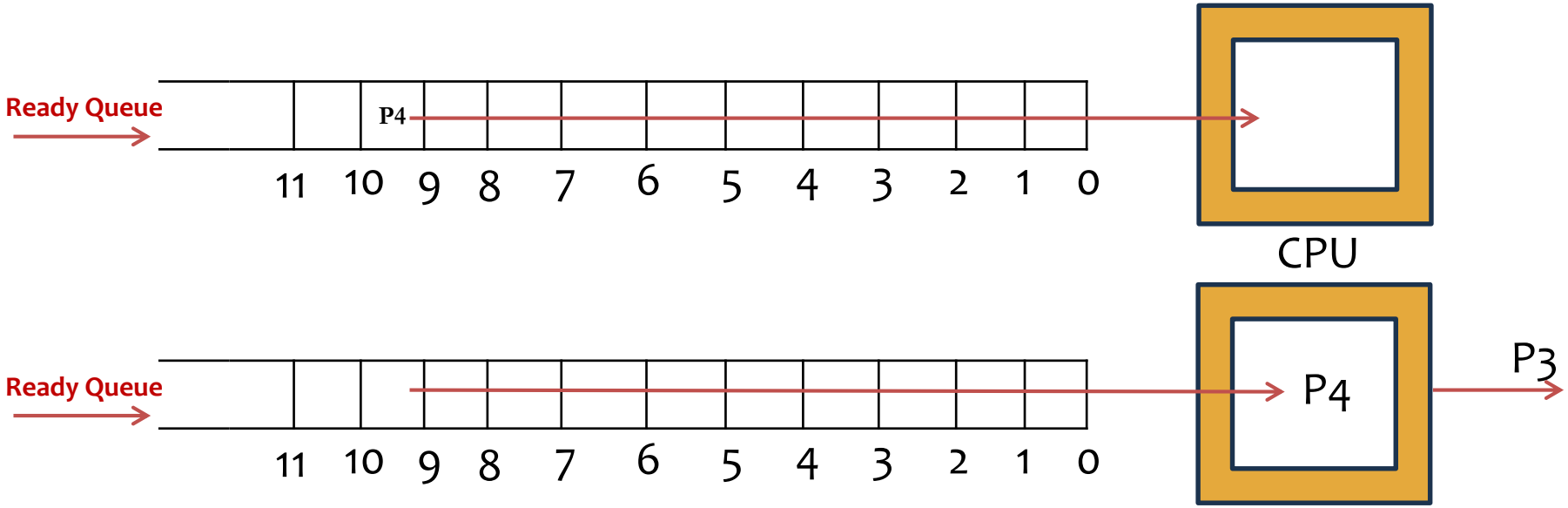


At T=15, P3 Terminates
P4 is the only process in RQ
P4 gets CPU, P4 runs the remaining cycle

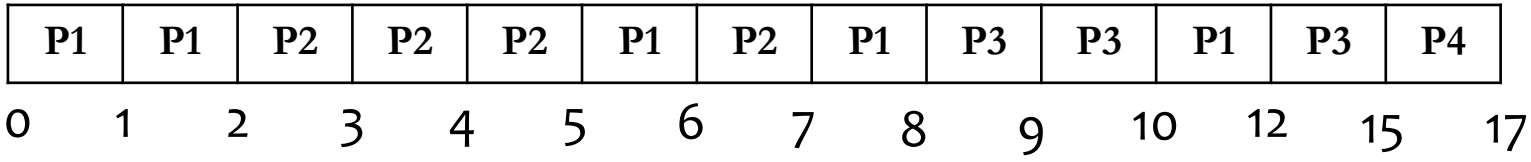


Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4

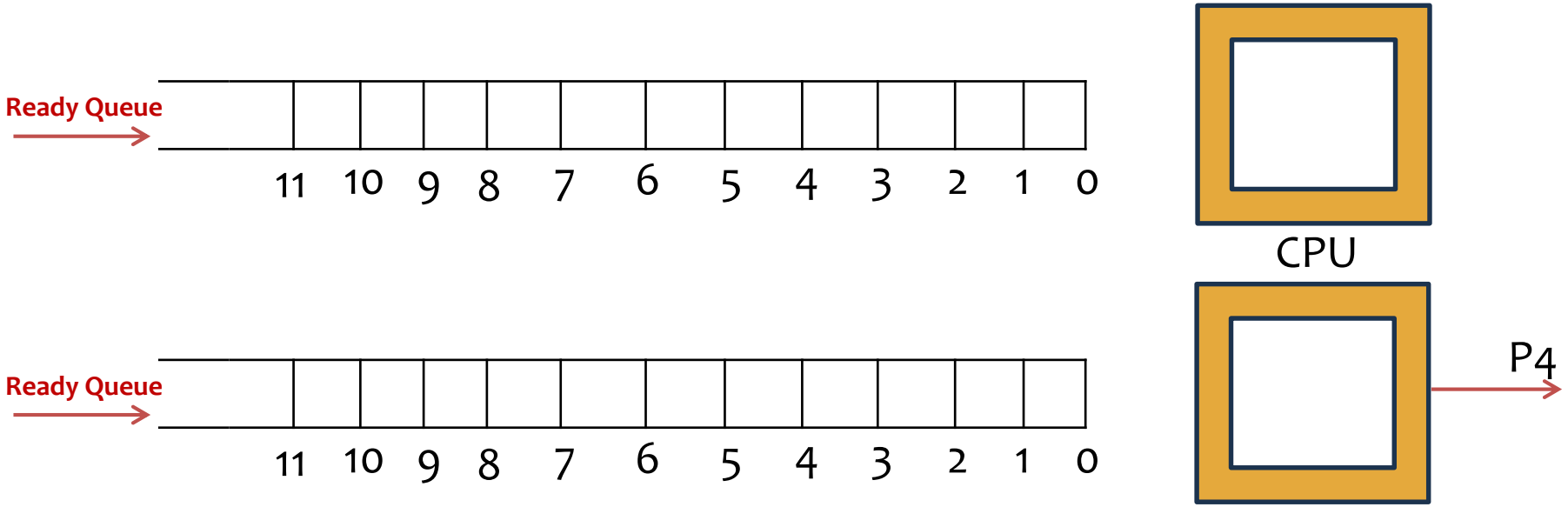


At T=15, P3 Terminates
P4 is the only process in RQ
P4 gets CPU, P4 runs the remaining cycle



Example CPU Scheduling – with I/O Burst -Priority

PID	AT	BT 1	I/O Time	BT 2	Priority
P1	0	4	2	2	2
P2	2	3	1	1	1
P3	5	5			3
P4	9	2			4



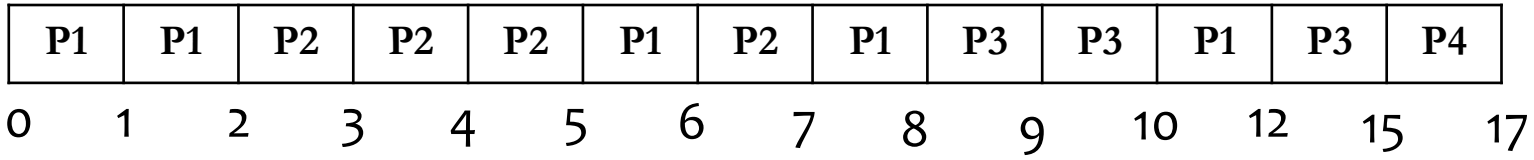
At T=17, P4 Terminates
No process in RQ

P1	P1	P2	P2	P2	P1	P2	P1	P3	P3	P1	P3	P4

0 1 2 3 4 5 6 7 8 9 10 12 15 17

Example CPU Scheduling – with I/O Burst -Priority

Performance of the Priority



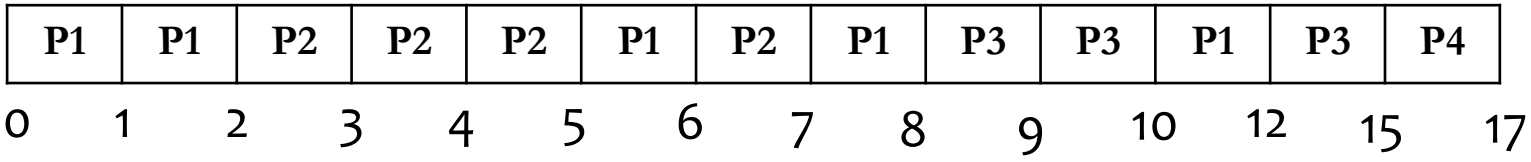
Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU Start Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	0	6	0	12	12	6	0
P2	2	4	2	7	5	1	0
P3	5	5	8	15	10	5	3
P4	9	2	15	17	8	6	6
SUM					**25**	**18**	**9**
AVG					**25/4= 6.25 ms**	**18 / 4 = 4.5 ms**	**9/4 = 2.25 ms**

Example CPU Scheduling – with I/O Burst -Priority

Performance of the Priority

Total CPU Burst Time

$17ms$



Total System Time

17 ms

CPU utilization → 17/17 * 100 = 100%

Throughput= Number of Processes / Total System Time

$$= \frac{4}{17} = 0.235$$

Throughput =0.2 processes/ms

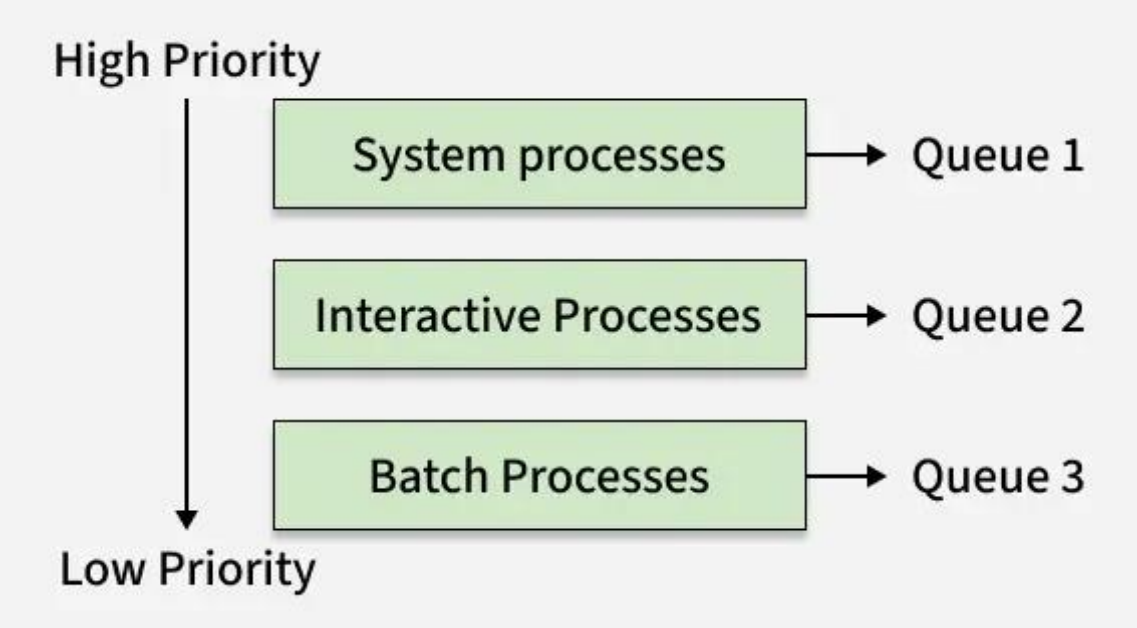
No. of Context switches in Priority – 8

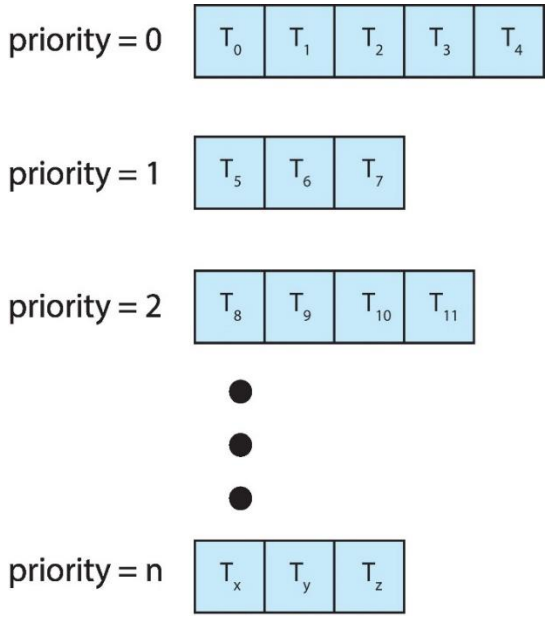
CPU Scheduling – Multilevel Queue Scheduling

- MLQ - applied at the operating system level with the aim of sectioning types of processes and then being able to manage them properly.
- Multilevel Queue Scheduling divides processes into multiple queues arranged in a hierarchy.
- Each queue has its own priority level and process type.
- Different scheduling algorithms can be applied to different queues.
- Once assigned, a process is permanently mapped to a specific queue.
- Assignment is usually based on factors like priority, resources, or process characteristics.

CPU Scheduling – Multilevel Queue Scheduling

- **Ready Queue** is divided into separate queues for each class of processes.
- For example, let us take three different types of processes System processes, Interactive processes, and Batch Processes. All three processes have their own queue.





- All three different type of processes have their own queue. Each queue has its own Scheduling algorithm.
- For example, queue 1 and queue 2 use **Round Robin** while queue 3 can use **FCFS** to schedule their processes.

CPU Scheduling – Multilevel Queue Scheduling

Features

- **Multiple Queues:** Processes are divided into queues based on priority (e.g., system, interactive, batch).
- **Priorities:** Higher-priority processes get higher queues; lower ones go to lower queues.
- **Preemption**: A high-priority process can interrupt a low-priority one.
- **Independent Scheduling (Different Algorithms)**: Each queue uses its own scheduling method (e.g., RR for interactive, FCFS for batch).
- **Feedback**: Priority may be adjusted if a process waits too long.
- **Efficient CPU Use**: High-priority processes run first, but low-priority ones also get CPU when higher queues are empty.

CPU Scheduling – Multilevel Queue Scheduling

Scheduling among the queues

- What will happen if all the queues have some processes?
- Which process should get the CPU?
- To determine this Scheduling among the queues is necessary.
- There are two ways

  1.Fixed priority preemptive scheduling method: Each queue has absolute priority over the lower priority queue.

  - Let us consider the following priority order **queue 1 > queue 2 > queue 3**.
  - According to this algorithm, no process in the batch queue (queue 3) can run unless queues 1 and 2 are empty. If any batch process (queue 3) is running and any system (queue 1) or Interactive process(queue 2) entered the ready queue the batch process is preempted.

  2.Time slicing: In this method, each queue gets a certain portion of CPU time and can use it to schedule its own processes. For instance, queue 1 takes 50 percent of CPU time queue 2 takes 30 percent and queue 3 gets 20 percent of CPU time.

CPU Scheduling – Multilevel Queue Scheduling

Example:

Consider 3 processes:

P1: Uses Round Robin (RR) in Queue 1

P2: Uses First-Come-First-Serve (FCFS) in Queue 2

P3: Uses Shortest Job First (SJF) in Queue 3

Process	Queue(scheduling)	Arrival Time	Burst Time
P1	Queue 1 (RR , q=2)	0 ms	5 ms
P2	Queue 2 (FCFS)	1 ms	4 ms
P3	Queue 3 (SJF)	2 ms	3 ms

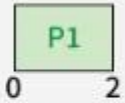
At time = 0 ms

P1 arrives (Queue 1: RR).

CPU executes P1 for 2 ms (time quantum = 2).

Remaining burst of P1 = 5 − 2 = 3 ms.

Timeline: 0 → 2 : P1



GANTT chart

At time = 2 ms

At t = 1, P2 arrived (Queue 2: FCFS).

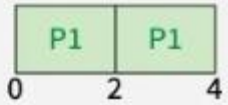
At t = 2, P3 arrived (Queue 3: SJF).

Still, Queue 1 has P1 (3 ms left) → highest priority.

CPU executes P1 again for 2 ms.

Remaining burst of P1 = 3 − 2 = 1 ms.

Timeline: 2 → 4 : P1



GANTT chart

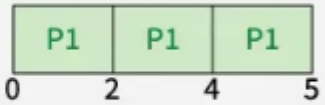
At time = 4 ms

Queue 1 still has P1 (1 ms left).

CPU executes P1 for final 1 ms.

P1 finishes at t = 5 ms.

Timeline: 4 → 5 : P1 (Finished)



GANTT chart

CPU Scheduling – Multilevel Queue Scheduling

Example:

Consider 3 processes:

P1: Uses Round Robin (RR) in Queue 1

P2: Uses First-Come-First-Serve (FCFS) in Queue 2

P3: Uses Shortest Job First (SJF) in Queue 3

Process	Queue(scheduling)	Arrival Time	Burst Time
P1	Queue 1 (RR , q=2)	0 ms	5 ms
P2	Queue 2 (FCFS)	1 ms	4 ms
P3	Queue 3 (SJF)	2 ms	3 ms

At time = 5 ms

Now Queue 1 is empty.

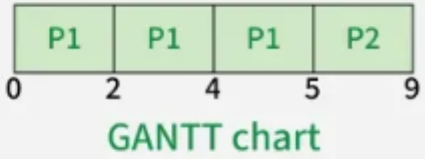
Next highest priority = Queue 2 (P2, FCFS).

CPU executes P2 fully
(since FCFS is non-preemptive).

Burst = 4 ms.

P2 finishes at t = 9 ms.

Timeline: 5 → 9 : P2 (Finished)



GANTT chart

At time = 9 ms

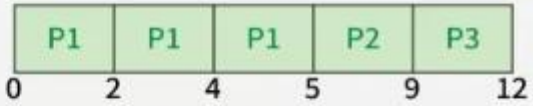
Queue 3 (P3) is now ready.

Scheduling = SJF, only one process.

CPU executes P3 fully (3 ms).

P3 finishes at t = 12 ms.

Timeline: 9 → 12 : P3 (Finished)



GANTT chart

In Multilevel Queue Scheduling, processes are divided into different queues based on priority or process type.

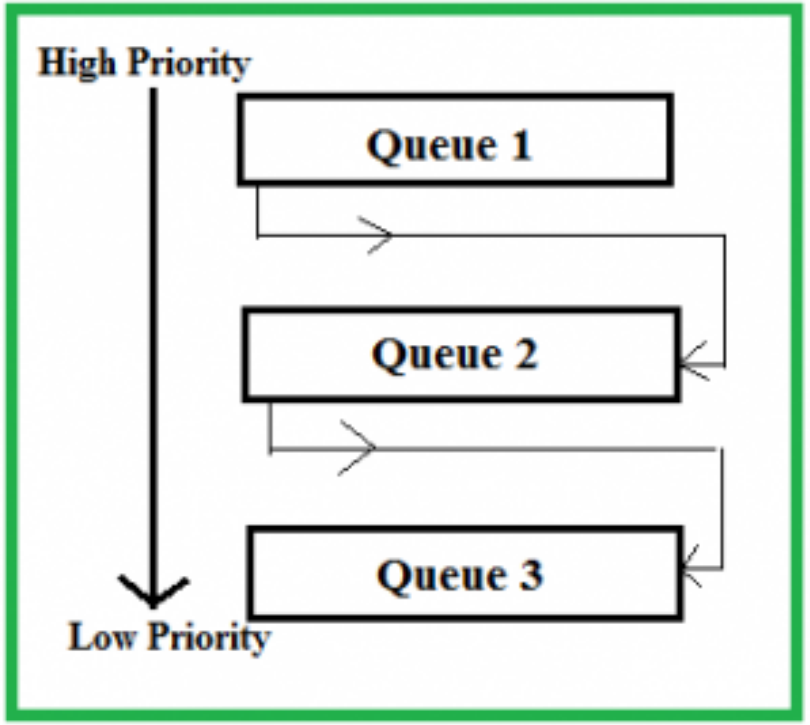
- Each queue uses a different scheduling algorithm (e.g., Round Robin, FCFS, SJF).
- Higher priority queues are always served first.
- Once a queue is empty, the CPU moves to the next lower-priority queue.

CPU Scheduling – Multilevel Queue Scheduling

Advantages	Disadvantages
Separates processes (e.g., system, interactive, batch) into distinct queues, making management more organized.	If high-priority queues are underloaded, CPU may stay idle while lower-priority processes wait.
Since queue priorities are fixed, system administrators can easily predict how processes will be scheduled.	Once placed in a queue, processes cannot move between queues, leading to inefficiency if workload type changes.
Interactive and system processes get higher priority, leading to improved response times for user-centric tasks.	Handling multiple queues requires more bookkeeping and resources.
Suitable for systems handling a wide variety of process types, as multiple queues can be designed for different workloads.	Some queues may dominate CPU time while others rarely get a chance
Ensures critical processes are not delayed behind non-essential tasks.	In real-world workloads where process behavior changes, **fixed queues** reduce flexibility and adaptability.

CPU Scheduling – Multilevel Feedback Queue Scheduling

- Multilevel Feedback Queue Scheduling (**MLFQ**) CPU Scheduling - like Multilevel Queue(MLQ) Scheduling but - process can move between the queues.
- much more efficient than multilevel queue scheduling.



Characteristics of Multilevel Feedback Queue Scheduling:

- Processes can move between queues based on their CPU usage or I/O behavior.
- Processes are NOT permanently assigned to any queue.
- Priority changes dynamically depending on process performance.

CPU Scheduling – Multilevel Feedback Queue Scheduling

Features of Multilevel Feedback Queue Scheduling (MLFQ) CPU Scheduling

Multiple Queues

- MLFQ has multiple priority queues, and processes can move between them.
- Higher-priority queues get CPU first.
- Movement depends on how the process behaves.

Dynamic Priority

- A process's priority changes based on CPU usage or I/O wait.
- CPU-heavy processes move down.
- I/O-bound or short processes move up.

Time-Slicing

- Each queue has its own time slice.
- High-priority queues have shorter slices.
- Processes using their slice may get demoted.

Feedback Mechanism

- Processes are promoted or demoted based on performance.
- Short or interactive processes rise to higher queues.
- CPU-bound processes move to lower queues.

Preemption

- Preemption is allowed in MLFQ scheduling, so higher-priority processes can interrupt lower-priority ones.
- This ensures urgent or interactive tasks get CPU quickly.
- Lower-priority processes resume later when the CPU is free.
- Multilevel Feedback Queue Scheduling (MLFQ) dynamically adjusts the priority of processes based on their behavior.

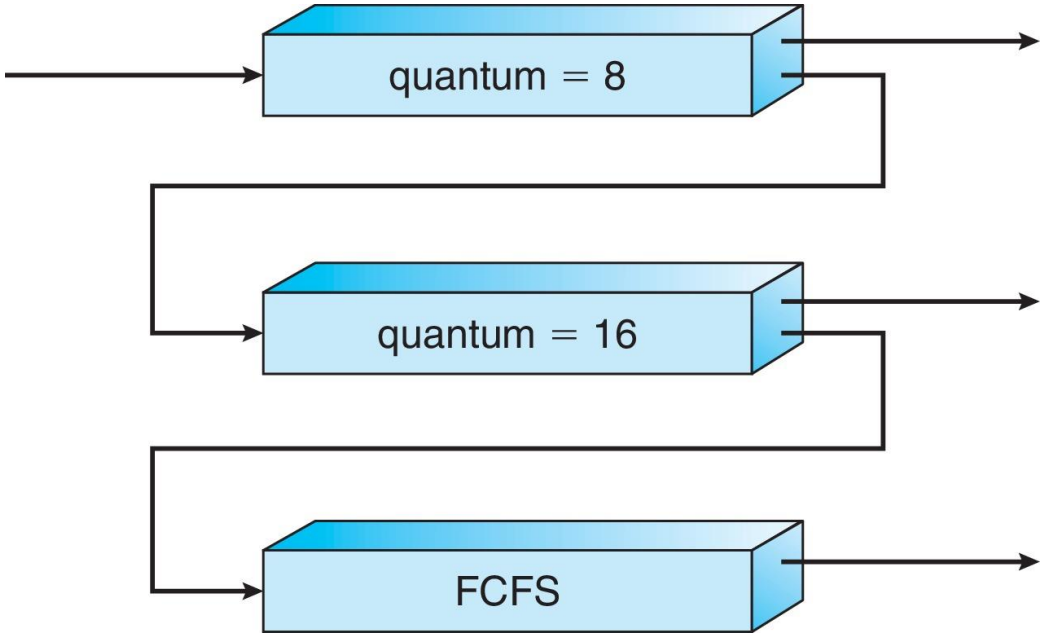
CPU Scheduling – Multilevel Feedback Queue Scheduling

Three queues:

Q_0 – RR with time quantum 8 milliseconds
Q_1 – RR time quantum 16 milliseconds
Q_2 – FCFS

How it works?

- A new process enters queue Q_0 which is served in RR
- When it gains CPU, the process receives 8 milliseconds
- If it does not complete in 8 milliseconds, the process is moved to queue Q_1
- At Q_1 job is again served in RR and receives 16 additional milliseconds
- If it still does not complete, it is preempted and moved to queue Q_2



if it completes in these 8 ms or it gives CPU for I/O operation in these 8 ms then the priority of this process does not change and if it again comes in the ready queue then it again starts its execution in Q_0.

CPU Scheduling – Multilevel Feedback Queue Scheduling

Example: Consider a system that has a CPU-bound process, which requires a burst time of 40 seconds. The multilevel Feed Back Queue scheduling algorithm is used and the queue time quantum '2' seconds and in each level it is incremented by '5' seconds
Q1 = 2, Q2 = 7, Q3 = 12, Q4 = 17, Q5 = 22
Then how many times the process will be interrupted and in which queue the process will terminate the execution?

Solution:

- Process P needs 40 Seconds for total execution.
- At Queue 1 it is executed for 2 seconds and then interrupted and shifted to queue 2.
- At Queue 2 it is executed for 7 seconds and then interrupted and shifted to queue 3.
- At Queue 3 it is executed for 12 seconds and then interrupted and shifted to queue 4.
- At Queue 4 it is executed for 17 seconds and then interrupted and shifted to queue 5.
- At Queue 5 it executes for 2 seconds and then it completes.
- Hence the process is interrupted 4 times and completed on queue 5.

CPU Scheduling – Multilevel Feedback Queue Scheduling

Example:

Queue Structure & Priorities:

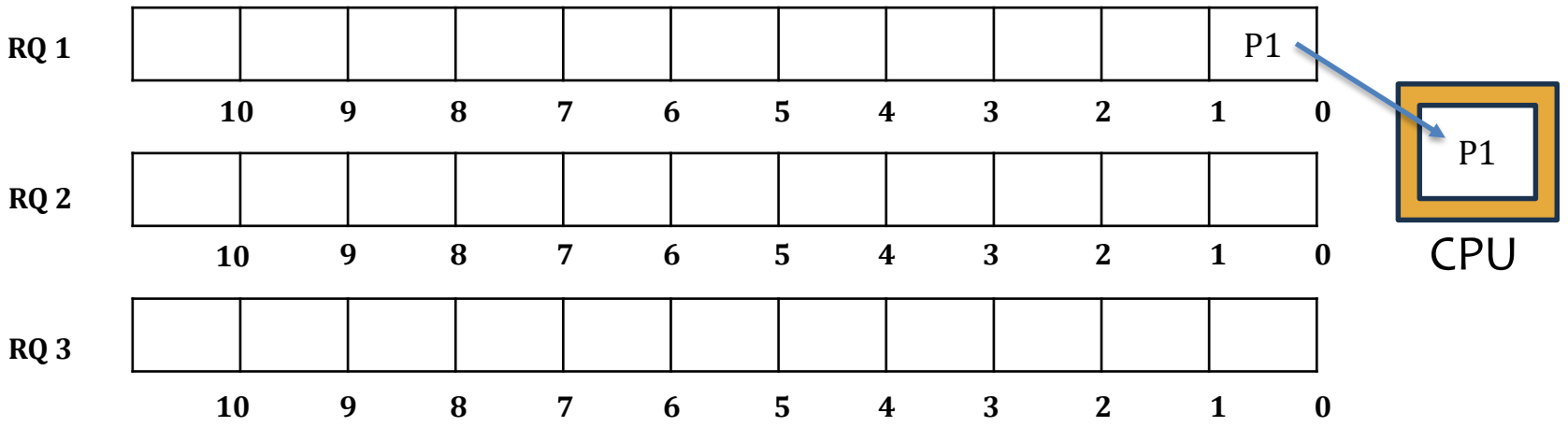
- Queue 1 (Q1): Highest Priority. Uses Round Robin (RR) with a Time Quantum (TQ) = 2 ms.
- Queue 2 (Q2): Medium Priority. Uses Round Robin (RR) with a Time Quantum (TQ) = 4 ms.
- Queue 3 (Q3): Lowest Priority. Uses First-Come, First-Served (FCFS).

Downgrade Rules:

- All new processes initially arrive in Q1.
- If a process consumes its entire Time Quantum in its current queue without finishing, it is preempted and downgraded to the next lower-priority queue (Q1 → Q2, Q2 → Q3).
- If a process finishes its burst time before or exactly when the quantum expires, it leaves the system.
- The CPU always processes tasks in Q1 first.
- Q2 is only processed if Q1 is empty, and
- Q3 is only processed if both Q1 and Q2 are empty.

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



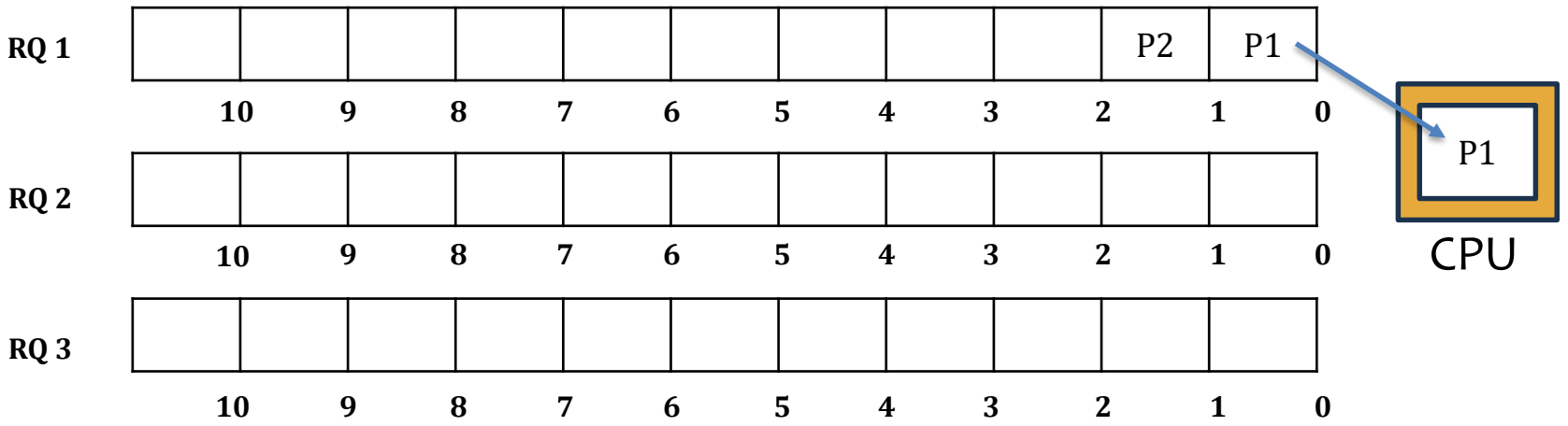
At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.

P1																							

0 **2**

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



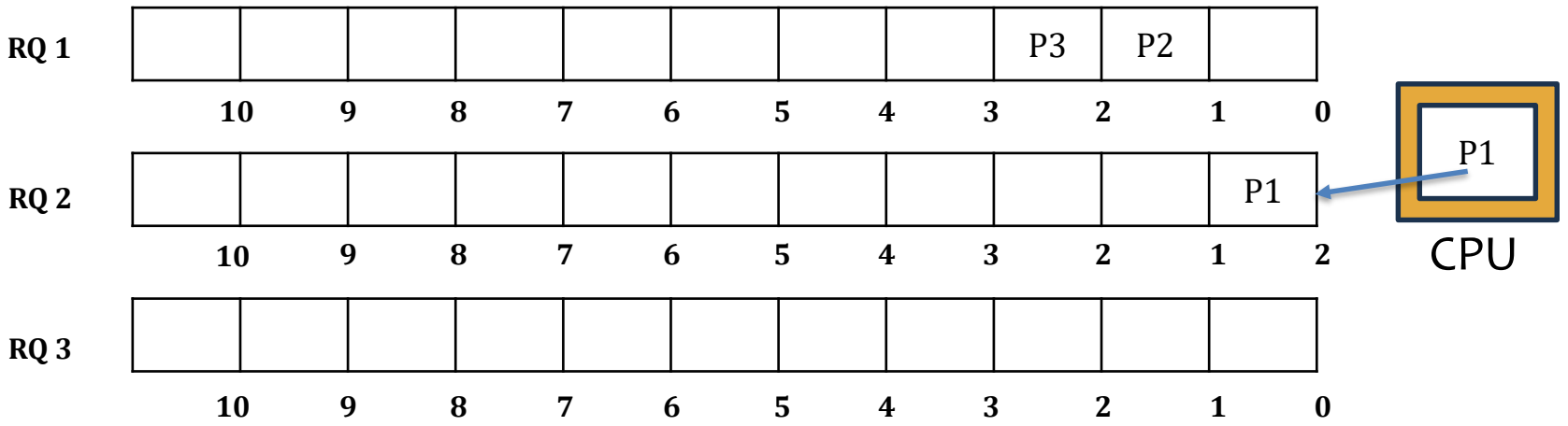
At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.
At t = 1, P2 arrives in RQ 1	P1 continues Execution	

P1																							

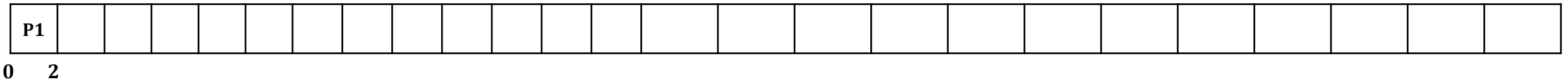
0 2

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

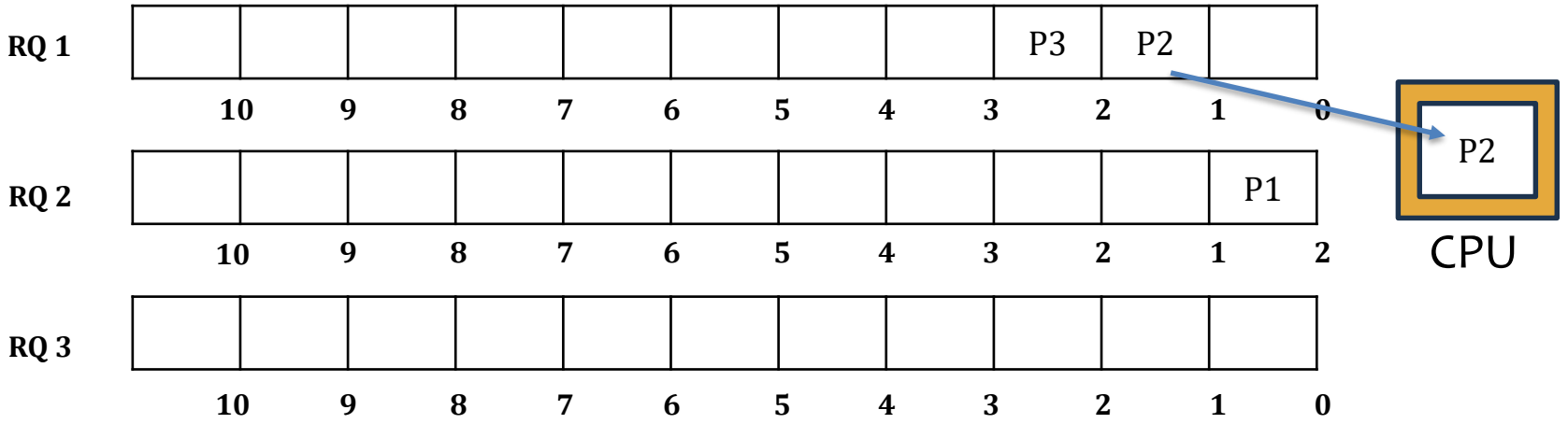


At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.
At t = 1, P2 arrives in RQ 1	P1 continues Execution	
At t = 2, P3 arrives in RQ 1	P1 gets suspended and moves to RQ 2	

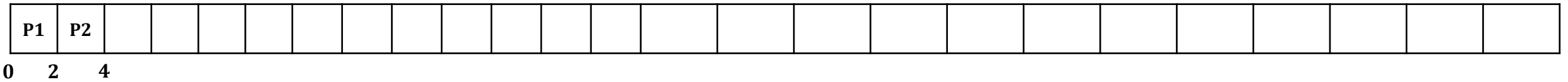


CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

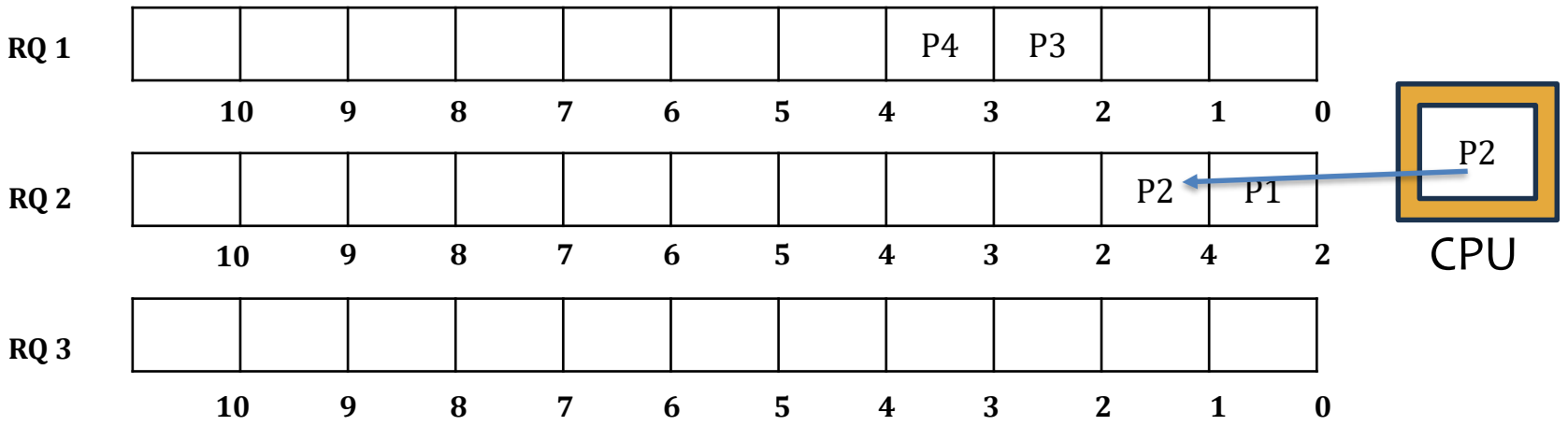


At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.
At t = 1, P2 arrives in RQ 1	P1 continues Execution	
At t = 2, P3 arrives in RQ 1	P1 gets suspended and moves to RQ 2	P2 gets the CPU and executes till 4 consuming its 2 TQ



CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



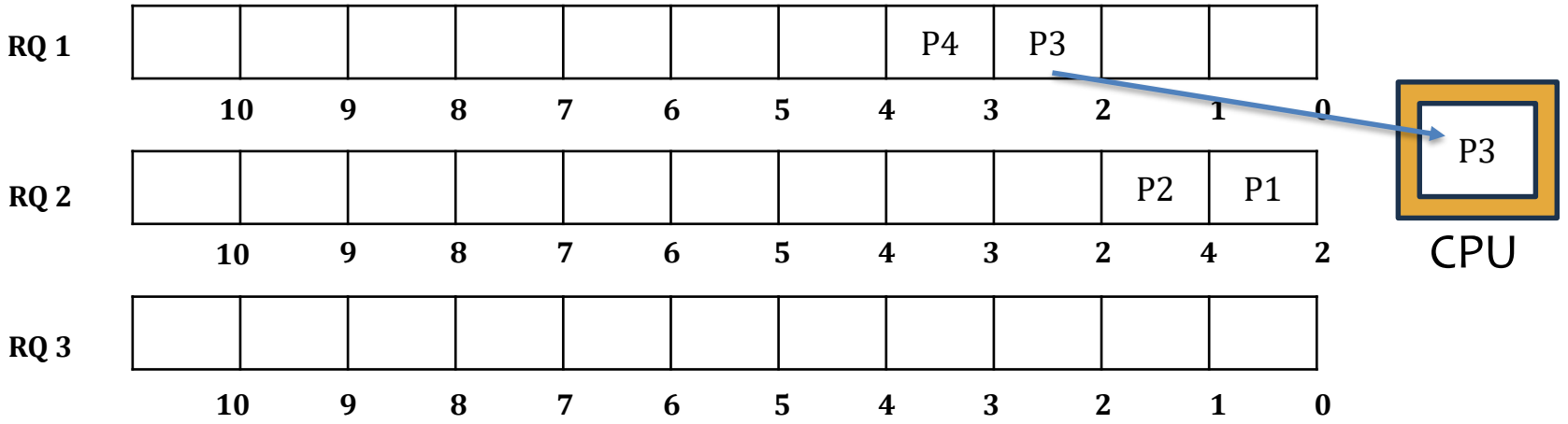
At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.
At t = 1, P2 arrives in RQ 1	P1 continues Execution	
At t = 2, P3 arrives in RQ 1	P1 gets suspended and moves to RQ 2	P2 gets the CPU and executes till 4 consuming its 2 TQ
At t = 4, P4 arrives in RQ 1	P2 gets suspended and moves to RQ 2	

P1	P2																						

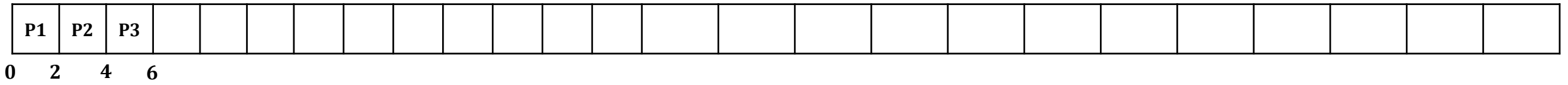
0 2 4

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

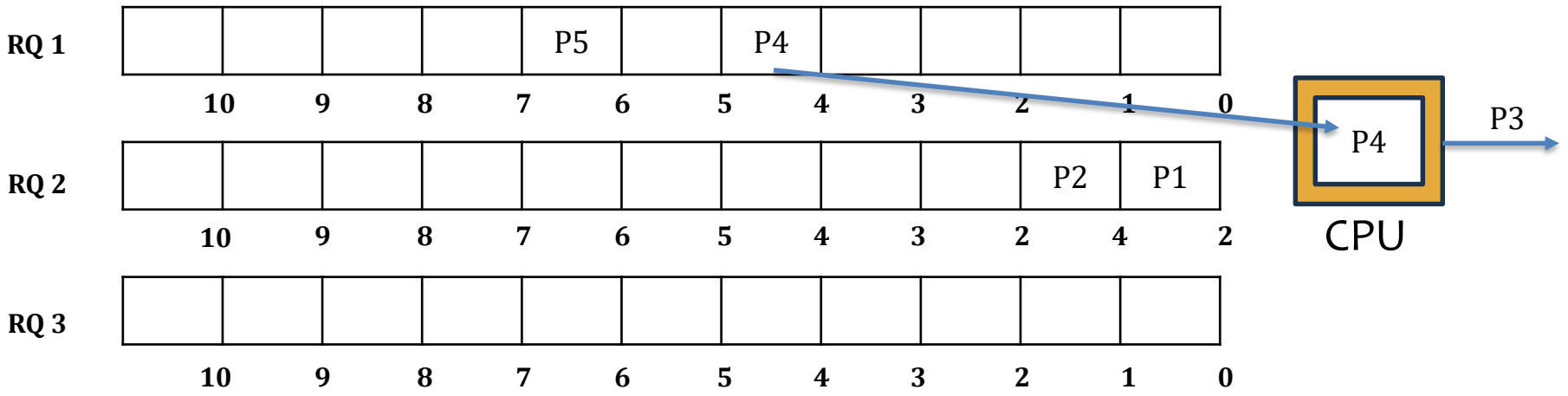


At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.
At t = 1, P2 arrives in RQ 1	P1 continues Execution	
At t = 2, P3 arrives in RQ 1	P1 gets suspended and moves to RQ 2	P2 gets the CPU and executes till 4 consuming its 2 TQ
At t = 4, P4 arrives in RQ 1	P2 gets suspended and moves to RQ 2	P3 gets the CPU and executes till 6 consuming its 2 TQ

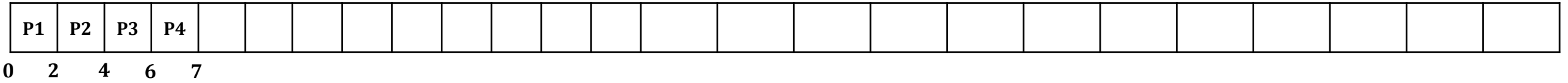


CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

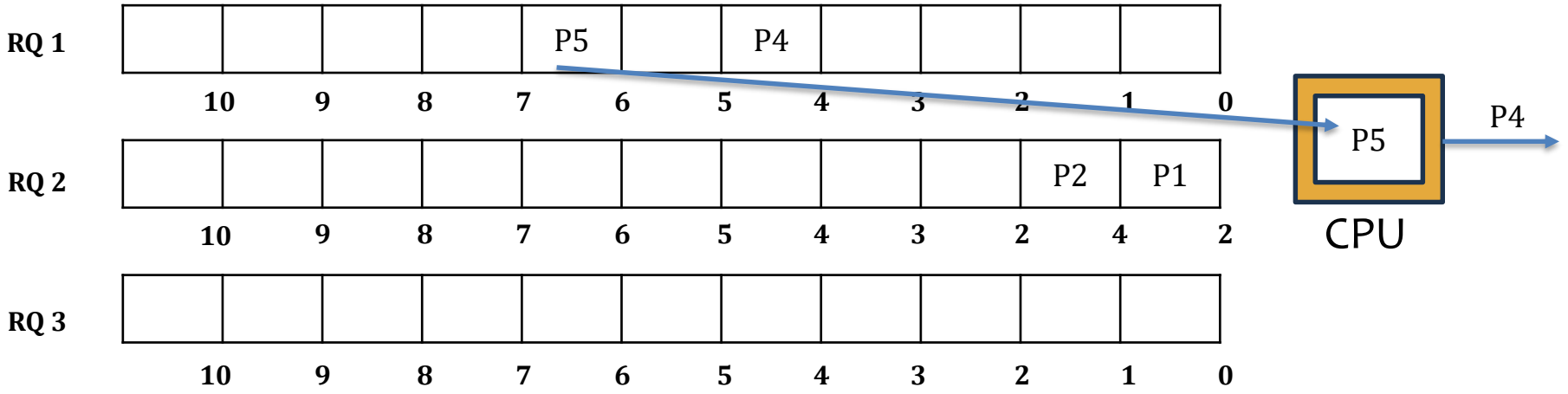


At t=0, P1 arrives in RQ 1	P1 is assigned to CPU instantly	P1 runs for 2 TQ.
At t = 1, P2 arrives in RQ 1	P1 continues Execution	
At t = 2, P3 arrives in RQ 1	P1 gets suspended and moves to RQ 2	P2 gets the CPU and executes till 4 consuming its 2 TQ
At t = 4, P4 arrives in RQ 1	P2 gets suspended and moves to RQ 2	P3 gets the CPU and executes till 6 consuming its 2 TQ
At t = 6, P5 Arrives in RQ 1	P3 completes it BT and Terminates	P4 gets the CPU and executes till 7 consuming its 2 TQ



CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



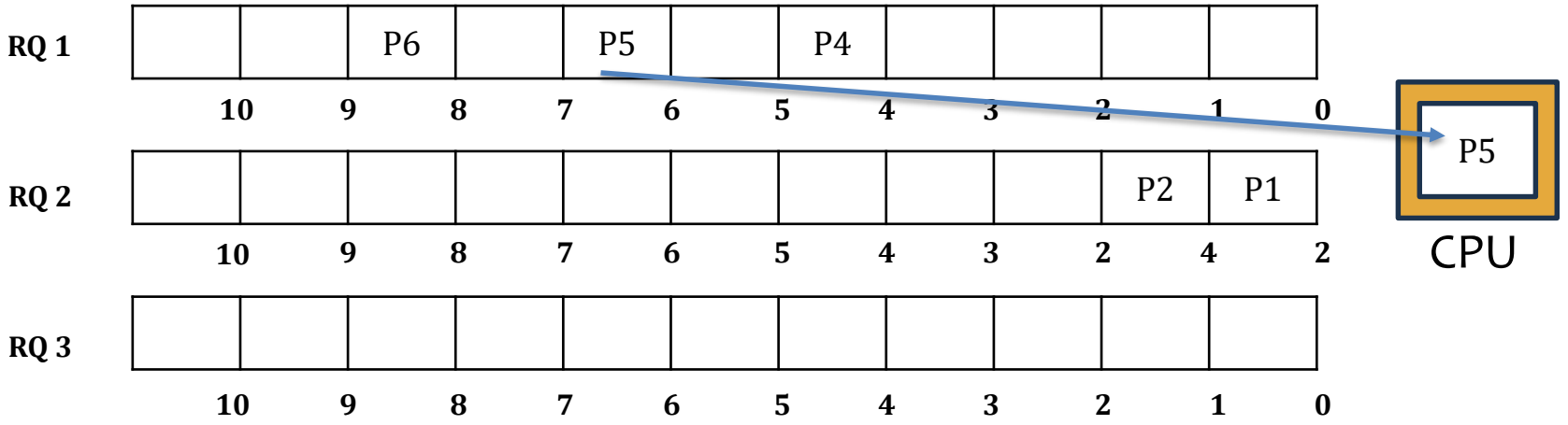
At t = 7	P4 terminates	P5 runs for 2 TQ till 9.

P1	P2	P3	P4	P5																		

0 2 4 6 7 9

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



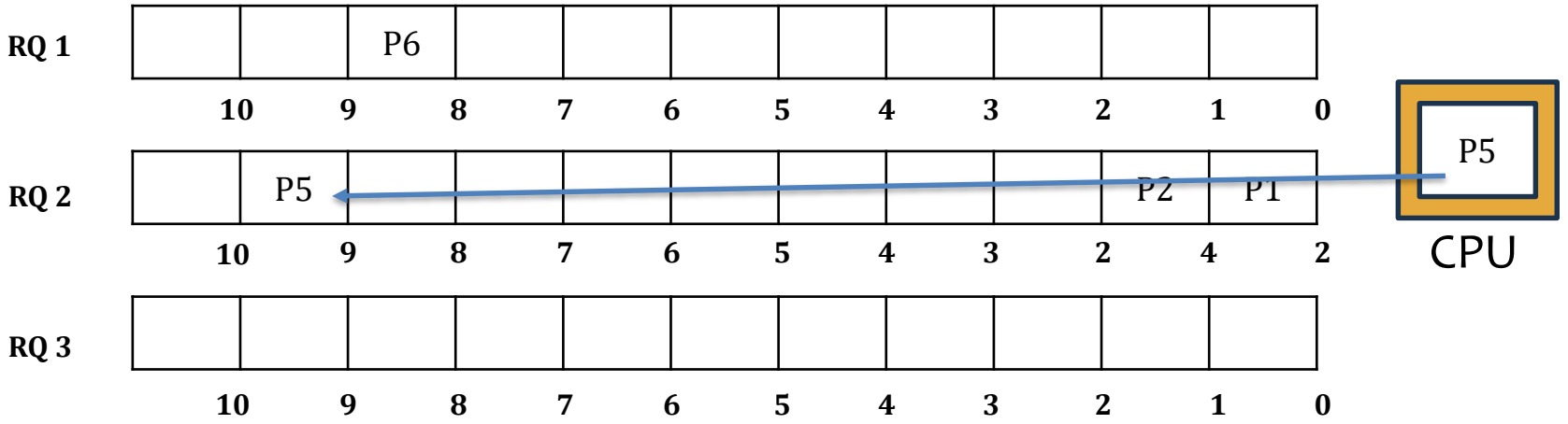
At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	

P1	P2	P3	P4	P5																		

0 2 4 6 7 9

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



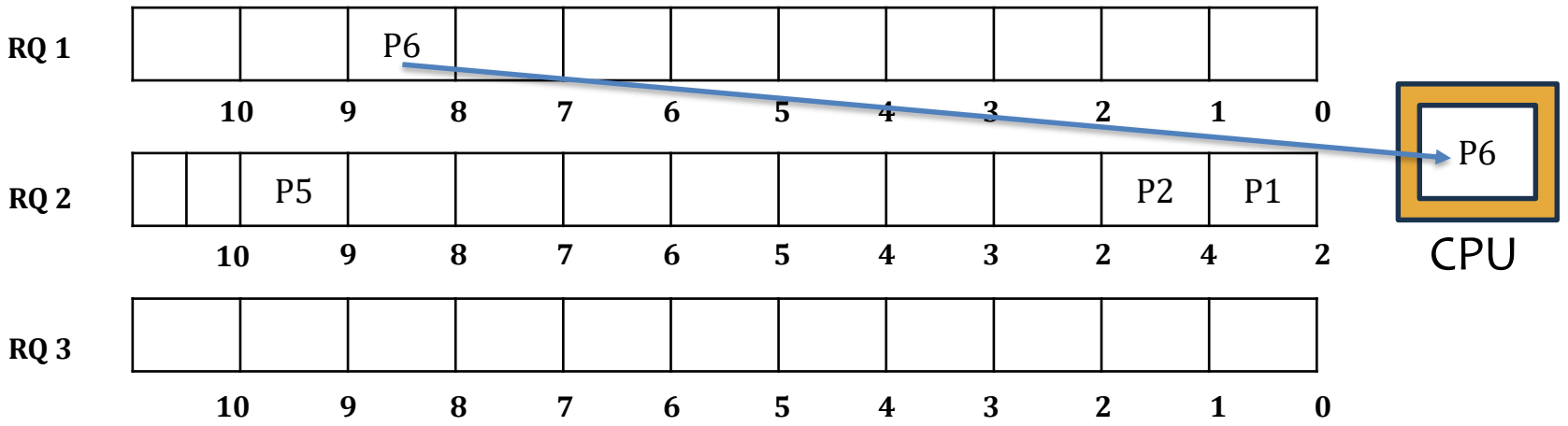
At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	
At t = 9	P5 gets suspended and moves to RQ 2	

P1	P2	P3	P4	P5																		

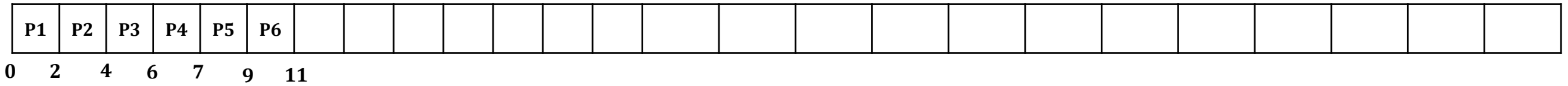
0 2 4 6 7 9

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	
At t = 9	P5 gets suspended and moves to RQ 2	P6 gets the CPU and executes till 11 consuming its 2 TQ



CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



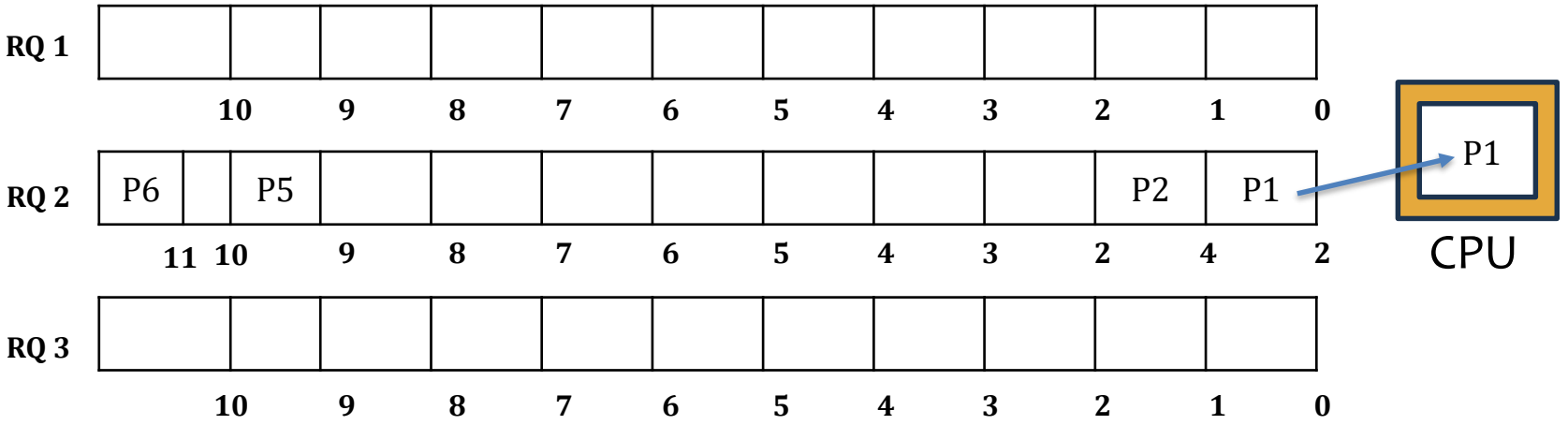
At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	
At t = 9	P5 gets suspended and moves to RQ 2	P6 gets the CPU and executes till 11 consuming its 2 TQ
At t = 11	P6 gets suspended and moves to RQ 2	

P1	P2	P3	P4	P5	P6																	

0 2 4 6 7 9 11

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

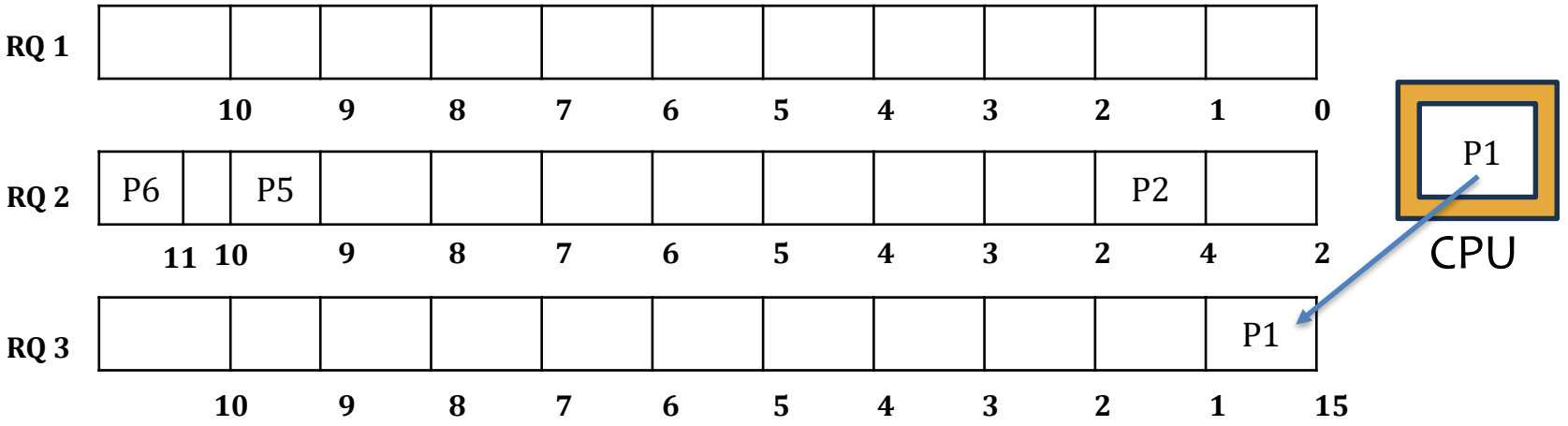


At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	
At t = 9	P5 gets suspended and moves to RQ 2	P6 gets the CPU and executes till 11 consuming its 2 TQ
At t = 11	P6 gets suspended and moves to RQ 2	P1 gets the CPU executes for 4 TQ till 15



CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

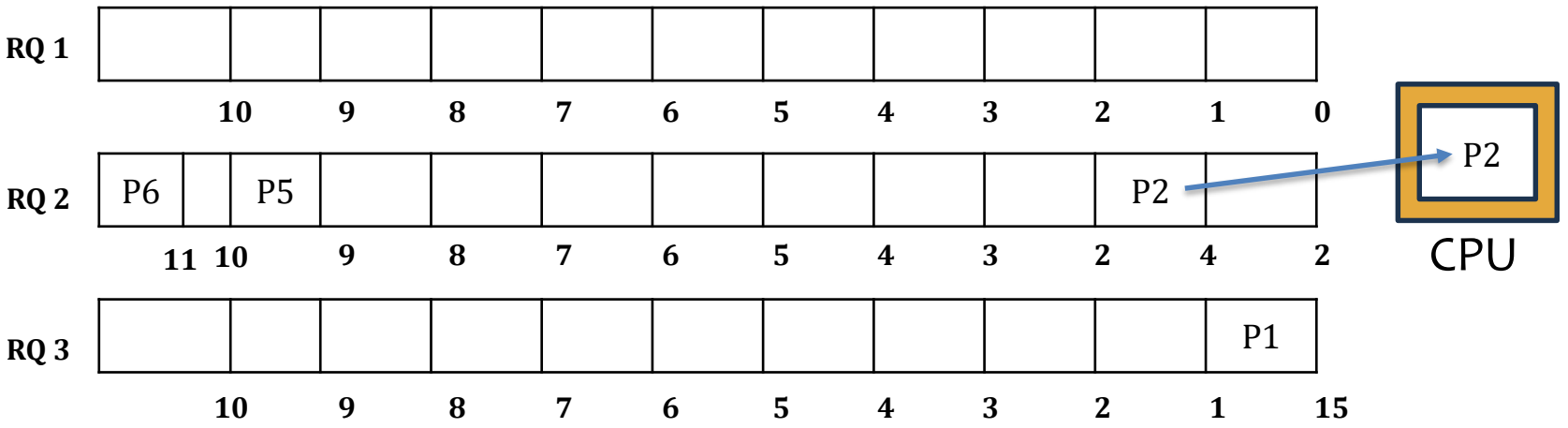


At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	
At t = 9	P5 gets suspended and moves to RQ 2	P6 gets the CPU and executes till 11 consuming its 2 TQ
At t = 11	P6 gets suspended and moves to RQ 2	P1 gets the CPU executes for 4 TQ till 15
At t = 15	P1 gets suspended and moves to RQ 3	

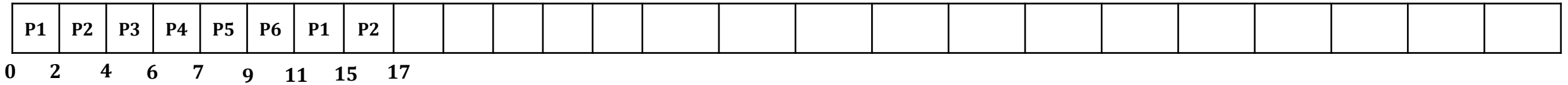


CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



At t = 7	P4 terminates	P5 runs for 2 TQ till 9.
At t = 8, P6 arrives in RQ 1	P5 continues Execution	
At t = 9	P5 gets suspended and moves to RQ 2	P6 gets the CPU and executes till 11 consuming its 2 TQ
At t = 11	P6 gets suspended and moves to RQ 2	P1 gets the CPU executes for 4 TQ till 15
At t = 15	P1 gets suspended and moves to RQ 3	P2 gets the CPU executes for 4 TQ till 17



CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

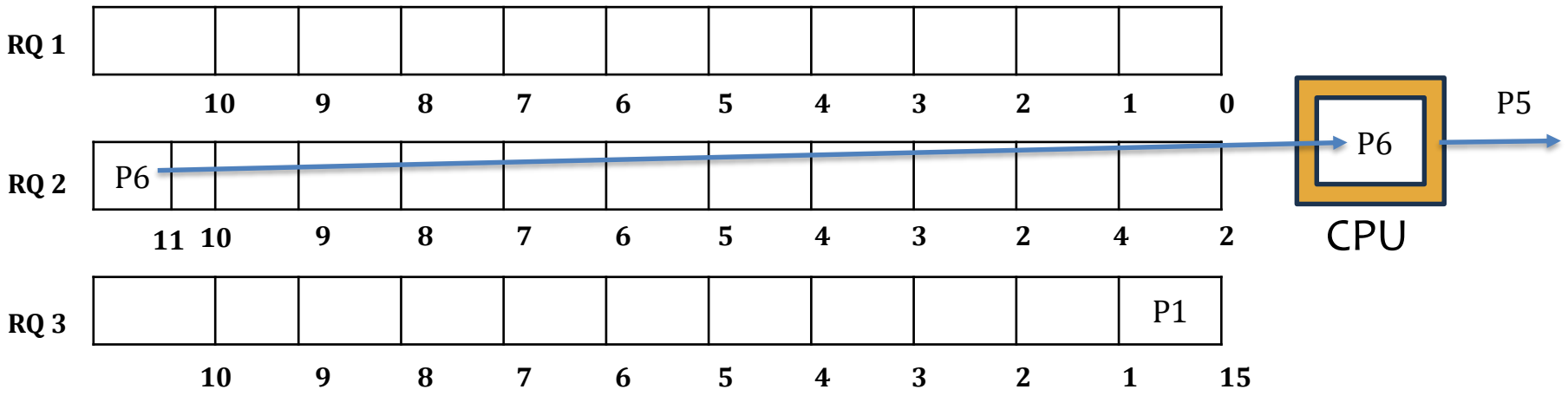


At t = 17	P2 terminates	P5 gets the CPU and executes for 4 TQ till 20



CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



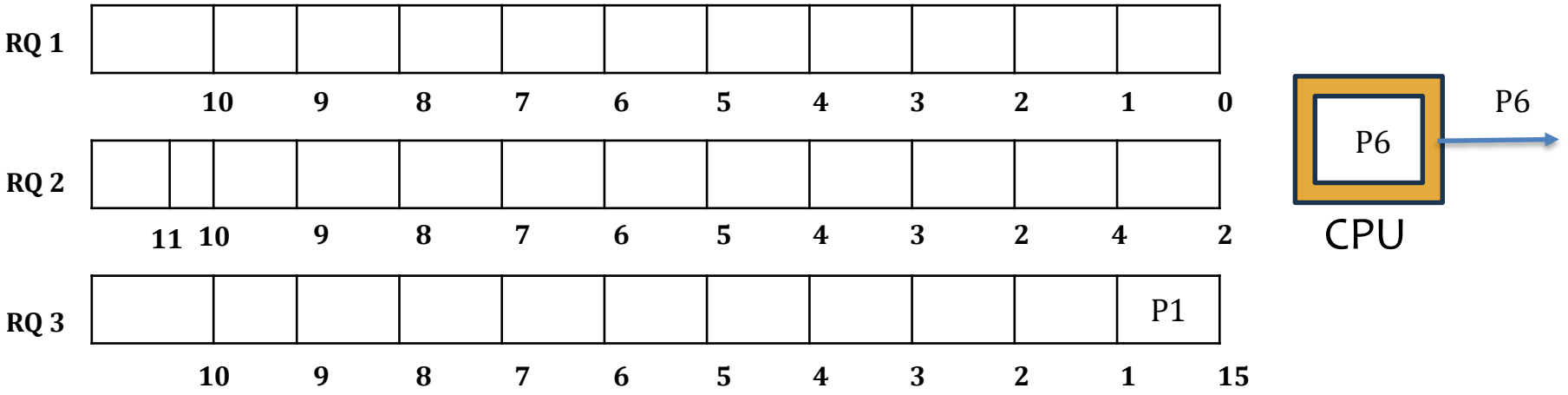
At t = 17	P2 terminates	P5 gets the CPU and executes for 4 TQ till 20
At t = 20	P5 Terminates	P6 gets the CPU and executes for 4 TQ till 21

P1	P2	P3	P4	P5	P6	P1	P2	P5	P6													

0 2 4 6 7 9 11 15 17 20 21

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



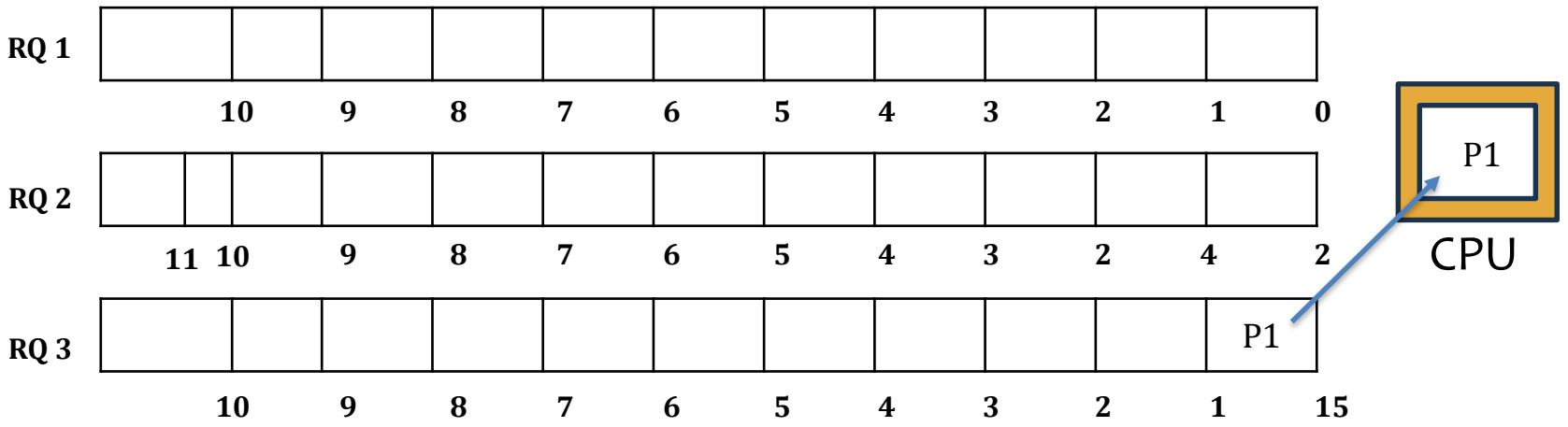
At t = 17	P2 terminates	P5 gets the CPU and executes for 4 TQ till 20
At t = 20	P5 Terminates	P6 gets the CPU and executes for 4 TQ till 21
At t = 21	P6 terminates	

P1	P2	P3	P4	P5	P6	P1	P2	P5	P6													

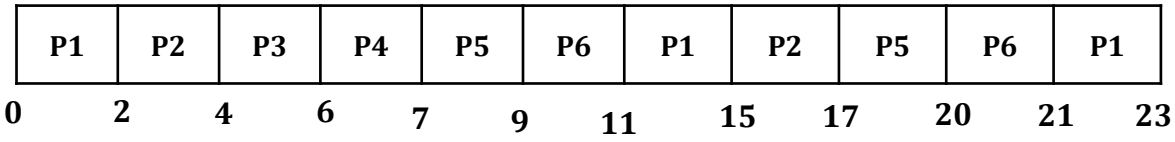
0 2 4 6 7 9 11 15 17 20 21

CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms

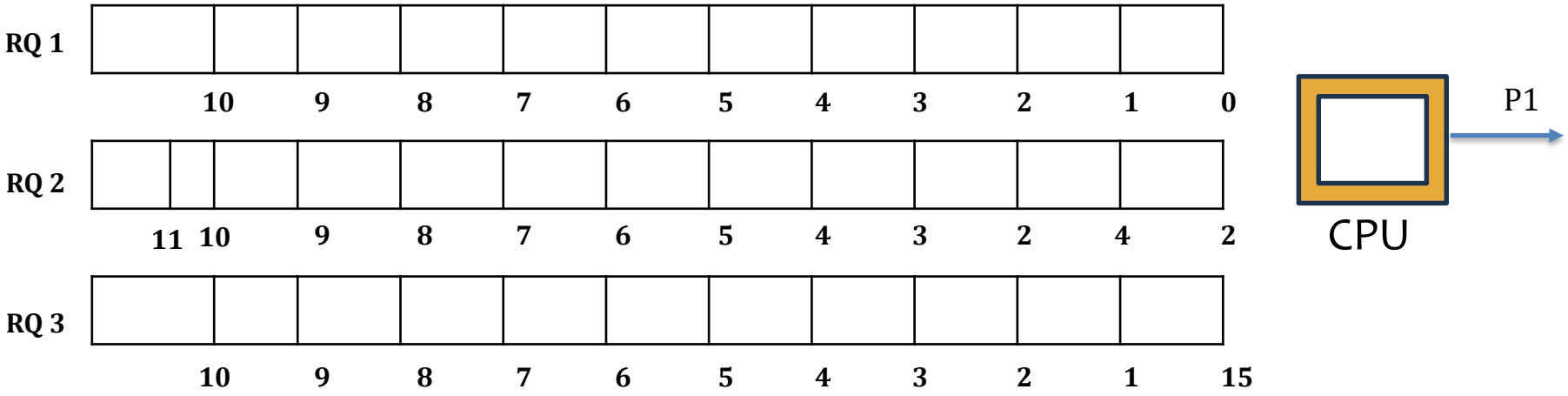


At t = 17	P2 terminates	P5 gets the CPU and executes for 4 TQ till 20
At t = 20	P5 Terminates	P6 gets the CPU and executes for 4 TQ till 21
At t = 21	P6 terminates	P1 gets the CPU and executes its remaining BT till 23

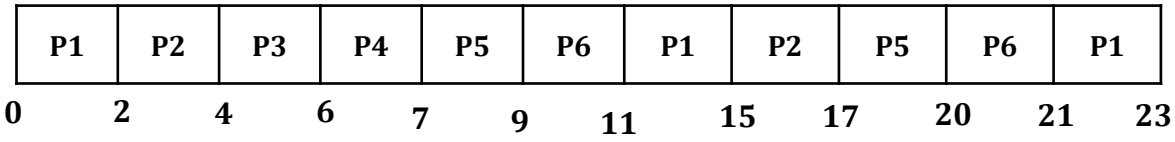


CPU Scheduling – Multilevel Feedback Queue Scheduling

Process	Arrival Time (AT)	Burst Time (BT)
P1	0 ms	8 ms
P2	1 ms	4 ms
P3	2 ms	2 ms
P4	4 ms	1 ms
P5	6 ms	5 ms
P6	8 ms	3 ms



At t = 17	P2 terminates	P5 gets the CPU and executes for 4 TQ till 20
At t = 20	P5 Terminates	P6 gets the CPU and executes for 4 TQ till 21
At t = 21	P6 terminates	P1 gets the CPU and executes its remaining BT till 23
At t = 23	P1 Terminates	No Process in any of the RQs



CPU Scheduling – Multilevel Feedback Queue Scheduling

Performance of the MLFQ

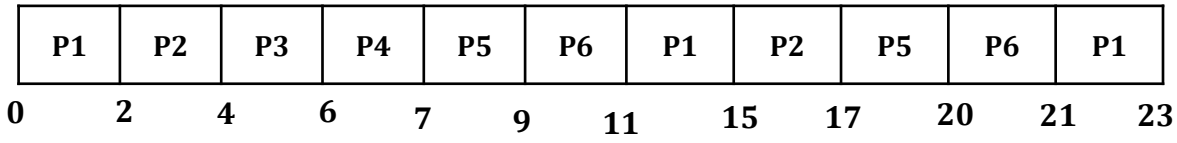
Process	Arrival Time (AT)	CPU Burst Time (BT)	*First CPU Start Time* (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P1	0	8	0	23	23	15	0
P2	1	4	2	17	16	12	1
P3	2	2	4	6	4	2	2
P4	4	1	6	7	3	2	2
P5	6	5	7	20	14	9	1
P6	8	3	9	21	13	10	1
SUM					**73**	**50**	**7**
AVG					**73 / 6 = 12.16**	**50 / 6 = 8.33**	**7 / 6 = 1.16**

CPU Scheduling – Multilevel Feedback Queue Scheduling

Performance of the MLFQ

Total CPU Burst Time

$23ms$



Total System Time

23 ms

CPU utilization → 23/23 * 100 = 100%

Throughput= Number of Processes / Total System Time

$$= \frac{6}{23} = 0.2607$$

Throughput =0.26 processes/ms

No. of Context switches in MLFQ – 10

CPU Scheduling – Multi Processors Scheduling

- When a system contains more than one CPU, Multiple-Processor Scheduling involves distributing tasks across multiple processors. This enables several tasks to be processed in parallel, improving performance significantly.
- **Key Challenges:**
  - Deciding which CPU handles which task.
  - Balancing workloads between processors to avoid idle CPUs or overloaded processors.

Note: *In multiple processor scheduling, there are cases when the processors are identical i.e. Homogenous, in terms of their functionality, we can use any processor available to run any process in the queue.*

CPU Scheduling – Multi Processors Scheduling

Important Concepts in Multiprocessor Scheduling

1. Processor Affinity

A process prefers to run on the same processor it previously ran on.

Benefits: Data stays in the CPU cache & reduces cache misses and improves performance.

Types of Affinity:

- Soft Affinity: OS attempts (but does not guarantee) to keep a process on the same CPU.
- Hard Affinity: Restricts the process to run only on a specific subset of processors using system calls (e.g., sched_setaffinity() in Linux).

2. Load Balancing

- Avoids - Overloaded processors with long queues and
  - Idle processors while other processors are busy.

Two approaches:

1.Push Migration: A processor actively moves tasks from itself to less busy processors to balance load.

2.Pull Migration: An idle processor pulls tasks from busy processors to balance its workload.

CPU Scheduling – Multi Processors Scheduling

Important Concepts in Multiprocessor Scheduling

3. Multicore Processors

- Multiple processor cores on a single physical chip.
- Each core appears as an independent processor to the OS.

Challenge: Memory Stall

- Happens when a processor waits for data not present in cache -
- Up to 50% of time can be wasted waiting for memory.

Solution: Multithreading within Cores

- **Coarse-Grained Multithreading**: Switch thread when long latency events occur (e.g., cache miss).
- **Fine-Grained Multithreading**: Switch threads every instruction cycle, keeping the pipeline filled with minimal overhead.

4. Virtualization and Threading

- Single physical CPU appears as multiple virtual CPUs to virtual machines.
- Host OS manages multiple guest OS instances.

Problem: Time-slice distortion - guest OS time slices may not match real CPU cycles due to virtualization overhead.

CPU Scheduling – Multi Processors Scheduling

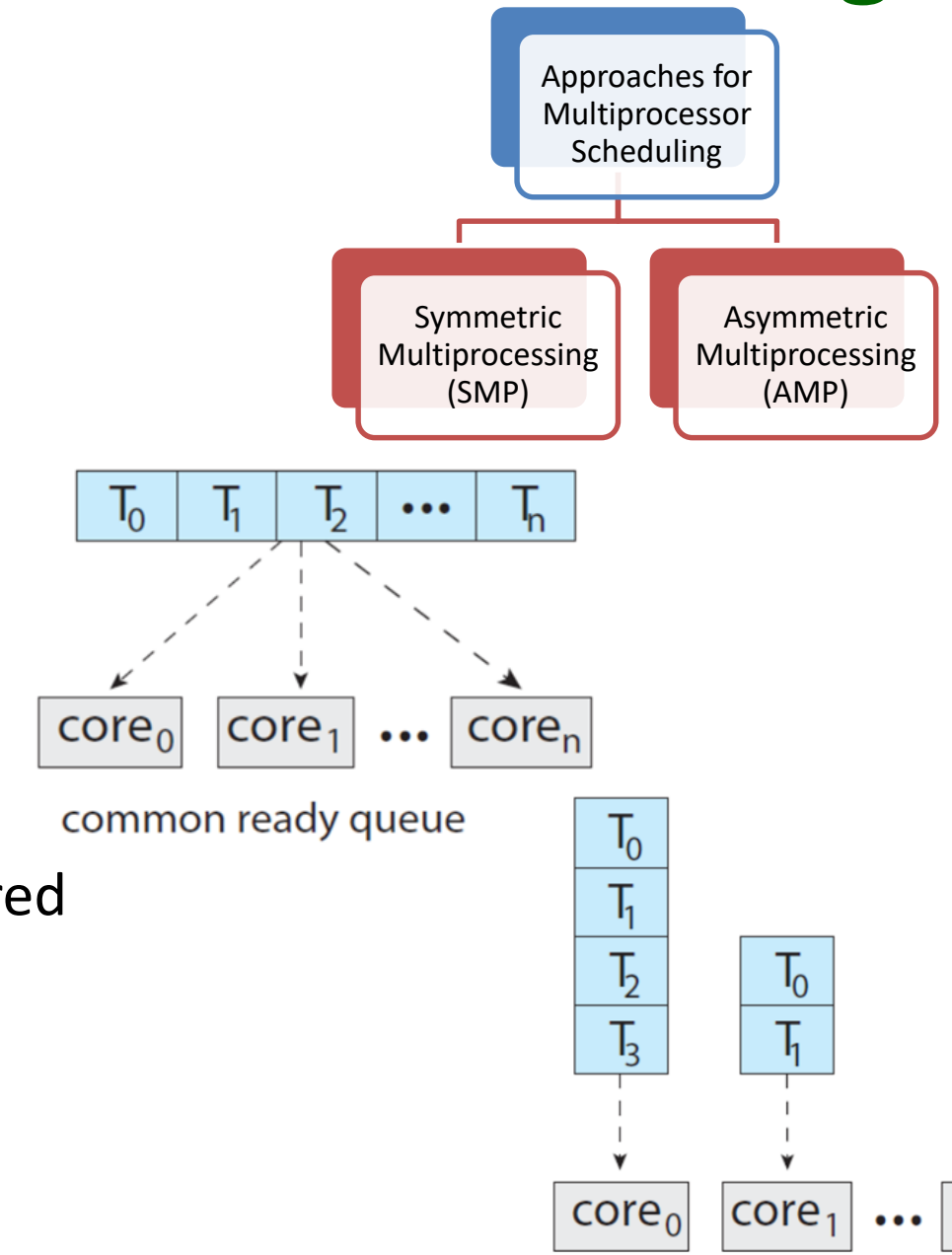
Approaches to Multiple-Processor Scheduling

1. Asymmetric Multiprocessing (AMP)

- One processor - acts as a Master Server handling scheduling decisions and I/O operations.
- Other processors - execute only user code.
- Simple design, reduces data-sharing complexity.

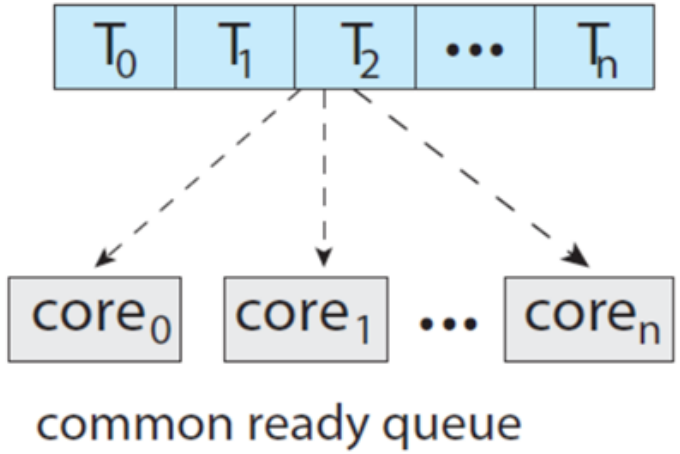
2. Symmetric Multiprocessing (SMP)

- Each processor can schedule tasks independently.
- Two models of task queues:
  - Common Ready Queue: All CPUs access a shared queue of ready processes.
  - Private Ready Queues: Each CPU has its own queue of ready processes.



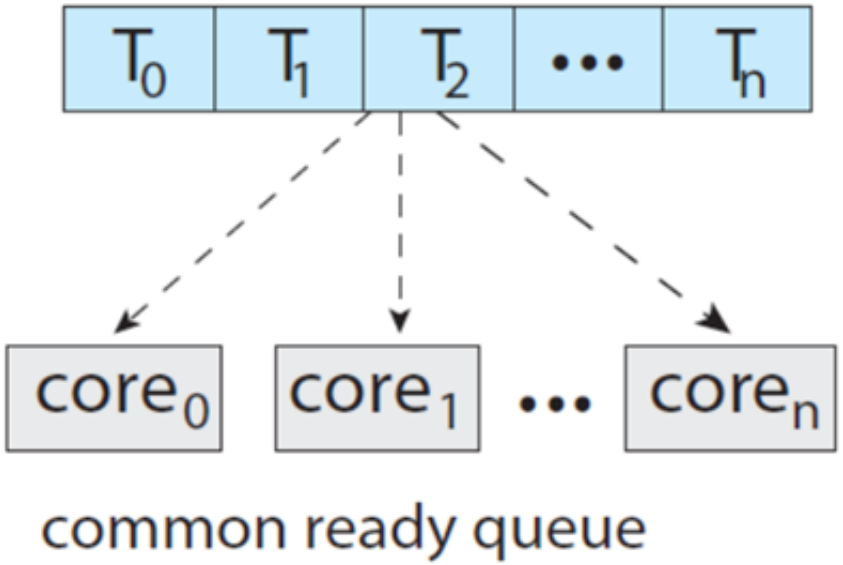
CPU Scheduling – Multi Processors Scheduling - AMP

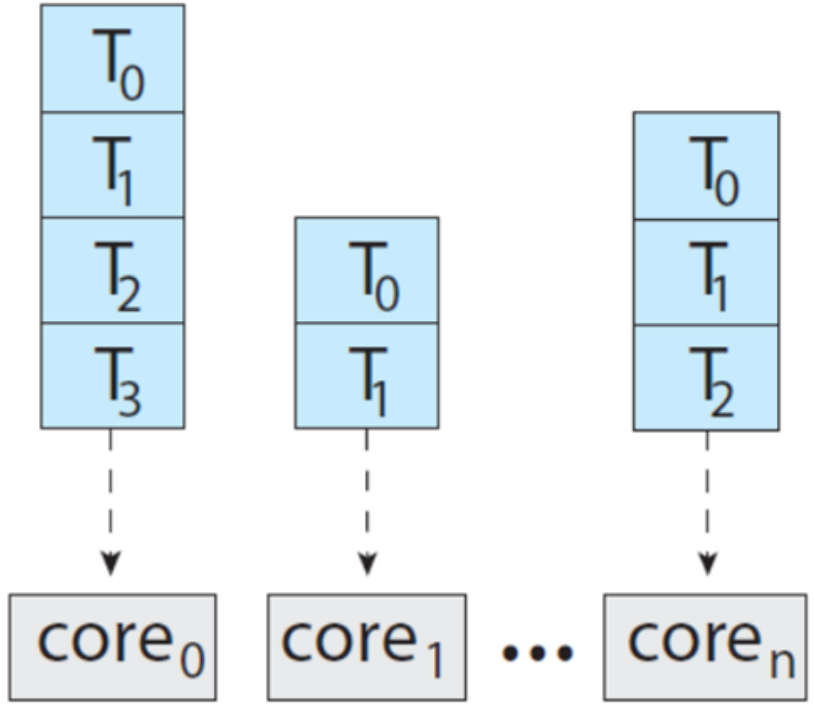
- Master – Slave approach
- Master - scheduling decisions
  - I/O operations
- Advantage: if one of the slave servers goes down, the rest of the system keeps working.
- Disadvantage: This is a simple approach because all the complex scheduling logic is handled by one processor, but it can create a bottleneck if the master processor gets overloaded.
- If the master server goes down, the whole system comes to a halt.



CPU Scheduling – Multi Processors Scheduling - SMP

- Multiple processors are installed and have an equal access to the system and memory resources of the system.
- Every processor works individually, while doing its work and communicating with the operating system.





CPU Scheduling – Multi Processors Scheduling - AMP

Example

Hardware & OS Rules:

Core 0 (Master): The only core permitted to execute Operating System code. It handles scheduling and exclusively processes all I/O requests.

Cores 1, 2, and 3 (Workers): Dedicated entirely to executing user code (CPU bursts). They cannot perform I/O.

Scheduling: The Master assigns CPU bursts to Workers using **First-Come, First-Served** (FCFS).

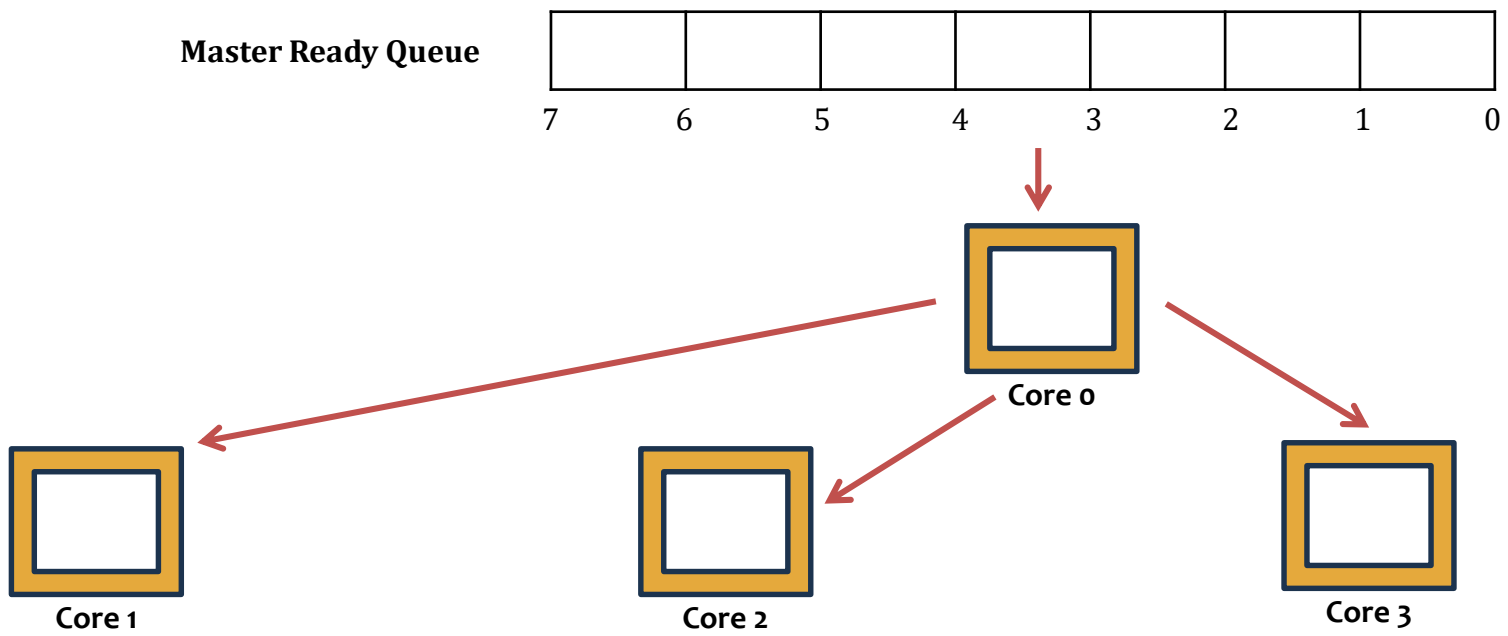
The AMP Bottleneck: Since **only the Master can handle I/O**, if multiple processes request I/O at the same time, they must wait in an I/O Queue processed by the Master using FCFS.

CPU Scheduling – Multi Processors Scheduling - AMP

Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms



I/O Queue

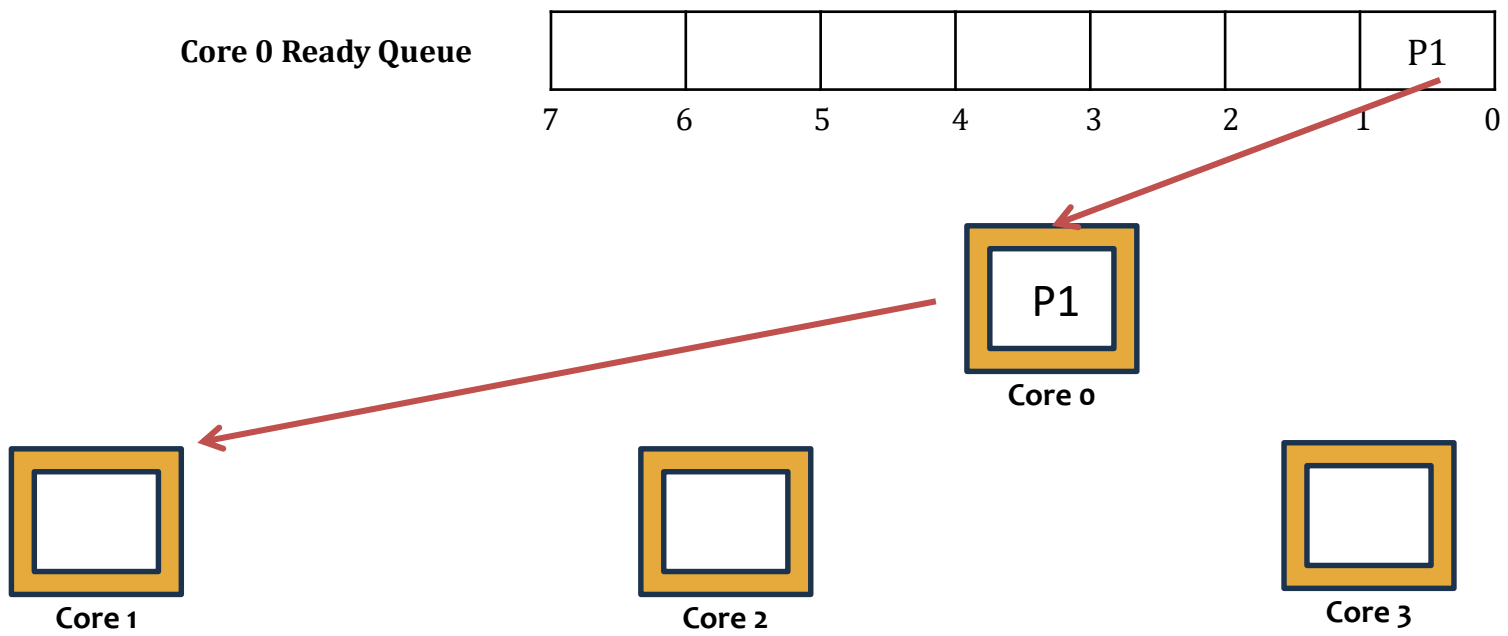
CPU Scheduling – Multi Processors Scheduling - AMP

Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms

At t = 0, P1 Arrives, Core 0 allocates it to Core 1



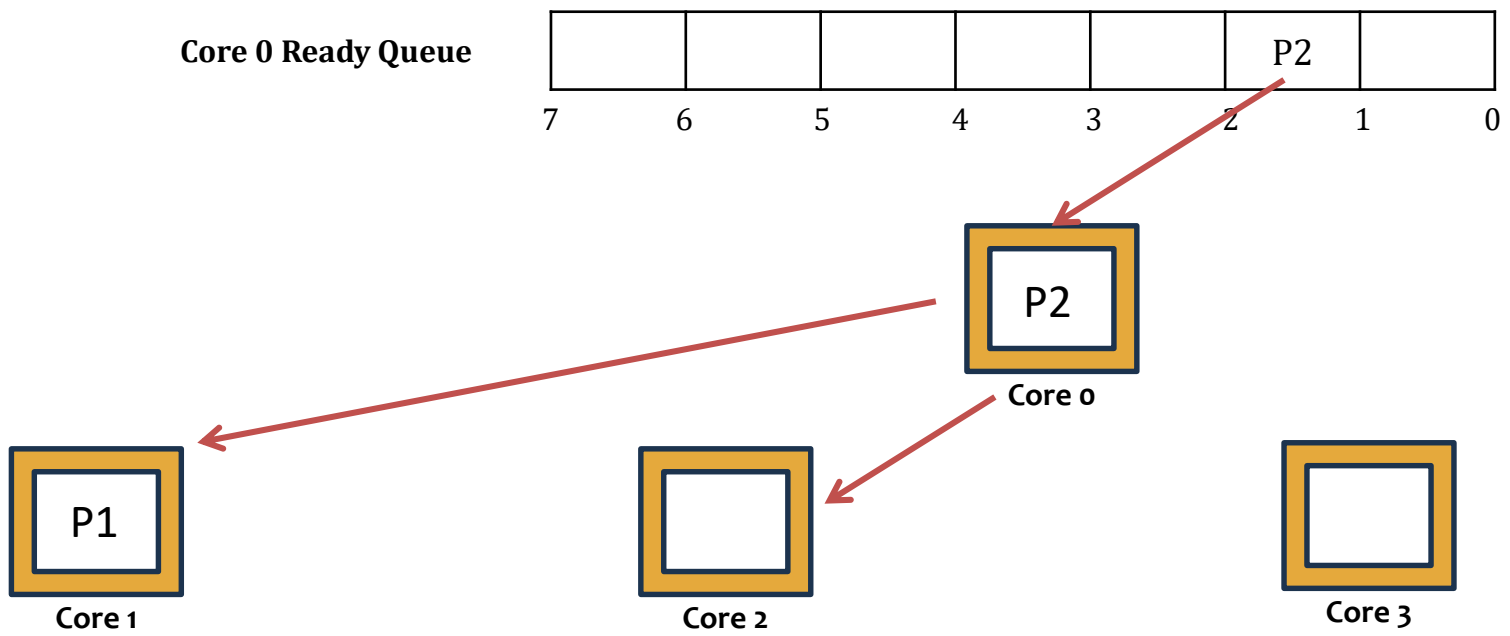
I/O Queue

CPU Scheduling – Multi Processors Scheduling - AMP

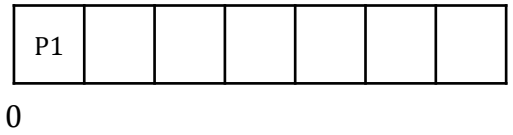
Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms



At t = 0, P1 Arrives, Core 0 allocates it to Core 1
At t = 1, P2 Arrives, Core 0 allocates it to Core 2



I/O Queue

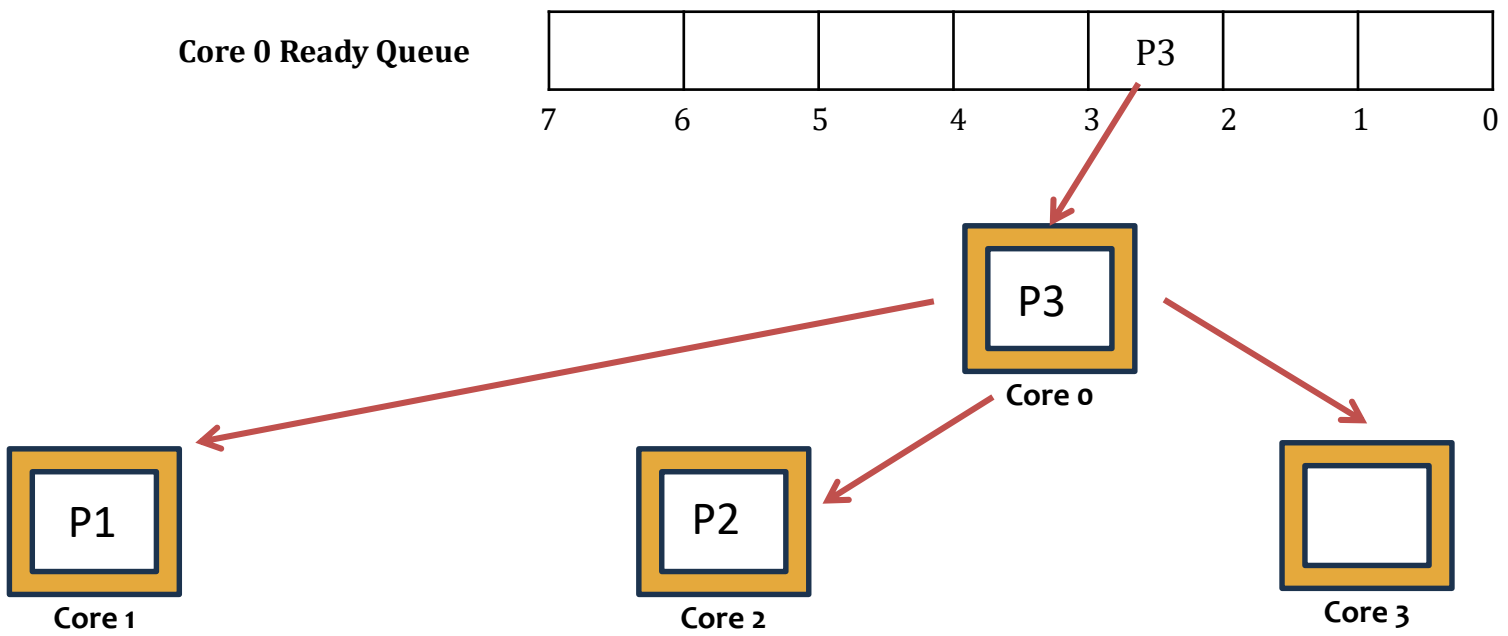
CPU Scheduling – Multi Processors Scheduling - AMP

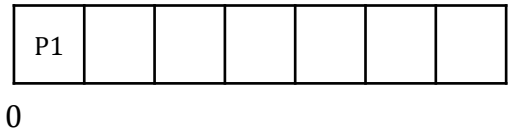
Example

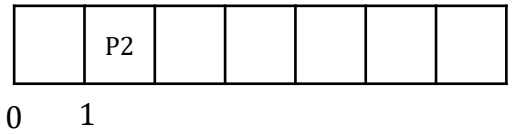
Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms

At t = 2, P3 Arrives, Core 0 allocates it to Core 3







I/O Queue

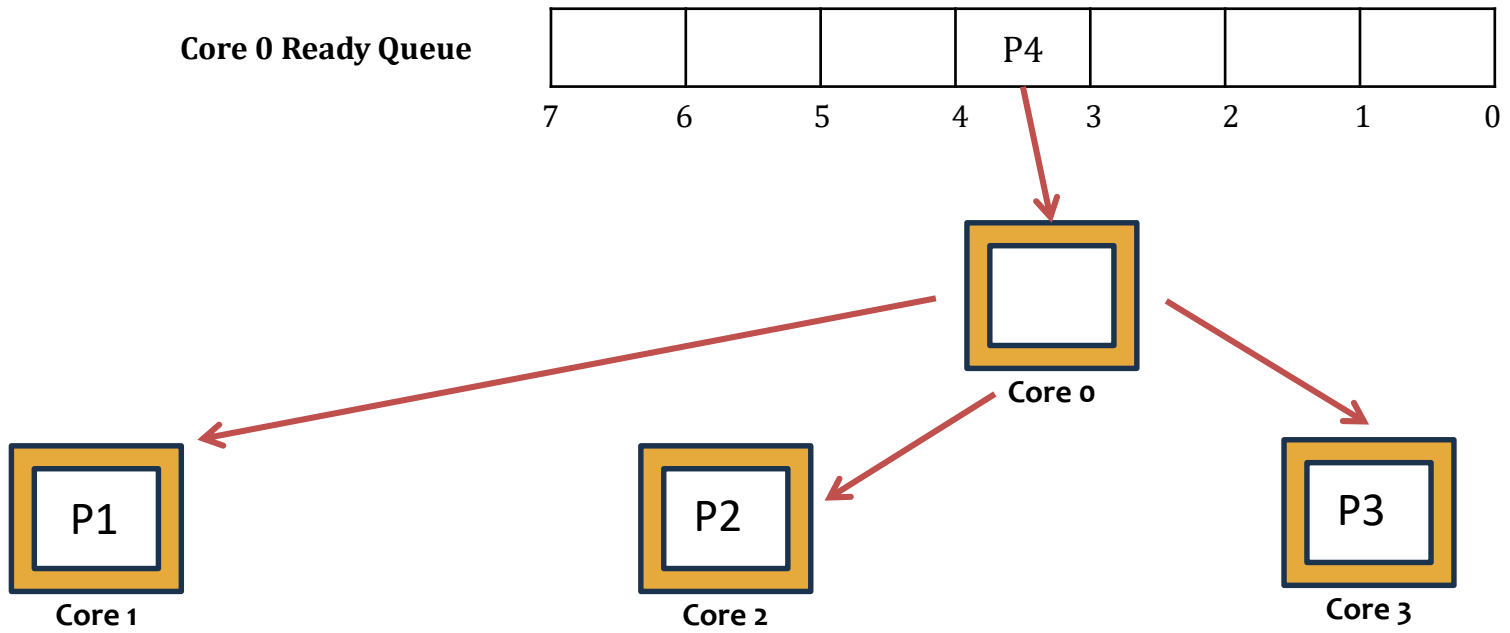
CPU Scheduling – Multi Processors Scheduling - AMP

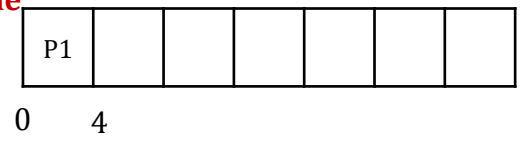
Example

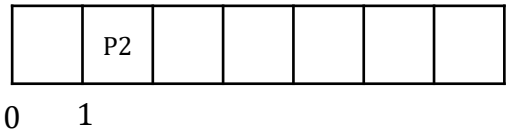
Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

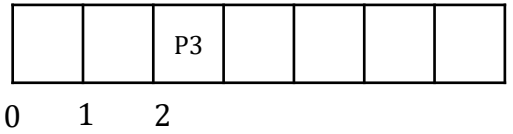
Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms

At t = 2, P3 Arrives, Core 0 allocates it to Core 3
At t = 3. P4 arrives, All cores are busy, hence P4 waits in Ready Queue









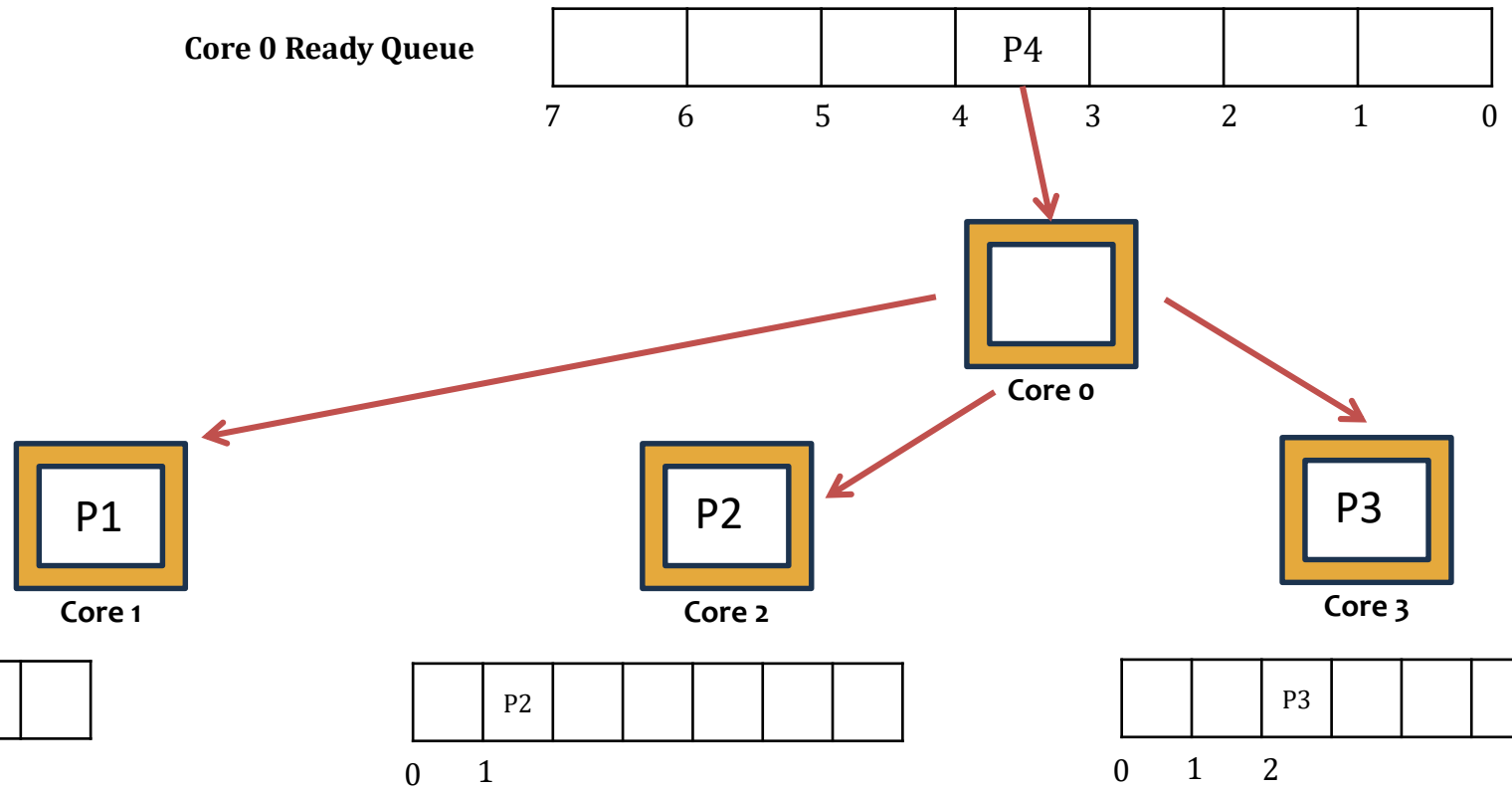
I/O Queue

CPU Scheduling – Multi Processors Scheduling - AMP

Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms



Core 2 executes P2 till time 4

I/O Queue

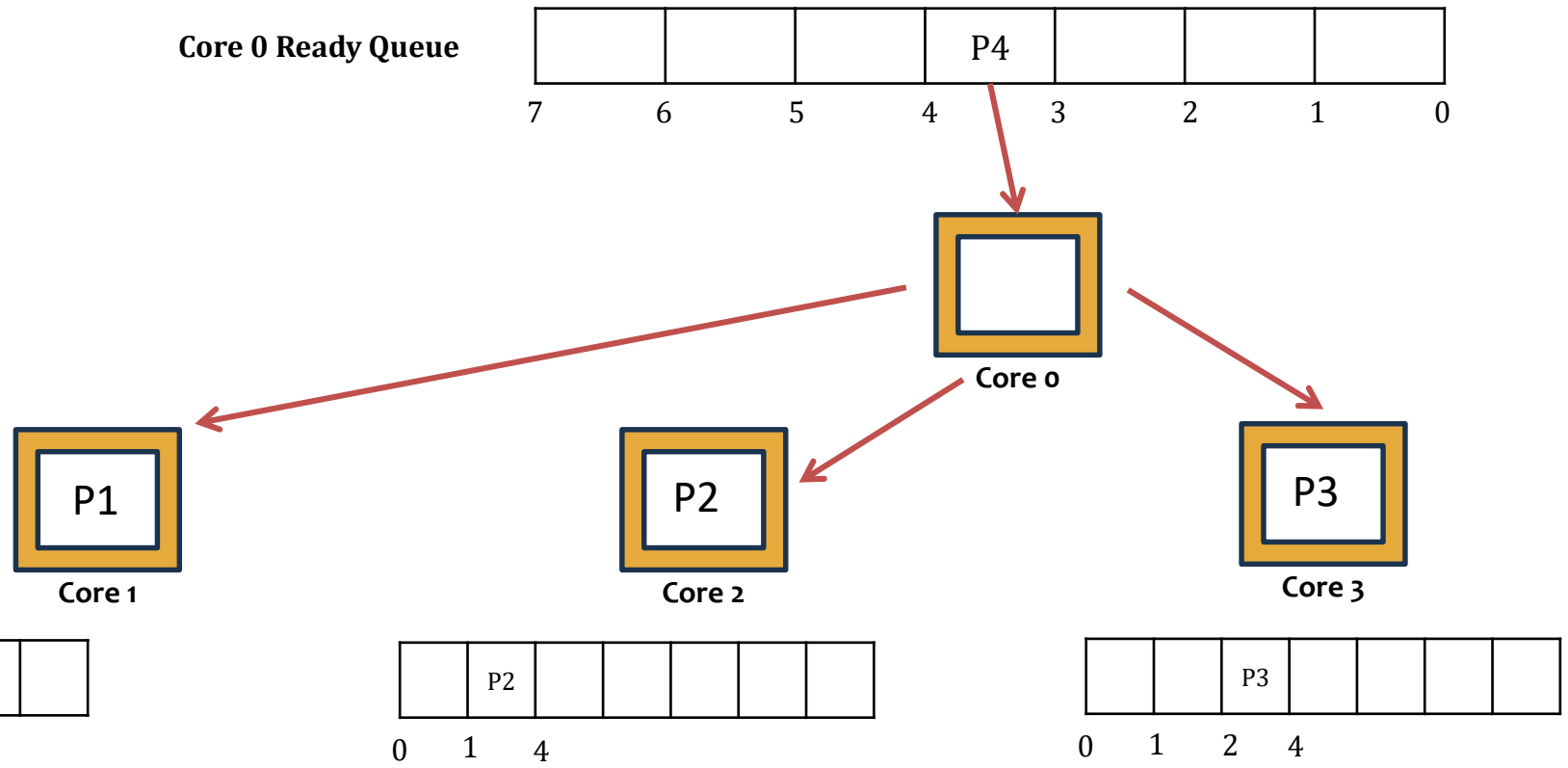
CPU Scheduling – Multi Processors Scheduling - AMP

Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms

Core 2 executes P2 till time 4
Core 3 executes P3 till time 4



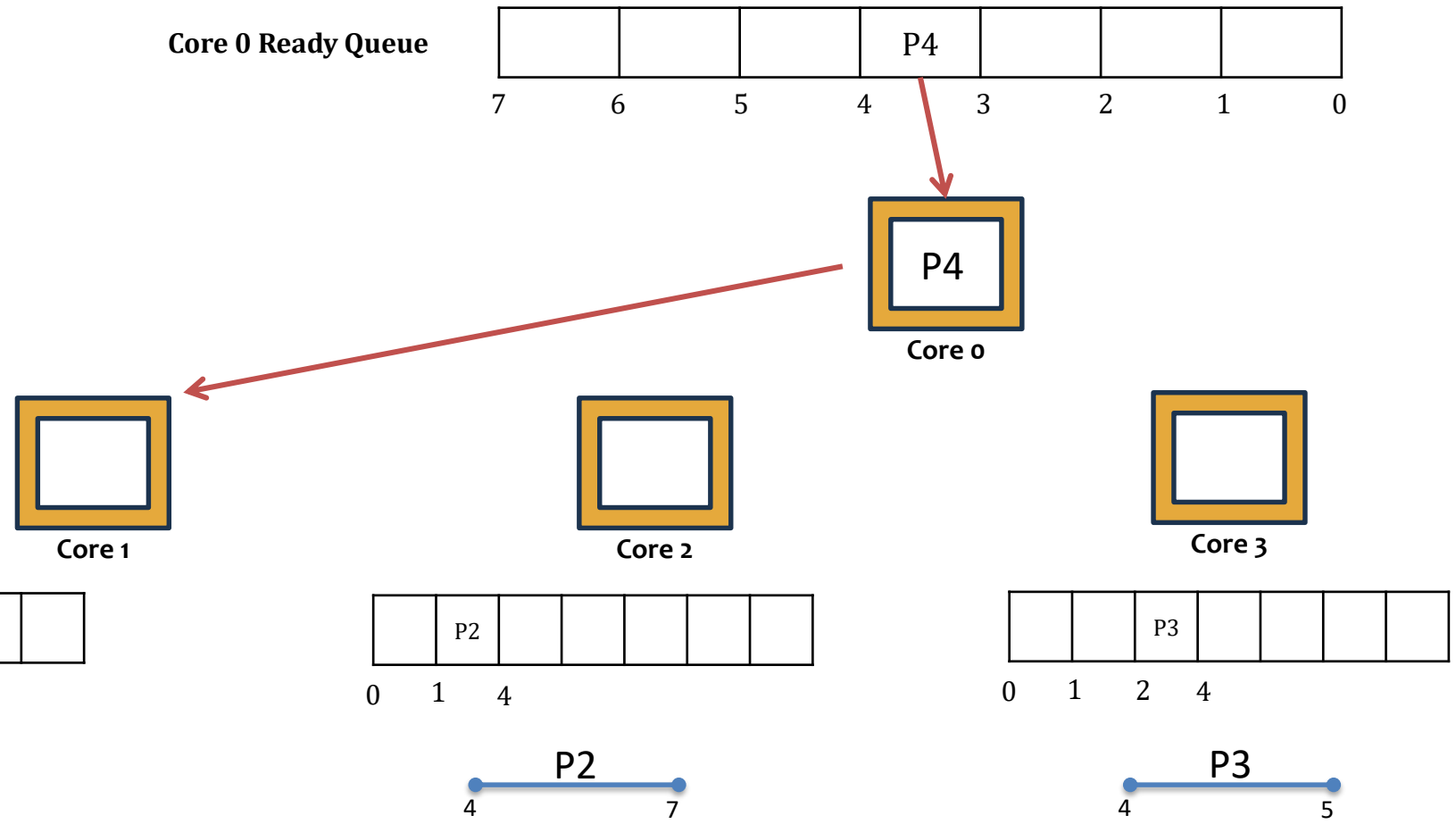
I/O Queue

CPU Scheduling – Multi Processors Scheduling - AMP

Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms

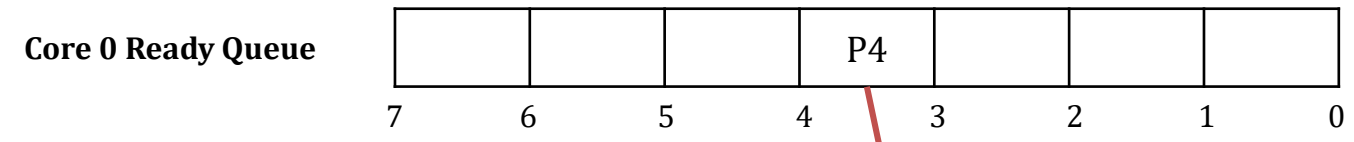


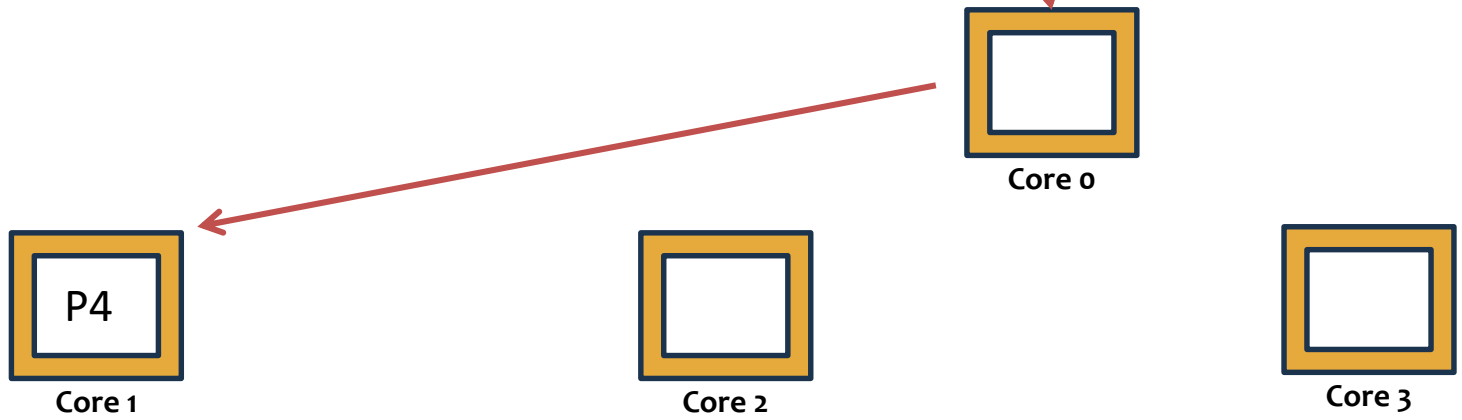
CPU Scheduling – Multi Processors Scheduling - AMP

Example

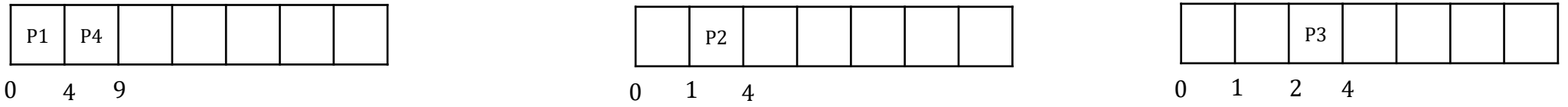
Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms





At t = 4, P1 switches to I/O leaving Core 1 Free
P4 is allocated to Core 1
P4 runs till 9



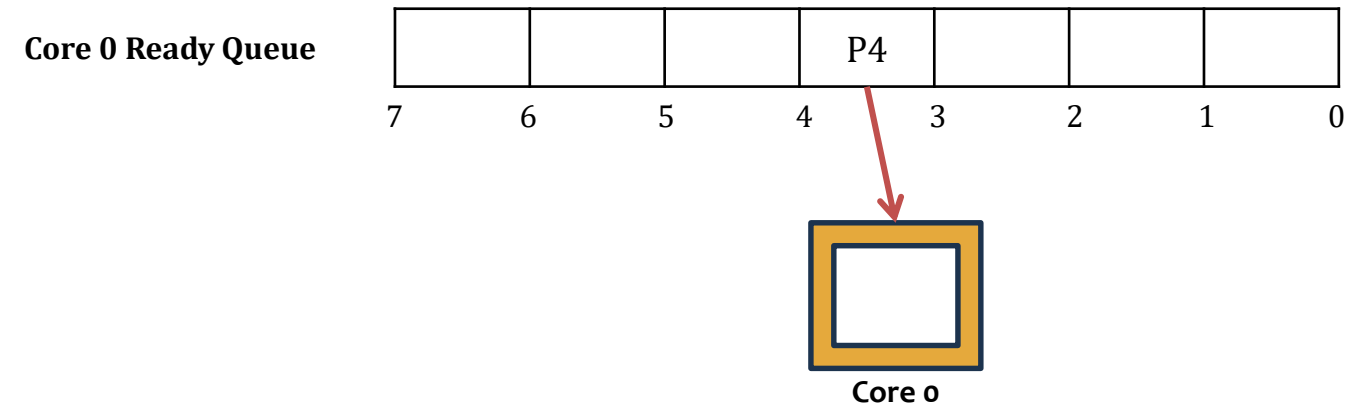


CPU Scheduling – Multi Processors Scheduling - AMP

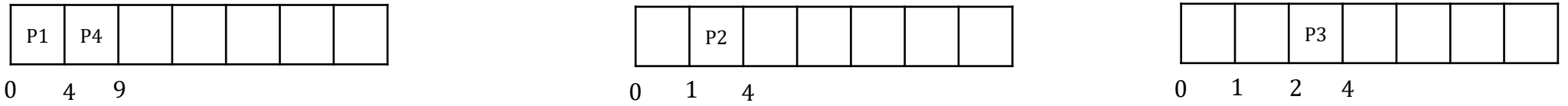
Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)
P_1	0 ms	4 ms	2 ms
P_2	1 ms	3 ms	3 ms
P_3	2 ms	2 ms	1 ms
P_4	3 ms	5 ms	2 ms



At t = 9, P4 switches to I/O leaving Core 1 Free





CPU Scheduling – Multi Processors Scheduling - AMP

Example

Each process requires a CPU Compute burst (executed on a Worker) followed immediately by an I/O burst (executed on the Master).

Process	Arrival Time (AT)	CPU Burst (Workers)	I/O Burst (Master)	$First\ CPU\ Start\ Time$ (ST)	Completion Time (CT)	Turnaround Time (TAT) = CT – AT	Waiting Time (WT) = TAT – BT	Response Time (RT) = ST - AT
P_1	0 ms	4 ms	2 ms	0	4	4	0	0
P_2	1 ms	3 ms	3 ms	1	4	3	0	0
P_3	2 ms	2 ms	1 ms	2	4	2	0	0
P_4	3 ms	5 ms	2 ms	4	9	6	1	1

CPU Scheduling – Multi Processors Scheduling - SMP

Example

2 identical cores, each with its own ready queue. Processes are split across cores

P1, P3 → Core-1;

P2, P4 → Core-2, and each core independently runs FCFS on its own queue.

Process	Arrival Time (AT)	CPU Burst (Workers)
P_1	0 ms	4 ms
P_2	1 ms	3 ms
P_3	2 ms	5 ms
P_4	3 ms	2 ms

Execution timeline:

Core-1: P1 runs **0→4**, then P3 runs **4→9**

Core-2: P2 runs **1→4**, then P4 runs **4→6**

PID	Core	AT	BT	Start	CT	TAT	WT	RT
P1	Core-1	0	4	0	4	4	0	0
P2	Core-2	1	3	1	4	3	0	0
P3	Core-1	2	5	4	9	7	2	2
P4	Core-2	3	2	4	6	3	1	1

Average Performance Metrics:

Average Turnaround Time = **4.25**

Average Waiting Time = **0.75**

Average Response Time = **0.75**

CPU Scheduling – Multi Processors Scheduling

Pros of Multi Processor Scheduling

- High throughput by parallel execution of tasks.
- Better resource utilization.
- Scalability for large applications and real-time systems.

Cons of Multi Processor Scheduling

- Complex implementation and maintenance.
- Difficult load balancing.
- Increased overhead due to process migration and affinity management.
- Virtualization may affect real-time guarantees.